

Power your Research
with
Artificial Intelligence-ready Advanced Infrastructure
for
Ab Initio Molecular Dynamics

User Manual

Version 3.0.0
Nov 12, 2026

Jingbai Li

Hoffmann Institute of Advanced Materials
Shenzhen Polytechnic University

Citation

The methods implemented in PyRAI3MD are described in the following papers. Please cite the ones that cover the techniques you used, together with the papers recommended by the engines, force fields and functionals that carried out your calculations.

Jingbai Li, Patrick Reiser, Benjamin R. Boswell, Andre Eberhard, Noah Z. Burns, Pascal Friederich, and Steven A. Lopez, "Automatic discovery of photoisomerization mechanisms with nanosecond machine learning photodynamics simulations", Chem. Sci. 2021, 12, 5302-5314. DOI:10.1039/D0SC05610C

Jingbai Li, Rachel Stein, Daniel Adrion, Steven A. Lopez, "Machine-learning photodynamics simulations uncover the role of substituent effects on the photochemical formation of cubanes", J. Am. Chem. Soc. 2021, 143, 48, 20166-20175. DOI:10.1021/jacs.1c07725

Jingbai Li, Steven A. Lopez, "Excited-state distortions promote the reactivities and regioselectivities of photochemical 4pi-electrocyclizations of fluorobenzenes", Chem. Eur. J. 2022, 28, e202200651. DOI:10.1002/chem.202200651

Jingbai Li, Steven A. Lopez, "A Look Inside the Black Box of Machine Learning Photodynamics Simulations", Acc. Chem. Res. 2022, 55, 1972-1984. DOI:10.1021/acs.accounts.2c00288

L. Wang, C. Salguero, S. A. Lopez, J. Li, "Machine learning photodynamics uncover blocked non-radiative mechanisms in aggregation-induced emission", Chem 2024, 10, 2295-2310. DOI:10.1016/j.chempr.2024.04.017

Z. Li, F. J. Hernandez, C. Salguero, R. Crespo-Otero, S. A. Lopez, J. Li, "Machine learning photodynamics decode multiple singlet fission channels in pentacene crystal", Nat. Commun. 2025, 16, 1194. DOI:10.1038/s41467-025-56480-y

Z. Li, H. Fu, S. A. Lopez, J. Li, "Machine learning photodynamics reveal intersystem-crossing-driven ladderdiene ring opening", Chem. Sci. 2025, 16, 13031-13041. DOI:10.1039/D4SC07395A

L. Wang, J. Li, "Modeling the Nonradiative Decay of Cyclooctatetrathiophene in Solution and Crystal Phases Using Machine Learning", J. Chem. Theory Comput. 2025, 21, 19, 9300-9308. DOI:10.1021/acs.jctc.5c01136

Contents

1. What is PyRAI3MD 6
1.1. Framework and skills 6
1.2. Relation to PyRAI2MD 6
1.3. How to read this manual 7
1.4. Citation and license 7
2. Features 8
2.1. Workflows 8
2.2. Calculation engines 8
2.3. Machine-learning potentials 9
2.4. Multiscale calculations 9
2.5. Dynamics and nonadiabatic dynamics 10
2.6. Periodic systems and constraints 10
2.7. Parallel and remote execution 10
2.8. Output, testing and tools 10
3. Installation 10
3.1. System requirements 11
3.2. Installing the package 11
3.3. Configuration layers 12
3.4. Setting up calculation engines 12
3.5. Verifying the installation 13
3.6. Installing on a cluster 14
4. Input files and conventions 14
4.1. Input carriers 14
4.2. Classic text input 15
4.3. YAML input 16
4.4. Agent JSON 18
4.5. How structure and guesses are resolved 19
4.6. Conventions 19
4.7. A first calculation in three carriers 20
5. Keyword sections 22
5.1. CONTROL 23

5.2. SYSTEM (MOLECULE) 25
5.3. DYNAMICS 34
5.4. OPTIMIZE 46
5.5. HESSIAN 53
5.6. SEARCH 56
5.7. ADAPTIVE 59
5.8. TRAIN 72
5.9. PREDICTION 73
5.10. READ 74
5.11. COMPOSITE 76
5.12. ENVIRONMENT and SLURM 76
5.13. CONSTRAINT 82
5.14. Common keywords in skill sections 88
5.15. PYSCF 90
5.16. MOLCAS 101
5.17. BAGEL 113
5.18. ORCA 124
5.19. OPENQP 133
5.20. XTB 143
5.21. CP2K 149
5.22. LAMMPS 161
5.23. QDM 171
5.24. NN_PES 179
6. Workflows 188
6.1. Job types and workflow resolution 188
6.2. Single-point and property calculations 189
6.3. Geometry optimization 190
6.4. Reaction paths (MEP, IRC, NEB) 192
6.5. Ab initio MD and NAMD 194
6.6. Hessian, frequencies and thermochemistry 196
6.7. Reading finished calculations 198
6.8. Artifacts and restart per workflow 198
7. Multiscale calculations 199

7.1. Additive QM/MM 199
7.2. ONIOM2 and ONIOM3 201
7.3. Multi-centre plans 201
7.4. Embedding charges and self-consistency 201
7.5. Limitations 202
8. Periodic systems 202
8.1. Cell, lattice and periodic flags 203
8.2. Stress and variable-cell optimization 203
8.3. Dynamics under PBC 204
8.4. Which engines support PBC 204
9. Constraints 205
9.1. Constraint families 205
9.2. Writing the block 205
9.3. Where constraints act 207
9.4. Refused combinations 207
10. Machine learning potentials 208
10.1. Training data 208
10.2. Training 209
10.3. Prediction 210
10.4. Hyperparameter search 210
10.5. Adaptive sampling 211
10.6. Data tools 212
11. Parallel and remote execution 212
11.1. The two dispatch axes 212
11.2. Slurm child runtimes 213
11.3. Runtime files and restart 214
12. Output and analysis 215
12.1. The run log 215
12.2. Result and trajectory files 216
12.3. Trajectory and structure tools 216
13. Examples and tutorials 217
13.1. The shipped examples 217
13.2. Tutorial: energy, optimization and frequencies 220

13.3. Tutorial: surface hopping 222
13.4. Tutorial: QM/MM 224
13.5. Tutorial: adaptive sampling 226
14. Qualification and verification 228
14.1. The qualification round 228
14.2. Remote rounds 229
14.3. Post-round verification 230
15. Extending PyRAI3MD 231
15.1. The skill contract 231
15.2. Writing a skill 232
15.3. Where to read more 233
A. Units and sign conventions 233
B. Index bases and derived quantities 236
C. Keyword status index 238
D. Refusal and limitation matrix 242
E. Migrating PyRAI2MD inputs 244
F. Output file index 246
G. Troubleshooting 248
H. Glossary 250

1. What is PyRAI3MD

PyRAI3MD is a framework for molecular and periodic simulations that mixes quantum chemistry, classical force fields, machine-learning potentials and nonadiabatic dynamics in one input file. It computes energies, gradients, couplings and spectra with ten interchangeable engines, propagates trajectories with surface hopping, optimizes minima, transition states and conical intersections, searches reaction paths, trains neural-network potentials and improves them by adaptive sampling.

1.1. Framework and skills

The program is split into a framework and a set of skills. The framework owns everything that is not physics: the input schema, the planning layer that turns a request into a list of tasks, the workflow loops, the logging, the parallel dispatch and the protocols every engine implements. A skill owns one scientific engine: its configuration schema, its input builder, its runner, its output parser and the conversion of its native units into the framework units.

The split is enforced by a one-way import rule. The framework never imports an engine module, and a skill never imports the framework planning or workflow layers; the two meet at the skill manifest, the query module and the SkillConfig schema. This is what lets a calculation run with PySCF today and with OpenMolcas tomorrow by editing one word of the method queue, and it is what allows an engine to be added without touching the workflow code.

The same tree serves three entry points: the classic or YAML command pyrai3md, the agent command pyrai3md_agent for sectioned JSON, and the single-skill command pyrai3md_skill. A child runtime that the framework writes for a parallel or remote task goes through the same pipeline, so a calculation behaves the same whether it runs on the login node or inside a scheduler job.

1.2. Relation to PyRAI2MD

PyRAI3MD is the successor of PyRAI2MD. The scientific core is the same: the neural-network potentials, the trajectory surface hopping algorithms and the adaptive-sampling idea were developed and validated in the PyRAI2MD papers listed in Section Section 1.4. What changed is the architecture and the reach of the program.

The engines became skills, so the set of methods is open rather than fixed. The input gained two structured carriers beside the classic text format, and the configuration is layered over a home file. Planning became an explicit step that resolves capabilities, units and partition before a single calculation starts, which is why an unsupported request is reported as a refusal instead of failing halfway through a trajectory. The workflow set grew to optimization, path search, Hessians, thermochemistry, grid search and label extraction. Multiscale work is expressed as a method queue, and periodic systems, constraints and scheduler dispatch are first-class rather than bolt-ons.

Input files written for PyRAI2MD are not expected to run unchanged. Appendix Section E is the migration table: it pairs each retired spelling with its replacement and marks the few constructs, such as the hop jobtype, that were removed rather than renamed.

1.3. How to read this manual

Chapter 2 lists the features and the engines. Chapter 3 installs the package and sets up engines. Chapter 4 explains the three ways to write an input and the conventions behind them. Chapter 5 is the keyword reference, one section per input section and one per skill, with the default of every keyword in red. Chapters 6 to 12 explain what each workflow does and how the pieces fit together. Chapter 13 walks through complete examples, Chapters 14 and 15 cover testing and extension, and the appendices hold the unit table, the index bases, the keyword status index and the refusal matrix.

In the keyword tables a blue keyword is followed by its default in red and a one-line description. The marks used in those tables and in the prose are: a dagger for a keyword that is accepted but has no effect in v3.0.0, a double dagger for a keyword that honours only one of its values, a section sign for an alias, and an asterisk for a derived quantity that is not read from the input. Defaults are quoted from the configuration model the program loads, so they describe version 3.0.0 and not an older release.

Code appears in grey boxes. Coordinates are in Angstrom, energies in Hartree and gradients in Hartree per Bohr unless a box says otherwise; Section Section 4.6 states the six conventions a reader needs and Appendix Section A gives the full table.

1.4. Citation and license

PyRAI3MD is distributed under the PyRAI3MD Research License 1.0; the LICENSE file in the repository is the authoritative text. The methods it implements are described in the following papers. Please cite the ones that cover the techniques you used.

Jingbai Li, Patrick Reiser, Benjamin R. Boswell, Andre Eberhard, Noah Z. Burns, Pascal Friederich, and Steven A. Lopez, "Automatic discovery of photoisomerization mechanisms with nanosecond machine learning photodynamics simulations", Chem. Sci. 2021, 12, 5302-5314. DOI:10.1039/D0SC05610C

Jingbai Li, Rachel Stein, Daniel Adrion, Steven A. Lopez, "Machine-learning photodynamics simulations uncover the role of substituent effects on the photochemical formation of cubanes", J. Am. Chem. Soc. 2021, 143, 48, 20166-20175. DOI:10.1021/jacs.1c07725

Jingbai Li, Steven A. Lopez, "Excited-state distortions promote the reactivities and regioselectivities of photochemical 4pi-electrocyclizations of fluorobenzenes", Chem. Eur. J. 2022, 28, e202200651. DOI:10.1002/chem.202200651

Jingbai Li, Steven A. Lopez, "A Look Inside the Black Box of Machine Learning Photodynamics Simulations", Acc. Chem. Res. 2022, 55, 1972-1984. DOI:10.1021/acs.accounts.2c00288

L. Wang, C. Salguero, S. A. Lopez, J. Li, "Machine learning photodynamics uncover blocked non-radiative mechanisms in aggregation-induced emission", Chem 2024, 10, 2295-2310. DOI:10.1016/j.chempr.2024.04.017

Z. Li, F. J. Hernandez, C. Salguero, R. Crespo-Otero, S. A. Lopez, J. Li, "Machine learning photodynamics decode multiple singlet fission channels in pentacene crystal", Nat. Commun. 2025, 16, 1194. DOI:10.1038/s41467-025-56480-y

Z. Li, H. Fu, S. A. Lopez, J. Li, "Machine learning photodynamics reveal intersystem-crossing-driven ladderdiene ring opening", Chem. Sci. 2025, 16, 13031-13041. DOI:10.1039/D4SC07395A

L. Wang, J. Li, "Modeling the Nonradiative Decay of Cyclooctatetrathiophene in Solution and Crystal Phases Using Machine Learning", J. Chem. Theory Comput. 2025, 21, 19, 9300-9308. DOI:10.1021/acs.jctc.5c01136

The programs that PyRAI3MD drives must be cited according to their own instructions, and the force fields and functionals used through CP2K, LAMMPS, xTB or the quantum-chemistry engines carry their own recommended references.

2. Features

This chapter summarises what the program does. Each item is one or two sentences and points to the chapter that explains it.

2.1. Workflows

The jobtype selects the workflow. A property jobtype evaluates one capability set, while an advanced jobtype owns an iteration. The table lists every jobtype the validator accepts; Section Section 6.1 explains when to use which.

Jobtype	What it does
energy, gradient, nac, soc	single evaluation of the requested properties
dipole, transition, absorption	transition moments and spectra
hessian, frequency	force constants, vibrations and thermochemistry
num_hess, num_freq	numerical force constants from displaced gradients
optimize	minima, GDIIS, transition state, MECI, MECP, variable cell
optimize with mep/irc/neb	reaction paths and bands
md, surface_hopping	ab initio MD and nonadiabatic dynamics
adaptive	iterative reference labelling, training and screening
train, prediction	machine-learning potential fitting and scoring
search	hyperparameter grid evaluation and ranking
read	label extraction from finished calculations

2.2. Calculation engines

Ten skills ship with the package. The capability list is the vocabulary the planner uses to decide whether a request can be served; a jobtype that needs a capability the chosen engine does not declare is refused before the run starts. The role column shows where an engine can sit in a method queue: qm is the high layer, qm2 the middle layer and mm the classical outer layer.

Engine	Capabilities	Roles	PBC	Profiles
pyscf	energy, gradient, frequency, geometry_optimization, ir_spectrum, uv_spectrum, nmr_spectrum, analysis	qm, qm2	no	39

Engine	Capabilities	Roles	PBC	Profiles
molcas	energy, gradient, nonadiabatic_coupling, geometry_optimization, abinitio_dynamics, surface_hopping, absorption_spectrum, soc_coupling, vibrational_frequency	qm	no	15
bagel	energy, gradient, nonadiabatic_coupling, geometry_optimization, vibrational_frequency, dipole, transition_dipole	qm	no	14
orca	energy, gradient, dipole, transition_dipole, nonadiabatic_coupling, soc_coupling, geometry_optimization, vibrational_frequency	qm	no	14
openqp	energy, gradient, nonadiabatic_coupling, soc_coupling, geometry_optimization, nonadiabatic_dynamics, vibrational_frequency	qm	no	17
xtb	energy, gradient, stress	qm, qm2, mm	yes	8
cp2k	energy, gradient, stress, transition_dipole, geometry_optimization, abinitio_dynamics	qm	yes	9
lammps	energy, gradient, stress, geometry_optimization, classical_dynamics	mm	yes	9
qdm	energy, gradient, nac, md	qm	no	24
nn_pes	energy, gradient, nac, soc	qm	no	25

The engines divide into three groups. PySCF, OpenMolcas, BAGEL, ORCA and OpenQP are molecular quantum-chemistry codes. xTB and CP2K add semi-empirical and periodic density functional work. LAMMPS is the classical engine and also supplies the QM/MM field, while QDM provides the analytic two-state models used to validate the surface-hopping code. The tenth skill, nn_pes, evaluates a trained neural-network potential. Sections Section 5.15 to Section 5.24 document each skill.

2.3. Machine-learning potentials

The nn_pes skill trains an ensemble of neural networks on energies, gradients and, when requested, nonadiabatic and spin-orbit couplings. It serves the ensemble inside any workflow that needs a potential, and it reports the spread between members as an uncertainty. Training, prediction and hyperparameter search are workflows of their own, and adaptive sampling closes the loop by running trajectories on the current model, selecting uncertain structures, labelling them with a reference engine and retraining. Chapter 10 covers the data format and the loop.

2.4. Multiscale calculations

A method queue of two entries is an ONIOM2 composite, or additive QM/MM when multiscale_type is qmmm; three entries are ONIOM3. Regions come from highlevel and midlevel, embedding charges come from point charges or a charge file, and a multi-centre run repeats the whole plan for each centre with optional charge self-consistency. Chapter 7 gives the energy expressions and the supported jobtypes.

2.5. Dynamics and nonadiabatic dynamics

The md workflow integrates with velocity Verlet in NVE, NVT or NPT, with Nose-Hoover, Nose-Hoover-chain or CSVR thermostats and Berendsen, Parrinello-Rahman, MTTK or stochastic-cell-rescaling barostats. Initial conditions can be read or sampled from a frequency file. Three surface-hopping algorithms are available: fewest switches with explicit or curvature-driven couplings, Granucci-Persico style hopping on all-state gradients, and Landau-Zener hopping that needs energies only. Guards stop a trajectory when a population, an energy, a gradient, a coupling or a tracked geometry leaves its window. Section Section 6.5 explains the step order and every switch.

2.6. Periodic systems and constraints

A cell is given by pbc, cell and lattice_vector. Engines that declare periodic support run minimum-image dynamics and can supply stress, which lets the optimizer relax the cell at a target pressure. Constraints are a separate block: harmonic biases, confining walls, fixed atoms, SHAKE and metadynamics act on dynamics and, where meaningful, on optimization. Chapters 8 and 9 cover both.

2.7. Parallel and remote execution

Two independent axes control execution. multitask runs independent units in a local process pool; remotetask runs them as child runtimes, normally through Slurm, with a generated script. multipool sets the worker count of a stage, and the adaptive workflow can override both axes per stage. Chapter 11 describes the axes, the scheduler section and the runtime files that make a run restartable.

2.8. Output, testing and tools

Every run writes a log named after its title and, depending on the workflow, trajectory, status, model or summary files. Chapter 12 lists them and the analysis tools that read them. The package ships a qualification suite that runs framework tests, shipped examples, skill certification cases and benchmarks through the same entry points a user uses; Chapter 14 explains how to run it locally or on a cluster and how to verify a finished round.

3. Installation

This chapter installs the package, points it at the engines it will drive, and checks the result. The order matters: the three commands appear only after the install, and an engine is found only after its section in the configuration file names it.

3.1. System requirements

PyRAI3MD is a pure Python package. It runs under Python 3.9 or newer and is tested with 3.11. Four pip libraries are needed by every run: numpy >= 1.20 for the array core, pydantic >= 2 for the configuration model and its validation, PyYAML >= 6 for the YAML carrier and the environment file, and matplotlib >= 3.5 for the population plots of the benchmark calibrations.

Two optional packages extend a run without being needed by it. numba >= 0.56 accelerates the FSSH propagator kernels; the NumPy reference runs without it. TensorFlow >= 2.20 is needed only by the neural network potential skill, stays out of the default install because of its size, and is reported when missing. PyTorch >= 2.9 is optional on the same terms.

Engines are external installations, with two exceptions: PySCF runs in process and is pip-installed, and the QDM models are analytic and need only numpy. Section 3.4 lists what each engine needs on the host.

Dependency	Required	Needed for
Python >= 3.9	Yes	Interpreter; 3.11 is tested
numpy >= 1.20	Yes	Array core of framework and skills
pydantic >= 2	Yes	Configuration model and validation
PyYAML >= 6	Yes	YAML carrier and environment file
matplotlib >= 3.5	Yes	Benchmark population plots
numba >= 0.56	No	Accelerated FSSH kernels
TensorFlow >= 2.20	No	The nn_pes skill
PyTorch >= 2.9	No	Neural-network calculations
Engine installations	No	Engine jobs; see Section 3.4

3.2. Installing the package

Download the repository and enter it.

```
git clone <repository-url>
cd PyRAI3MD
```

Install into the active interpreter. The install creates three commands: pyrai3md for YAML and classic text, pyrai3md_agent for sectioned JSON, and pyrai3md_skill for one skill launched from a flat JSON config.

```
pip install .
```

The three commands enter the same pipeline at different carriers (Section 4). A remote install by the deploy tool follows the same sequence (Section 14).

3.3. Configuration layers

Configuration is layered, and a run resolves each key from the highest layer that sets it: the environment and slurm sections of the run input, then ~/.pyrai3md/pmd_env.yaml, then the package template pyrai3md/qualification/pmd_env.yaml. The precedence is input > home > package.

The package template ships the defaults and is not edited. The home file is created on the first pyrai3md test run and holds engine paths, skill execution modes and shell lines, the scheduler, the deploy target and the switches.

The merge is per key: a key in the home file replaces the package value and leaves its siblings alone. An empty value counts as unset and falls back, while false and 0 are values. An input section merges the same way for that run alone.

```
environment:
  python: python3
  scheduler: local
  shell:
    - module load compiler/gcc/gcc-11.3.0
skills:
  molcas_skill:
    molcas_path: /opt/molcas-26
    execution_mode: local
deploy:
  host: cluster.example.org
  username: user
  install_path: /home/user/pyrai3md
```

The environment section holds the interpreter, the conda environment, the scheduler and the shell and export lists; slurm holds the directives a dispatched child or grouped job asks for. A user file that names only what a host changes is normal. Section 5.12 documents both.

3.4. Setting up calculation engines

Each engine is a skill, and the framework never imports one. The skill reads its installation path, executable and parallel settings from its own section; the framework only forwards those keys. PySCF runs from the interpreter and the QDM models are analytic, so neither needs an installation.

Engine modules and the site base environment are loaded by the skill, not the framework. A skill section carries a shell list of module load lines and exports, and the skill copies that list into every script it writes, so a host states the CP2K compiler and MPI modules once, in the cp2k_skill section.

Engine	Must be on the host	Key that names it	Notes
pyscf	PySCF importable in the interpreter	pyscf_path	Set to import for the current environment; no launcher.

Engine	Must be on the host	Key that names it	Notes
molcas	OpenMolcas tree with pymolcas in it	molcas_path	Needs the site module that declares the base directory.
bagel	BAGEL tree with $BAGEL/bin/BAGEL and its library variables	bagel_path	The library variables are exported in the shell list.
orca	ORCA 5.0 or newer with an orca executable	orca_path	A parallel build also names its MPI library.
openqp	An openqp command in PATH, or OPENQP_ROOT	openqp_path	An MPI launcher only with use_mpi.
xtb	An xTB tree, the binary, or xtb in PATH	xtb_path	Threads and memory are separate keys.
cp2k	A CP2K installation with an executable	cp2k_path	cp2k_exe overrides it; setup_script is sourced first.
lammps	A LAMMPS tree, the binary, or lmp in PATH	lammps_path	The launch directory must hold a topology file.
qdm	Python and numpy only	none	Analytic models; two states by construction.
nn_pes	TensorFlow >= 2.20 in the interpreter	none; modeldir names the folder	Discovered without TensorFlow; refuses a run by naming it.

OpenMolcas is the one engine whose setup is more than a path. Its launcher resolves the installation from the environment the site module exports, so the shell list loads that module and the MOLCAS variable it exports, while molcas_path names the same tree.

PySCF is the engine to start with: a pip install, no module, and pyscf_path: import means the running interpreter is the installation. The nn_pes modeldir is empty on purpose, because it names a folder a training run produced on a particular machine. Section 5.15 to Section 5.24 document each skill section.

3.5. Verifying the installation

Two commands check an installation. The info switch parses the input, resolves every layer and prints the configuration without executing anything.

```
pyrai3md energy.yaml --info
```

The qualification round then exercises the installation end to end.

```
pyrai3md test
```

The round plans from ~/.pyrai3md/pmd_env.yaml, opens a directory named tmp_YYYYMMDD_HHMMSS in the current directory, and writes two verdict files: results.json, one record per task, and status.json, the terminal state (pass, fail, error or skip). A local round leaves one row fragment per task under jobs/; a grouped Slurm round leaves the job specs, batch scripts and fragments there.

The work directory is disposable; the next round makes its own. What to run is decided in the home file rather than on the command line, through the _runall master, the family and skill switches and the per-case keys; a host that carries PySCF alone should leave the other families closed. Section 14 explains the categories, switches and verifier.

3.6. Installing on a cluster

A cluster installation is the same install done remotely, and the deploy tool performs it: it syncs the checkout to the configured host, installs it, starts a round and collects the result. The round directory stays on the host; only the log comes back as test_<stamp>.log.

```
python tools/deploy.py
python tools/deploy.py --force
```

The deploy section names the host, the user, the install path and optionally a jump host; environment.scheduler runs the round in one local process, or as one grouped Slurm job per carrier when it reads slurm. With Slurm, environment.max_in_flight caps unfinished jobs, and 0 submits the whole round. The force form cancels the jobs a Slurm round recorded and stops the orchestrator; work a skill submitted itself follows that skill section. Section 14 gives the sequence and what each step leaves.

4. Input files and conventions

This chapter describes the files a user writes and the conventions every carrier shares. PyRAI3MD reads three user carriers - classic &SECTION text, sectioned YAML and agent JSON - and writes two serialized payloads of its own, .rt runtime files and .rtt term files. All five lead to one validated configuration or to the state built from it; the carrier decides how sections and values are spelled, never what a keyword means.

4.1. Input carriers

Classic text is the historical PyRAI2MD style, YAML is the form the shipped examples use, and agent JSON is the form the agent and the programmatic interface consume. The two runtime suffixes are written by the program and are not input formats.

Carrier	Entry command	First line	Used by
classic text	pyrai3md water.input	&CONTROL	command line; generated example carriers
sectioned YAML	pyrai3md water.yaml	control:	command line; shipped examples
agent JSON	pyrai3md_agent water.json	{	agent command line and PyRAI3Agent
.rt runtime	pyrai3md water.rt	pickled PyRAIState	restart and remote dispatch
.rtt term payload	pyrai3md water.rtt	pickled term payload	one remote term worker

The three user carriers enter one pipeline: parse, build the configuration, normalize, validate, build the runtime state and execute. Classic text and YAML enter through interface.parser, agent JSON through interface.loader; from the configuration model onward the route is identical. A keyword means the same thing and a mistake is refused at the same stage whichever file carried it. The same pipeline therefore serves the command line, the agent and child runtimes, which differ only in where the state comes from.

A .rt file is a serialized runtime state written for restart or remote dispatch; it re-enters execution at the restored state without parsing an input file. A .rtt file is a single-term payload for a remote worker; it executes one backend term and writes term_outcome.json. Neither suffix is a user input format.

4.2. Classic text input

A line whose first non-blank character is & opens a section and names it; the name is case-insensitive. A line &end closes the current section, and any key after it is ignored until the next section header. Inside a section each line is a keyword followed by one or more whitespace-separated values: the first token is the key, the rest of the line is its value. Keys are case-insensitive, blank lines are skipped, and a # starts a comment that runs to the end of the line.

Several values on one line are kept as one space-joined string, which the section reader splits according to the keyword type. A keyword repeated inside one section is different: all its values are kept, in order, as a list. Nothing is silently overwritten; a section reader that cannot use several values refuses the input and names the key. Booleans are true, false, 1 or 0. The aliases qm, abinit, temp and lattice are read as theory, reference, temperature and lattice_vector, and the log shows the modern names.

The input below is one complete water single point at the Hartree-Fock/STO-3G level. The comments mark the three parts every run needs: control, the molecular specification and one skill section. &MOLECULE is the historical spelling of the system section; &PYSCF is the skill section, and &PYSCF_SKILL is accepted as the same section. Unlike YAML and JSON, classic text has no inline structure, so the run reads water.xyz beside the input, or the path named by &MOLECULE coord.

```
# water single point, classic text carrier
&CONTROL
title        water_energy     # names <title>.log and the scratch tree
jobtype      energy          # one evaluation; advanced jobs stand alone
theory       pyscf           # the method queue, high to low
driver       main            # the framework owns the loop

&MOLECULE
charges      0               # total charge
multiplicity 1               # singlet
ci           1               # number of states per multiplicity
spin         0               # total spin per state

&PYSCF
method       hf              # the skill reads this key in its own section
basis        sto3g
&end
```

Three separators carry structure inside a value. Whitespace separates the keyword from its values and separates several values from each other. A comma separates the two state indices of one pair. A semicolon separates centres in the multi-centre keywords. The table collects these conventions with the two line-level tokens.

Separator	Meaning	Example
space	keyword from value; several values in one string	theory pyscf molcas
, (comma)	state indices inside one pair	coupling 1 2, 2 3
; (semicolon)	centres, in the multi-centre keywords	ci 2 2; 2 2
#	comment to the end of the line	title water # run name
blank line	skipped; it has no meaning	
&end	closes the current section	&end

Note. *A repeated key is a list, not an overwrite. The section reader refuses a second value it cannot use instead of discarding one.*

4.3. YAML input

A YAML input is one top-level mapping. Each framework section is a mapping named by its lower-case section name: control, molecule, md, environment, and so on. The parser lower-cases top-level keys, and native YAML types are preserved, so a scalar stays a scalar and a list stays a list. Comments use # and run to the end of the line.

The control mapping carries title, jobtype and theory; theory is a list because a method queue is a list even when it holds one entry. The structure arrays are top-level keys, atoms and coordinates, beside control and not inside molecule. The coordinate resolver returns early when a top-level atoms key exists, so an atoms list nested under molecule is never seen by that check; the file then needs a coord path or a same-basename .xyz, and otherwise reaches the no-coordinates error. Section 4.5 states the full order.

A skill section is named after the skill: both pyscf: and pyscf_skill: resolve to the PySCF skill, and the short spelling is the one the shipped examples use. Lists keep their native shape, which matters for ci, spin and target_states. A pair field written as a string, for example coupling: "1 2", remains external and 1-based; a typed nested list is already the normalized 0-based entry.

The water single point in YAML keeps the same three parts as the classic file, with the structure written inline.

```
# water single point, YAML carrier
control:
  title: water_energy
  jobtype: energy
  theory: [pyscf]

atoms: [O, H, H]
coordinates:
  - [0.000, 0.000, 0.000]
  - [0.000, 0.757, 0.587]
  - [0.000, -0.757, 0.587]

molecule:
  charges: 0
  multiplicity: 1
  ci: [1]
  spin: [0]

pyscf:
  method: hf
  basis: sto3g
```

A dynamics input adds an md mapping. The example runs ten steps of FSSH on two singlet states under an NVT thermostat. ci and spin are lists; coupling is a string, so its pair is read as external 1-based indices; root is 1-based in every external spelling. The skill section selects TDA with PBE and asks for two states.

```
# two-state FSSH dynamics, YAML carrier
control:
  title: water_namd
  jobtype: md
  theory: [pyscf]

atoms: [O, H, H]
coordinates:
  - [0.000, 0.000, 0.000]
  - [0.000, 0.757, 0.587]
  - [0.000, -0.757, 0.587]

molecule:
  charges: 0
  multiplicity: 1
  ci: "2"
  spin: "0"
  coupling: "1 2"
```

```
md:
  ensemble: nvt
  thermostat: nose-hoover
  sfhp: fssh
  root: 1
  step: 10
  size: 20.67

pyscf:
  method: tda
  functional: pbe
  basis: sto-3g
  nstate: 2
  target_states: [0, 1]
```

4.4. Agent JSON

Agent JSON is a sectioned JSON document. Its top-level keys are the section names of the other carriers plus one section per skill, named by the skill package, for example pyscf_skill. The loader lower-cases top-level keys and does no type conversion; build, normalize, validate and state construction then follow the same pipeline. Because JSON has no comments, annotation has to live in the surrounding text.

The command line is pyrai3md_agent config.json. In Python, PyRAI3Agent.from_file(path) returns an agent that runs the same pipeline; initialize() is optional because run() initializes on first use. The programmatic run() writes no log, because a library caller owns its output. The command line calls run_cli(), which goes through the framework RunLogger and writes the same <title>.log and closing status line as pyrai3md.

```
{
  "control": {
    "title": "water_energy",
    "jobtype": "energy",
    "theory": "pyscf"
  },
  "molecule": {
    "atoms": ["O", "H", "H"],
    "coordinates": [[0.000, 0.000, 0.000],
                    [0.000, 0.757, 0.587],
                    [0.000, -0.757, 0.587]],
    "charges": 0,
    "multiplicity": 1,
    "ci": [1],
    "spin": [0]
  },
  "pyscf_skill": {
    "method": "hf",
    "basis": "sto3g",
    "execution_mode": "local"
  }
}
```

Note. *from_file(...).run() is the library route; pyrai3md_agent config.json calls run_cli(). Only run_cli() writes the log.*

4.5. How structure and guesses are resolved

Coordinates and velocities are resolved in the interface layer before the configuration is built. The routes are tried in a fixed order and the first that applies wins. The table states each route as the user writes it and as the program reads it.

Route	What you write	What is read
1	top-level atoms and coordinates in YAML, JSON or agent input	used as they stand; no file is opened
2	molecule coord <file.xyz>	the first value that names an existing file
3	a same-basename .xyz beside the input	water.xyz for water.input, water.yaml or water.json
4	molecule velo <file>; otherwise <coord base>.velo or <coord base>.velocity.xyz	that velocity file; with none, zero velocities
5	the skill-declared initial guess {title}<suffix> or <input stem><suffix>	the first candidate present, stored for that skill field
6	nothing that supplies a structure	a hard error naming the three coordinate routes

Route 4 is separate from the coordinate choice: the velocity path is read from the molecule section if it names an existing file, otherwise the coordinate base supplies .velo or .velocity.xyz, and otherwise the run starts from rest and prints an informational line with the atom count.

Route 5 is driven by the skill manifest. A skill that starts from a converged file declares an initial_guess suffix and field; a skill that declares nothing is never consulted. The candidates are checked in the input file directory, title first and input stem second, and the resolved path is stored with its skill and field. The task queue fills that field only when the input left it empty, so an explicit path in the skill section always wins.

When no route supplies atoms or coordinates, the error is raised before any engine starts and names three ways forward: a molecule coord path, a same-basename .xyz file, or atoms and coordinates in the input itself. Route 1 tests a top-level atoms key; an atoms list nested under molecule does not satisfy it, so the file falls through to routes 2 and 3 and, if neither finds a file, reaches route 6.

4.6. Conventions

Six conventions apply to every carrier. They belong to the framework, not to an engine or a file format, so a number keeps its meaning after a switch from classic text to YAML.

Coordinates are in Angstrom, and so are cell lengths, while cell angles are in degrees; velocities travel in Bohr per atomic unit of time. Energies are in Hartree. Gradients are in Hartree/Bohr and

are the derivative dU/dR, not a force: an engine that prints a force block, in its own energy-per-length unit and with the opposite sign, is converted by the parser before the values reach the framework.

The MD keyword size is a time step in atomic units of time, not in femtoseconds. One atomic unit is 0.02418884254 fs, so the common value 20.67 au is about 0.5 fs. The hopping thresholds gap and gapsoc are in eV and are converted to Hartree internally. Other keywords carry the unit named in their own table in Section 5.

Atom indices are 1-based in every carrier, including the region keywords, the coupling pairs and the constraint block. State indices are mixed: the initial state root is 1-based externally, while current_state, opt_state_i and opt_state_j are 0-based. A dynamics run started with root 1 therefore begins on the state that current_state 0 names. Section B tabulates every index-bearing keyword and its base.

At the framework boundary the units are Hartree, Hartree/Bohr and Hartree/Bohr^3. A skill receives and returns values in these units and converts its native units before returning: electronvolts, kcal/mol, Angstrom, femtoseconds and engine-specific force blocks are converted inside the skill or by the parser that reads its output. The framework never inspects an engine unit system.

4.7. A first calculation in three carriers

The same water single point can be written in each carrier. Save the three input files as water.input, water.yaml and water.json, with water.xyz beside water.input: O at the origin and two H at (0, 0.757, 0.587) and (0, -0.757, 0.587) Angstrom. Only the carrier changes; the title, jobtype, structure and PySCF keywords are the same.

```
&CONTROL
title        water_energy
jobtype      energy
theory       pyscf

&MOLECULE
charges      0
multiplicity 1
ci           1
spin         0

&PYSCF
method       hf
basis        sto3g
&end
```

```
control:
  title: water_energy
  jobtype: energy
  theory: [pyscf]
```

```
atoms: [O, H, H]
coordinates:
  - [0.000, 0.000, 0.000]
  - [0.000, 0.757, 0.587]
  - [0.000, -0.757, 0.587]

molecule:
  charges: 0
  multiplicity: 1
  ci: [1]
  spin: [0]

pyscf:
  method: hf
  basis: sto3g
```

```
{
  "control": {"title": "water_energy", "jobtype": "energy",
              "theory": "pyscf"},
  "molecule": {
    "atoms": ["O", "H", "H"],
    "coordinates": [[0.000, 0.000, 0.000],
                    [0.000, 0.757, 0.587],
                    [0.000, -0.757, 0.587]],
    "charges": 0, "multiplicity": 1, "ci": [1], "spin": [0]
  },
  "pyscf_skill": {"method": "hf", "basis": "sto3g"}
}
```

Run the classic and YAML files with pyrai3md water.input and pyrai3md water.yaml; python -m pyrai3md water.yaml is equivalent. Run the JSON file with pyrai3md_agent water.json. Adding --info parses the file, builds and validates the configuration and prints a compact resolved summary without starting an engine:

```
title:     water_energy
jobtype:   energy
theory:    pyscf
reference: -
driver:    main
natom:     3
tasks:     theory 1, reference 0
```

Without --info the run writes <title>.log in the launch directory, or under the scratch path when the environment section sets one. The log opens with the start time and logo, echoes the control, system and environment sections, then prints the result block. For this run the block reports the three coordinates and one electronic energy; the value below is the Hartree-Fock/STO-3G result at the geometry above, and the tail line carries the wall time and the final status.

```
  &electronic energy in Hartree
```

```
-------------------------------------------------------------------------
-
  State   1        -74.9631499176452110
-------------------------------------------------------------------------
-

PyRAI3MD End: Nov 12 2026  17:08:31 Total: 00:00:00:02 Status: completed
```

The same title, jobtype and energy appear for all three carriers. A validation error stops the run before the log is opened, so a missing structure or an unknown keyword produces a message, not a half-written log. Section 6 describes the workflows that replace jobtype energy, and Section 12.2 indexes every file a run can leave behind.

5. Keyword sections

This chapter is the reference for every input keyword. Each section is introduced by a short paragraph, followed by a table of keywords with their defaults and one-line descriptions, and then by a paragraph for each keyword. The tables are generated from the configuration model that the program itself loads, so the default column shows exactly the value a run uses when the keyword is absent.

In the tables the keyword is printed in blue and its default in red. The classic text spelling of a section is given in the table header, for example &CONTROL or &MD. In YAML and agent JSON the same section is a mapping with the lower-case name, for example control: or md:. The section names SYSTEM and DYNAMICS are also accepted as &MOLECULE and &MD; MOLECULE is the historical name and remains the spelling used by most examples.

Four marks qualify a keyword. A dagger marks a keyword that is accepted but has no effect in v3.0.0: it is kept so that older input files still parse. A double dagger marks a keyword that accepts several values but honours only one of them in this version. A section sign marks an alias, and the description names the modern keyword. An asterisk marks a quantity that is derived rather than read; derived quantities are listed in Appendix Section B. Retired keywords are not tabulated: they are refused at validation with a message that names their replacement, and Appendix Section E lists them together with the migration from PyRAI2MD inputs.

The sections are read in any order. CONTROL and the molecular specification are the only parts every run needs; the workflow sections are read only by the workflow that the jobtype selects, and a skill section is read only when that skill is in the theory or reference queue. A key written in the wrong section is not silently moved: the owning model does not see it and validation reports the unknown name.

Section 5.1 to Section 5.12 cover the framework. Section 5.13 covers the structured constraint block, Section 5.14 covers the keys that all skills share, and Section 5.15 to Section 5.24 cover the ten calculation skills.

5.1. CONTROL

The control section names the run, chooses the workflow, and queues the calculation methods. It is required in every input file. The classic spelling of the section is &CONTROL; in YAML and agent JSON it is the control mapping.

The method queue is written high to low. One method is a single calculation, two methods form an ONIOM2 or QM/MM composite, and three methods form an ONIOM3 composite. Section 7 explains the composite rules and Section 4 shows the same job in the three input carriers.

The keywords, default values, and short descriptions are listed below.

&CONTROL		
title	pyrai3md	Sets the name of the run. The run log, the trajectory files, the molecular charge file, the runtime file and the frequency files are all named after…
jobtype	energy	Sets the type of job. Basic tokens name a property and may be combined in one run: energy, gradient, nac, soc, frequency, absorption, dipole…
theory	[]	Sets the electronic-structure method queue, written high to low. Each entry is a skill label such as pyscf, molcas, orca, bagel, openqp, xtb, cp2k…
reference	[]	Sets the reference method queue used to label structures during adaptive sampling. It accepts the same labels as theory except nn. At most three…
driver	main	Chooses who owns the iteration. main lets the framework run the optimizer or the MD loop and ask the skill for single points; skill, or the name of…
multiscale_type	''	Names the two-entry composite scheme: an empty value keeps the legacy ONIOM2 meaning, qmmm selects additive QM/MM with ESPF embedding and oniom…
gl_seed	1	Sets the global random number seed. It seeds the initial-condition sampling and, per trajectory, the ensemble draw as gl_seed plus the trajectory…
verbose	2	Sets the amount of information written to the run log. The levels are cumulative: 0

&CONTROL		
		prints progress and error blocks, 1 adds structures, 2 adds…
silent	1	Controls the screen output. 1 writes the run log only and is the default; 0 also prints the log blocks to the terminal. The log file is written in…
debug	false	Turns on the global test switch. It enables run-health monitoring and early interruption in every module, and it is forwarded to the skill…
skill_name	''	Binds the run to one skill in agent mode. When it is empty and theory is set, the first theory entry supplies it. A name that is not in the skill…
skill_path	''	Records the backend module path of the bound skill. It is written by the certification converter and carried as metadata; dispatch always resolves…

Full descriptions for all available keywords are summarized below.

title Sets the name of the run. The run log, the trajectory files, the molecular charge file, the runtime file and the frequency files are all named after it, for example <title>.log and <title>.md.xyz. An empty value is reset to pyrai3md.

jobtype Sets the type of job and therefore the workflow. Basic tokens name a property and may be combined in one run: energy, gradient, nac, soc, frequency, absorption, dipole, transition, hessian, num_freq and num_hess. Advanced tokens select a workflow and must be used alone: md (also spelled namd), adaptive, train, prediction, search, optimize, surface_hopping and read. The aliases sp, opt, freq, hess, numfreq, numhess, namd and predict are normalized to their canonical spelling. Mixing basic and advanced tokens, or naming two advanced tokens, is refused. The retired tokens hop and hybrid are refused with a message that names the replacement: surface hopping is md with a surface-hopping algorithm, and a hybrid calculation is a composite run.

theory Sets the electronic-structure method queue, written high to low. Each entry is a skill label: pyscf, molcas, orca, bagel, openqp, xtb, cp2k, lammps, qdm or nn. The package spelling (pyscf_skill) also resolves, and the retired prefixes in qm.pyscf or abinit.orca are stripped once. One entry runs a single calculation. Two entries select an ONIOM2 composite, or additive QM/MM when multiscale_type is qmmm; for QM/MM the first entry provides the QM field and the second owns the classical surface and the cell. Three entries select ONIOM3. Unknown labels are refused at validation.

reference Sets the reference method queue used to label structures during adaptive sampling. It accepts the same labels as theory except nn. At most three layers are allowed, and a run with a layered theory queue accepts one reference layer unless the jobtype is adaptive. Its regions are given by highref and midref. The legacy spelling abinit is read as reference.

driver Chooses who owns the iteration. With main the framework runs the optimizer or the MD loop and asks the skill for single-point energies and gradients; this is the default and the only mode in which a path search, a variable-cell search, or a framework numerical Hessian is available. With skill, or with the label of the first theory skill, the skill runs its own loop and returns the final structure through the fin_coord channel and its convergence through opt_converge; the framework records one evaluation and settles the run from those two fields. A non-main driver requires exactly one theory skill, must name that skill when a name is given, and is refused with a path method or with optimize_cell.

multiscale_type Names the two-entry composite scheme. An empty string keeps the legacy ONIOM2 meaning of a two-method queue. qmmm selects additive QM/MM: the QM story is theory[0], the classical story is theory[1], the QM region comes from highlevel and the embedding field is handed between them. oniom spells the ONIOM2 selection explicitly. The key is only meaningful with two or three theory entries: three entries are always ONIOM3, and qmmm with three entries is refused.

gl_seed Sets the global random number seed. Initial-condition sampling draws from a generator seeded with this value, and each trajectory of a multi-trajectory run is seeded with gl_seed plus its index, so surface hopping and Wigner sampling are reproducible run to run. Zero is a legal explicit value and is distinguished from an absent key. A restarted runtime file carries the generator state instead of re-seeding.

verbose Sets the detail level of the run log and of each result block. Levels are cumulative: 0 prints progress and error blocks only, 1 adds structures, 2 adds the property arrays and is the default, 3 adds the Hessian and an explicit not-computed marker for every absent property. A value of 4 or more additionally dumps the per-step FSSH amplitudes to <title>.fssh.

silent Controls the screen output. 1 writes the run log only and is the default; 0 also prints the log blocks to the terminal. The log file is written in both cases.

debug Turns on the global test switch and is the one control field that reaches into the skill layer: it enables run-health monitoring, gating and early interruption, and it is applied as the lowest layer of the skill-configuration merge, so it outranks a skill's own debug key. It writes extra diagnostics and can stop a run at the first inconsistent record; leave it false for production.

skill_name Binds the run to one skill. It is the agent-mode field: an agent configuration that names a skill here does not need a theory queue. When it is empty and theory is set, the first theory entry supplies the name. If it is set it must be a registered skill label, otherwise validation refuses the input.

skill_path Records the backend module path of the bound skill. It is written by the certification converter and carried with the configuration as metadata. Dispatch resolves the real module path from the skill manifest, so the value documents where a skill lives but does not redirect execution.

The aliases qm and abinit are accepted anywhere theory and reference are accepted; both are stored under the modern names and appear as such in the run log.

5.2. SYSTEM (MOLECULE)

The system section describes the molecule: the atoms and their geometry, the electronic states a run computes, the regions of a multiscale calculation, and the periodic cell. The classic spelling is &MOLECULE; in YAML and agent JSON it is the system mapping, and the historical name molecule selects the same model.

The structure may be written in the section or read from a file: the coord key names a coordinate file, a same-basename .xyz beside the input is used when no coordinate file is named, and a top-

level atoms list in YAML or agent JSON is taken as written. Section 4 compares the three carriers and Section 7 explains how the region keys divide a multiscale system.

The keywords, default values, and short descriptions are listed below. A † marks a keyword that is accepted but has no effect in v3.0.0; ‡ marks one value that is honoured; § marks an alias; * marks a derived quantity.

&MOLECULE		
atoms	[]	Lists the element symbols, one per atom, in the framework atom order that every other per-atom array follows. The value is normally resolved by the…
coordinates	[]	Gives the Cartesian position of every atom in Angstrom, one row per atom in the order of atoms. It is filled from the coord key or from the…
velocities	[]	Gives the Cartesian velocity of every atom in Bohr per atomic unit of time, one row per atom. It is read from the velo key or from the <coord>.velo…
mass	[]	Gives the atomic mass of every atom in amu. When the list is empty or its length disagrees with the atom count, the masses are derived from the…
natom	0	Records the number of atoms. It is derived from the atom list whenever the key is absent, and a value that disagrees with the length of atoms…
charges	0	Sets the total charge of the system in units of the elementary charge. A scalar is the single-center value and applies to every center, while a list…
multiplicity	1	Sets the spin multiplicity 2S+1 and must be at least 1 for every center. A scalar applies to every center and a list gives one value per center. The…
ci	[1]	Sets the number of electronic states computed in each spin multiplicity. One entry per multiplicity: ci 1 asks for one singlet and ci 1 1 asks for…
spin	[0]	Sets the spin of each multiplicity as its number of unpaired electrons, so 0 is a singlet, 1 a doublet and 2 a triplet. The entries run parallel to…

&MOLECULE		
coupling	[]	Lists the state pairs whose couplings are computed. Each pair is two state numbers written 1-based, pairs are separated by a comma and centers by a…
current_state	0	Sets the electronic state the run is on, counted from 0. It is the ledger a surface hop rewrites during a trajectory. When it is left at its default…
highlevel	[]	Selects the atoms of the high level, which is the QM region of a QMMM run and the innermost layer of an ONIOM run, as 1-based atom numbers. Ranges…
midlevel	[]	Selects the atoms the middle level adds in an ONIOM3 run, as 1-based atom numbers in the same forms highlevel accepts. The middle layer covers…
highref	[]	Selects the high layer of the reference partition that adaptive sampling labels with, as 1-based atom numbers. It is parsed exactly like highlevel…
midref	[]	Selects the middle layer of the adaptive reference partition, as 1-based atom numbers. It is disjoint from highref and is read only when the…
embedding	false	Turns on charge embedding: the high level is computed in the field of the charges of the rest of the system instead of in a mechanical, isolated…
read_charge	false	Turns on reading {title}.charge as a background field. The first line of the file declares what its rows are: point angstrom external for arbitrary…
q_qm	[]	Sets the zero-order charge of each QM atom, one value per QM atom in the global order of the QM set. The classical stage of a QMMM run seeds its…
qm_contains_field_gradient	true	Declares who owns the zero-order field gradient term, the charge times the field. The default true is the ESPF contract, in which the QM analytic…

&MOLECULE		
sc_charge	false	Turns on charge self-consistency for a multi-center QMMM run: the charges of the other centers are converged in the field before the wavefunction is…
sc_charge_maxiter	10	Sets the maximum number of charge rounds of the self-consistent multi-center QMMM iteration and must be at least 1. The iteration stops at the first…
sc_charge_threshold	0.01	Sets the convergence threshold of the charge iteration in units of the elementary charge, as a positive number. The default 0.01 accepts a round in…
sc_charge_damping	1	Sets the weight of each round fresh charge fit in the charges the next round is solved in, in the interval from above zero to 1 inclusive. A value of…
qmmm_key	None	Carries the retired QMMM declaration of the archive. A non-empty value is refused unless multiscale_type is qmmm, and under QMMM the key no longer…
qmmm_xyz †	Input	Records the legacy name of the QMMM coordinate input, Input by default. The value is carried with the system block; the region and the embedding are…
point_charges	[]	Supplies point charges as rows of q x y z, one row per atom in framework atom order, with the charge in units of the elementary charge and the…
external_charges	''	Names a legacy .charge file of arbitrary external charges, whose layout is a charge count, a comment line and then q x y z rows. The rows are a…
external_potential	[]	Holds the external electrostatic potential and field as rows of V Fx Fy Fz, one row per QM atom, in atomic units. It is the channel an ESPF potential…
constraints	None	Holds the structured constraint block: named groups, collective variables, harmonic biases, walls, frozen Cartesian components, strict geometric…

&MOLECULE		
center_groups	[]	Partitions the system into fragment groups, one group of atom numbers per fragment. The atom numbers here are 0-based, unlike every other atom…
track_type	None	Selects the geometry tracking of an MD trajectory: dist compares each atom pair named in track_index and frag compares the centres of exactly two…
track_index	[]	Lists the atom pairs of a dist declaration or the two atom groups of a frag declaration whose distance is tracked. Atom numbers are 1-based in the…
track_thrhd	[]	Sets the distance bound of each tracked object in Angstrom. The comparison reports when the value is greater than the bound, so equality does not…
track_stop	0	Chooses what a crossed bound does: 0 reports in the run log and 1 ends the trajectory after the offending frame is written. The legacy value 2, which…
solvent †	gas	Records the legacy environment label of the run, gas by default. It is a label of the archive input and names no solvation model in the framework; a…
pbc	[0, 0, 0]	Sets the periodicity of the three axes as three flags, each 0 or 1. Only full three-dimensional periodicity, 1 1 1, or none, 0 0 0, is accepted, and…
cell	[99, 99, 99, 90, 90, 90]	Sets the simulation cell as six parameters, a b c alpha beta gamma in Angstrom and degrees, or as the nine components of the lattice matrix. The…
lattice_vector	[[0, 0, 0], [0, 0, 0], [0, 0, 0]]	Sets the lattice as three row vectors in Angstrom, which is the form the runtime reads. The legacy spelling lattice takes the six cell parameters and…
kpoint	[1, 1, 1]	Sets the k-point sampling grid of a periodic calculation as three integers, one per reciprocal axis. The default 1 1 1 is the Gamma-point…
multitask	None	Sets the molecule-scope dispatch of backend terms: true fans the terms of

&MOLECULE		
		one evaluation out to a multiprocessing pool and false keeps them in…
remotetask	None	Sets the molecule-scope dispatch of backend terms to Slurm: true submits each term as its own batch job. An empty value inherits the declarations of…
multipool	None	Sets the pool size of the molecule-scope term fan-out, an integer of at least 1. An empty value inherits, which is distinct from the serial default…

Full descriptions for all available keywords are summarized below.

atoms Lists the element symbols, one per atom, in the framework atom order that every other per-atom array follows. The value is normally resolved by the input parser from a coordinate file rather than written in the section, and an isotope label is stripped when masses are derived from it. An input with no atom list and no resolvable coordinate file is refused before any backend is started.

coordinates Gives the Cartesian position of every atom in Angstrom, one row per atom in the order of atoms. It is filled from the coord key or from the same-basename .xyz file when the key is absent. A row count that disagrees with atoms is refused.

velocities Gives the Cartesian velocity of every atom in Bohr per atomic unit of time, one row per atom. It is read from the velo key or from the <coord>.velo or <coord>.velocity.xyz file beside the geometry, and it is filled with zeros with an informational message when no velocity file exists.

mass Gives the atomic mass of every atom in amu. When the list is empty or its length disagrees with the atom count, the masses are derived from the periodic table; an explicit list of the right length always wins. The integrators, thermostats and barostats read this array, and a run without masses never moves.

natom Records the number of atoms. It is derived from the atom list whenever the key is absent, and a value that disagrees with the length of atoms, coordinates or mass is refused.

charges Sets the total charge of the system in units of the elementary charge. It is not an atom index and is never decremented. A scalar is the single-center form and applies to every center; the per-center list is the authoritative spelling of a multi-center run, and the full-system charge that the low layer sees is the sum of the center charges. The derived electron count is built from this value together with multiplicity, and a run whose unpaired electrons exceed the electron count is refused at validation.

multiplicity Sets the spin multiplicity 2S+1, at least 1 for every center. A scalar is the single-center form and applies to every center, while a list gives one multiplicity per center. The per-center value is the electronic structure of that center region, and the full-system multiplicity is the paired aggregate the lowest layer sees. The value is read together with ci and spin: the state ladders are checked against it, and a disagreement in length is refused.

ci Sets the number of electronic states computed in each spin multiplicity, one entry per multiplicity. Thus ci 1 with spin 0 asks for one singlet, and ci 1 1 with spin 0 1 asks for one singlet plus one triplet. The

flat form is the single-center spelling and the nested form carries one such list per center. The state count, the state labels, the pair maps of the couplings and the range that the root and the optimize state indices are checked against are all derived from ci and coupling.

spin Sets the spin of each multiplicity as its number of unpaired electrons: 0 is a singlet, 1 a doublet and 2 a triplet. The entries sit beside the entries of ci and must agree with them in length, in the flat and in the per-center nested form. Together with multiplicity the list decides how many electrons are unpaired in each ladder, and a ladder that asks for more unpaired electrons than the system has is refused.

coupling Lists the state pairs whose couplings are computed. State numbers are written 1-based, a comma separates the two states of a pair, a semicolon separates the pairs of one center from the next, and the word none gives a center no pairs at all. Each pair must name two distinct states inside the ladder, and the same unordered pair may not appear twice. The NAC and SOC pair maps, and therefore what each step asks a backend to compute, are derived from this list.

current_state Sets the electronic state the run is on, counted from 0. It is the ledger a surface hop rewrites during a trajectory. When it is left at its default it is filled from the dynamics root, and an explicitly declared non-default value is never overridden.

highlevel Selects the atoms of the high level, as 1-based atom numbers. Ranges such as 1-3 are accepted, a semicolon separates the region of one center from the next, and a nested list per center is the multi-center spelling. The region must be non-empty, free of duplicates and inside the system, and the high regions of two centers may not overlap. For QMMM the high level is the QM region and must leave a non-empty classical region; for ONIOM it is the innermost layer, and the middle layer of a three-layer run is described by midlevel.

midlevel Selects the atoms that the middle level adds in an ONIOM3 run, as 1-based atom numbers in the flat or the per-center nested form. The middle layer covers midlevel and highlevel together, so midlevel is disjoint from highlevel while the middle region is a superset of the high one. A three-layer run refuses an empty, duplicate or out-of-range middle region, and a two-layer run has no middle level to read it with.

highref Selects the high layer of the reference partition that adaptive sampling labels with, as 1-based atom numbers. It is parsed exactly like highlevel but never enters the theory center plan. When it is empty and the theory queue is layered, highlevel supplies it.

midref Selects the middle layer of the adaptive reference partition, as 1-based atom numbers. It is disjoint from highref and is read only when the reference queue has three layers; the reference partition is single-center only.

embedding Turns on charge embedding, so that the high level is computed in the field of the charges of its surroundings rather than in an isolated mechanical calculation. Classic text reads true or false, and because a shared namespace can also hand the keyword a skill word, charge, field and legacy fold to true while none folds to false; potential is refused with a pointer to field. An ONIOM run with the switch on needs a highlevel region, and the three-layer form a midlevel as well, so an empty pair is refused. ONIOM2 with a classical outer method refuses the switch outright and points to multiscale_type qmmm, which is the additive combination it describes.

read_charge Turns on reading {title}.charge as a background field, with no computed field at all in a standalone run. The first line of the file declares its rows, and one file carries one kind: point angstrom external for arbitrary external points, point angstrom atom for one row per mid-shell atom of an ONIOM run, or field au for one potential and field row per QM atom. A standalone run refuses the atom-indexed kind because it has no mid shell, an ONIOM run refuses a field document, and a QMMM

run clears the keyword because the composite supplies the field every step. The switch is mutually exclusive with external_charges.

q_qm Sets the zero-order charge of each QM atom, one value per QM atom in the global order of the QM set. The classical stage of a QMMM run seeds its field from these charges, and a QM backend that returns computed charges overrides them. The length must equal the QM set, and only a single-center QMMM run may pin them.

qm_contains_field_gradient Declares who owns the zero-order field gradient term, the charge times the field. The default true is the ESPF contract, in which the QM analytic gradient already contains the term and the composite removes it from the classical correction. False declares that the QM provider does not contain it, so the term stays on the classical side; the switch never changes an energy.

sc_charge Turns on charge self-consistency for a multi-center QMMM run: the charges of the other centers are converged in the field before the wavefunction is solved. It needs at least two centers and embedding true, because a single center has no other charge in its field to iterate over.

sc_charge_maxiter Sets the maximum number of charge rounds of the self-consistent multi-center QMMM iteration and must be at least 1. The iteration stops at the first round that meets sc_charge_threshold or at this bound, and the run log records which of the two stopped it.

sc_charge_threshold Sets the convergence threshold of the charge iteration in units of the elementary charge, as a positive number. The default 0.01 accepts a round in which no center charge moved by more than that amount since the previous round.

sc_charge_damping Sets the weight of each round fresh charge fit in the charges the next round is solved in, in the interval from above zero to 1 inclusive. A value of 1.0 accepts the fit as it is and a smaller value damps an iteration that oscillates instead of settling. The convergence test is taken on the raw fit either way.

qmmm_key Carries the retired QMMM declaration of the archive. A non-empty value is refused unless multiscale_type is qmmm, and under QMMM the key no longer selects anything: the scheme comes from multiscale_type and the region from highlevel.

qmmm_xyz Records the legacy name of the QMMM coordinate input, Input by default. The value is carried with the system block; the region and the embedding are declared by highlevel and embedding. This keyword is accepted but has no effect in v3.0.0.

point_charges Supplies point charges as rows of q x y z, one row per atom in framework atom order, with the charge in units of the elementary charge and the position in Angstrom. The array is whole-system, so row i is atom i, and a composite task reads its own region slice rather than the whole array. A row count that disagrees with the atom count is refused.

external_charges Names a legacy .charge file of arbitrary external charges, whose layout is a charge count, a comment line and then q x y z rows. The rows are a background field for the first theory task and the first reference task; they are never sliced and carry no periodic images. It is refused together with highlevel or midlevel and with any periodic run, and it may not be combined with read_charge.

external_potential Holds the external electrostatic potential and field as rows of V Fx Fy Fz, one row per QM atom, in atomic units. It is the channel an ESPF potential embedding reads in place of the point-charge channel. A standalone read_charge file and the QMMM field handoff fill it, and a task that carries both this array and external charges is refused.

constraints Holds the structured constraint block: named groups, collective variables, harmonic biases, walls, frozen Cartesian components, strict geometric constraints and metadynamics. In classic text the block is its own &constraint section, one keyword per line, and in YAML and agent JSON it is

the constraints mapping of the system section. An empty value means no constraint plan and no change of behaviour; the keyword inventory is in Section 9.

center_groups Partitions the system into fragment groups, one list of atom numbers per fragment, and is the cue the composite planner reads for a fragment task queue. The atom numbers are 0-based, deliberately unlike every other atom selection in the input, because the planner has always read them that way; center_groups 0 1 followed by 2 makes group 0 out of atoms 1 and 2 and group 1 out of atom 3. The name keeps the center_ prefix because the field once shared the name groups with the constraint block and the two were read as each other. A multi-center run refuses center_groups beside the legacy qmmm_key.

track_type Selects the geometry tracking of an MD trajectory: dist compares each atom pair named in track_index and frag compares the centres of exactly two groups. It needs track_index and at least one track_thrhd, and leaving it empty monitors nothing.

track_index Lists the atom pairs of a dist declaration or the two atom groups of a frag declaration whose distance is tracked. Atom numbers are 1-based in the input and decremented once; a frag entry that does not name exactly two groups is refused.

track_thrhd Sets the distance bound of each tracked object in Angstrom. The comparison reports when the value is greater than the bound, so equality does not report. A list shorter than the number of tracked objects is broadcast with its last value, and an empty list is refused.

track_stop Chooses what a crossed bound does: 0 reports in the run log and 1 ends the trajectory after the offending frame is written. The legacy value 2, which suppressed surface hopping, is not implemented and is refused.

solvent Records the legacy environment label of the run, gas by default. It is a label of the archive input and names no solvation model in the framework; a continuum solvent is requested by the solvent keyword of the skill section. This keyword is accepted but has no effect in v3.0.0.

pbc Sets the periodicity of the three axes as three flags, each 0 or 1, and requires exactly three of them. Only full three-dimensional periodicity, 1 1 1, or none, 0 0 0, is supported in this version; a mixed setting is refused because the low-dimensional solvers are not defined. pbc is the single periodicity authority after normalization, so a non-zero lattice alone no longer implies a periodic run, and a periodic run needs a real cell and lattice. Every provider of a periodic task must declare that it supports periodic boundaries.

cell Sets the simulation cell either as six parameters, a b c alpha beta gamma in Angstrom and degrees, or as the nine components of the lattice matrix. The default 99 99 99 90 90 90 is a sentinel meaning not filled in: no lattice is derived from it, so a non-periodic run never carries a fake 99 Angstrom cube. The six-parameter form is triangularized into the row-vector lattice that the runtime reads, a nine-component form and a lattice_vector must agree or the input is refused, and the cell volume is derived from the matrix unless it is given.

lattice_vector Sets the lattice as three row vectors in Angstrom, which is the form the runtime reads. The legacy spelling lattice takes the six cell parameters and is converted to this matrix. A matrix that disagrees with cell is refused, and the cell volume is derived from it.

kpoint Sets the k-point sampling grid of a periodic calculation as three integers, one per reciprocal axis. The default 1 1 1 is the Gamma-point calculation, and the value is carried with the system block for the periodic backend.

multitask Sets the molecule-scope dispatch of backend terms: true fans the terms of one evaluation out to a multiprocessing pool and false keeps them in process. The value is tri-state, so an

empty value inherits the declarations of the skills in the queue and an explicit value overrides them. It is refused together with a composite executor of thread when it is false.

remotetask Sets the molecule-scope dispatch of backend terms to Slurm: true submits each term as its own batch job. An empty value inherits the declarations of the skills in the queue. Per-term remote execution is refused for QMMM, which runs its terms in process.

multipool Sets the pool size of the molecule-scope term fan-out, an integer of at least 1. An empty value inherits, which is distinct from the serial default of 1 that the workflow sections use. A value that disagrees with an explicit composite max_workers is refused.

The per-atom arrays are whole-system arrays: row i is atom i in the order of atoms, in the input file, in every trajectory frame and in every output table. A composite task reads the slice of its own region, so an array that is not sized to the whole system is refused rather than guessed at.

The derived quantities of this section, among them the electron count, the state labels, the lattice volume and the cell-crossing ledger, are not input keywords; Appendix B lists them with the runtime fields of the state.

5.3. DYNAMICS

The dynamics section configures the MD and surface-hopping loop: the thermodynamic ensemble, the initial conditions, the number of steps and the timestep, the hopping algorithm and its couplings, and the guards that end a trajectory. The classic spelling is &MD; in YAML and agent JSON it is the dynamics mapping, and the historical name md selects the same model.

The section is read by a run whose jobtype is md or surface_hopping, and by the md stage of an adaptive run; every other workflow ignores it. Section 6 gives the step order of the integrator and Section 8 the ensemble algorithms behind the thermostat and barostat keywords.

The keywords, default values, and short descriptions are listed below. A † marks a keyword that is accepted but has no effect in v3.0.0; ‡ marks one value that is honoured; § marks an alias; * marks a derived quantity.

&MD		
ensemble	nve	Selects the thermodynamic ensemble: nve integrates the microcanonical equations, nvt adds a particle thermostat and npt adds a barostat as well. The…
thermostat	off	Selects the particle thermostat: off, the default, nose-hoover, nose-hoover-chain or csvr, with the aliases nh and nose for the first, nhc for the…
thermostat_chain	1	Sets the length of the Nose-Hoover chain, an integer of at least 1. A value of 1 is a single Nose-Hoover thermostat and every further link adds one…
thermostat_tau	100	Sets the coupling time of the particle thermostat in femtoseconds, must be

&MD		
		positive. It fixes the thermostat mass through the number of degrees of…
thermostat_mass	None	Overrides the mass of the first thermostat link in atomic units. An empty value derives the mass from thermostat_tau and the number of degrees of…
thermodelay	200	Sets how many steps the thermostat stays off after the trajectory leaves an excited state. It is read together with dissipate and counts steps of the…
dissipate	0	Turns on the gated thermostat of an excited-state trajectory: 1 holds the thermostat off while the run is on an excited state and for thermodelay…
barostat	berendsen	Selects the cell algorithm of an npt run: off, berendsen, the default, parrinello-rahman, mttk or scr, with the aliases pr for the second and…
barostat_mode	isotropic	Selects the cell coupling: isotropic scales the volume with one scalar coordinate and anisotropic relaxes the full lattice tensor. The default is…
barostat_tau	1000	Sets the coupling time of the barostat in femtoseconds, must be positive. It fixes the cell mass together with the compressibility unless…
barostat_mass	None	Overrides the cell mass in atomic units. An empty value derives it from the instantaneous volume, the compressibility and barostat_tau. scr carries…
compressibility	4.5e-05	Sets the isothermal compressibility of the sample in inverse bar, must be positive. The default 4.5e-5 is the value of liquid water, and the barostat…
temperature	300	Sets the target temperature in kelvin and must be at least 0. It is the target of the thermostat, the temperature of a Maxwell-Boltzmann velocity…
pressure	1	Sets the target pressure of an npt run in bar. It is the reference the barostat

&MD		
		drives the internal pressure to, and it has no effect outside npt.
root	0	Sets the initial electronic state, written 1-based so that 1 is the ground state and stored 0-based. A scalar is the single-center value and a list…
activestate	0	Selects which gradients a step requests: 0 asks for every state and 1 asks for the active state only. Global surface hopping needs the all-state…
initcond	0	Selects the source of the initial conditions: 0 or none uses the geometry and velocities the state already carries, read or load takes frames from…
ninitcond	1	Sets the number of trajectories, one per initial condition, and must be at least 1. Each trajectory is planned as its own task with its own runtime…
format	molden	Selects the format of the frequency record that Wigner and Boltzmann sampling read: molden, the default, g16, orca or bagel. The file is…
method †	wigner	Records the legacy sampling method name, wigner by default. The sampling is selected by initcond, which supersedes this keyword.
randvelo	0	Turns on a Maxwell-Boltzmann velocity draw at the start of a trajectory: 1 draws every Cartesian component with a width of the square root of the…
excess	0	Sets the kinetic energy to add to the first step, in Hartree. It is one of three spellings of the same request and may not be combined with scale or…
scale	1	Sets the factor applied to the kinetic energy of the first step, dimensionless, with 1.0 leaving the velocities untouched. It may not be combined…
target	0	Sets the kinetic energy that the first step is brought to, in Hartree. It may not be combined with excess or scale: the three keywords describe one…

&MD		
reset	0	Turns on the removal of the rigid rotation of the velocities: 1 diagonalizes the mass-weighted inertia tensor and removes the rotational part, 0…
resetstep	0	Sets the cadence of the rotational reset in steps: 0 resets on the first step only and a non-zero value resets on every step whose number is a…
step	10	Sets the number of steps this process integrates, a non-negative integer. A resumed run adds it to the step its runtime file carries, so the same…
size	20.67	Sets the integration timestep in atomic units of time, must be positive. The default 20.67 au is about 0.5 fs, and the clock columns of the output…
history	4	Sets the number of state snapshots kept for the integrators. It must be at least 1, and any value below 4 is raised to 4 because the electronic…
sfhp	nosh	Selects the hopping algorithm: nosh, the default, runs no hopping, gsh selects global surface hopping, fssh fewest-switches surface hopping and lzsh…
nactype	ktdc	Selects the coupling source of the fewest-switches integrator: nac uses the nonadiabatic coupling vectors and ktdc the curvature form built from the…
phasecheck	1	Turns on the phase correction of the couplings, 1 by default. Each step compares the overlap of a coupling pair with the previous step and flips the…
substep	20	Sets the number of electronic substeps per nuclear step, at least 1. The electronic equations are propagated with a step of the nuclear timestep…
rk4	0	Selects the electronic propagator: the string 0, the default, takes the first-order local step, and any other value uses fourth-order Runge-Kutta.
integrate	0	Selects how the electronic propagation treats the accumulated hopping matrix:

&MD		
		0, the default, resets it at every substep and a non-zero value…
deco	0.1	Sets the decoherence parameter of the fewest-switches kernel, in Hartree, and reads a number as a string. The default 0.1 enters the coherence time…
adjust	1	Selects the velocity policy after a hop: 0 leaves the velocity unchanged, 1 rescales the whole velocity so that the total energy is conserved, and 2…
reflect	1	Selects what a frustrated hop does, that is a hop that lacks the kinetic energy to reach the upper state: 1 reverses the whole velocity and 2…
maxh	10	Limits how far a hop may jump, counted in state indices: only a target whose index differs from the active one by at most this value is eligible. The…
gap	0.5	Sets the energy gap below which an internal-conversion hop is considered, in eV. The value is converted to Hartree when the integrator is built, and…
gapsoc	0.5	Sets the energy gap below which an intersystem-crossing hop is considered, in eV. It is the spin-forbidden counterpart of gap and is converted to…
addsoc	0	Adds a constant bias to every spin-orbit coupling element of one evaluation, in inverse centimetres. The bias is applied while the result is…
dosoc	0	Turns on the spin-orbit couplings: 1 adds the coupling capability to the plan of every step and 0 leaves it out. It is also the gate of addsoc, so…
dodipole	0	Turns on the dipole of every evaluation: 1 adds the dipole capability to the plan. The dipoles are records of the step and do not enter the…
radiative	0	Requests transition dipoles and oscillator strengths when its value is 1, true or yes. Any other value leaves them out of the plan. Like dodipole it…

&MD		
fssh_rk4	numpy	Selects the implementation of the Runge-Kutta kernel, numpy for the reference form or numba for the compiled form. It is one of five per-function…
fssh_dpdt	numpy	Selects the implementation of the density-derivative kernel, numpy or numba. It is the kernel that propagates the electronic density matrix of one…
fssh_matb	numpy	Selects the implementation of the hopping-matrix kernel, numpy or numba. It is the kernel that builds the switching probabilities of one substep.
fssh_ktdc	numpy	Selects the implementation of the curvature-coupling kernel, numpy or numba. It is read only by a run whose nactype is ktdc.
fssh_decoherence	numpy	Selects the implementation of the decoherence kernel, numpy or numba. It is read only by a run whose deco value enables decoherence.
fssh_force_hop	0	Sets the hopping random number to zero so that any positive probability hops. It is a debug switch, never used in production runs, and is isolated…
fssh_profile	0	Turns on the function-level timing of the electronic kernels and writes the implementation table beside the run log. 0, the default, times nothing.
perf_profile	0	Turns on the per-step phase table, which is written to pyrai3md_step.perf in the working directory. 0, the default, writes no table.
md_maxenergy	None	Sets the energy deviation above which the trajectory stops, in Hartree, when the backend reports a prediction deviation. An empty value leaves the…
md_maxgrad	None	Sets the gradient deviation above which the trajectory stops, in Hartree per Bohr. An empty value leaves the channel unjudged, and an adaptive md…

&MD		
md_maxnac	None	Sets the coupling deviation above which the trajectory stops, in Hartree per Bohr. An empty value leaves the channel unjudged, and an adaptive md…
md_maxsoc	None	Sets the spin-orbit deviation above which the trajectory stops, in inverse centimetres. An empty value leaves the channel unjudged, and an adaptive…
md_maxpop	0.01	Sets the population tolerance: a population outside the interval from minus the tolerance to one plus the tolerance ends the trajectory after the…
direct	2000	Sets the number of steps recorded one by one from the start: up to this step every frame is written. After it buffer decides the cadence, and the…
buffer	500	Sets the recording cadence after the direct window: every buffer-th step is written, counted from the end of that window. The default is 500, the…
checkpoint	0	Sets the cadence of the runtime file <title>.rt in steps: a step whose absolute number is a multiple of the value writes the file, and the last step…
multitask	None	Turns on the multiprocessing fan-out of this workflow units: true runs the trajectories in a spawn pool and false runs them one after another in…
remotetask	None	Submits each trajectory of this workflow to Slurm: true writes a batch script per unit and collects its result record. It needs sbatch and sacct on…
multipool	1	Sets the number of pool workers of this workflow fan-out, at least 1. It is the width of the pool and not the number of units; the default 1 keeps…

Full descriptions for all available keywords are summarized below.

ensemble Selects the thermodynamic ensemble and therefore which of the thermostat and barostat keywords are read. nve is the default microcanonical run: it requires a thermostat of off and refuses dissipate, because there is no thermostat to delay. nvt adds a particle thermostat and ignores the barostat; npt requires both a barostat and a thermostat, and it also needs a stress from the backend,

which in practice means a periodic provider. The ensemble files <title>.md.ensemble and <title>.sh.ensemble are created for nvt and npt runs only.

thermostat Selects the particle thermostat of an nvt or npt run. The values are off, nose-hoover, nose-hoover-chain and csvr, with nh, nose, nhc and csnr accepted as aliases; the same vocabulary is written in barostat terms by the pairing table, which refuses a combination with no derivation. parrinello-rahman and mttk are extended-cell methods whose cell dynamics carries a Nose-Hoover chain, so they accept only the two Nose-Hoover thermostats; berendsen and scr accept any thermostat that is not off. The default off belongs to nve, and nvt or npt refuses it.

thermostat_chain Sets the length of the Nose-Hoover chain, an integer of at least 1. A value of 1 is a single Nose-Hoover thermostat and every further link adds one thermostat to the chain. An MTTK run thermalizes the cell momentum with a chain of the same length.

thermostat_tau Sets the coupling time of the particle thermostat in femtoseconds, must be positive. It fixes the thermostat mass through the number of degrees of freedom and the target temperature unless thermostat_mass overrides it. A chain coupled faster than the motion it thermostats pumps that motion instead of damping it.

thermostat_mass Overrides the mass of the first thermostat link in atomic units. An empty value derives the mass from thermostat_tau and the number of degrees of freedom, which is what a normal run uses; the further links of a chain keep their derived masses.

thermodelay Sets how many steps the thermostat stays off after the trajectory leaves an excited state. It is read together with dissipate and counts steps of the run, not steps on the excited state.

dissipate Turns on the gated thermostat of an excited-state trajectory: 1 holds the thermostat off while the run is on an excited state and for thermodelay steps after it lands on the ground state, so the excess energy leaves with the nuclear motion. The barostat is not gated. nve refuses a non-zero value because it has no thermostat to hold.

barostat Selects the cell algorithm of an npt run and is inert in any other ensemble. berendsen, the default, is a weak-coupling relaxation with no ensemble claim and is the right choice for equilibration. parrinello-rahman and mttk are deterministic extended-Lagrangian cell dynamics and need a Nose-Hoover thermostat on the particles; mttk adds a chain on the cell momentum. scr is a first-order stochastic cell equation with no cell momentum, derived to pair with any thermostat, and it is isotropic only.

barostat_mode Selects the cell coupling: isotropic scales the volume with one scalar coordinate and anisotropic relaxes the full lattice tensor. The default is isotropic. scr supports the isotropic form only and refuses anisotropic, because its tensor form shears a liquid with no restoring modulus.

barostat_tau Sets the coupling time of the barostat in femtoseconds, must be positive. It fixes the cell mass together with the compressibility unless barostat_mass overrides it. For scr it is the volume relaxation time of a first-order cell equation, so it is not an oscillation period there.

barostat_mass Overrides the cell mass in atomic units. An empty value derives it from the instantaneous volume, the compressibility and barostat_tau. scr carries no cell mass at all, so the keyword does not apply to it.

compressibility Sets the isothermal compressibility of the sample in inverse bar, must be positive. The default 4.5e-5 is the value of liquid water, and the barostat derives its cell mass from it.

temperature Sets the target temperature of the run in kelvin, and it is the one value three mechanisms read: the thermostat drives the instantaneous temperature to it, randvelo draws velocities from a Maxwell-Boltzmann distribution at it, and the Wigner and Boltzmann initial condition samplers

populate the vibrational states at it. Zero is accepted and a negative value is refused. An nve run has no thermostat to read it, so the keyword there only seeds the draws.

pressure Sets the target pressure of an npt run in bar. It is the reference the barostat drives the internal pressure to, and it has no effect outside npt.

root Sets the initial electronic state of the trajectory. It is written 1-based, so 1 is the ground state, and the adapter decrements it once into the 0-based index the run uses; a value below 1 in the input and a state outside the ladder built from ci are both refused. A scalar is the single-center spelling and a list gives one initial state per center of a multi-center run. The run fills the current state of the system from root when that field is left at its default. A hop rewrites that current state and never root, so this keyword records where the trajectory started while the runtime ledger records where it is.

activestate Selects which gradients a step requests: 0 asks for every state and 1 asks for the active state only. Global surface hopping needs the all-state form, Landau-Zener hopping needs energies only, and fewest-switches hopping works with either.

initcond Selects where the initial conditions come from. 0, the default, and none use the geometry and velocities the state already carries, and with more than one trajectory the same condition is repeated. read and load take ninitcond frames from <title>.init.xyz, whose atom lines are an element symbol followed by six numbers, position in Angstrom and velocity in Bohr per atomic unit; fewer frames than asked for is an error and extra frames are truncated. wigner and boltzmann sample the frequency record <title>.freq.<format> at the target temperature, seeded by the global seed of the control section, and the legacy value 1 is read as wigner.

ninitcond Sets the number of trajectories, one per initial condition, and must be at least 1. Each trajectory is planned as its own task with its own runtime directory. With initcond 0 the trajectories repeat the same initial condition byte for byte.

format Selects the format of the frequency record that Wigner and Boltzmann sampling read: molden, the default, g16, orca or bagel. The file is <title>.freq.<format> in the working directory, and the title comes from the control section.

method Records the legacy sampling method name, wigner by default. The sampling is selected by initcond, which supersedes this keyword. This keyword is accepted but has no effect in v3.0.0.

randvelo Turns on a Maxwell-Boltzmann velocity draw at the start of a trajectory: 1 draws every Cartesian component with a width of the square root of the thermal energy over the atomic mass. The draw replaces the velocities the state carries and needs no frequency record, which is what separates it from vibrational sampling.

excess Sets the kinetic energy to add to the first step, in Hartree. It is one of three spellings of the same request and may not be combined with scale or target. A restarted segment applies neither this nor the velocity draw.

scale Sets the factor applied to the kinetic energy of the first step, dimensionless, with 1.0 leaving the velocities untouched. It may not be combined with excess or target.

target Sets the kinetic energy that the first step is brought to, in Hartree. It may not be combined with excess or scale: the three keywords describe one request and only one of them may be written.

reset Turns on the removal of the rigid rotation of the velocities: 1 diagonalizes the mass-weighted inertia tensor and removes the rotational part, 0 leaves the velocities as they are. The translation is subtracted on every step of every run, and a periodic run never takes the rotational half.

resetstep Sets the cadence of the rotational reset in steps: 0 resets on the first step only and a non-zero value resets on every step whose number is a multiple of it. It is read only when reset is 1.

step Sets the number of steps this process integrates, a non-negative integer, and the default 10 is a short smoke run rather than a production length. It bounds the loop directly, so the number of recorded frames is a function of it together with direct and buffer. A resumed segment adds its own step count to the step the runtime file carries: step is the length of one process, not of the whole trajectory.

size Sets the integration timestep in atomic units of time, which is the unit of the velocity array; 20.67 au is about half a femtosecond and is the default. The trajectory clock printed in the energies and population files is the step number minus one times this value, so step 1 sits at time zero. A value must be positive, and a timestep large enough to break the electronic substeps is a physical choice rather than something the framework rescales.

history Sets the number of state snapshots kept for the integrators. It must be at least 1, and any value below 4 is raised to 4 because the electronic dynamics needs the last four steps.

sfhp Selects the hopping algorithm of the trajectory. nosh, the default, propagates the nuclei on one surface and runs no hopping at all. gsh is global surface hopping, which needs all-state gradients and rebuilds a pure-state density, and it uses neither decoherence nor electronic substeps. fssh is fewest-switches surface hopping, the algorithm that reads nactype, substep, rk4, deco, adjust, reflect and maxh. lzsh is Landau-Zener surface hopping, which needs state energies only and rescales the velocity isotropically on every hop. The algorithm is a framework keyword and is never passed to the skill.

nactype Selects the coupling source that the fewest-switches integrator consumes. nac uses the nonadiabatic coupling vectors, which the plan requests as their own capability; ktdc, the default, uses the curvature form built from the energy gap and the gradients, so a step that needs no coupling vectors can still hop. The integrator accepts only these two values and refuses anything else when it is initialized. The overlap-based dcm spelling is known to the planner, which then asks for an overlap capability, but no integrator implements it and the run fails at that same point.

phasecheck Turns on the phase correction of the couplings, 1 by default. Each step compares the overlap of a coupling pair with the previous step and flips the sign of the pairs that came out negative, while a pair whose overlap is at the noise level is left alone.

substep Sets the number of electronic substeps per nuclear step, at least 1. The electronic equations are propagated with a step of the nuclear timestep divided by this number, and the default 20 resolves the phase within one nuclear step.

rk4 Selects the electronic propagator: the string 0, the default, takes the first-order local step, and any other value uses fourth-order Runge-Kutta.

integrate Selects how the electronic propagation treats the accumulated hopping matrix: 0, the default, resets it at every substep and a non-zero value accumulates it across the substeps of one nuclear step.

deco Sets the decoherence parameter of the fewest-switches kernel, in Hartree, and it is read as a string so that the word off can turn the correction off. The parameter enters the coherence time of a pair as the inverse gap times one plus the parameter over the kinetic energy, which is the OpenMolcas form of the correction. The default 0.1 is the legacy value, and the kernel that applies it is selected separately by fssh_decoherence.

adjust Selects the velocity policy after a hop: 0 leaves the velocity unchanged, 1 rescales the whole velocity so that the total energy is conserved, and 2 rescales the component along the coupling vector. The default is 1.

reflect Selects what a frustrated hop does, that is a hop that lacks the kinetic energy to reach the upper state: 1 reverses the whole velocity and 2 reflects it along the coupling vector. The default is 1.

maxh Limits how far a hop may jump, counted in state indices: only a target whose index differs from the active one by at most this value is eligible. The default 10 leaves every state of a small ladder reachable.

gap Sets the energy gap below which an internal-conversion hop is considered, in eV. The value is converted to Hartree when the integrator is built, and the default 0.5 eV is the legacy threshold.

gapsoc Sets the energy gap below which an intersystem-crossing hop is considered, in eV. It is the spin-forbidden counterpart of gap and is converted to Hartree when the integrator is built.

addsoc Adds a constant bias to every spin-orbit coupling element of one evaluation, in inverse centimetres. The bias is applied while the result is finalized, so the matrix the step records is the biased one, and it is applied only to a run that asked for couplings with dosoc.

dosoc Turns on the spin-orbit couplings: 1 adds the coupling capability to the plan of every step and 0 leaves it out. It is also the gate of addsoc, so without it nothing is computed and nothing is biased.

dodipole Turns on the dipole of every evaluation: 1 adds the dipole capability to the plan. The dipoles are records of the step and do not enter the propagation.

radiative Requests transition dipoles and oscillator strengths when its value is 1, true or yes. Any other value leaves them out of the plan. Like dodipole it extends what a step computes rather than how it propagates.

fssh_rk4 Selects the implementation of the Runge-Kutta kernel, numpy for the reference form or numba for the compiled form. It is one of five per-function kernel switches and changes the speed of a trajectory, not its result.

fssh_dpdt Selects the implementation of the density-derivative kernel, numpy or numba. It is the kernel that propagates the electronic density matrix of one substep.

fssh_matb Selects the implementation of the hopping-matrix kernel, numpy or numba. It is the kernel that builds the switching probabilities of one substep.

fssh_ktdc Selects the implementation of the curvature-coupling kernel, numpy or numba. It is read only by a run whose nactype is ktdc.

fssh_decoherence Selects the implementation of the decoherence kernel, numpy or numba. It is read only by a run whose deco value enables decoherence.

fssh_force_hop Sets the hopping random number to zero so that any positive probability hops. It is a debug switch, never used in production runs, and is isolated from the electronic integration mode of integrate.

fssh_profile Turns on the function-level timing of the electronic kernels and writes the implementation table beside the run log. 0, the default, times nothing.

perf_profile Turns on the per-step phase table, which is written to pyrai3md_step.perf in the working directory. 0, the default, writes no table.

md_maxenergy Sets the energy deviation above which the trajectory stops, in Hartree, when the backend reports a prediction deviation. An empty value leaves the channel unjudged, and the md stage of an adaptive run has it overwritten from the adaptive section.

md_maxgrad Sets the gradient deviation above which the trajectory stops, in Hartree per Bohr. An empty value leaves the channel unjudged, and an adaptive md child reads the bound from its adaptive section instead.

md_maxnac Sets the coupling deviation above which the trajectory stops, in Hartree per Bohr. An empty value leaves the channel unjudged, and an adaptive md child reads the bound from its adaptive section instead.

md_maxsoc Sets the spin-orbit deviation above which the trajectory stops, in inverse centimetres. An empty value leaves the channel unjudged, and an adaptive md child reads the bound from its adaptive section instead.

md_maxpop Sets the population tolerance of the resident guard, which is the one detection channel that is on by default. The interval read is minus the tolerance to one plus the tolerance, and the first population outside it ends the trajectory after the offending frame is written, with stop code 1. An empty value turns the guard off, and a negative or non-finite value is refused rather than reinterpreted, because a negative tolerance would move both ends inward and refuse every physical population. Unlike the four deviation thresholds, this bound is not overwritten in the md child of an adaptive run: the parent reads the stop code and only decides whether to collect the structure.

direct Sets the number of steps recorded one by one from the start: up to this step every frame is written. After it buffer decides the cadence, and the default is 2000.

buffer Sets the recording cadence after the direct window: every buffer-th step is written, counted from the end of that window. The default is 500, the last step of a run is always written, and hop frames are never throttled.

checkpoint Sets the cadence of the runtime file <title>.rt in steps: a step whose absolute number is a multiple of the value writes the file, and the last step of a run always writes it. 0, the default, writes no file during the run.

multitask Turns on the multiprocessing fan-out of this workflow units: true runs the trajectories in a spawn pool and false runs them one after another in process. An empty value keeps the workflow default. It decides how the units run, never how many; ninitcond sets the number.

remotetask Submits each trajectory of this workflow to Slurm: true writes a batch script per unit and collects its result record. It needs sbatch and sacct on the path and a slurm section or the environment keys of the input. An empty value keeps the workflow default.

multipool Sets the number of pool workers of this workflow fan-out, at least 1. It is the width of the pool and not the number of units; the default 1 keeps the fan-out serial.

The four deviation thresholds md_maxenergy, md_maxgrad, md_maxnac and md_maxsoc judge only the deviation keys a backend reports, so a molecular-mechanics or quantum-chemistry run whose provider reports none leaves them all unjudged. The last three keywords of the table are shared with every other workflow section: multitask and remotetask choose how the units of a run are executed and multipool chooses the width of the pool.

The retired spellings of this section, among them restart, addstep, record, thermo and popreset, are not accepted: each is refused with the keyword that replaced it, and the differences are collected in Appendix E.

5.4. OPTIMIZE

The optimize section configures the geometry search started by jobtype optimize. It names the algorithm, the five thresholds that judge the atomic step, the step bound, and the options that make the lattice a variable. The section is read only by an optimize run: the other workflows keep their own sections.

The path methods mep, irc and neb are written as values of opt_method, but no optimizer steps them. A run that names one is routed to the Path workflow, which walks a sequence of constrained points or relaxes a band of images between two endpoint files, and the path keywords of this section are read there. The single-structure algorithms are minima, gdiis, ts, meci and mecp.

The keywords, default values, and short descriptions are listed below.

&OPTIMIZE		
multipool	1	Sets the number of workers of the dispatch fan-out of this workflow node. A NEB band fans its image evaluations out over this pool, while the…
opt_method	minima	Sets the search algorithm. minima descends to the nearest minimum, gdiis accelerates the same descent with a DIIS extrapolation, ts walks to a…
opt_maxit	100	Sets the maximum number of optimization steps. A run that reaches this count without meeting the thresholds is reported as failed with the message…
opt_energy	1e-06	Sets the energy convergence threshold, in Hartree. The step is converged when the change in energy between two successive steps is smaller than this…
opt_grms	0.0003	Sets the convergence threshold of the RMS gradient, in Hartree per Bohr. The five thresholds of the atomic step must all be met at once. The default…
opt_gmax	0.00045	Sets the largest allowed component of the gradient, in Hartree per Bohr. The default is 4.5e-4.
opt_drms	0.0012	Sets the convergence threshold of the RMS displacement of the last step, in Angstrom. The default is 1.2e-3.
opt_dmax	0.0018	Sets the largest allowed component of the displacement of the last step, in Angstrom. The default is 1.8e-3.

&OPTIMIZE		
opt_csrms	40	Sets the convergence threshold of the RMS cell stress, in bar. It is judged only when optimize_cell is true, and the cell half of the run is…
opt_csmax	60	Sets the largest allowed component of the cell stress, in bar. It is read only with optimize_cell. The default is 60.0.
opt_cerms	5e-05	Sets the convergence threshold of the RMS cell strain, which is the displacement of a lattice row divided by the length of that row and is therefore…
opt_cemax	7e-05	Sets the largest allowed component of the cell strain, dimensionless. It is read only with optimize_cell. The default is 7e-05.
opt_pressure	0	Sets the external pressure the variable-cell search works against, in bar and tensile positive. The objective becomes the enthalpy E plus pV, so the…
opt_cell_coupling	anisotropic	Chooses how many lattice directions one pressure value may move. isotropic scales the box as a whole and anisotropic lets each of the six relaxed…
opt_trust_radius	None	Sets the step bound of the search, in Bohr. The bound clips the joint nuclear and lattice step, so it decides whether a pressure band is reachable…
opt_use_rfo	false	Requests a rational function optimization step from the augmented Hessian instead of the quasi-Newton step. The minima search honours it, gdiis has…
opt_state_i	0	Sets the electronic state the search follows, counted from zero. A multi-center run may give one index per center, and a single value is read by…
opt_state_j	None	Sets the second surface of a two-state search, counted from zero, for meci and mecp. Unset means there is no pair and the run follows one surface…
optimize_cell	false	Makes the six lattice components part of the optimized vector, so the box relaxes

&OPTIMIZE		
		together with the atoms. The run needs a backend that reports…
opt_path_constraint	sphere	Chooses the algorithm that holds a path point on its hypersphere. sphere takes the mass-weighted Gonzalez-Schlegel constrained step and relu…
opt_path_radius	0.1	Sets the arc length between two path points, in Bohr, which is also the radius of the hypersphere a point is found on. The value must be positive…
opt_path_inner_maxit	20	Sets the number of probe steps the constrained minimization of one path point may take. The value must be at least 1. The default is 20.
opt_path_inner_tol	0.0003	Sets the convergence tolerance of the probe that finds one path point. Left at its default the run uses opt_grms instead, so one keyword drives a…
opt_irc_direction	both	Chooses the side of the saddle an intrinsic reaction coordinate walks: backward, forward, or both, which runs one walker per side from the same…
opt_irc_mode	hessian	Chooses where the imaginary mode that seeds the IRC comes from. hessian computes it with the framework Hessian and molden reads it from a frequency…
opt_irc_mode_file	''	Names the frequency file the imaginary mode is read from when opt_irc_mode is molden. It is required in that case and is not read otherwise. The…
opt_neb_reactant	''	Names the file that holds the starting structure of a band. It is required for neb, together with opt_neb_product, and the atom order of the two…
opt_neb_product	''	Names the file that holds the final structure of a band. It is required for neb, together with opt_neb_reactant. The default is empty.
opt_neb_ts	''	Names an optional file that holds an intermediate guess, which is placed in the band between the two endpoints. It is read only for neb. The default…

&OPTIMIZE		
opt_neb_images	8	Sets the number of interior images of a band. The band itself carries this number plus the two endpoints, and the value must be at least 1. The…
opt_neb_interpolate	linear	Chooses how the interior images are placed. linear uses fractional coordinates for a periodic band, and geodesic measures distances in a Morse scaled…
opt_neb_geodesic_threshold	3	Sets the pair cutoff radius of the geodesic metric, in Angstrom. It is read only with geodesic interpolation and must be positive. The default is 3.0.
opt_neb_geodesic_scaler	1.7	Sets the alpha parameter of the Morse metric that scales the geodesic distances. It is read only with geodesic interpolation and must be positive…
opt_neb_geodesic_min_neighbors	4	Sets the minimum coordination number of an atom in the pair list the geodesic metric is built from. The value must be at least 1. The default is 4.
opt_neb_spring	0.01	Sets the spring constant between neighbouring images, in Hartree per Bohr squared. A value of zero is allowed. The default is 0.01.
opt_neb_climb	false	Turns the highest image into a climbing image once the band is relaxed enough, so that the image converges on the saddle instead of on the chord…
opt_neb_climb_threshold	0.01	Sets the largest perpendicular force below which the climbing image is armed. It is read only with opt_neb_climb and is refused without it. The…
opt_neb_step	0.1	Sets the initial timestep of the FIRE relaxation of the band, in atomic units. The value must be positive. The default is 0.1.

Full descriptions for all available keywords are summarized below.

multipool Sets the number of workers of the dispatch fan-out of this workflow node. A NEB band fans its image evaluations out over this pool, while the single-structure loop steps one geometry at a time. The default is 1, which keeps execution serial.

opt_method Sets the search algorithm, and with it the loop the run takes. minima is an inverse-Hessian BFGS descent with a trust radius, gdiis adds a DIIS extrapolation on top of a BFGS preconditioner, ts follows the lowest mode from a partitioned RFO step to a first-order saddle, meci locates a minimum energy conical intersection, and mecp locates a minimum energy crossing point. The last two read the state pair from opt_state_i and opt_state_j, and meci also uses the derivative coupling. The three path names mep, irc and neb are values of this keyword, but the framework routes them to the Path workflow rather than to an optimizer, and the Optimize workflow refuses a path name that reaches it.

opt_maxit Sets the maximum number of steps of the search. The loop stops at this count whether or not the thresholds were met, and a run that ends there is marked failed with Maximum optimization steps reached rather than reported as converged. Only a resumed runtime file that carries its own step budget can lower the bound; a run that says nothing else is bounded by this keyword alone. The default is 100.

opt_energy Sets the energy convergence threshold, in Hartree. The step is converged when the change in energy between two successive steps is smaller than this value. The default is 1e-6.

opt_grms Sets the convergence threshold of the RMS gradient, in Hartree per Bohr. The five thresholds of the atomic step must all be met at once. The default is 3e-4.

opt_gmax Sets the largest allowed component of the gradient, in Hartree per Bohr. The default is 4.5e-4.

opt_drms Sets the convergence threshold of the RMS displacement of the last step, in Angstrom. The default is 1.2e-3.

opt_dmax Sets the largest allowed component of the displacement of the last step, in Angstrom. The default is 1.8e-3.

opt_csrms Sets the convergence threshold of the RMS cell stress, in bar. It is judged only when optimize_cell is true, and the cell half of the run is converged only when all four of its thresholds are met. The default is 40.0.

opt_csmax Sets the largest allowed component of the cell stress, in bar. It is read only with optimize_cell. The default is 60.0.

opt_cerms Sets the convergence threshold of the RMS cell strain, which is the displacement of a lattice row divided by the length of that row and is therefore dimensionless. It is read only with optimize_cell. The default is 5e-05.

opt_cemax Sets the largest allowed component of the cell strain, dimensionless. It is read only with optimize_cell. The default is 7e-05.

opt_pressure Sets the external pressure of a variable-cell run, in bar and tensile positive like the stress it is added to. The objective becomes the enthalpy E plus pV, so the relaxed box is the one whose stress equals minus this pressure. The keyword is refused without optimize_cell, because a held box has no lattice derivative for a pressure to act on; a box under pressure without a relaxation is a dynamics run with a barostat. The default is 0.0, which asks for a stress-free box.

opt_cell_coupling Chooses how many lattice directions one pressure value may move. anisotropic, the default, lets each of the six relaxed components answer for itself. isotropic scales the box, so only the mean normal stress and the single scale of the step are judged, and the deviatoric and shear readings are reported but not held to a threshold, because a shape-preserving step has no direction to

relieve them. The two names are the names of the dynamics barostat modes. The keyword needs optimize_cell.

opt_trust_radius Sets the step bound of the search, in Bohr. The radius clips the joint nuclear and lattice step, so the pressure band a box is judged against is reachable or not depending on this number rather than on opt_maxit. An unset value is not a number: it leaves each algorithm on its own starting radius, 0.3 Bohr for minima and gdiis and 0.05 Bohr for the transition-state search, which is what the shipped examples run. A value of zero is refused, because it would freeze a search that then reports that it moved nothing.

opt_use_rfo Requests the rational function optimization step from the augmented Hessian in place of the plain quasi-Newton step. Only the minima search honours the flag: gdiis has no RFO variant and discards it, and the transition-state search always forms a partitioned RFO step because mode following needs the partitioned curvature. The step is clipped by the same trust radius as the quasi-Newton step. The default is false.

opt_state_i Sets the electronic state the search follows, counted from zero. A multi-center run may give one index per center, and a single value is read by every center. The default is 0.

opt_state_j Sets the second surface of a two-state search, counted from zero, for meci and mecp. Unset means there is no pair and the run follows one surface. The default is unset.

optimize_cell Makes the box a variable: the six movable lattice components join the optimized vector, the stress conjugate to them joins the gradient, and the atoms ride the box with their fractional coordinates held. The planning layer derives the stress capability from this keyword and passes the whole-system stress tensor to the skill, so a backend without that capability is refused rather than relaxed in the box it was given. The keyword is refused with a driver other than main, because an engine that owns the steps commits no lattice, and with a vibrational analysis, because the lattice entries have no mass to be weighted by.

opt_path_constraint Chooses the algorithm that holds a path point on the hypersphere of radius opt_path_radius around the accepted point. sphere is the mass-weighted Gonzalez-Schlegel constrained step, whose rotation shrinks when the probe energy fails to fall. relu minimizes the energy plus a one-sided penalty that acts only outside the shell. The keyword belongs to mep and irc and is refused on a band, which is a chain of images rather than a walk on a sphere.

opt_path_radius Sets the arc length between two path points, in Bohr, which is also the radius of the hypersphere a point is found on. The value must be positive. The default is 0.1.

opt_path_inner_maxit Sets the number of probe steps the constrained minimization of one path point may take. The value must be at least 1. The default is 20.

opt_path_inner_tol Sets the convergence tolerance of the probe that finds one path point. Left at its default the run uses opt_grms instead, so one keyword drives a point and the walk alike. The value must be positive and defaults to 3e-04.

opt_irc_direction Chooses the side of the saddle an intrinsic reaction coordinate walks: backward, forward, or both, which runs one walker per side from the same saddle. It is read only for irc. The default is both.

opt_irc_mode Chooses where the imaginary mode that seeds the IRC comes from. hessian computes it with the framework Hessian and molden reads it from a frequency file named by opt_irc_mode_file. It is read only for irc and defaults to hessian.

opt_irc_mode_file Names the frequency file the imaginary mode is read from when opt_irc_mode is molden. It is required in that case and is not read otherwise. The default is empty.

opt_neb_reactant Names the file that holds the starting structure of a band, one frame in the format the molecular section accepts. A neb run is refused without both endpoints, because the band is interpolated between them and a run that names none has nothing to relax. The atom count and atom order must agree with the product file, and a periodic endpoint file must carry its cell comment.

opt_neb_product Names the file that holds the final structure of a band. It is read together with opt_neb_reactant when the band is built, and a neb run that names only one endpoint is refused. The atom order and, for a variable-cell run, the lattice of the two files are checked at that point, because the section validator has no file to look at.

opt_neb_ts Names an optional file that holds an intermediate guess for the band. When it is given, the guess becomes an interior image instead of a point the interpolation invents, which is how a band is kept from crossing a barrier the endpoints alone would not suggest. It is read only for neb and defaults to empty, which leaves the band to the interpolation.

opt_neb_images Sets the number of interior images of the band. The band carries this number plus the two endpoints, so the default of 8 means ten frames, and the count also keeps every band dispatch above the single-unit identity allocation. The value must be at least 1, and geodesic interpolation asks for at least 2, because one interior image is a chord with nothing between it and its neighbours to smooth.

opt_neb_interpolate Chooses how the interior images are placed between the two endpoints. linear, the default, is a straight interpolation in Cartesian coordinates and uses fractional coordinates for a periodic band. geodesic smooths the band in a Morse scaled internal-coordinate metric, which keeps a rotation from being read as a barrier. It needs at least two interior images and is refused for a band above 200 atoms.

opt_neb_geodesic_threshold Sets the pair cutoff radius of the geodesic metric, in Angstrom. Every atom pair closer than this radius enters the internal coordinate list the metric is measured on. The value must be positive and is read only when opt_neb_interpolate is geodesic. The default is 3.0.

opt_neb_geodesic_scaler Sets the alpha parameter of the Morse potential whose distances the geodesic metric uses, so that a stretched pair contributes less than a compressed one. It must be positive and is read only with geodesic interpolation. The default is 1.7.

opt_neb_geodesic_min_neighbors Sets the minimum coordination number an atom keeps in the pair list of the geodesic metric, which stops a sparse or unusual structure from leaving an atom with no distance to anything. The value must be at least 1 and is read only with geodesic interpolation. The default is 4.

opt_neb_spring Sets the spring constant that keeps neighbouring images apart, in Hartree per Bohr squared, so that the constant times a segment length in Bohr is a force in the unit the true force is read in. The springs act along the band only, and the true force acts perpendicular to it, which is what lets an image relax without sliding into a neighbour. A value of zero is allowed and leaves the band to the perpendicular force. The default is 0.01.

opt_neb_climb Turns the highest image into a climbing image once the band is relaxed enough. The climbing image inverts the component of the true force along the band, so it climbs to the saddle instead of settling at the middle of the chord, and the latch is one-way: once armed, the image stays a climbing image for the rest of the run. The default is false.

opt_neb_climb_threshold Sets the largest perpendicular force, in Hartree per Bohr, below which the climbing image is armed. The keyword is read only with opt_neb_climb and is refused without it, because the threshold is the reading that arms an image the run did not ask for. The default is 0.01.

opt_neb_step Sets the initial timestep of the FIRE relaxation of the band, in atomic units. It is the timestep of the integrator that moves the images, not a step bound, and FIRE adjusts it as the band settles. The value must be positive. The default is 0.1.

Each path keyword answers a question exactly one of the three walks asks, so a key declared on a method that cannot read it is refused rather than carried: the hypersphere keys opt_path_constraint, opt_path_radius, opt_path_inner_maxit and opt_path_inner_tol belong to mep and irc, the IRC keys belong to irc, and the band keys belong to neb. A band also needs both endpoints and a climbing threshold only with a climbing image.

5.5. HESSIAN

The hessian section configures the curvature request of jobtype hessian, frequency, num_hess and num_freq. Two questions are answered separately: hess_driver says who produces the matrix, the framework displacing the geometry or the backend computing it for itself, and hess_mode says how it is produced, by an analytical second derivative or by a finite difference. The section also decides whether the matrix is read into normal modes and thermochemical corrections and which atoms are displaced.

The section is always present with its defaults, so hess_driver and hess_mode are unset by default. An unset value lets the jobtype token choose the route, which is what keeps a backend matrix (hessian, frequency) and a framework displacement batch (num_hess, num_freq) apart without a second keyword. The pairings that have no producer behind them are refused where the input is still cheap to change.

The keywords, default values, and short descriptions are listed below.

&HESSIAN		
multitask	None	Runs the independent displaced evaluations of the batch concurrently when true and one at a time when false. Unset leaves the choice to the workflow…
remotetask	None	Submits each displaced evaluation to the cluster through the slurm section of the environment when true. Unset leaves the choice to the workflow…
multipool	1	Sets the number of workers of the dispatch fan-out of this workflow node. A numerical Hessian is 6N independent displaced evaluations, which is the…
hess_driver	None	Chooses who produces the force constant matrix. main makes the framework displace the geometry and difference the gradients, and skill asks the…

&HESSIAN		
hess_mode	None	Chooses how the force constants are produced. analytic asks for the analytical second derivative of the backend, numerical displaces the coordinates…
hess_step	0.005	Sets the displacement of each mobile Cartesian coordinate, in Angstrom. The gradient is differenced in Hartree per Bohr, so the quotient carries the…
hess_vibration	true	Reads the matrix into vibrational frequencies and normal modes. False leaves the run with the force constants alone. The default is true.
hess_thermo	true	Carries the modes on into the thermochemical corrections at the declared temperature and pressure. It is read only when the vibrational analysis…
hess_molden	false	Writes the frequencies and normal modes to <title>.freq.molden in the run directory. That name is the record the initial-condition sampler looks for…
hess_atoms	[]	Selects the mobile atoms of a partial Hessian, counted from one; the atoms it does not name are held. An empty list displaces every atom. The value…
hess_temperature	298.15	Sets the temperature of the thermochemical corrections, in kelvin. The default is 298.15.
hess_pressure	1	Sets the standard-state pressure of the thermochemical corrections, in bar. It enters the translational partition function through the factor kT over…
hess_scale	1	Sets the scaling factor applied to every harmonic frequency before the modes are read or written out. The default is 1.0.
hess_cutoff	100	Sets the wavenumber, in cm-1, below which the vibrational entropy starts to blend into the free-rotor form. Zero turns the blending off and leaves…

&HESSIAN		
hess_alpha	4	Sets the exponent of the quasi-harmonic blend below hess_cutoff. The default is 4.0.
hess_symmetry	1	Sets the rotational symmetry number of the molecule in the thermochemical corrections. The default is 1, so an uncorrected symmetry leaves the…
hess_linear	None	Declares the structure collinear, which costs the rotor one degree of freedom. Unset lets the analysis detect it from the geometry.

Full descriptions for all available keywords are summarized below.

multitask Runs the independent displaced evaluations of the batch concurrently when true and one at a time when false. Unset leaves the choice to the workflow class default.

remotetask Submits each displaced evaluation to the cluster through the slurm section of the environment when true. Unset leaves the choice to the workflow class default.

multipool Sets the number of workers of the dispatch fan-out of this workflow node. A numerical Hessian is 6N independent displaced evaluations, which is the case this pool exists for. The default is 1, which keeps execution serial.

hess_driver Chooses who produces the force constant matrix, one of main and skill. main displaces the geometry and differences the gradients the framework is already entitled to, one child runtime per displaced point; skill asks the backend for the matrix it computes for itself, and whether that backend answers with an analytical second derivative or with displacements of its own is the backend's keyword, so from this side the matrix simply arrives in the result. An unset value lets the jobtype token decide, num_hess and num_freq naming the framework and hessian and frequency naming the backend. main with hess_mode analytic or auto is refused, because the framework has no analytical second derivative and auto is the statement that no producer has been settled on.

hess_mode Chooses how the force constants are produced, one of analytic, numerical and auto. analytic asks the backend for its own second derivative, numerical displaces each mobile coordinate by hess_step and differences the gradients, and auto takes the analytical matrix when the evaluation returns one and falls back to the displacement route when it does not, which the run log states. Two pairings are refused: main with analytic and main with auto. skill with numerical is legal and means the engine displaces its own atoms. An unset value lets the jobtype token choose.

hess_step Sets the displacement of each mobile Cartesian coordinate, in Angstrom. The numerical route evaluates the gradient at plus and minus this displacement and forms the central difference, which is 2 times 3N evaluations for a full Hessian, each in its own child runtime. The gradients are Hartree per Bohr, so the difference quotient carries the factor between the two units. The default is 0.005.

hess_vibration Reads the matrix into vibrational frequencies, normal modes, reduced masses and rotational constants. The vibrational problem is built from the atoms the batch displaced, the six rigid directions are removed by rank, and an imaginary mode is reported as a negative wavenumber. Setting

it false leaves the run with the force constants alone, which is what a search that only wants the curvature asks for. The default is true.

hess_thermo Carries the modes on into the thermochemical corrections: zero point energy, internal energy, enthalpy, entropy, free energy and the two heat capacities, all in Hartree or Hartree per kelvin, with the pieces they were summed from kept beside them. The analysis is a rigid-rotor harmonic-oscillator sum whose vibrational entropy blends into a free-rotor form below hess_cutoff, damped by hess_alpha. It is read only when the vibrational analysis runs. The default is true.

hess_molden Writes the frequencies and normal modes to <title>.freq.molden in the run directory. The exact name matters: it is the record the initial-condition sampler reads when a dynamics run asks for frequencies. The file is written only when this keyword is true. The default is false.

hess_atoms Selects the atoms of a partial Hessian, counted from one as every atom selection in an input is; the atoms it does not name are held. The analysis re-reads the mobile set from the displacement rows, so a held atom does not appear as a soft mode. During a search the frozen set belongs to the constraint plan: a hess_atoms set that matches the plan is accepted, one that clashes with it is refused, and one written with no fix behind it is refused because the matrix would hold rows of zeros on atoms the run may move. An empty list displaces every atom.

hess_temperature Sets the temperature of the thermochemical corrections, in kelvin. The default is 298.15.

hess_pressure Sets the standard-state pressure of the thermochemical corrections, in bar. It enters the translational partition function through the factor kT over p. The default is 1.0.

hess_scale Sets the scaling factor applied to every harmonic frequency before the modes are read or written out. The default is 1.0.

hess_cutoff Sets the wavenumber, in cm-1, below which the vibrational entropy starts to blend into the free-rotor form. Zero turns the blending off and leaves the plain harmonic oscillator. The default is 100.0.

hess_alpha Sets the exponent of the quasi-harmonic blend below hess_cutoff. The default is 4.0.

hess_symmetry Sets the rotational symmetry number of the molecule in the thermochemical corrections. The default is 1, so an uncorrected symmetry leaves the entropy high by k ln sigma.

hess_linear Declares the structure collinear, which costs the rotor one degree of freedom. Unset lets the analysis detect it from the geometry.

A vibrational analysis and a variable cell are refused together, because the six lattice entries have no mass to be weighted by, and a backend matrix is refused for a queue of more than one backend. The matrix returned by a backend and the matrix assembled by the framework are written to the same result key, so a reader cannot tell which producer answered.

5.6. SEARCH

The search section configures the hyperparameter grid scan of jobtype search. Its grid maps the keywords a skill declares as tunable to the candidate values to try, and the scan is the Cartesian product of those lists. The framework never interprets the vocabulary: which keyword may vary and what a candidate means is the declaration of the skill that would train with it. The section also says how the points are executed, where their directories live, and how the finished folders are ranked.

A scan trains one model per grid point and ranks the finished folders; it does not judge them against a threshold, and only the fit error of a point is comparable across points. A rerun into the same grid_root skips the folders that are already complete and ranks them again, and retrieve reads the folders of a previous scan without submitting anything.

The keywords, default values, and short descriptions are listed below.

&SEARCH		
grid	{}	Maps each tunable keyword to the list of candidate values, for example depth to [1, 2] and nn_size to [8, 16]. The scan is the Cartesian product of…
retrieve	0	Ranks the folders a previous scan left under grid_root instead of submitting anything. A value of 0 runs the scan and a non-zero value reads the disk…
sort_by	''	Names the error channel the points are ranked on. Empty takes the first channel the trained profile reports. The default is empty.
sort_stat	max	Chooses the statistic of a point: max ranks by the worst ensemble member and mean by the average over the members. A misspelled value is refused. The…
sort_order	asc	Chooses the order of the ranking: asc puts the smallest error first and desc the largest. The default is asc.
grid_root	./grid-search	Sets the root directory of the scan. Each point gets one model directory under it, named NN-<title>-<key> after its values, so a restart re-derives…
max_grid	0	Refuses a scan that expands to more than this many points. A value of 0 states no limit. The default is 0.
multitask	None	Fans the points of the scan out over a local worker pool when true and runs them one at a time in this process when false. Unset leaves the choice to…
remotetask	None	Submits each point as a remote child runtime through the slurm section of the environment when true. Unset leaves the choice to the workflow class…

&SEARCH		
multipool	1	Sets the number of points that run at once. It is a concurrency count, not a core count: the thread and rank counts of one training stay in the skill…

Full descriptions for all available keywords are summarized below.

grid Maps each tunable keyword to the list of candidate values, and the scan is the Cartesian product of those lists. The keyword vocabulary belongs to the skill: a name the skill does not declare as searchable, a name written twice, and an entry with no candidate are all refused when the scan is planned, which is the one place that can see the declaration. The classic carrier writes the whole grid on one keyword with semicolons between the entries, and one grid line per keyword reaches the same mapping because a repeated key keeps both values. There is no start, stop and step generator: the values to try are written out.

retrieve Reads the folders a previous scan left under grid_root and ranks them, submitting nothing. It is the mode to use when the trainings are already on disk and only the table is wanted, and it is the same reader the parent process falls back to for a point whose own child died, so a folder read back and a folder reported at the end of its own fit cannot disagree. A value of 0 runs the scan instead.

sort_by Names the error channel the points are ranked on, for example the energy channel or a gradient channel of the trained profile. An empty value takes the first channel the profile reports, which the plan derives rather than the user guessing it. A point that reports no value for the channel is ranked last in either order, because it has no error rather than a large one. The default is empty.

sort_stat Chooses the statistic of a point: max ranks by the worst ensemble member and mean by the average over the members. A misspelled value is refused. The default is max.

sort_order Chooses the order of the ranking: asc puts the smallest error first and desc the largest. The default is asc.

grid_root Sets the root directory of the scan. Each point gets one model directory under it, named NN-<title>-<key> after its values, so a restart re-derives the folder rather than guessing it. The default is ./grid-search.

max_grid Refuses a scan that expands to more than this many points. A Cartesian product grows quietly, so the limit is a guard against submitting a scan larger than the machine or the account can carry. A value of 0 states no limit. The refusal names the size the grid expanded to and the limit the run declared.

multitask Fans the points of the scan out over a local worker pool when true and runs them one at a time in this process when false. Unset leaves the choice to the workflow class default.

remotetask Submits each point as a remote child runtime through the slurm section of the environment when true. Unset leaves the choice to the workflow class default.

multipool Sets the number of points that run at once. It is a concurrency count, not a core count: the thread and rank counts of one training stay in the skill section. The default is 1.

A scan ranks, it never judges: there is no acceptance threshold, and a run whose points all completed is reported as completed even when a few points failed. A scan is also refused under a retraining session, because a continuation cannot change the architecture a grid point would vary.

5.7. ADAPTIVE

The adaptive section steers the iterative sampling loop of the adaptive jobtype. It fixes the iteration budget, the screening window and its thresholds, the property channels the labels carry, the analysis and exploration options, and the dispatch flags of the three stages. The classic spelling of the section is &ADAPTIVE; in YAML and agent JSON it is the adaptive mapping.

Iteration 1 is the external starting point: the training data named in the input and the pretrained ensemble are taken as given, the MD trajectories run on that network, and their retained history is screened. Every iteration k of 2 and above labels the structures of iteration k-1 with the reference queue, appends those labels to the dataset, trains the iteration k network from the appended dataset, runs the MD trajectories on the new network, and screens them again. The loop stops when a round selects nothing, when the budget set by maxiter is spent, when a saturation test fires, or when a missed data check ends the round.

The parent workflow never evaluates the potential: it holds no backend, and the reference, training and MD stages are child runtimes materialized from the parent state, each with its own calculator. Section 10 describes the loop and its products, and Section 11 describes the dispatch layer the stage flags belong to.

The keywords, default values, and short descriptions are listed below. A † marks a keyword that is accepted but has no effect in v3.0.0; ‡ marks one value that is honoured; § marks an alias; * marks a derived quantity.

&ADAPTIVE		
maxiter	1	Sets the number of iterations. A value of 1 runs the external start only, and 0 ends the run before the first trajectory with the reason maxiter is…
maxsample	1	Caps the number of structures one trajectory contributes to the reference calculation in one iteration. The cap follows the distance filter in the…
load ‡	1	Chooses where iteration 1 takes its data and network. Only 1 is accepted: the run starts from the training data and the pretrained ensemble named in…
transfer ‡	0	Requests transfer learning from the previous network. Only 0 is accepted; transfer learning is not ported in this build and any other value stops the…
remote_train	0	Sets the route of the training child. 0 runs the fit locally; a non-zero value submits it through the remote route. Each iteration trains one…

&ADAPTIVE		
keep_md	1	Keeps the round MD directory. 1 keeps iter-<k>/md after the round finishes, and 0 deletes it, which leaves the screening record as the only trace of…
record_step	10	Sets the screening window: the last N retained history frames of every trajectory are screened each iteration. 0 is normalized to 10, and the MD…
check_population	1	Collects the structure the MD loop stopped on when its own population guard ended the trajectory with stop code 1. 1 collects that frame, 0 leaves it…
check_energy_order	1	Turns on the same-spin level-order check. With 1 a frame whose same-spin level lies below an earlier one by more than gap_tolerance is a trigger and…
gap_tolerance	1e-06	Sets the tolerance of the same-spin level-order check, in Hartree. An inversion larger than this value flags the frame, and a smaller one is treated…
sample_energy ‡	1	Asks for energy labels and energy deviations. Only 1 is accepted: the energy is half of the training baseline and validation refuses 0.
sample_grad ‡	1	Asks for gradient labels and gradient deviations. Only 1 is accepted: the gradient is half of the training baseline and validation refuses 0.
sample_nac	0	Asks for NAC labels and derivative-coupling deviations, labelled in Hartree/Bohr. 0 and 1 are accepted; turning it on switches &MD nactype to nac, a…
sample_soc	0	Asks for SOC labels, labelled in cm-1. 0 and 1 are accepted; turning it on switches &MD dosoc on, and the reference queue must plan the soc…
sample_dipole ‡	0	Asks for dipole labels. Only 0 is accepted in this build, which labels energy, gradient, NAC and SOC only: an input that turns this key or &MD…

&ADAPTIVE		
sample_transition_dipole ‡	0	Asks for transition-dipole labels. Only 0 is accepted in this build: the label extractor stops at energy, gradient, NAC and SOC, so an input that…
sample_oscillator_strength ‡	0	Asks for oscillator-strength labels. Only 0 is accepted in this build, for the same reason as sample_transition_dipole: the channel is not labelled…
predict_mode	0	Chooses the ensemble member that serves the MD trajectories. 0 returns the member mean; k above 0 returns the 1-based member k as the primary…
maxmae	0	Sets the validation energy MAE ceiling of the training-data quality gate, in eV. 0 disables the gate, so every fit stands.
maxenergy	0.05	Sets the energy deviation that triggers a frame, in Hartree, and is stamped on every MD child of the round as md_maxenergy, replacing the value &MD…
minenergy	0.02	Sets the energy deviation that puts a frame in the candidate pool, in Hartree. A frame above it is selected when the trajectory triggered anywhere in…
dynenergy †	0.1	Belongs to the legacy dynamic-threshold family of the energy channel. The value is stored on the section and no adaptive stage reads it.
inienergy †	0.3	Belongs to the legacy dynamic-threshold family of the energy channel. The value is stored on the section and no adaptive stage reads it.
fwdenergy †	1	Belongs to the legacy dynamic-threshold family of the energy channel. The value is stored on the section and no adaptive stage reads it.
bckenergy †	1	Belongs to the legacy dynamic-threshold family of the energy channel. The value is stored on the section and no adaptive stage reads it.
maxgrad	0.15	Sets the gradient deviation that triggers a frame, in Hartree/Bohr, and is stamped

&ADAPTIVE		
		on every MD child of the round as md_maxgrad. Validation requires…
mingrad	0.06	Sets the gradient deviation that puts a frame in the candidate pool, in Hartree/Bohr. Validation requires it not to exceed maxgrad.
dyngrad †	0.1	Belongs to the legacy dynamic-threshold family of the gradient channel. The value is stored on the section and no adaptive stage reads it.
inigrad †	0.3	Belongs to the legacy dynamic-threshold family of the gradient channel. The value is stored on the section and no adaptive stage reads it.
fwdgrad †	1	Belongs to the legacy dynamic-threshold family of the gradient channel. The value is stored on the section and no adaptive stage reads it.
bckgrad †	1	Belongs to the legacy dynamic-threshold family of the gradient channel. The value is stored on the section and no adaptive stage reads it.
maxnac	0.15	Sets the NAC deviation that triggers a frame, in Hartree/Bohr, and is stamped on every MD child of the round as md_maxnac. Validation requires it to…
minnac	0.06	Sets the NAC deviation that puts a frame in the candidate pool, in Hartree/Bohr. Validation requires it not to exceed maxnac.
dynnac †	0.1	Belongs to the legacy dynamic-threshold family of the NAC channel. The value is stored on the section and no adaptive stage reads it.
ininac †	0.3	Belongs to the legacy dynamic-threshold family of the NAC channel. The value is stored on the section and no adaptive stage reads it.
fwdnac †	1	Belongs to the legacy dynamic-threshold family of the NAC channel. The value is stored on the section and no adaptive stage reads it.

&ADAPTIVE		
bcknac †	1	Belongs to the legacy dynamic-threshold family of the NAC channel. The value is stored on the section and no adaptive stage reads it.
maxsoc	50	Sets the SOC deviation that triggers a frame, in cm-1, and is stamped on every MD child of the round as md_maxsoc. Validation requires it to be at…
minsoc	20	Sets the SOC deviation that puts a frame in the candidate pool, in cm-1. Validation requires it not to exceed maxsoc.
dynsoc †	0.1	Belongs to the legacy dynamic-threshold family of the SOC channel. The value is stored on the section and no adaptive stage reads it.
inisoc †	0.3	Belongs to the legacy dynamic-threshold family of the SOC channel. The value is stored on the section and no adaptive stage reads it.
fwdsoc †	1	Belongs to the legacy dynamic-threshold family of the SOC channel. The value is stored on the section and no adaptive stage reads it.
bcksoc †	1	Belongs to the legacy dynamic-threshold family of the SOC channel. The value is stored on the section and no adaptive stage reads it.
refine ‡	0	Requests the legacy geometry refinement of the selected structures. It is not ported: only 0 is accepted, and any other value stops the run at…
refine_num †	4	Belongs to the legacy refinement family. The value is stored on the section and no adaptive stage reads it.
refine_start †	0	Belongs to the legacy refinement family. The value is stored on the section and no adaptive stage reads it.
refine_end †	200	Belongs to the legacy refinement family. The value is stored on the section and no adaptive stage reads it.

&ADAPTIVE		
refine_gap †	0.3	Belongs to the legacy refinement family, where it named the gap criterion of a refinement window. The value is stored on the section and no adaptive…
explore_udd	0	Turns on UDD exploration, which builds a bias from the covariance of the ensemble members. It needs predict_mode above 0, at least two members and a…
explore_gap	0	Turns on gap-driven exploration, which needs at least two electronic states and marks its trajectories as exploration, outside the production…
gap_threshold	0.03	Sets the gap, in Hartree, at which gap driving starts. At or below this value the correction is exactly zero; above it the cv potential grows and…
udd_mode	force	Chooses what the UDD bias changes: force modifies the nuclear force only, potential adds the bias energy to the active state as well. The gradient…
udd_amplitude	0	Sets the height of the UDD bias, in Hartree. 0 selects the automatic value, 5.0e-3 Hartree.
udd_width	0	Sets the width of the UDD bias, in Hartree. 0 selects the automatic value, one tenth of the ensemble standard deviation.
udd_start_iteration	1	Delays the UDD bias to the given iteration, counted from 1. Earlier rounds run unbiased and report no bias work.
gap_method	cv	Chooses the gap-driving method. cv splits the selected pair with an antisymmetric potential that leaves the pair mean unchanged; paper replaces the…
gap_pair	[]	Names the state pair the gap bias acts on, as two indices such as 0 1. An empty value takes the pair from the state ladder, and a list that is…

&ADAPTIVE		
gap_height	0	Sets the height of the saturating cv gap potential, in Hartree. 0 selects the automatic value, 1.0e-3 Hartree.
gap_width	0	Sets the decay width of the cv gap potential, in Hartree. 0 selects the automatic value, one third of the gap being driven.
analyze_pes	0	Turns on the QM-side PES analysis of the dataset. It reads the reference labels already in the data and never an NN frame, and reports point-pair…
analyze_gap	off	Selects the scope of the gap statistics: off, s0s1, adjacent or all. The analysis reads the unbiased predicted state energies and reports the minimum…
analyze_trace	0	Collects the scalar trajectory trace through the MD child: maximum force, population, energy drift and the minimum inter-atomic distance. A short…
analyze_coverage	0	Turns on the descriptor coverage report and the saturation test. Coverage is measured over a bounded sample of the dataset, at most 200 frames and…
analyze_window	3	Sets the number of rounds in the moving average of the saturation test, at least 1. The test needs one round more of validation MAE than the window.
analyze_local_tolerance	0	Sets the per-round validation-MAE improvement, in eV, that still counts as progress. An improvement above it keeps the round unsaturated even when…
analyze_global_tolerance	0	Sets the validation-MAE improvement, in eV, below which the fit counts as saturated. With both tolerances at 0 the saturation stop is disabled.
dynsample ‡	0	Requests the dynamic threshold update of the legacy loop. It is not ported: only 0 is accepted, and any other value stops the run at start-up with a…

&ADAPTIVE		
maxdiscard †	0	Belongs to the legacy discard budget. The value is stored on the section and no adaptive stage reads it.
pop_step †	200	Belongs to the legacy population-report cadence. The value is stored on the section and no adaptive stage reads it.
multitask	None	Sets the generic fan-out flag of the adaptive node. It reaches the MD and reference stages as the fallback of md_multitask and reference_multitask…
remotetask	None	Sets the generic remote flag of the adaptive node. It reaches the MD stage only, as the fallback of md_remotetask: training follows remote_train and…
multipool	1	Sets the size of the adaptive node own worker pool, at least 1. Each stage batch runs under its own pool, md_multipool for the trajectories and…
md_multitask	None	Overrides the fan-out flag of the MD stage. Unset falls back to multitask and then to the workflow default, which is serial, and one trajectory runs…
md_remotetask	None	Overrides the remote flag of the MD stage. Unset falls back to remotetask; a remote MD child is serialized and submitted by the framework, with the…
md_multipool	None	Sets the worker pool of the MD stage, at least 1, with one trajectory per worker. Unset means 1, and the generic multipool does not raise it.
reference_multitask	None	Overrides the fan-out flag of the reference stage. Unset falls back to multitask and then to the workflow default, which is serial, and the stage…
reference_remotetask	None	Switches the reference skill itself to its slurm execution mode. The reference child stays local and the skill writes its own submission script, so…
reference_multipool	None	Sets the worker pool of the reference stage, at least 1. Unset means 1, and the generic multipool does not raise it.

Full descriptions for all available keywords are summarized below.

maxiter Sets the number of iterations. A value of 1 runs the external start only, and 0 ends the run before the first trajectory with the reason maxiter is 0. The loop also stops with maxiter exhausted when a round selects structures and the budget is spent, so the last selection has no reference consumer. A terminal iteration still labels the previous selection, trains its network and runs MD; only its own selection goes unused.

maxsample Caps the number of structures one trajectory contributes to the reference calculation in one iteration. The candidate pool is the union of the frames above the minimum thresholds, the physically bad frames and the stopping frame when it is collected, taken newest first; the distance filter runs over the whole pool and the cap is applied after it. A value of 0 therefore selects nothing, and the round then ends with the reason no sample selected.

load Chooses where iteration 1 takes its data and network. Only 1 is accepted: the run starts from the training data and the pretrained ensemble named in the input, and any other value stops the run at start-up with a message that asks for a jobtype train run first. The key is kept because the legacy loop could also start from raw data, which this build does not implement.

transfer Requests transfer learning from the previous network. Only 0 is accepted; transfer learning is not ported in this build and any other value stops the run at start-up.

remote_train Sets the route of the training child. 0 runs the fit locally; a non-zero value submits it through the remote route. Each iteration trains one network, so the key decides the route of one child runtime and not a fan-out.

keep_md Keeps the round MD directory. 1 keeps iter-<k>/md after the round finishes, and 0 deletes it, which leaves the screening record as the only trace of the trajectories.

record_step Sets the screening window: the last N retained history frames of every trajectory are screened each iteration. 0 is normalized to 10, and the MD child keeps at least four frames of history whatever the value. A trajectory shorter than the window is screened on the frames it has, and its completed flag follows the last recorded step against the planned MD length.

check_population Collects the structure the MD loop stopped on when its own population guard ended the trajectory with stop code 1. The tolerance is not adaptive: &MD md_maxpop owns the judgment, the adaptive child inherits that value like any other MD run, and this switch only decides whether the stopping frame goes to the reference calculation. A frame whose stored density looks poor but whose run did not stop is not collected for that reason, and a failed calculation (stop code 4) is never collected.

check_energy_order Turns on the same-spin level-order check. With 1 a frame whose same-spin level lies below an earlier one by more than gap_tolerance is a trigger and a candidate; the check reads energy and state-spin labels, and a trajectory that returns neither is reported as missing.

gap_tolerance Sets the tolerance of the same-spin level-order check, in Hartree. An inversion larger than this value flags the frame, and a smaller one is treated as numerical noise.

sample_energy Asks for energy labels and energy deviations. Only 1 is accepted: the energy is half of the training baseline and validation refuses 0.

sample_grad Asks for gradient labels and gradient deviations. Only 1 is accepted: the gradient is half of the training baseline and validation refuses 0.

sample_nac Asks for NAC labels and derivative-coupling deviations, labelled in Hartree/Bohr. Turning it on switches &MD nactype to nac, and a run whose nactype is already nac has this key switched on by normalization, so the two declarations cannot disagree. The reference queue must plan

the nac capability, and an initial dataset that already carries NAC labels must agree with it, or start-up refuses the run.

sample_soc Asks for SOC labels, labelled in cm-1. 0 and 1 are accepted; turning it on switches &MD dosoc on, and the reference queue must plan the soc capability.

sample_dipole Asks for dipole labels. Only 0 is accepted in this build, which labels energy, gradient, NAC and SOC only: an input that turns this key or &MD dodipole on is refused by normalization.

sample_transition_dipole Asks for transition-dipole labels. Only 0 is accepted in this build: the label extractor stops at energy, gradient, NAC and SOC, so an input that turns this key or &MD radiative on is refused by normalization.

sample_oscillator_strength Asks for oscillator-strength labels. Only 0 is accepted in this build, for the same reason as sample_transition_dipole: the channel is not labelled yet.

predict_mode Chooses the ensemble member that serves the MD trajectories. 0 returns the member mean; k above 0 returns the 1-based member k as the primary prediction and writes the other members to the alternative channels of the predictor comparison. A value above the member count, or any positive value with a one-member ensemble, is refused.

maxmae Sets the validation energy MAE ceiling of the training-data quality gate, in eV. With a positive value, a fit whose validation energy MAE exceeds it is retried after the worst outlier among the frames added in this round is removed, one frame per attempt and one retraining per removal. A removal stays only when it lowers the MAE; when it does not, or when every new frame is an outlier, the run stops before MD with the reason recorded in the iteration record. A value of 0 disables the gate, so every fit stands.

maxenergy Sets the energy deviation that triggers a frame, in Hartree, and is stamped on every MD child of the round as md_maxenergy, replacing the value &MD declared. Validation requires it to be at least minenergy.

minenergy Sets the energy deviation that puts a frame in the candidate pool, in Hartree. A frame above it is selected when the trajectory triggered anywhere in its window, and validation requires it not to exceed maxenergy.

dynenergy Belongs to the legacy dynamic-threshold family of the energy channel. The value is stored on the section and no adaptive stage reads it. This keyword is accepted but has no effect in v3.0.0.

inienergy Belongs to the legacy dynamic-threshold family of the energy channel. The value is stored on the section and no adaptive stage reads it. This keyword is accepted but has no effect in v3.0.0.

fwdenergy Belongs to the legacy dynamic-threshold family of the energy channel. The value is stored on the section and no adaptive stage reads it. This keyword is accepted but has no effect in v3.0.0.

bckenergy Belongs to the legacy dynamic-threshold family of the energy channel. The value is stored on the section and no adaptive stage reads it. This keyword is accepted but has no effect in v3.0.0.

maxgrad Sets the gradient deviation that triggers a frame, in Hartree/Bohr, and is stamped on every MD child of the round as md_maxgrad. Validation requires it to be at least mingrad.

mingrad Sets the gradient deviation that puts a frame in the candidate pool, in Hartree/Bohr. Validation requires it not to exceed maxgrad.

dyngrad Belongs to the legacy dynamic-threshold family of the gradient channel. The value is stored on the section and no adaptive stage reads it. This keyword is accepted but has no effect in v3.0.0.

inigrad Belongs to the legacy dynamic-threshold family of the gradient channel. The value is stored on the section and no adaptive stage reads it. This keyword is accepted but has no effect in v3.0.0.

fwdgrad Belongs to the legacy dynamic-threshold family of the gradient channel. The value is stored on the section and no adaptive stage reads it. This keyword is accepted but has no effect in v3.0.0.

bckgrad Belongs to the legacy dynamic-threshold family of the gradient channel. The value is stored on the section and no adaptive stage reads it. This keyword is accepted but has no effect in v3.0.0.

maxnac Sets the NAC deviation that triggers a frame, in Hartree/Bohr, and is stamped on every MD child of the round as md_maxnac. Validation requires it to be at least minnac.

minnac Sets the NAC deviation that puts a frame in the candidate pool, in Hartree/Bohr. Validation requires it not to exceed maxnac.

dynnac Belongs to the legacy dynamic-threshold family of the NAC channel. The value is stored on the section and no adaptive stage reads it. This keyword is accepted but has no effect in v3.0.0.

ininac Belongs to the legacy dynamic-threshold family of the NAC channel. The value is stored on the section and no adaptive stage reads it. This keyword is accepted but has no effect in v3.0.0.

fwdnac Belongs to the legacy dynamic-threshold family of the NAC channel. The value is stored on the section and no adaptive stage reads it. This keyword is accepted but has no effect in v3.0.0.

bcknac Belongs to the legacy dynamic-threshold family of the NAC channel. The value is stored on the section and no adaptive stage reads it. This keyword is accepted but has no effect in v3.0.0.

maxsoc Sets the SOC deviation that triggers a frame, in cm-1, and is stamped on every MD child of the round as md_maxsoc. Validation requires it to be at least minsoc.

minsoc Sets the SOC deviation that puts a frame in the candidate pool, in cm-1. Validation requires it not to exceed maxsoc.

dynsoc Belongs to the legacy dynamic-threshold family of the SOC channel. The value is stored on the section and no adaptive stage reads it. This keyword is accepted but has no effect in v3.0.0.

inisoc Belongs to the legacy dynamic-threshold family of the SOC channel. The value is stored on the section and no adaptive stage reads it. This keyword is accepted but has no effect in v3.0.0.

fwdsoc Belongs to the legacy dynamic-threshold family of the SOC channel. The value is stored on the section and no adaptive stage reads it. This keyword is accepted but has no effect in v3.0.0.

bcksoc Belongs to the legacy dynamic-threshold family of the SOC channel. The value is stored on the section and no adaptive stage reads it. This keyword is accepted but has no effect in v3.0.0.

refine Requests the legacy geometry refinement of the selected structures. It is not ported: only 0 is accepted, and any other value stops the run at start-up with a message that asks for 0.

refine_num Belongs to the legacy refinement family. The value is stored on the section and no adaptive stage reads it. This keyword is accepted but has no effect in v3.0.0.

refine_start Belongs to the legacy refinement family. The value is stored on the section and no adaptive stage reads it. This keyword is accepted but has no effect in v3.0.0.

refine_end Belongs to the legacy refinement family. The value is stored on the section and no adaptive stage reads it. This keyword is accepted but has no effect in v3.0.0.

refine_gap Belongs to the legacy refinement family, where it named the gap criterion of a refinement window. The value is stored on the section and no adaptive stage reads it. This keyword is accepted but has no effect in v3.0.0.

explore_udd Turns on UDD exploration, which builds a bias from the covariance of the ensemble members. It needs predict_mode above 0, at least two members and a single-center run, and it follows the active state, so a hop moves the target. The bias starts at udd_start_iteration, and exploration trajectories are recorded but kept out of the production statistics and the population judgment.

explore_gap Turns on gap-driven exploration, which needs at least two electronic states and marks its trajectories as exploration, outside the production statistics and the population judgment. With gap_method cv the selected pair is split by an antisymmetric potential above gap_threshold, so the pair mean is unchanged and the correction vanishes at the threshold. With gap_method paper the active propagation gradient becomes the upper-minus-lower difference, the excess energy is paid by rescaling the velocities, and the next round starts from the last pre-trigger frame, half on the lower and half on the upper state.

gap_threshold Sets the gap, in Hartree, at which gap driving starts. At or below this value the correction is exactly zero, and above it the cv potential grows monotonically and saturates at gap_height. The gap analysis counts crossings of the same value, and a negative gap stops the exploration, which keeps the bias from driving an already inverted ladder.

udd_mode Chooses what the UDD bias changes: force modifies the nuclear force only, potential adds the bias energy to the active state as well. The gradient channel is modified in both modes.

udd_amplitude Sets the height of the UDD bias, in Hartree. 0 selects the automatic value, 5.0e-3 Hartree.

udd_width Sets the width of the UDD bias, in Hartree. 0 selects the automatic value, one tenth of the ensemble standard deviation.

udd_start_iteration Delays the UDD bias to the given iteration, counted from 1. Earlier rounds run unbiased and report no bias work.

gap_method Chooses the gap-driving method. cv splits the selected pair with an antisymmetric potential that leaves the pair mean unchanged; paper replaces the active propagation gradient and rescales the velocities when the excess energy is payable. paper requires sfhp nosh and is refused together with explore_udd.

gap_pair Names the state pair the gap bias acts on, as two indices such as 0 1. An empty value takes the pair from the state ladder, and a list that is neither empty nor two indices long, or that names a state outside the ladder, is refused.

gap_height Sets the height of the saturating cv gap potential, in Hartree. 0 selects the automatic value, 1.0e-3 Hartree.

gap_width Sets the decay width of the cv gap potential, in Hartree. 0 selects the automatic value, one third of the gap being driven.

analyze_pes Turns on the QM-side PES analysis of the dataset. It reads the reference labels already in the data and never an NN frame: point-pair curvature and gradient-change magnitude, the energy-gradient residual, a low-rank local gradient reconstruction with its numerical rank and leave-one-out check, and the resolution curve that gives the geometry spacing at which the reconstruction stays within tolerance. Periodic datasets are not applicable in this version, and a thin neighbourhood reports the point count and nearest distance it would have needed.

analyze_gap Selects the scope of the gap statistics: off, s0s1, adjacent or all. The analysis reads the unbiased predicted state energies of the screened frames and reports the minimum gap with its pair, the first crossing below gap_threshold and the fraction of frames below it; a frame without state energies counts as missing data. The same setting tells the MD child which gap trace to record, so a switched-on scope costs per-step work in the trajectory.

analyze_trace Collects the scalar trajectory trace through the MD child: maximum force, population, energy drift and the minimum inter-atomic distance. A short trajectory with no rows reports not_applicable, and a trace that every trajectory omits stops the round.

analyze_coverage Turns on the descriptor coverage report and the saturation test. Coverage is measured over a bounded sample of the dataset, at most 200 frames and 200 atoms; saturation is the moving average of the validation MAE over analyze_window rounds, and crossing the tolerance stops the run with the reason saturated.

analyze_window Sets the number of rounds in the moving average of the saturation test, at least 1. The test needs one round more of validation MAE than the window.

analyze_local_tolerance Sets the per-round validation-MAE improvement, in eV, that still counts as progress. An improvement above it keeps the round unsaturated even when the global tolerance looks loose; 0 disables the local test.

analyze_global_tolerance Sets the validation-MAE improvement, in eV, below which the fit counts as saturated. With both tolerances at 0 the saturation stop is disabled.

dynsample Requests the dynamic threshold update of the legacy loop. It is not ported: only 0 is accepted, and any other value stops the run at start-up with a message that asks for 0.

maxdiscard Belongs to the legacy discard budget. The value is stored on the section and no adaptive stage reads it. This keyword is accepted but has no effect in v3.0.0.

pop_step Belongs to the legacy population-report cadence. The value is stored on the section and no adaptive stage reads it. This keyword is accepted but has no effect in v3.0.0.

multitask Sets the generic fan-out flag of the adaptive node. It reaches the MD and reference stages as the fallback of md_multitask and reference_multitask, while the training child is always one runtime. An unset value keeps the workflow default, which is serial.

remotetask Sets the generic remote flag of the adaptive node. It reaches the MD stage only, as the fallback of md_remotetask: training follows remote_train and the reference stage does not inherit it. An unset value keeps the workflow default, which is local.

multipool Sets the size of the adaptive node own worker pool, at least 1. Each stage batch runs under its own pool, md_multipool for the trajectories and reference_multipool for the labels, so this key does not widen a stage fan-out.

md_multitask Overrides the fan-out flag of the MD stage. Unset falls back to multitask and then to the workflow default, which is serial, and one trajectory runs per worker of md_multipool.

md_remotetask Overrides the remote flag of the MD stage. Unset falls back to remotetask; a remote MD child is serialized and submitted by the framework, with the serving skill running locally inside it.

md_multipool Sets the worker pool of the MD stage, at least 1, with one trajectory per worker. Unset means 1, and the generic multipool does not raise it.

reference_multitask Overrides the fan-out flag of the reference stage. Unset falls back to multitask and then to the workflow default, which is serial, and the stage then runs over reference_multipool workers. The stage labels one selected structure per child runtime, keeping the full reference queue and its own composite layers from the highref and midref partition. A remote reference stage is a different key: reference_remotetask switches the skill itself to its slurm mode.

reference_remotetask Switches the reference skill itself to its slurm execution mode. The reference child stays local and the skill writes its own submission script, so this key does not submit the framework runtime; unset means 0 and there is no generic fallback.

reference_multipool Sets the worker pool of the reference stage, at least 1. Unset means 1, and the generic multipool does not raise it.

The loop owns three of the four MD guards: the max* energy, gradient, NAC and SOC values are stamped on every MD child of the round, so they replace the md_max* values that &MD declared, and a channel the run does not label leaves its child value in place. The population guard is not overwritten: &MD md_maxpop keeps the value the MD section gave it, and check_population only decides what the loop does with the structure that guard stopped on.

The retired key pop_tolerance is not read: an input that still carries it is refused at validation with a message that points at &MD md_maxpop. The legacy threshold family dyn*, ini*, fwd* and bck* parses and does nothing; the screen compares against max* and min* only, and the analysis, exploration and dispatch keys are read through the adaptive section alone, never through &MD.

The one list-typed key of the section is gap_pair, whose absent value is the empty list, that is no pair selected; the table prints the field declaration of the model for that default.

5.8. TRAIN

The train section carries only the execution layer of jobtype train. The data file, the hyperparameters and the continuation switch belong to the skill section, so this section says how the one long call is executed and names the one preprocessing step the framework performs on its input. It exists because the fit is the longest call of a run and is therefore the call a cluster submission is usually for.

A training run reports whether the record arrived, not how well the model converged: the fit error is a reading the ranking of a grid scan compares, and this section sets nothing that changes it.

The keywords, default values, and short descriptions are listed below.

&TRAIN		
multitask	None	Runs the training call through a local worker pool when true and in this process when false. Unset leaves the choice to the workflow class default.

&TRAIN		
remotetask	None	Submits the training call to the cluster through the slurm section of the environment when true, which is what this section exists for. Unset leaves…
multipool	1	Sets the number of workers of the dispatch fan-out of this workflow node. It is a concurrency count, not a core count: the thread count of one fit…
permutation_map	''	Names a legacy atom-order file that expands the training set before the fit. Every non-comment row is a permutation of the atom list, counted from…

Full descriptions for all available keywords are summarized below.

multitask Runs the training call through a local worker pool when true and in this process when false. Unset leaves the choice to the workflow class default.

remotetask Submits the training call to the cluster through the slurm section of the environment when true, which is what this section exists for. Unset leaves the choice to the workflow class default.

multipool Sets the number of workers of the dispatch fan-out of this workflow node. It is a concurrency count, not a core count: the thread count of one fit stays in the skill section. The default is 1.

permutation_map Names a legacy atom-order file that expands the training set before the skill backend is initialized, so the skill receives frames and never a map. The file holds one row per additional permutation, whitespace separated and counted from one; every row must be a permutation of the atom list and, when symbols are present, must preserve the element order. Only the atom-indexed channels are re-indexed, the expanded frames are written as an inline dataset, and the operation is idempotent. The values No, None and an empty value disable it.

5.9. PREDICTION

The prediction section carries only the execution layer of jobtype prediction. The test file, the error statistics and the raw comparison products belong to the skill section, so this section says how the one evaluation is executed and nothing else. It has the same three keywords as the execution layer of the train section.

A prediction serves a trained model folder over every frame of the test file and reports the errors of the served values. The statistics are read from the result, never from the human log.

The keywords, default values, and short descriptions are listed below.

&PREDICTION		
multitask	None	Runs the evaluation through a local worker pool when true and in this process when false. Unset leaves the choice to the workflow class default.
remotetask	None	Submits the evaluation to the cluster through the slurm section of the environment when true. Unset leaves the choice to the workflow class default.
multipool	1	Sets the number of workers of the dispatch fan-out of this workflow node. A prediction is one evaluation, so this is not where cores are spent: the…

Full descriptions for all available keywords are summarized below.

multitask Runs the evaluation through a local worker pool when true and in this process when false. Unset leaves the choice to the workflow class default.

remotetask Submits the evaluation to the cluster through the slurm section of the environment when true. Unset leaves the choice to the workflow class default.

multipool Sets the number of workers of the dispatch fan-out of this workflow node. A prediction is one evaluation, so this is not where cores are spent: the thread count stays in the skill section. The default is 1.

5.10. READ

The read section names finished calculation artifacts and the labels to extract from them. A read run never starts an engine and writes no data artifact: it hands each path to the parser of the single theory skill bound to the run and returns the frames to its caller, and the run log is bypassed so the working directory is left untouched.

A read run binds exactly one theory skill, uses driver main, and has no reference skill and no multiscale partition. At least one path and at least one label are required, because a read that asks for nothing would return a dataset with no content.

The keywords, default values, and short descriptions are listed below.

&READ		
path	''	Names a calculation directory, a calculation output file, or a file that lists one path per line. A path list is anchored to the directory of the…
project	''	Sets the file stem of the artifacts to read. Empty takes the unique artifact stem

&READ		
		found in a directory, or the file stem when path names a file, and…
profile	''	Names the skill profile the parser uses to read iterative artifacts such as an MD or an optimize trajectory. Empty derives the profile from the…
energy	true	Asks the parser for energies. At least one of the four label switches must be on. The default is true.
gradient	true	Asks the parser for gradients. The default is true.
nac	false	Asks the parser for nonadiabatic couplings. The default is false.
soc	false	Asks the parser for spin-orbit couplings. The default is false.

Full descriptions for all available keywords are summarized below.

path Names a calculation directory, a calculation output file, or a file that lists one path per line. A single file whose non-comment lines all resolve to existing paths is read as a path list, and its entries are anchored to the directory of the list file, so a portable list can carry relative paths. Every other entry is a calculation directory or an output file, whose parent directory and file stem locate the artifacts. A path that does not exist is reported with the entry that failed, and a run that asks for no path is refused.

project Sets the file stem of the artifacts to read. Empty takes the unique artifact stem found in a directory, or the file stem when path names a file, and falls back to the directory basename. The default is empty.

profile Names the skill profile the parser uses, which is how an iterative artifact such as an MD or an optimize trajectory is read: the profile says which frames and which channels the file holds. An empty value derives the profile from the labels the run asks for, so the common case needs no profile. The value is passed to the reading function of the skill, not to a backend object, because a machine holding only finished artifacts has no engine to install.

energy Asks the parser for energies. At least one of the four label switches must be on. The default is true.

gradient Asks the parser for gradients. The default is true.

nac Asks the parser for nonadiabatic couplings. The default is false.

soc Asks the parser for spin-orbit couplings. The default is false.

The extracted frames are returned to the caller, which owns the filtering, the statistics and the persistence. A label that the parser cannot find is reported as missing in that frame rather than invented.

5.11. COMPOSITE

The composite section controls how the terms of a multiscale calculation are executed. The terms of one composite evaluation are the layers of an ONIOM or fragment calculation and the cross terms between them, and this section chooses whether they run one after another in this process or in a thread pool. The layers themselves are named in the control and molecular sections, not here.

The two keywords describe the mechanism and its capacity. They are the backend-term layer of the execution graph and are independent of the outer fan-out of the workflow node, so an ONIOM layer may keep its base node serial while its own terms run in a pool.

The keywords, default values, and short descriptions are listed below.

&COMPOSITE		
executor	serial	Chooses the execution mode of the composite terms. serial runs them one after another in this process and thread runs them in a pool. Thread mode…
max_workers	1	Sets the size of the thread pool. It is read only when executor is thread, and the value must be at least 1. The default is 1.

Full descriptions for all available keywords are summarized below.

executor Chooses the execution mode of the composite terms. serial runs them one after another in this process and thread runs them in a pool. Thread mode asks each backend whether it may run concurrently, a backend that refuses stops the run by name, and the default is serial.

max_workers Sets the size of the thread pool. It is read only when executor is thread, and the value must be at least 1. The default is 1.

A thread pool that the molecular section forbids, and a worker count that disagrees with the multipool of the molecular section, are both refused before any backend directory, thread or submission exists, because the two spellings describe one capacity.

5.12. ENVIRONMENT and SLURM

The environment section records the execution metadata that travels with the state: the shell lines, the free environment block, the scratch root and the scheduler fields. In v3.0.0 a calculation reads shell, environment and scratch_path; the remaining keys are carried so that a serialized runtime still describes the machine it came from, and the table marks them accordingly. The directives that a submission actually uses come from the slurm section, whose keys follow.

The environment section describes the machine a run starts on and the allocation a submitted child asks for. The engine paths, the shell setup and the scheduler settings normally live in the user file ~/.pyrai3md/pmd_env.yaml, which is created on first use and read in every session. This

section overrides those values for one run, and the merge order is the package defaults, then the user file, then this input.

The overrides belong to the run that declares them: they are applied before the workflows are planned and rolled back before the next input is built in the same process, so an input can only override a setting it declares. The values travel with the run state, which is what lets the dispatch subsystem read one source instead of the process environment.

Only four of the keys below reach the framework: shell, environment, slurm and scratch_path. The remaining scalars are the historical spellings of settings that moved into the skill sections and the slurm mapping, and they are accepted so that older input files still parse.

The keywords, default values, and short descriptions are listed below. A † marks a keyword that is accepted but has no effect in v3.0.0; ‡ marks one value that is honoured; § marks an alias; * marks a derived quantity.

&ENVIRONMENT		
shell	''	Sets raw shell commands, one per line, that are copied verbatim into the submission scripts of the main framework dispatch. The classic carrier…
environment	{}	Holds the exported environment variables as a name to value mapping, for example OMP_NUM_THREADS. The values are written under the export key of this…
program †	''	Names the program label of a legacy input. It is parsed for compatibility only: the engine of a calculation is the skill named in the theory queue…
version †	''	Names the program version of a legacy input. It is parsed for compatibility only: a skill resolves its own executable. The default is empty.
device †	cpu	Names a device selector of a legacy input. It is parsed for compatibility only: a device is chosen in the skill section that runs the calculation…
program_path †	''	Names an executable path of a legacy input. It is parsed for compatibility only: each skill has its own path keyword. The default is empty.
scratch_path	''	Sets the directory the run works in. The framework opens <title>.log here, and a

&ENVIRONMENT		
		skill without a scratch directory of its own creates its calculation…
slurm_time †	01:00:00	Sets a wall time in the legacy scalar spelling. The submission script reads the time directive of the slurm mapping of this section, so this key…
slurm_partition †	default	Sets a queue name in the legacy scalar spelling. The submission script reads the partition directive of the slurm mapping. The default is default.
slurm_ntasks †	1	Sets a task count in the legacy scalar spelling. The submission script reads ntasks from the slurm mapping. The default is 1.
num_gpu †	0	Sets a GPU count in the legacy scalar spelling. It is parsed for compatibility only: a GPU is asked for through the gres directive of the slurm…
slurm_memory †	4000	Sets a memory request in the legacy scalar spelling. The submission script reads mem from the slurm mapping. The default is 4000.
slurm_command †	''	Sets a submission command in the legacy scalar spelling. The framework submits through its own Slurm launcher, which builds the command itself. The…
slurm	{}	Merges Slurm directives into the allocation a submitted child asks for, as a mapping of directive names to values, for example partition, time, mem…

Full descriptions for all available keywords are summarized below.

shell Sets raw shell commands, one per line, that are copied verbatim into the submission scripts the main framework dispatch builds, which is where a module load or a conda activation goes. The classic carrier writes them on one line separated by semicolons. The key serves the main framework scripts only: a skill consumes its own shell keyword, so a command written here does not reach a skill subprocess. The default is empty.

environment Holds the exported environment variables as a name to value mapping. The values are written under the export key of this section or of a pmd_env.yaml file, and the merged mapping is flattened into this field with one value per name. They are exported in the batch script of a remote child and set in the subprocess environment of a local one, which is how a run pins for example OMP_NUM_THREADS or MPI_NP. The default is empty.

program Names the program label of a legacy input. It is parsed for compatibility only: the engine of a calculation is the skill named in the theory queue. The default is empty. This keyword is accepted but has no effect in v3.0.0.

version Names the program version of a legacy input. It is parsed for compatibility only: a skill resolves its own executable. The default is empty. This keyword is accepted but has no effect in v3.0.0.

device Names a device selector of a legacy input. It is parsed for compatibility only: a device is chosen in the skill section that runs the calculation. The default is cpu. This keyword is accepted but has no effect in v3.0.0.

program_path Names an executable path of a legacy input. It is parsed for compatibility only: each skill has its own path keyword. The default is empty. This keyword is accepted but has no effect in v3.0.0.

scratch_path Sets the directory the run works in. The framework opens <title>.log here, and a skill without a scratch directory of its own creates its calculation directories under it, so one keyword moves the whole run off a fast local disk or onto a shared one. A relative value is resolved against the launch directory, and an empty value keeps the historical behaviour of writing beside the launch directory.

slurm_time Sets a wall time in the legacy scalar spelling. The submission script reads the time directive of the slurm mapping of this section, so this key changes no script. The default is 01:00:00. This keyword is accepted but has no effect in v3.0.0.

slurm_partition Sets a queue name in the legacy scalar spelling. The submission script reads the partition directive of the slurm mapping. The default is default. This keyword is accepted but has no effect in v3.0.0.

slurm_ntasks Sets a task count in the legacy scalar spelling. The submission script reads ntasks from the slurm mapping. The default is 1. This keyword is accepted but has no effect in v3.0.0.

num_gpu Sets a GPU count in the legacy scalar spelling. It is parsed for compatibility only: a GPU is asked for through the gres directive of the slurm mapping. The default is 0. This keyword is accepted but has no effect in v3.0.0.

slurm_memory Sets a memory request in the legacy scalar spelling. The submission script reads mem from the slurm mapping. The default is 4000. This keyword is accepted but has no effect in v3.0.0.

slurm_command Sets a submission command in the legacy scalar spelling. The framework submits through its own Slurm launcher, which builds the command itself. The default is empty. This keyword is accepted but has no effect in v3.0.0.

slurm Merges Slurm directives into the allocation a submitted child asks for, as a mapping of directive names to values, for example partition, nodes, ntasks, cpus_per_task, mem, time, job_name, output, error, account, qos, exclusive, constraint, gres, mail_type, mail_user, array and extra. An empty value writes no directive, exclusive false writes none either, and extra is copied verbatim as the only pass-through. Unknown names are refused by the script builder rather than silently dropped. The directives are read when remotetask is true, and the merge order is the package file, then the user file, then this section.

The retired keys of this section, execution_mode, workflow_execution, workflow_workers and num_thread, are refused at validation with a message that names where each setting belongs now. Thread and rank counts are properties of a calculation, so they are written in the skill section as <skill>_thread and <skill>_nproc, and a workflow section refuses them by name.

The slurm section supplies the directives of the submission scripts the framework writes for child runtimes and for skill self-submission. The same keys can be written in three places: the slurm block of the home pmd_env.yaml, a &SLURM section in the classic input, and the slurm mapping of YAML or agent input. The merge is per key and the input wins.

An empty value writes no directive. Keys that are not listed here are refused by the script builder, so a misspelt option is reported instead of being ignored; the extra key is the only free pass-through and is copied verbatim into the script.

The keywords, default values, and short descriptions are listed below.

&SLURM		
partition	long,short,express	Sets the scheduler partition or comma-separated list of partitions for the child jobs.
nodes	1	Sets the number of nodes requested by a child job.
ntasks	10	Sets the number of tasks per node; an engine that runs with MPI uses this value as its rank count.
cpus_per_task	1	Sets the number of CPUs per task requested from the scheduler.
mem	16G	Sets the memory per node requested by the child job, for example 16G.
mem_per_cpu	''	Sets the memory per CPU instead of the memory per node; only one of mem and mem_per_cpu should be non-empty.
time	1:00:00	Sets the wall-clock limit of the child job in hh:mm:ss.
job_name	pyrai3md	Sets the job name written as the --job-name directive.
output	slurm-%j.out	Sets the standard-output file of the child job; the default uses the scheduler job identifier.
error	slurm-%j.err	Sets the standard-error file of the child job; the default uses the scheduler job identifier.
account	''	Sets the accounting project passed as --account.
qos	''	Sets the quality-of-service string passed as --qos.
exclusive	false	Requests exclusive node access when true.
constraint	''	Sets the scheduler node constraint string passed as --constraint.

&SLURM		
gres	''	Requests generic resources such as a GPU with --gres; it is also the trigger for the GPU submission template of the training skill.
mail_type	''	Sets the --mail-type events for scheduler notifications.
mail_user	''	Sets the --mail-user address for scheduler notifications.
array	''	Sets a scheduler array specification; a manual run normally leaves it empty because the framework submits one job per unit.
extra	''	Copies a free-form string into the script verbatim; it is the only pass-through directive and must be valid script syntax.

Full descriptions for all available keywords are summarized below.

partition Sets the scheduler partition or comma-separated list of partitions for every child job the run submits. A list lets the scheduler place each job where capacity exists. An empty value writes no --partition directive.

nodes Sets the number of nodes requested by a child job.

ntasks Sets the number of tasks per node. An engine that runs with MPI uses this value as its rank count, and the framework also uses it to size a grouped submission. It does not change the number of concurrent jobs, which is a dispatch setting.

cpus_per_task Sets the number of CPUs per task requested from the scheduler.

mem Sets the memory per node requested for a child job, for example 16G. It is written as --mem; mem_per_cpu is the alternative spelling and only one of the two should carry a value.

mem_per_cpu Sets the memory per CPU instead of the memory per node; only one of mem and mem_per_cpu should be non-empty.

time Sets the wall-clock limit of the child job in hh:mm:ss.

job_name Sets the job name written as the --job-name directive.

output Sets the standard-output file of the child job; the default uses the scheduler job identifier.

error Sets the standard-error file of the child job; the default uses the scheduler job identifier.

account Sets the accounting project passed as --account.

qos Sets the quality-of-service string passed as --qos.

exclusive Requests exclusive node access when true.

constraint Sets the scheduler node constraint string passed as --constraint.

gres Requests generic resources such as a GPU. It is written as --gres and doubles as the switch that makes the training skill use its GPU submission template instead of the plain one.

mail_type Sets the --mail-type events for scheduler notifications.

mail_user Sets the --mail-user address for scheduler notifications.

array Sets a scheduler array specification; a manual run normally leaves it empty because the framework submits one job per unit.

extra Copies a free-form string into the generated script verbatim. It is the only key that is not validated against the directive list, so anything written here must already be valid script syntax.

5.13. CONSTRAINT

The constraint block adds potential terms, hard constraints and collective-variable bias to a dynamics or optimization run. In YAML it is written under molecule.constraints or as a top-level constraint section; in classic input it is the &CONSTRAINT section. Every atom number in this block is 1-based in all three carriers and is converted once when the state is built.

The block has eight subsections. units fixes the units of the lengths, angles and energies that the block reads. groups names atom selections that other subsections refer to by name. cvs defines collective variables. biases adds harmonic or flat-bottomed potentials on bonds, angles, dihedrals or a named collective variable. walls adds confining potentials. fixes holds atoms or directions. constraints adds hard bond, angle and dihedral constraints solved during integration. metadynamics adds history-dependent bias. notes is free text carried with the block.

A typical block defines a group and then uses it. The example below restrains two bonds and walls the whole solute into a sphere.

```
molecule:
  constraints:
    units:
      length: angstrom
      energy: hartree
    groups:
      - name: solute
        atoms: 1-6
    biases:
      - id: b1
        atoms: [1, 2]
        target: 1.5
        force_constant: 0.1
    walls:
      - id: w1
        group: solute
        geometry: sphere
        center: [0.0, 0.0, 0.0]
        radius: 8.0
        law: soft
        force_constant: 1.0
```

units

&CONSTRAINT units		
length	angstrom	Sets the unit of every length in the block: angstrom or bohr.
angle	degree	Sets the unit of every angle in the block: degree or radian.
energy	hartree	Sets the unit of every energy-like value in the block: hartree or kjmol.

groups

Each entry defines one named selection. The selector keys atoms, element and group are mutually exclusive: an entry names atoms by index or range, by element symbol, or by an already defined group.

&CONSTRAINT groups		
atoms	[]	Selects atoms by 1-based index or index range, for example 1-3 7.
element	[]	Selects atoms by element symbol.
group	''	Selects the atoms of another named group.
name	''	Sets the name other subsections use to refer to this group.

cvs

A collective variable is a named geometric coordinate used by biases and metadynamics. Three kinds are defined by the atoms list: bond (two atoms), angle (three atoms) and dihedral (four atoms).

&CONSTRAINT cvs		
name	''	Sets the name that biases and metadynamics use for this variable.
kind	''	Sets the variable kind: bond, angle or dihedral.
atoms	[]	Sets the 1-based atoms that define the variable; two, three or four atoms according to the kind.

biases

A bias adds a harmonic or flat-bottomed potential on a bond, angle, dihedral or named collective variable. Either atoms or cv must be given, not both. The region selects where the potential acts: all, inside a range, or outside a range; the inside and outside spellings are refused and the range is given by two bounds.

&CONSTRAINT biases		
atoms	[]	Sets the 1-based atoms of the biased bond, angle or dihedral.
id	''	Sets an identifier used in logs and in ramp schedules.
cv	''	Names a collective variable defined in cvs instead of giving atoms.
target	None	Sets the target value of the biased coordinate, in the length or angle unit of the block.
force_constant	None	Sets the harmonic force constant; it must be positive.
region	all	Selects where the bias acts: all, or a range of the coordinate.

walls

A wall confines atoms inside or outside a geometric region. The geometry key chooses the region and therefore which geometry keys are read: sphere reads radius, ellipsoid reads semiaxes, box reads lengths, and triclinic reads cell. A soft wall needs force constant and power and cannot be used in a periodic system; a hard wall reflects.

&CONSTRAINT walls		
atoms	[]	Selects the confined atoms by 1-based index or range.
element	[]	Selects the confined atoms by element symbol.
group	''	Selects the confined atoms by the name of a defined group.
id	''	Sets an identifier used in logs and in ramp schedules.
geometry	sphere	Sets the wall shape: sphere, ellipsoid, box or triclinic.
center	[]	Sets the centre of the wall, in the length unit of the block.

&CONSTRAINT walls		
ref_atoms	[]	Selects atoms whose centre of mass is used instead of an explicit centre.
molecules	[]	Sets how many molecules each group member contains, for group-based walls on molecular systems.
law	hard	Sets the wall law: hard reflection or a soft potential.
radius	None	Sets the radius of a spherical wall.
semiaxes	[]	Sets the three semiaxes of an ellipsoidal wall.
lengths	[]	Sets the three box lengths of a box wall.
cell	[]	Sets the six cell parameters of a triclinic wall.
power	2	Sets the exponent of the soft wall potential; it must be at least 2.
force_constant	None	Sets the strength of the soft wall potential.
target	atom	Selects whether the wall acts on atoms or on the centre of a group: atom or center.

fixes

A fix removes selected coordinates from the dynamics or from an optimization step. The direction key uses any combination of x, y and z.

&CONSTRAINT fixes		
atoms	[]	Selects the fixed atoms by 1-based index or range.
element	[]	Selects the fixed atoms by element symbol.
group	''	Selects the fixed atoms by the name of a defined group.
id	''	Sets an identifier used in logs.
direction	xyz	Sets the frozen Cartesian directions, for example xz.

constraints

A hard constraint keeps bond lengths, angles or dihedrals at a target during integration. SHAKE is applied after each velocity update and can be paired with a velocity constraint, or the positions

can be projected without touching velocities. A bond_type entry expands to every bond of that type found in the topology.

&CONSTRAINT constraints		
id	''	Sets an identifier used in logs.
kind	bond	Sets the constrained coordinate: bond, angle or dihedral.
bonds	[]	Sets the 1-based atoms of each constrained coordinate.
bond_type	''	Expands to every bond of the given element pair.
target	None	Sets the constrained value; when absent the starting value is kept.
tolerance	1e-08	Sets the convergence tolerance of the constraint solver.
max_iterations	200	Sets the maximum number of solver iterations per step.
velocity_constraint	none	Selects whether velocities are also constrained: none or rattle.

metadynamics

Metadynamics accumulates Gaussian bias along one or more collective variables. Each entry needs a height and a width per variable, or exactly one width for an RMSD variable. A well-tempered run sets bias_factor greater than one together with a positive temperature.

&CONSTRAINT metadynamics		
atoms	[]	Sets the 1-based reference atoms of an RMSD variable.
id	''	Sets an identifier used in logs.
kind	gaussian	Sets the bias construction: gaussian or rmsd.
cvs	[]	Names the collective variables the bias is deposited along.
height	None	Sets the Gaussian height in the energy unit of the block.
width	[]	Sets the Gaussian width, one value per variable.
pace	100	Sets the number of steps between two Gaussian depositions.
reference	[]	Sets the reference structure used by an RMSD variable.

&CONSTRAINT metadynamics		
weight	uniform	Sets the weighting of atoms in an RMSD variable: uniform or mass.
bias_factor	0	Sets the well-tempered bias factor; 0 disables well-tempered metadynamics.
temperature	0	Sets the temperature used by the well-tempered update, in kelvin.

Ramp schedules

Any numeric parameter of the block can be ramped instead of held fixed. A schedule is written as a small mapping with a suffix on the parameter name: _scale multiplies the base value, _start and _end give the ramp endpoints, _delay postpones the start, and _stop ends the ramp. _scale cannot be combined with _start and _end; _delay and _stop require the endpoints. Endpoints take the shape of the base parameter, and a positive parameter may not ramp to a non-positive value.

```
biases:
  - id: b1
    atoms: [1, 2]
    target: 3.0
    force_constant: 0.1
    force_constant_start: 0.1
    force_constant_end: 1.0
    force_constant_delay: 100
    force_constant_stop: 1000
```

Legacy constraint spellings

The classic input also accepts the flat constraint tokens of PyRAI2MD. Each token maps onto one subsection entry. The table lists the tokens; the structured spelling above is the recommended one because it can name entries and carry schedules.

&CONSTRAINT classic tokens		
group_name	→	groups: name
cv_bond	→	cvs: kind bond
cv_angle	→	cvs: kind angle
cv_dihedral	→	cvs: kind dihedral
shake_bond	→	constraints: kind bond
shake_bond_type	→	constraints: bond_type
shake_angle	→	constraints: kind angle
shake_dihedral	→	constraints: kind dihedral
bias_bond	→	biases: atoms (2)

&CONSTRAINT classic tokens		
bias_angle	→	biases: atoms (3)
bias_dihedral	→	biases: atoms (4)
bias_cv	→	biases: cv
wall_sphere_radius	→	walls: geometry sphere
wall_ellipsoid_semiaxes	→	walls: geometry ellipsoid
wall_box_lengths	→	walls: geometry box
wall_triclinic_cell	→	walls: geometry triclinic
fix_atoms	→	fixes: atoms
fix_element	→	fixes: element
fix_group	→	fixes: group
meta_cv	→	metadynamics: cvs
meta_rmsd	→	metadynamics: kind rmsd

The retired structured spellings are refused with an error that names the replacement. Appendix Section D lists the refusal together with the constraint combinations that validation rejects, such as a soft wall in a periodic system, a dihedral bias with an inside or outside region, and a ramp that drives a positive parameter to zero.

5.14. Common keywords in skill sections

Every skill section accepts the same small set of keys. They are documented once here and repeated in each skill table with the same meaning, so a section can be read on its own.

The first group controls execution: where the run happens, what the calculation directory is called, and which profile and method the skill uses. The second group is the framework state the skill receives: charge, multiplicity, the state ladder of ci and spin, the requested coupling pairs, and the periodic cell. The third group is resources: threads, scratch retention and the directives of a skill self-submission.

A key written in a skill section wins over the same key in the home pmd_env.yaml. Framework state written in the skill section is also accepted but is normally written in the molecule section, where the planner can see it before the skill is bound.

all skill sections		
execution_mode	local	Chooses how the skill runs: local as a subprocess on this machine, or slurm as a child submitted to the scheduler.
project	engine-specific	Sets the file stem of the skill calculation directory; each skill has its own default such as PYSCF or MOLCAS.

all skill sections		
profile	engine-specific	Selects the computation chain of the skill; the profile tables in Sections 5.15 to 5.24 list every value.
method	engine-specific	Selects the electronic-structure method inside the profile; an incompatible method and profile pair is refused.
charges	0	Total charge of the system, read from the molecule section.
multiplicity	1	Spin multiplicity of the system, read from the molecule section.
ci	[1]	Number of states per spin multiplicity, read from the molecule section.
spin	[0]	Total spin S per multiplicity, read from the molecule section.
coupling	[]	State pairs that request a coupling, read from the molecule section.
pbc	[0, 0, 0]	Periodic flags of the three axes, read from the molecule section.
cell	[99, 99, 99, 90, 90, 90]	Cell parameters, read from the molecule section.
lattice_vector	zero matrix	Lattice vectors, read from the molecule section.
kpoint	[1, 1, 1]	k-point grid, read from the molecule section.
thread	1	Number of OpenMP threads requested from the engine; a process-wide setting, so the first value written wins.
keep_tmp	0	Keeps the skill calculation directory after the run when set to 1.
slurm_time	01:00:00	Wall-clock limit of a skill self-submission.
slurm_partition	''	Partition of a skill self-submission; an empty value writes no directive.
slurm_ntasks	1	Task count of a skill self-submission.
slurm_memory	4000	Memory in MB of a skill self-submission.
slurm_command	''	Extra sbatch command appended to a skill self-submission.

Full descriptions for all available keywords are summarized below.

execution_mode Chooses how the skill runs: local as a subprocess on this machine, or slurm as a child submitted to the scheduler.

project Sets the file stem of the skill calculation directory; each skill has its own default such as PYSCF or MOLCAS.

profile Selects the computation chain of the skill; the profile tables in Sections 5.15 to 5.24 list every value.

method Selects the electronic-structure method inside the profile; an incompatible method and profile pair is refused.

charges Total charge of the system, read from the molecule section.

multiplicity Spin multiplicity of the system, read from the molecule section.

ci Number of states per spin multiplicity, read from the molecule section.

spin Total spin S per multiplicity, read from the molecule section.

coupling State pairs that request a coupling, read from the molecule section.

pbc Periodic flags of the three axes, read from the molecule section.

cell Cell parameters, read from the molecule section.

lattice_vector Lattice vectors, read from the molecule section.

kpoint k-point grid, read from the molecule section.

thread Number of OpenMP threads requested from the engine; a process-wide setting, so the first value written wins.

keep_tmp Keeps the skill calculation directory after the run when set to 1.

slurm_time Wall-clock limit of a skill self-submission.

slurm_partition Partition of a skill self-submission; an empty value writes no directive.

slurm_ntasks Task count of a skill self-submission.

slurm_memory Memory in MB of a skill self-submission.

slurm_command Extra sbatch command appended to a skill self-submission.

The defaults above are the values most skills ship. A skill may override one of them, and its table in the following sections shows the value that skill actually uses; the slurm partition, for example, is engine-specific.

5.15. PYSCF

The pyscf section configures the PySCF skill, the in-process backend for Hartree-Fock, Kohn-Sham density functional theory, MP2, TDDFT and TDA. It returns ground- and excited-state energies, analytic gradients, numerical Hessians and frequencies, optimized minima and transition states, MO, ESP and population analysis, IR and NMR spectra, and UV/vis absorption for the time-dependent methods. Solvation runs through the PySCF solvent models, so a named solvent selects IEF-PCM while gas, vacuum, none or an empty value leaves the calculation in the gas phase.

PySCF is a Python library rather than a program: no input deck is written and no log is parsed, so the numbers come straight from live PySCF objects. The section therefore has no executable to point at, and pyscf_path stays import.

The thirty-nine profiles choose the chain and each one locks its method. The hf and dft families provide energy, gradient, hessian, opt_min, opt_ts, mo, esp, population, ir and nmr; the mp2 family provides energy, gradient and the three analysis chains; the tddft and tda families provide energy, gradient, opt_min, uv_vis and the three analysis chains. The prefix of profile and the value of method must agree.

Nothing outside the Python environment is required: the interpreter that runs the framework must be able to import pyscf, and the optional pyscf/properties extension supplies the infrared and shielding modules behind the ir and nmr profiles. No auxiliary input file is staged; chk_file names an SCF checkpoint whose contents become the initial guess whenever the file already exists. The remaining keys at the end of the table are shared with the framework sections, and the skill receives them from &MOLECULE and &MD.

The skill exposes the following profiles. A profile fixes the method and the set of properties a calculation returns.

Profile	Capability	Method
hf_energy	energy	hf
hf_gradient	gradient	hf
hf_hessian	frequency	hf
hf_opt_min	geometry_optimization	hf
hf_opt_ts	geometry_optimization	hf
hf_mo	analysis	hf
hf_esp	analysis	hf
hf_population	analysis	hf
hf_ir	ir_spectrum	hf
hf_nmr	nmr_spectrum	hf
dft_energy	energy	dft
dft_gradient	gradient	dft
dft_hessian	frequency	dft
dft_opt_min	geometry_optimization	dft
dft_opt_ts	geometry_optimization	dft
dft_mo	analysis	dft
dft_esp	analysis	dft
dft_population	analysis	dft
dft_ir	ir_spectrum	dft
dft_nmr	nmr_spectrum	dft
mp2_energy	energy	mp2
mp2_gradient	gradient	mp2
mp2_mo	analysis	mp2
mp2_esp	analysis	mp2
mp2_population	analysis	mp2
tddft_energy	energy	tddft
tddft_gradient	gradient	tddft
tddft_opt_min	geometry_optimization	tddft
tddft_uv_vis	uv_spectrum	tddft
tddft_mo	analysis	tddft
tddft_esp	analysis	tddft
tddft_population	analysis	tddft
tda_energy	energy	tda

Profile	Capability	Method
tda_gradient	gradient	tda
tda_opt_min	geometry_optimization	tda
tda_uv_vis	uv_spectrum	tda
tda_mo	analysis	tda
tda_esp	analysis	tda
tda_population	analysis	tda

The keywords, default values, and short descriptions are listed below. A † marks a keyword that is accepted but has no effect in v3.0.0; ‡ marks one value that is honoured; § marks an alias; * marks a derived quantity.

&PYSCF		
execution_mode	local	Selects the run arm: local starts the calculation in a subprocess on the current machine, and slurm writes a batch script and submits it. The default…
project	PYSCF	Sets the artifact stem. The log, checkpoint, orbital and geometry files are named after it, for example <project>.log, <project>.chk…
profile	dft_energy	Selects one of the thirty-nine profiles, named <method>_<task>, for example dft_energy or tddft_gradient. The profile fixes the complete step chain…
method	dft	Sets the electronic-structure method: hf, dft, mp2, tddft or tda. It selects the SCF reference class, the post-SCF service and the property modules a…
pyscf_path	import	Names how PySCF is reached. import loads the library from the running interpreter, which is the only mode the framework uses; there is no binary…
pyscf_project	PYSCF	Carries the project name in the spelling the framework forwards. A non-empty value overwrites project and the two are kept in step.
scratch_path	''	Sets the parent directory of the per-term calculation directory. An empty value leaves the placement to the framework scratch root.
thread	1	Sets the OpenMP thread count written to OMP_NUM_THREADS. The variable is

&PYSCF		
		latched the first time the process needs it, so a later term asking for…
pyscf_thread	1	Carries the thread count in the spelling the framework forwards. The larger of pyscf_thread and thread is applied, and the retired spelling…
slurm_time	01:00:00	Sets the wall-clock limit of the batch job in HH:MM:SS. The default is 01:00:00.
slurm_partition	''	Names the SLURM partition. An empty value writes no --partition directive, so the cluster default answers.
slurm_ntasks	1	Sets the number of SLURM tasks of the batch job. The default is 1.
slurm_memory	4000	Sets the memory request of the batch job in megabytes. The default is 4000.
slurm_command	''	Adds extra commands to the submission line of the batch job, for example an account or a constraint.
keep_tmp	0	0 removes the calculation directory once the results are collected; 1 keeps the log, checkpoint and Molden files.
functional	pbe0	Sets the exchange-correlation functional for a dft, tddft or tda run from the 140 names in the skill list, for example pbe0 or b3lyp. The hf and mp2…
basis	ccpvdz	Sets the orbital basis from the 362 names in the skill list. Dashes and spaces are stripped, so cc-pvdz is written ccpvdz and 6-31g* becomes 631g*.
density_fitting	true	Turns the resolution-of-the-identity approximation on or off through mf.density_fit(). It is switched off automatically for the tddft_gradient and…
non_local_correction	true	Turns the non-local correlation correction of the functional on through mf.nlc. Given false, mf.nlc is disabled and the functional runs without the…
dispersion_correction	''	Sets the Grimme dispersion correction handed to PySCF as mf.disp: d3bj,

&PYSCF		
		d3zero, d3bjm, d3zerom, d3op or d4. An empty value adds no correction, and…
scf_converge	1e-09	Sets the SCF energy convergence threshold in Hartree and is handed to PySCF as conv_tol. The default is 1e-9.
maxit_scf	100	Sets the SCF cycle cap and is handed to PySCF as max_cycle. The default is 100 iterations.
td_converge	1e-06	Sets the convergence threshold of the TDDFT or TDA response iterations. The default is 1e-6.
chk_file	''	Names the SCF checkpoint file. An existing file is read as the initial guess; an empty value writes…
save_mo	false	Writes <project>.mo.molden with the orbitals of the last SCF step when true. The default false skips the file.
save_norm_mode	false	Writes <project>.freq.molden with the frequencies and normal modes of a hessian, ir or nmr run when true. The default false skips the file.
opt_converge	false	Reports whether the geometry optimization converged. The skill writes it as a result and the framework reads it as the run verdict; it is not a…
opt_energy	1e-06	Sets the energy-change threshold of the geometry optimizer in Hartree, passed as convergence_energy. The default is 1e-6.
opt_grms	0.0003	Sets the root-mean-square gradient threshold of the optimizer in Hartree/Bohr, passed as convergence_grms. The default is 3e-4.
opt_gmax	0.00045	Sets the largest-gradient threshold of the optimizer in Hartree/Bohr, passed as convergence_gmax. The default is 4.5e-4.
opt_drms	0.0012	Sets the root-mean-square displacement threshold of the optimizer in Bohr, passed as convergence_drms. The default is 1.2e-3.

&PYSCF		
opt_dmax	0.0018	Sets the largest-displacement threshold of the optimizer in Bohr, passed as convergence_dmax. The default is 1.8e-3.
opt_sp	true	Runs a single-point energy after the optimization so the same record carries the final energy and dipole; given false the run returns the optimized…
opt_freq	false	Runs a numerical Hessian and frequency analysis after a converged ground-state optimization. It fires only for target state 0 and only when the…
maxit_opt	100	Sets the step cap of the geometry optimizer used by the opt_min and opt_ts profiles. The default is 100 steps.
maxit_md	1	Sets the step cap of a molecular-dynamics run. The skill registers no dynamics profile, so the value is recorded for the contract and no MD loop is…
grid_level	3	Selects the DFT integration grid level of the Kohn-Sham steps: 1 to 5 are the conventional values and 3 is the shipped default.
ir_width	100	Sets the half-width of the Gaussian broadening of the IR trace in cm-1. The default is 100.0.
ir_scale	1	Sets the scaling factor applied to the harmonic frequencies before the IR trace is drawn. The default 1.0 leaves them unscaled.
uv_width	100	Sets the half-width of the Gaussian broadening of the UV/vis spectrum in nm. The default is 100.0.
nics_ring †	[0]	Reserved NICS request: the 1-based ring atom indices whose axis would receive ghost centres. The key is declared for the NMR contract and documented…
nics_scan †	[0]	Reserved NICS request: the distances in Angstrom along the ring axis of

&PYSCF		
		nics_ring. The shipped profiles leave it at [0.0].
thermostat †	nose-hoover	Selects the thermostat of a dynamics run, nose-hoover by default, with nve and berendsen as the other names. The skill registers no dynamics profile…
barostat †	berendsen	Selects the barostat of a constant-pressure dynamics run, berendsen by default. The skill registers no dynamics profile, so the value never reaches a…
ensemble †	nve	Names the thermodynamic ensemble of a dynamics run: nve, nvt or npt. The skill registers no dynamics profile, so the value never reaches a…
charges	0	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
multiplicity	1	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
ci	[1]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
spin	[0]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
coupling	[]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
pbc	[false, false, false]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
cell	[99, 99, 99, 90, 90, 90]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
lattice_vector	[[0, 0, 0], [0, 0, 0], [0, 0, 0]]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
kpoint	[1, 1, 1]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

&PYSCF		
target_states	[0]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
target_nac_pair	[[]]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
target_soc_pair	[[]]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
verbose	4	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
temperature	298.15	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.3.
pressure	1	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.3.
solvent	gas	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
shape	ellipsoid	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
factor	40	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
scale	1	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

Full descriptions for all available keywords are summarized below.

execution_mode Selects the run arm: local starts the calculation in a subprocess on the current machine, and slurm writes a batch script and submits it. The default is local.

project Sets the artifact stem. The log, checkpoint, orbital and geometry files are named after it, for example <project>.log, <project>.chk, <project>.mo.molden and <project>.opt.xyz.

profile Selects the computation chain. The name is <method>_<task>, and the thirty-nine registered chains are hf_energy, hf_gradient, hf_hessian, hf_opt_min, hf_opt_ts, hf_mo, hf_esp, hf_population, hf_ir, hf_nmr, the same ten for dft, mp2_energy, mp2_gradient, mp2_mo, mp2_esp, mp2_population, and the seven tddft_ and tda_ chains: energy, gradient, opt_min, uv_vis, mo, esp and population. The profile alone decides which steps run, from the molecule and reference steps through

gradient, hessian, optimize, optimize_ts, the analysis chains and the spectra. A profile whose prefix does not match method is refused, and the nac, soc, dipole and stress jobtypes have no profile here.

method Sets the electronic-structure level and, with it, the reference class the chain starts from. hf runs a Hartree-Fock reference; dft runs the same reference with the exchange-correlation functional from functional; mp2 adds a second-order Moller-Plesset correction on the reference; tddft and tda run a time-dependent response on it, tda within the Tamm-Dancoff approximation. The capability table behind the profiles decides what each method may ask for: hf and dft have no UV/vis chain, mp2 has no Hessian or optimization chain, and tddft and tda have neither a Hessian, a transition-state optimization, IR nor NMR.

pyscf_path Names how PySCF is reached. import loads the library from the running interpreter, which is the only mode the framework uses; there is no binary whose path could be given here.

pyscf_project Carries the project name in the spelling the framework forwards. A non-empty value overwrites project and the two are kept in step.

scratch_path Sets the parent directory of the per-term calculation directory. An empty value leaves the placement to the framework scratch root.

thread Sets the thread count of the in-process run. The value is written to OMP_NUM_THREADS the first time the process needs the variable, because PySCF reads the setting once per process; a later term that asks for a different count keeps the latched value and says so on stderr rather than silently oversubscribing the node. The two spellings thread and pyscf_thread fold to their larger value, so neither can shrink a request made in the other.

pyscf_thread Carries the thread count in the spelling the framework forwards. The larger of pyscf_thread and thread is applied, and the retired spelling num_thread is refused.

slurm_time Sets the wall-clock limit of the batch job in HH:MM:SS. The default is 01:00:00.

slurm_partition Names the SLURM partition. An empty value writes no --partition directive, so the cluster default answers.

slurm_ntasks Sets the number of SLURM tasks of the batch job. The default is 1.

slurm_memory Sets the memory request of the batch job in megabytes. The default is 4000.

slurm_command Adds extra commands to the submission line of the batch job, for example an account or a constraint.

keep_tmp 0 removes the calculation directory once the results are collected; 1 keeps the log, checkpoint and Molden files.

functional Sets the exchange-correlation functional of a dft, tddft or tda run. The skill keeps a list of 140 names covering the LDA, GGA, meta-GGA, hybrid and range-separated functionals PySCF assembles, among them pbe, pbe0, b3lyp, b3lyp5, b3pw91, camb3lyp and wb97x; the list holds decomposed exchange and correlation parts as well, which is how the composite names are spelled without dashes. The value is compared case-insensitively. hf and mp2 carry no functional flag, so the key is ignored there.

basis Sets the orbital basis. The skill keeps a list of 362 names, written without dashes because PySCF spells them that way: ccpvdz, augccpvtz, ccpvtzpp, def2svp, def2tzvpp, 631g*, 6311g**, sto3g, and the auxiliary jkfit and ri sets of the same families. A dummy centre X used for a ghost atom receives the minao basis. The value is compared case-insensitively.

density_fitting Turns the resolution-of-the-identity approximation on or off through mf.density_fit(). It is switched off automatically for the tddft_gradient and tda_gradient chains, whose analytic gradients PySCF does not support under density fitting.

non_local_correction Turns the non-local correlation correction of the functional on through mf.nlc. Given false, mf.nlc is disabled and the functional runs without the VV10 term.

dispersion_correction Sets the dispersion correction model that PySCF dispatches on mf.disp. The accepted names are the versions the dftd3 and dftd4 packages provide: d3bj and d3zero for the Becke-Johnson and zero-damping D3 forms, d3bjm and d3zerom for their three-body variants, d3op for the optimised-power form, and d4 for D4. An empty value leaves the correction off; the model must not contradict a dispersion suffix carried by the functional name.

scf_converge Sets the SCF energy convergence threshold in Hartree and is handed to PySCF as conv_tol. The default is 1e-9.

maxit_scf Sets the SCF cycle cap and is handed to PySCF as max_cycle. The default is 100 iterations.

td_converge Sets the convergence threshold of the TDDFT or TDA response iterations. The default is 1e-6.

chk_file Names the SCF checkpoint. The path is expanded for a leading tilde and handed to PySCF as mf.chkfile; when the file already exists it is read as the initial guess, which is how a converged orbital set is carried from one run to the next. An empty value lets the skill build the name from project, profile, method, functional and basis inside the scratch directory.

save_mo Writes <project>.mo.molden with the orbitals of the last SCF step when true. The default false skips the file.

save_norm_mode Writes the frequency record as <project>.freq.molden. The writer does not copy PySCF's own convention: PySCF reports mass-weighted normalized modes (sqrt(amu) Bohr), while the framework's Molden reader and the initial-condition sampler follow the Molcas convention of unnormalized, un-mass-weighted Cartesian displacements, so each mode vector is divided by the square root of the atomic mass before it is written. The file also serves the front-end visualization.

opt_converge Reports whether the geometry optimization converged. The skill writes it as a result and the framework reads it as the run verdict; it is not a setting that steers the optimizer.

opt_energy Sets the energy-change threshold of the geometry optimizer in Hartree, passed as convergence_energy. The default is 1e-6.

opt_grms Sets the root-mean-square gradient threshold of the optimizer in Hartree/Bohr, passed as convergence_grms. The default is 3e-4.

opt_gmax Sets the largest-gradient threshold of the optimizer in Hartree/Bohr, passed as convergence_gmax. The default is 4.5e-4.

opt_drms Sets the root-mean-square displacement threshold of the optimizer in Bohr, passed as convergence_drms. The default is 1.2e-3.

opt_dmax Sets the largest-displacement threshold of the optimizer in Bohr, passed as convergence_dmax. The default is 1.8e-3.

opt_sp Runs a single-point energy after the optimization so the same record carries the final energy and dipole; given false the run returns the optimized structure alone.

opt_freq Runs a numerical Hessian and frequency analysis after a converged ground-state optimization. It fires only for target state 0 and only when the optimization converged.

maxit_opt Sets the step cap of the geometry optimizer used by the opt_min and opt_ts profiles. The default is 100 steps.

maxit_md Sets the step cap of a molecular-dynamics run. The skill registers no dynamics profile, so the value is recorded for the contract and no MD loop is advanced here.

grid_level Selects the DFT integration grid level of the Kohn-Sham steps: 1 to 5 are the conventional values and 3 is the shipped default.

ir_width Sets the half-width of the Gaussian broadening of the IR trace in cm-1. The default is 100.0.

ir_scale Sets the scaling factor applied to the harmonic frequencies before the IR trace is drawn. The default 1.0 leaves them unscaled.

uv_width Sets the half-width of the Gaussian broadening of the UV/vis spectrum in nm. The default is 100.0.

nics_ring Reserved NICS request: the 1-based ring atom indices whose axis would receive ghost centres. The key is declared for the NMR contract and documented here for completeness, and the shipped profiles leave it at [0]. This keyword is accepted but has no effect in v3.0.0.

nics_scan Reserved NICS request: the distances in Angstrom along the ring axis of nics_ring. The shipped profiles leave it at [0.0]. This keyword is accepted but has no effect in v3.0.0.

thermostat Selects the thermostat of a dynamics run, nose-hoover by default, with nve and berendsen as the other names. The skill registers no dynamics profile, so the value never reaches a calculation. This keyword is accepted but has no effect in v3.0.0.

barostat Selects the barostat of a constant-pressure dynamics run, berendsen by default. The skill registers no dynamics profile, so the value never reaches a calculation. This keyword is accepted but has no effect in v3.0.0.

ensemble Names the thermodynamic ensemble of a dynamics run: nve, nvt or npt. The skill registers no dynamics profile, so the value never reaches a calculation. This keyword is accepted but has no effect in v3.0.0.

charges Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

multiplicity Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

ci Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

spin Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

coupling Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

pbc Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

cell Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

lattice_vector Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

kpoint Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

target_states Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

target_nac_pair Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

target_soc_pair Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

verbose Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

temperature Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.3.

pressure Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.3.

solvent Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

shape Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

factor Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

scale Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

The skill is molecular and non-periodic. Any true pbc is refused at validation, so cell, lattice_vector and kpoint never reach a calculation, and a periodic or extended workflow belongs to cp2k, xtb or lammps. There is no non-adiabatic coupling and no spin-orbit coupling, so target_nac_pair and target_soc_pair stay empty and the nac, soc, dipole and stress jobtypes are not registered.

The method families are narrow on purpose: mp2 has no Hessian and no optimization, hf and dft have no UV/vis profile, which needs tddft or tda, and tddft and tda have no Hessian, no transition-state optimization, no IR and no NMR. The ir and nmr profiles need the optional pyscf/properties extension and stop at the first call with a named error without it. thread is a process-wide setting rather than a per-term one, so the first term to run fixes it for the whole run.

5.16. MOLCAS

The molcas section drives OpenMolcas through pymolcas. It covers CASSCF, with RAS1/RAS2/RAS3 partitioning or an explicit ALTEr active space, and CASPT2 in its ss, ms, xms, rms and xdw forms, together with analytic gradients and non-adiabatic couplings through ALASKA, spin-orbit coupling through a dual singlet/triplet RASSCF and RASSI, absorption and transition dipoles through RASSI, SLAPAF geometry optimization of a single state, of a same-spin crossing seam or of a singlet-triplet crossing point, engine-internal ab initio dynamics and surface hopping through DYNAMIX and SURFACEHOP, and MCKINLEY vibrational frequencies.

The fifteen profiles choose the module chain: hf_energy; casscf_energy, casscf_gradient, casscf_nac, casscf_properties, casscf_geometry_optimization, casscf_abinitio_dynamics, casscf_surface_hopping, casscf_soc, casscf_absorption and casscf_frequency; and caspt2_energy, caspt2_properties, caspt2_absorption and caspt2_frequency. Each profile fixes its method, and the capability table behind it decides what may be asked: hf serves energy and frequency only, casscf serves every task, and caspt2 serves energy, gradient, nac, absorption, soc and frequency but not geometry optimization, dynamics or surface hopping.

Four files matter beside the installation. molcas_path must name the OpenMolcas root that holds bin/pymolcas; the geometry is staged as <project>.xyz and referenced from the GATEWAY block rather than inlined; a <title>.StrOrb orbital guess sitting next to the framework input is picked up automatically and staged as <project>.RasOrb; and a dynamics run needs initial velocities, without which the skill raises. Background charges arrive either as per-step arrays or through read_charge, which may name a file that declares its own point-charge or field representation.

The keys at the end of the table are shared with the framework sections, and the skill receives them from &MOLECULE and &MD. The dynamics keys in the middle, size, deco, thermostat and root, are the values the adapter forwards from &MD into the generated DYNAMIX and SURFACEHOP decks.

The skill exposes the following profiles. A profile fixes the method and the set of properties a calculation returns.

Profile	Capability	Method
hf_energy	energy	hf
casscf_energy	energy	casscf
casscf_gradient	gradient	casscf
casscf_nac	nonadiabatic_coupling	casscf
casscf_properties	energy	casscf
casscf_geometry_optimization	geometry_optimization	casscf
casscf_abinitio_dynamics	abinitio_dynamics	casscf
casscf_surface_hopping	surface_hopping	casscf
casscf_soc	soc_coupling	casscf
casscf_absorption	absorption_spectrum	casscf
caspt2_energy	energy	caspt2
caspt2_properties	energy	caspt2
caspt2_absorption	absorption_spectrum	caspt2
casscf_frequency	vibrational_frequency	casscf
caspt2_frequency	vibrational_frequency	caspt2

The keywords, default values, and short descriptions are listed below. A † marks a keyword that is accepted but has no effect in v3.0.0; ‡ marks one value that is honoured; § marks an alias; * marks a derived quantity.

&MOLCAS		
execution_mode	local	Selects the run arm: local writes and runs the pymolcas script on the current machine, and slurm writes a batch script and submits it. The default is…
shell	''	Adds skill-level shell commands to the run script, given as a list, a multi-line string or one line with semicolons. Module loads and environment…
project	molcas_calc	Sets the OpenMolcas project name, exported as MOLCAS_PROJECT. The

&MOLCAS		
		deck, the geometry, the orbital, the optimized structure and the trajectory files…
profile	casscf_energy	Selects one of the fifteen profiles: hf_energy, the ten casscf_* chains and the four caspt2_* chains, for example casscf_energy or caspt2_properties…
method	casscf	Sets the multiconfigurational method: hf, casscf or caspt2. It must equal the prefix of profile and it decides which modules may appear in the chain.
molcas_path	''	Names the OpenMolcas installation root. pymolcas is looked for at <path>/bin/pymolcas and then at <path>/pymolcas, and the directory must exist at…
molcas_project	MOLCAS	Carries the project name in the spelling the framework forwards. A non-empty value overwrites project and the two are kept in step.
molcas_mem	4000	Sets the memory exported as MOLCAS_MEM in megabytes. The default is 4000 MB.
molcas_nproc	1	Carries the MPI rank count in the spelling the framework forwards. The larger of molcas_nproc and nproc is exported as MOLCAS_NPROCS.
molcas_print	3	Sets the print level exported as MOLCAS_PRINT. The default 3 writes the standard module output, lower values quieten the deck and higher values add…
nproc	1	Sets the MPI rank count of a local run. The larger of nproc and molcas_nproc wins, so neither spelling can shrink a request made in the other.
thread	1	Sets the OpenMP thread count exported as OMP_NUM_THREADS, one thread count per rank. The larger of thread and molcas_thread wins, and the retired…

&MOLCAS		
molcas_thread	1	Carries the thread count in the spelling the framework forwards; the larger of molcas_thread and thread is applied.
slurm_time	01:00:00	Sets the wall-clock limit of the batch job in HH:MM:SS. The default is 01:00:00.
slurm_partition	default	Names the SLURM partition of the batch job. The default is default.
slurm_ntasks	1	Sets the number of SLURM tasks, which becomes the rank count of the submitted job. The default is 1.
slurm_memory	4000	Sets the memory request of the batch job in megabytes. The default is 4000.
slurm_command	''	Adds extra commands to the submission line of the batch job.
keep_tmp	0	0 removes the calculation directory once the results are collected; 1 keeps the deck, the logs and the orbital files.
basis	ano-rcc-vdzp	Sets the orbital basis from the 24 names in the skill list: the Pople, ANO-S, ANO-L, ANO-RCC, correlation-consistent and def2 families. A per-element…
nact_elec	2	Sets the number of active electrons in the RASSCF, written as NactEl. The default is 2.
nact_orb	2	Sets the number of active orbitals in the RASSCF, written as Ras2 unless the RAS partition overrides it. The default is 2.
inactive_orb	0	Sets the number of inactive orbitals. Zero derives it from (N_elec - nact_elec)/2.
ras1_orb	0	Sets the size of the RAS1 subspace, the holes allowed above it. Zero leaves a plain CAS without RAS partitioning.
ras2_orb	0	Sets the size of the RAS2 subspace. Zero falls back to nact_orb, which is the plain CAS case.
ras3_orb	0	Sets the size of the RAS3 subspace, the particles allowed into it. Zero leaves a plain CAS without RAS partitioning.
active	[]	Lists the active orbitals explicitly, 1-based as the input spells them, and the

&MOLCAS		
		skill writes them as ALTEr pairs. An empty list takes the orbitals…
ci_roots	[1]	Sets the number of CI roots in each spin block, written as ROOT in the RASSCF block. One block is written per ci_spin entry, and the default [1]…
ci_spin	[0]	Sets 2S of each spin block, written as Spin in the RASSCF block: 0 for a singlet, 2 for a triplet. The default [0] runs one singlet block.
state_spin	[]	Labels the spin of each requested state as 2S, 0 for a singlet and 2 for a triplet. It decides which state sits in which block and which pairs count…
nstate	1	Sets the number of electronic states requested. It bounds the state ladder the framework builds and the roots the CI blocks must cover.
xms	true	Legacy multistate switch for CASPT2: true, the default, resolves an empty caspt2_type to xms and false resolves it to ss.
caspt2_type	''	Selects the CASPT2 variant: ss for single-state, ms for MULTISTATE all, xms for XMULTISTATE all, rms for RMULTISTATE all, and xdw for XMULTISTATE…
dwms_zeta	1	Sets the XDW-CASPT2 zeta parameter, written as DWMS for the xdw variant. It is a positive real number, 1.0 by default.
dwtype	3	Selects the exponent type of the XDW-CASPT2 reference weights, one of 1, 2 or 3, written as DWTYPE. The default is 3.
nocsf	true	Drops the CSF term from the non-adiabatic coupling when true, which is the default for both the CASSCF and the CASPT2 NAC. Given false the CSF…
caspt2_ipea_shift	None	Sets the CASPT2 IPEA shift in Hartree. None leaves the OpenMolcas default of 0.25; 0.0 emits IPEA 0.0, the portable setting for analytic gradient and…

&MOLCAS		
caspt2_real_shift	0	Sets the CASPT2 real level shift in Hartree, written as REAL. The default 0.0 applies no shift.
caspt2_imaginary_shift	0.2	Sets the CASPT2 imaginary level shift in Hartree, written as IMAG. The default is 0.2.
singlet_ci_roots	[3]	Sets the number of roots of the singlet RASSCF block in a SOC run. The default [3] asks for three singlets.
singlet_ci_spin	[0]	Sets 2S of the singlet block in a SOC run. The default [0] labels it a singlet.
triplet_ci_roots	[2]	Sets the number of roots of the triplet RASSCF block in a SOC run. The default [2] asks for two triplets.
triplet_ci_spin	[2]	Sets 2S of the triplet block in a SOC run. The default [2] labels it a triplet.
embedding	none	Selects the ESPF embedding: none, charge for point charges or field for an electrostatic field with its gradients. The retired value potential is…
read_charge	none	Selects the source of the embedding data: none reads the per-step point_charges and external_potential arrays, true reads <project>.charge, and any…
scf_converge	1e-08	Sets the SCF density and energy threshold. The default is 1e-8.
maxit_scf	200	Sets the first number of the RASSCF ITERATIONS card, the CI iteration cap; the second number, the super-CI orbital cap, stays at 100. The default is…
opt_maxit	100	Sets the SLAPAF relaxation cap and is exported as MOLCAS_MAXITER for the optimization loop. The default is 100 iterations.
maxit_md	1	Sets the number of MD steps and bounds the FOREACH loop that wraps SEWARD through DYNAMIX. The default is 1 step.
md_maxiter	0	Sets the MOLCAS_MAXITER export limit of a dynamics run, where 0 derives

&MOLCAS		
		ten times maxit_md. The value must exceed maxit_md.
size	20.67	Sets the DYNAMIX time step in atomic units, written as dt. The default 20.67 au is about 0.5 fs.
deco	0.1	Sets the decoherence parameter of the surface-hopping run, written as decoherence in the SURFACEHOP block. It is a string, 0.1 by default.
thermostat	off	Selects the DYNAMIX thermostat: off or nve runs NVE, and nose, nose-hoover or nvt run a Nose-Hoover NVT. Any other value raises.
root	1	Sets the initial state of the dynamics, spelled 1-based in the input; the MDRLXR card of the RASSCF block is this value.
opt_converge	false	Reports whether the optimization converged. The skill writes it as a result and the framework reads it as the run verdict.
opt_state_i	0	Sets the first optimization target state, 0-based. Equal to opt_state_j it relaxes that state alone; with a distinct same-spin partner it requests a…
opt_state_j	None	Sets the second optimization target state, 0-based, where None means a single-state relaxation. A same-spin pair gives a MECI and a singlet-triplet…
ir_width	100	Sets the half-width in cm-1 recorded with an IR spectrum. The parser reports band positions and intensities per mode, so the value carries the width…
ir_scale	1	Sets the frequency scaling factor recorded with an IR spectrum. The default 1.0 leaves the harmonic wavenumbers unscaled.
uv_width	100	Sets the half-width in nm recorded with an absorption spectrum. RASSI reports oscillator strengths per root, and the width carries the half-width of…

&MOLCAS		
nics_ring †	[0]	Reserved NICS request: the ring atom indices that would define the scan axis. No molcas profile consumes it.
nics_scan †	[0]	Reserved NICS request: the distances in Angstrom along the ring axis. No molcas profile consumes it.
charges	0	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
multiplicity	1	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
ci	[1]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
spin	[0]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
coupling	[]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
pbc	[0, 0, 0]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
cell	[99, 99, 99, 90, 90, 90]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
lattice_vector	[[0, 0, 0], [0, 0, 0], [0, 0, 0]]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
kpoint	[1, 1, 1]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
target_states	[0]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
verbose	1	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
temperature	298.15	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.3.

&MOLCAS		
solvent	gas	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
shape	ellipsoid	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
factor	40	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
scale	1	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

Full descriptions for all available keywords are summarized below.

execution_mode Selects the run arm: local writes and runs the pymolcas script on the current machine, and slurm writes a batch script and submits it. The default is local.

shell Adds skill-level shell commands to the run script, given as a list, a multi-line string or one line with semicolons. Module loads and environment activations belong here; the framework global shell is reserved for the main run.

project Sets the OpenMolcas project name, exported as MOLCAS_PROJECT. The deck, the geometry, the orbital, the optimized structure and the trajectory files are named after it.

profile Selects the module chain. casscf_energy runs GATEWAY, SEWARD and RASSCF; casscf_gradient and casscf_nac add ALASKA for the analytic gradient and for the coupling of a same-spin pair; casscf_properties runs GATEWAY, SEWARD, RASSCF, ALASKA and RASSI for a free combination of energy, gradient, nac and soc; casscf_geometry_optimization adds SLAPAF and derives single-state, MECI or MECP from opt_state_i, opt_state_j and state_spin; casscf_abinitio_dynamics and casscf_surface_hopping propagate the nuclei inside one Molcas run through DYNAMIX, the second with SURFACEHOP; casscf_soc runs the dual singlet/triplet RASSCF and RASSI; casscf_absorption and caspt2_absorption read RASSI oscillator strengths; and the two _frequency profiles add MCKINLEY. A generic _properties chain accepts only the tokens energy, gradient, nac and soc plus the dipole and transition modifiers, and refuses anything else.

method Sets the multiconfigurational level. hf runs a plain SCF and serves energy and frequency only, with no gradient, optimization, NAC, SOC or dynamics. casscf runs the complete set of profiles, from energy to surface hopping. caspt2 adds the perturbative correction on the CASSCF reference and serves energy, gradient, nac, absorption, soc and frequency, but no geometry optimization, no ab initio dynamics and no surface hopping, because the perturbative gradient path is single-shot. A profile whose prefix does not match method is refused before any deck is written.

molcas_path Names the OpenMolcas installation root. pymolcas is looked for at <path>/bin/pymolcas and then at <path>/pymolcas, and the directory must exist at validation.

molcas_project Carries the project name in the spelling the framework forwards. A non-empty value overwrites project and the two are kept in step.

molcas_mem Sets the memory exported as MOLCAS_MEM in megabytes. The default is 4000 MB.

molcas_nproc Carries the MPI rank count in the spelling the framework forwards. The larger of molcas_nproc and nproc is exported as MOLCAS_NPROCS.

molcas_print Sets the print level exported as MOLCAS_PRINT. The default 3 writes the standard module output, lower values quieten the deck and higher values add the orbital and integral detail OpenMolcas offers.

nproc Sets the MPI rank count of a local run. The larger of nproc and molcas_nproc wins, so neither spelling can shrink a request made in the other.

thread Sets the OpenMP thread count exported as OMP_NUM_THREADS, one thread count per rank. The larger of thread and molcas_thread wins, and the retired spelling num_thread is refused.

molcas_thread Carries the thread count in the spelling the framework forwards; the larger of molcas_thread and thread is applied.

slurm_time Sets the wall-clock limit of the batch job in HH:MM:SS. The default is 01:00:00.

slurm_partition Names the SLURM partition of the batch job. The default is default.

slurm_ntasks Sets the number of SLURM tasks, which becomes the rank count of the submitted job. The default is 1.

slurm_memory Sets the memory request of the batch job in megabytes. The default is 4000.

slurm_command Adds extra commands to the submission line of the batch job.

keep_tmp 0 removes the calculation directory once the results are collected; 1 keeps the deck, the logs and the orbital files.

basis Sets the orbital basis of the SEWARD step. The 24 accepted names cover the Pople sets 3-21g, 4-31g, 5-21g, 6-31g and 6-31g**, the ANO-S, ANO-L and ANO-RCC sequences, the Dunning correlation-consistent sets up to aug-cc-pvqz, and def2-svp, def2-tzvp and def2-tzvpp. A single name applies to every element; a per-element combination joined by commas, such as H.ano-rcc-vdzp, C.ano-rcc-vdzp, validates each component separately and is written element by element.

nact_elec Sets the number of active electrons in the RASSCF, written as NactEl. The default is 2.

nact_orb Sets the number of active orbitals in the RASSCF, written as Ras2 unless the RAS partition overrides it. The default is 2.

inactive_orb Sets the number of inactive orbitals. Zero derives it from (N_elec - nact_elec)/2.

ras1_orb Sets the size of the RAS1 subspace, the holes allowed above it. Zero leaves a plain CAS without RAS partitioning.

ras2_orb Sets the size of the RAS2 subspace. Zero falls back to nact_orb, which is the plain CAS case.

ras3_orb Sets the size of the RAS3 subspace, the particles allowed into it. Zero leaves a plain CAS without RAS partitioning.

active Lists the active orbitals explicitly, 1-based as the input spells them, and the skill writes them as ALTEr pairs. An empty list takes the orbitals from inactive_orb, nact_orb and the RAS partition.

ci_roots Sets the number of CI roots in each spin block, written as ROOT in the RASSCF block. One block is written per ci_spin entry, and the default [1] solves the lowest root.

ci_spin Sets 2S of each spin block, written as Spin in the RASSCF block: 0 for a singlet, 2 for a triplet. The default [0] runs one singlet block.

state_spin Labels the spin of each requested state as 2S, 0 for a singlet and 2 for a triplet. It decides which state sits in which block and which pairs count as same-spin for a NAC.

nstate Sets the number of electronic states requested. It bounds the state ladder the framework builds and the roots the CI blocks must cover.

xms Keeps the legacy multistate switch of the CASPT2 branch: it is the only source of the variant when caspt2_type is empty, resolving to xms when true and to ss when false. An explicit caspt2_type always wins, so the switch matters only for inputs written before the five names were accepted.

caspt2_type Selects how the CASPT2 step treats the reference states. ss writes a single-state calculation and is therefore one state per call, which makes it unable to serve a NAC and, for SOC, limited to one state per spin manifold; ms writes MULTISTATE all, xms writes XMULTISTATE all and is the resolution of the default switch, rms writes RMULTISTATE all, and xdw writes XMULTISTATE all with the DWMS zeta and DWTYPE exponent of the XDW variant. An empty value resolves from xms.

dwms_zeta Sets the zeta parameter of the XDW-CASPT2 weighting, written as DWMS only when caspt2_type is xdw. The value is a positive real number and defaults to 1.0, the vendor recommendation for the smooth interpolation between reference states.

dwtype Selects the exponent form of the XDW-CASPT2 weights, written as DWTYPE next to DWMS: 1, 2 or 3 are the three forms OpenMolcas accepts and 3 is the shipped default. Like dwms_zeta it only reaches a deck whose caspt2_type is xdw.

nocsf Controls the CSF contribution of the non-adiabatic coupling. With true, the default, one NOCSF card is added to the ALASKA block for both the CASSCF and the CASPT2 NAC, so the coupling carries the orbital-relaxation part only; with false the CSF term is kept.

caspt2_ipea_shift Sets the CASPT2 IPEA shift in Hartree. None leaves the OpenMolcas default of 0.25; 0.0 emits IPEA 0.0, the portable setting for analytic gradient and NAC runs because OpenMolcas v25.06 segfaults on the IPEA 0.25 plus DORT path.

caspt2_real_shift Sets the CASPT2 real level shift in Hartree, written as REAL. The default 0.0 applies no shift.

caspt2_imaginary_shift Sets the CASPT2 imaginary level shift in Hartree, written as IMAG. The default is 0.2.

singlet_ci_roots Sets the number of roots of the singlet RASSCF block in a SOC run. The default [3] asks for three singlets.

singlet_ci_spin Sets 2S of the singlet block in a SOC run. The default [0] labels it a singlet.

triplet_ci_roots Sets the number of roots of the triplet RASSCF block in a SOC run. The default [2] asks for two triplets.

triplet_ci_spin Sets 2S of the triplet block in a SOC run. The default [2] labels it a triplet.

embedding Selects the ESPF representation of the environment. charge writes an External block of point charges in Angstrom, field writes one row per atom with the potential and its three gradients in atomic units, and none writes no ESPF block unless read_charge names a file. ESPF is refused for the iterative profiles, the two MD profiles and the two frequency profiles, and for any method other than casscf or caspt2.

read_charge Selects where the embedding data comes from. none takes the per-step point_charges and external_potential arrays, true reads <project>.charge from the calculation or launch

directory, and any other value names a file that is fixed for the run. The first line of such a file declares point angstrom or point bohr with external or atom rows, or field au; older charge and potential spellings still read.

scf_converge Sets the SCF density and energy threshold. The default is 1e-8.

maxit_scf Sets the first number of the RASSCF ITERATIONS card, the CI iteration cap; the second number, the super-CI orbital cap, stays at 100. The default is 200.

opt_maxit Sets the SLAPAF relaxation cap and is exported as MOLCAS_MAXITER for the optimization loop. The default is 100 iterations.

maxit_md Sets the number of MD steps and bounds the FOREACH loop that wraps SEWARD through DYNAMIX. The default is 1 step.

md_maxiter Sets the MOLCAS_MAXITER export limit of a dynamics run, where 0 derives ten times maxit_md. The value must exceed maxit_md.

size Sets the DYNAMIX time step in atomic units, written as dt. The default 20.67 au is about 0.5 fs.

deco Sets the decoherence parameter of the surface-hopping run, written as decoherence in the SURFACEHOP block. It is a string, 0.1 by default.

thermostat Selects the DYNAMIX thermostat: off or nve runs NVE, and nose, nose-hoover or nvt run a Nose-Hoover NVT. Any other value raises.

root Sets the initial state of the dynamics, spelled 1-based in the input; the MDRLXR card of the RASSCF block is this value.

opt_converge Reports whether the optimization converged. The skill writes it as a result and the framework reads it as the run verdict.

opt_state_i Sets the first optimization target state, 0-based. Equal to opt_state_j it relaxes that state alone; with a distinct same-spin partner it requests a minimum-energy conical intersection.

opt_state_j Sets the second optimization target state, 0-based, where None means a single-state relaxation. A same-spin pair gives a MECI and a singlet-triplet pair a MECP, with opt_state_i the singlet.

ir_width Sets the half-width in cm-1 recorded with an IR spectrum. The parser reports band positions and intensities per mode, so the value carries the width of a broadened trace rather than a broadening applied inside the deck.

ir_scale Sets the frequency scaling factor recorded with an IR spectrum. The default 1.0 leaves the harmonic wavenumbers unscaled.

uv_width Sets the half-width in nm recorded with an absorption spectrum. RASSI reports oscillator strengths per root, and the width carries the half-width of the broadened trace.

nics_ring Reserved NICS request: the ring atom indices that would define the scan axis. No molcas profile consumes it. This keyword is accepted but has no effect in v3.0.0.

nics_scan Reserved NICS request: the distances in Angstrom along the ring axis. No molcas profile consumes it. This keyword is accepted but has no effect in v3.0.0.

charges Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

multiplicity Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

ci Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

spin Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

coupling Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

pbc Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

cell Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

lattice_vector Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

kpoint Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

target_states Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

verbose Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

temperature Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.3.

solvent Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

shape Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

factor Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

scale Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

OpenMolcas is a molecular code here: any true pbc is refused at validation, so cell, lattice_vector and kpoint never reach a deck, and a periodic or extended workflow belongs to cp2k, xtb or lammps. The method matrix is narrow in the same way: hf serves energy and frequency only, and CASPT2 serves no geometry optimization, no ab initio dynamics and no surface hopping, so an optimized or propagated CASPT2 trajectory is a sequence of single-point calls rather than an internal loop.

Several couplings are restricted. ss cannot serve a NAC at all, a NAC needs a same-spin pair, mixed spin manifolds need an explicit SOC request, and MECP needs opt_state_i on the singlet and opt_state_j on the triplet. ESPF is refused for the iterative profiles and omits the ESPF grid-point derivatives from its gradients, and under ESPF a CASPT2 gradient plus NAC becomes two passes with the NAC labelled a no-ESPF approximation. The returned energy is the pre-SOC CASPT2 energy, never a SO-RASSI eigenvalue, and a single-root spin block keeps its CASSCF diagonal in RASSI with a warning.

5.17. BAGEL

The bagel section drives BAGEL for multiconfigurational chemistry. It covers CASSCF and XMS-CASPT2 energies, analytic gradients, non-adiabatic coupling elements between states of one

spin manifold, Hessians and frequencies from finite differences of gradients, internal geometry optimization of a single state or of a minimum-energy conical intersection, and state and transition dipoles. Unlike the text-deck engines, BAGEL reads a JSON block list: the skill builds the molecule, casscf or smith blocks and the forces, hessian or optimize block from the section keys.

The fourteen profiles are casscf_energy, casscf_gradient, casscf_nac, casscf_frequency, casscf_geometry_optimization, casscf_dipole and casscf_transition_dipole, and the same seven with the xmscaspt2 prefix. Each profile fixes method to the prefix, and the nac chains carry the same-spin NACME of CASSCF or XMS-CASPT2 states; there is no spin-orbit chain at all.

BAGEL keeps its orbitals in .archive files rather than in a Molcas RasOrb or StrOrb. A casscf run writes <project>_converged.archive through its save_ref step, later steps copy that file in as <project>_guess.archive, and the load_ref block is written only when the archive is present, since a reference to a missing file aborts BAGEL. The installation supplies the basis data: <basis>.json and <df_basis>.json are read from <bagel_path>/share, and the run script exports BAGEL, BLAS, LAPACK, BOOST, MPI, OMP_NUM_THREADS, MKL_NUM_THREADS and BAGEL_NUM_THREADS before it launches the binary.

The keys at the end of the table are shared with the framework sections, and the skill receives them from &MOLECULE and &MD.

The skill exposes the following profiles. A profile fixes the method and the set of properties a calculation returns.

Profile	Capability	Method
casscf_energy	energy	casscf
casscf_gradient	gradient	casscf
casscf_nac	nonadiabatic_coupling	casscf
casscf_frequency	vibrational_frequency	casscf
casscf_geometry_optimization	geometry_optimization	casscf
casscf_dipole	dipole	casscf
casscf_transition_dipole	transition_dipole	casscf
xmscaspt2_energy	energy	xmscaspt2
xmscaspt2_gradient	gradient	xmscaspt2
xmscaspt2_nac	nonadiabatic_coupling	xmscaspt2
xmscaspt2_frequency	vibrational_frequency	xmscaspt2
xmscaspt2_geometry_optimization	geometry_optimization	xmscaspt2
xmscaspt2_dipole	dipole	xmscaspt2
xmscaspt2_transition_dipole	transition_dipole	xmscaspt2

The keywords, default values, and short descriptions are listed below. A † marks a keyword that is accepted but has no effect in v3.0.0; ‡ marks one value that is honoured; § marks an alias; * marks a derived quantity.

&BAGEL		
execution_mode	local	Selects the run arm: local runs BAGEL in a subprocess on the current machine,

&BAGEL		
		and slurm writes a batch script and submits it. The default is local.
shell	''	Adds skill-level shell commands to the run script, given as a list, a multi-line string or one line with semicolons. Module loads belong here; the…
project	bagel_calc	Sets the artifact stem. The JSON deck, the log and the archive files are named after it, for example <project>.json, <project>.log and…
profile	casscf_energy	Selects one of the fourteen profiles, named <method>_<task>, for example casscf_energy or xmscaspt2_gradient. The profile fixes the step chain and…
method	casscf	Sets the method: casscf or xmscaspt2. It must equal the prefix of profile and it decides whether the chain carries a smith CASPT2 block after the…
bagel_path	''	Names the BAGEL installation root. The executable is <path>/bin/BAGEL, the basis and density-fitting JSON files are read from <path>/share, and a run…
bagel_project	BAGEL	Carries the project name in the spelling the framework forwards. A non-empty value overwrites project and the two are kept in step.
blas	''	Sets the BLAS installation path, exported as BLAS and prepended to LD_LIBRARY_PATH before BAGEL starts. An empty value leaves the library search path…
lapack	''	Sets the LAPACK installation path, exported as LAPACK and prepended to LD_LIBRARY_PATH. An empty value changes nothing.
boost	''	Sets the Boost installation path, exported as BOOST, with <path>/lib joining LD_LIBRARY_PATH. An empty value changes nothing.
mpi	''	Sets the MPI installation root, exported as MPI, with <path>/lib joining LD_LIBRARY_PATH and <path>/bin

&BAGEL		
		joining PATH so that the launcher named by…
mkl	''	Names the MKL setup script sourced before the run, together with arch. An empty value sources nothing.
arch	''	Sets the architecture argument of the MKL setup script named by mkl, for example intel64. It is ignored when mkl is empty.
mpiexe	mpirun	Names the MPI launcher: mpirun by default, an absolute path or another spelling such as mpiexec.hydra when a site needs one. no or none runs…
bagel_nproc	1	Sets the MPI rank count of a local run. Slurm runs use the granted SLURM_NTASKS instead, and the larger of bagel_nproc and nproc wins in the local…
nproc	1	Sets the rank count of a local run in the spelling the skill reads. The larger of nproc and bagel_nproc is applied, and it is independent of thread…
thread	1	Sets the OpenMP thread count per rank, exported as OMP_NUM_THREADS, MKL_NUM_THREADS and BAGEL_NUM_THREADS in both arms. The larger of thread and…
bagel_thread	1	Carries the thread count in the spelling the framework forwards; the larger of bagel_thread and thread is applied.
slurm_time	01:00:00	Sets the wall-clock limit of the batch job in HH:MM:SS. The default is 01:00:00.
slurm_partition	default	Names the SLURM partition of the batch job. The default is default.
slurm_ntasks	1	Sets the number of SLURM tasks, which the batch script takes as its rank count. The default is 1.
slurm_memory	4000	Sets the memory request of the batch job in megabytes. The default is 4000.
slurm_command	''	Adds extra commands to the submission line of the batch job.

&BAGEL		
keep_tmp	0	0 removes the calculation directory once the results are collected; 1 keeps the deck, the log and the archive files.
basis	cc-pvdz	Sets the orbital basis from the 38 names in the skill list, for example cc-pvdz, cc-pvtz, aug-cc-pvdz, svp, tzvpp, ano-rcc or sto-3g. The basis is…
df_basis	''	Sets the density-fitting basis used by the CASSCF and CASPT2 blocks, one of the seven jkfit sets: svp-jkfit, tzvpp-jkfit, qzvpp-jkfit, cc-pvdz-jkfit…
nact_elec	2	Sets the number of active electrons in the casscf block, written as nactel. The default is 2.
nact_orb	2	Sets the number of active orbitals, written as nacta and derived with nclosed when active is empty. The default is 2.
nclosed	0	Sets the number of closed-shell orbitals of the casscf block. Together with nact_orb it defines the default active space, the orbitals nclosed + 1…
active	[]	Lists the active orbitals explicitly, 1-based as inputs spell them. An empty list takes them from nclosed and nact_orb.
ci_roots	[1]	Sets the number of CI roots solved in the casscf block. The default [1] solves the lowest root.
ci_spin	[0]	Sets 2S of the CI roots, 0 for a singlet and 2 for a triplet. The default [0] asks for singlet roots.
nstate	1	Sets the number of electronic states requested. The deck carries no spin keyword, so every root comes from one manifold whatever the state labels say.
scf_converge	1e-08	Sets the CASSCF convergence threshold and the thresh of the smith CASPT2 block. The default is 1e-8.
pt2_shift	0.2	Sets the XMS-CASPT2 level shift, written as shift in the smith block. The default is 0.2.

&BAGEL		
maxit_scf	500	Sets maxiter of every CASSCF and CASPT2 block, including the smith sub-block. The default is 500 cycles.
maxit_opt	100	Sets maxiter of the optimize block used by the two geometry_optimization profiles. The default is 100 steps.
opt_state_i	0	Sets the first optimization target state, 0-based. Equal to opt_state_j it writes opttype energy and relaxes that state alone.
opt_state_j	None	Sets the second optimization target state, 0-based, where None means a single-state relaxation and a distinct value requests the minimum-energy…
ir_width †	100	Sets the half-width in cm-1 recorded with an IR spectrum. The frequency profiles report wavenumbers and intensities, so the value is metadata for a…
ir_scale †	1	Sets the frequency scaling factor recorded with an IR spectrum. The default 1.0 leaves the harmonic wavenumbers unscaled, and no BAGEL block consumes…
uv_width †	100	Sets the half-width in nm recorded with an absorption spectrum. BAGEL reports energies and transition dipoles per state, and no block consumes the…
nics_ring †	[0]	Reserved NICS request: the ring atom indices that would define the scan axis. No bagel profile consumes it.
nics_scan †	[0]	Reserved NICS request: the distances in Angstrom along the ring axis. No bagel profile consumes it.
charges	0	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
multiplicity	1	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
ci	[1]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

&BAGEL		
spin	[0]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
coupling	[]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
pbc	[0, 0, 0]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
cell	[99, 99, 99, 90, 90, 90]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
lattice_vector	[[0, 0, 0], [0, 0, 0], [0, 0, 0]]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
kpoint	[1, 1, 1]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
target_states	[0]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
target_nac_pair	[[]]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
verbose	4	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
temperature	298.15	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.3.
solvent	gas	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
shape	ellipsoid	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
factor	40	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
scale	1	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

Full descriptions for all available keywords are summarized below.

execution_mode Selects the run arm: local runs BAGEL in a subprocess on the current machine, and slurm writes a batch script and submits it. The default is local.

shell Adds skill-level shell commands to the run script, given as a list, a multi-line string or one line with semicolons. Module loads belong here; the framework global shell is reserved for the main run.

project Sets the artifact stem. The JSON deck, the log and the archive files are named after it, for example <project>.json, <project>.log and <project>_converged.archive.

profile Selects the step chain, which is the sequence of blocks written into the JSON deck. casscf_energy runs molecule, load_ref, casscf and save_ref; casscf_gradient and casscf_nac run molecule, load_ref, casscf and then one forces block, with the nacme set for the coupling chain; casscf_frequency ends in a hessian block and casscf_geometry_optimization in an optimize block; and the two dipole chains end in one forces block carrying the dipole or the transition-dipole set. The xmscaspt2 chains repeat that shape around the smith CASPT2 level: xmscaspt2_energy runs molecule, load_ref, casscf and smith_caspt2; xmscaspt2_gradient, xmscaspt2_nac and the two dipole chains go from load_ref straight to a caspt2 forces block; and xmscaspt2_frequency runs molecule, load_ref, casscf, smith_caspt2 and hessian_caspt2, because a bare hessian_caspt2 job prints no energy in its own log. The framework jobtypes map onto these chains: md onto the gradient chain and dipole, transition and absorption onto the dipole chains.

method Sets the level of the chain. casscf builds the casscf block and either saves the orbitals through save_ref or consumes them; xmscaspt2 runs the same casscf block and then a smith block whose flags ms, xms and sssr plus shift carry the multistate treatment. BAGEL has no standalone caspt2-titled block, which is why an XMS-CASPT2 energy is always a casscf step followed by a smith step. A profile whose prefix does not match method is refused before the JSON deck is written.

bagel_path Names the BAGEL installation root. The executable is <path>/bin/BAGEL, the basis and density-fitting JSON files are read from <path>/share, and a run is refused when the directory or the binary is missing.

bagel_project Carries the project name in the spelling the framework forwards. A non-empty value overwrites project and the two are kept in step.

blas Sets the BLAS installation path, exported as BLAS and prepended to LD_LIBRARY_PATH before BAGEL starts. An empty value leaves the library search path as the machine set it.

lapack Sets the LAPACK installation path, exported as LAPACK and prepended to LD_LIBRARY_PATH. An empty value changes nothing.

boost Sets the Boost installation path, exported as BOOST, with <path>/lib joining LD_LIBRARY_PATH. An empty value changes nothing.

mpi Sets the MPI root that the run script exports as MPI, adds to LD_LIBRARY_PATH as <path>/lib and to PATH as <path>/bin. BAGEL itself is linked against this MPI, and the launcher named by mpiexe is found through the same PATH entry, so the two must name the same MPI installation. An empty value leaves both the library and the program search paths untouched, which is correct only when the machine already provides them.

mkl Names the MKL setup script that the run script sources before BAGEL starts, together with arch. Sourcing is what puts the MKL runtime on the path on a cluster whose login environment does not carry

it; an empty value writes no source line, and the value is ignored by a BAGEL built against another BLAS.

arch Sets the architecture argument passed to the MKL setup script, for example intel64 or intel64_and_ia32. It is only read together with mkl: when mkl is empty no setup script runs and arch has no effect, and BAGEL reports the MKL layer it actually loaded in its own log.

mpiexe Names the MPI launcher: mpirun by default, an absolute path or another spelling such as mpiexec.hydra when a site needs one. no or none runs <bagel_path>/bin/BAGEL directly as a single process that uses threads only.

bagel_nproc Sets the number of MPI ranks of a local run, handed to the launcher as mpiexe -np bagel_nproc, with the larger of bagel_nproc and nproc applied. The local arm deliberately reads this value rather than $SLURM_NTASKS, because a local job sits outside an allocation and an inherited SLURM_NTASKS would describe a different machine. The skill's own batch script takes the granted $SLURM_NTASKS instead, from the slurm_ntasks header. Ranks and threads are separate keywords: neither stands in for the other.

nproc Sets the rank count of a local run in the spelling the skill reads. The larger of nproc and bagel_nproc is applied, and it is independent of thread, which counts threads rather than ranks.

thread Sets the OpenMP thread count per rank, exported as OMP_NUM_THREADS, MKL_NUM_THREADS and BAGEL_NUM_THREADS in both arms. The larger of thread and bagel_thread wins, and the retired spelling num_thread is refused.

bagel_thread Carries the thread count in the spelling the framework forwards; the larger of bagel_thread and thread is applied.

slurm_time Sets the wall-clock limit of the batch job in HH:MM:SS. The default is 01:00:00.

slurm_partition Names the SLURM partition of the batch job. The default is default.

slurm_ntasks Sets the number of SLURM tasks, which the batch script takes as its rank count. The default is 1.

slurm_memory Sets the memory request of the batch job in megabytes. The default is 4000.

slurm_command Adds extra commands to the submission line of the batch job.

keep_tmp 0 removes the calculation directory once the results are collected; 1 keeps the deck, the log and the archive files.

basis Sets the orbital basis from the 38 names in the skill list, for example cc-pvdz, cc-pvtz, aug-cc-pvdz, svp, tzvpp, ano-rcc or sto-3g. The basis is read from <basis>.json in the BAGEL share directory, so only names shipped with the installation resolve.

df_basis Sets the density-fitting basis of the correlated blocks. The seven accepted sets are the jkfit auxiliary bases shipped with BAGEL: svp-jkfit, tzvpp-jkfit, qzvpp-jkfit, cc-pvdz-jkfit, cc-pvtz-jkfit, cc-pvqz-jkfit and cc-pv5z-jkfit, each read from <bagel_path>/share. An empty value lets the builder match the orbital basis by similarity and fall back to svp-jkfit when no match exists, and the retired spelling no or none is no longer accepted.

nact_elec Sets the number of active electrons in the casscf block, written as nactel. The default is 2.

nact_orb Sets the number of active orbitals, written as nacta and derived with nclosed when active is empty. The default is 2.

nclosed Sets the number of closed-shell orbitals of the casscf block. Together with nact_orb it defines the default active space, the orbitals nclosed + 1 through nclosed + nact_orb in the 1-based input numbering.

active Lists the active orbitals explicitly, 1-based as inputs spell them. An empty list takes them from nclosed and nact_orb.

ci_roots Sets the number of CI roots solved in the casscf block. The default [1] solves the lowest root.

ci_spin Sets 2S of the CI roots, 0 for a singlet and 2 for a triplet. The default [0] asks for singlet roots.

nstate Sets the number of electronic states requested. The deck carries no spin keyword, so every root comes from one manifold whatever the state labels say.

scf_converge Sets the CASSCF convergence threshold and the thresh of the smith CASPT2 block. The default is 1e-8.

pt2_shift Sets the XMS-CASPT2 level shift, written as shift in the smith block. The default is 0.2.

maxit_scf Sets maxiter of every CASSCF and CASPT2 block, including the smith sub-block. The default is 500 cycles.

maxit_opt Sets maxiter of the optimize block used by the two geometry_optimization profiles. The default is 100 steps.

opt_state_i Sets the first optimization target state, 0-based. Equal to opt_state_j it writes opttype energy and relaxes that state alone.

opt_state_j Sets the second optimization target state, 0-based, where None means a single-state relaxation and a distinct value requests the minimum-energy conical intersection of the pair (opttype meci).

ir_width Sets the half-width in cm-1 recorded with an IR spectrum. The frequency profiles report wavenumbers and intensities, so the value is metadata for a broadened trace and no BAGEL block consumes it. This keyword is accepted but has no effect in v3.0.0.

ir_scale Sets the frequency scaling factor recorded with an IR spectrum. The default 1.0 leaves the harmonic wavenumbers unscaled, and no BAGEL block consumes the value. This keyword is accepted but has no effect in v3.0.0.

uv_width Sets the half-width in nm recorded with an absorption spectrum. BAGEL reports energies and transition dipoles per state, and no block consumes the width. This keyword is accepted but has no effect in v3.0.0.

nics_ring Reserved NICS request: the ring atom indices that would define the scan axis. No bagel profile consumes it. This keyword is accepted but has no effect in v3.0.0.

nics_scan Reserved NICS request: the distances in Angstrom along the ring axis. No bagel profile consumes it. This keyword is accepted but has no effect in v3.0.0.

charges Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

multiplicity Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

ci Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

spin Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

coupling Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

pbc Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

cell Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

lattice_vector Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

kpoint Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

target_states Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

target_nac_pair Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

verbose Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

temperature Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.3.

solvent Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

shape Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

factor Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

scale Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

BAGEL supports CASSCF and XMS-CASPT2 and nothing else here: there is no coupled-cluster, no density functional and no implicit solvent, and the section carries no functional or solvent key. Spin-orbit coupling is absent at every level, so there is no SOC field to fill, and the non-adiabatic coupling is computed between states of one spin manifold only. The generated deck carries no spin keyword, so a run with more than one spin in state_spin is refused rather than misinterpreted, and every root of one job belongs to one manifold.

The backend is molecular: any true pbc is refused at validation, so a periodic or extended workflow belongs to cp2k, xtb or lammps. Optimization steps are capped at a maximum step of 0.05 Bohr because an initial analytic CASPT2 gradient can be very large, and a frequency run is a finite-difference Hessian of gradients rather than an analytic one. Orbitals continue through .archive files only, so a first CASPT2 step without an archive simply omits the load_ref block and starts from a fresh CASSCF.

5.18. ORCA

The orca section configures the ORCA layer of a run. It selects the profile, the functional and basis set, the SCF and TDDFT controls, and the runtime that launches ORCA, and the framework reads it only when orca appears in the theory queue of &CONTROL. The classic spelling of the section is &ORCA; in YAML and agent JSON it is the orca mapping.

The profile is the unit of work. Each profile compiles to a chain of ORCA steps, from a ground-state single point (dft_energy) to an excited-state spin-orbit calculation (tddft_soc), and the jobtype is settled from the capability the profile reports to the framework. Section 5 describes how a jobtype reaches a profile.

The table lists the execution and program keys first, then the method and model keys, then the spectrum-output keys, and the framework-parity keys last.

The skill exposes the following profiles. A profile fixes the method and the set of properties a calculation returns.

Profile	Capability	Method
dft_energy	energy	dft
dft_gradient	gradient	dft
dft_dipole	dipole	dft
dft_opt_min	geometry_optimization	dft
dft_opt_ts	geometry_optimization	dft
dft_frequency	vibrational_frequency	dft
tddft_energy	energy	tddft
tddft_gradient	gradient	tddft
tddft_transition_dipole	transition_dipole	tddft
tddft_nac	nonadiabatic_coupling	tddft
tddft_soc	soc_coupling	tddft
tddft_opt_min	geometry_optimization	tddft
tddft_opt_ts	geometry_optimization	tddft
tddft_frequency	vibrational_frequency	tddft

The keywords, default values, and short descriptions are listed below. A † marks a keyword that is accepted but has no effect in v3.0.0; ‡ marks one value that is honoured; § marks an alias; * marks a derived quantity.

&ORCA		
execution_mode	local	Selects the run arm. local writes a bash script and runs it in the calculation directory, and slurm writes a batch script and submits it with sbatch…
shell	''	Prepends extra shell commands, such as module loads, to the run script. A list is joined one command per line and a string is split on semicolons…

&ORCA		
project	orca_calc	Sets the file stem of the run. The input, the output and the point-charge side file are named <project>.inp, <project>.out and <project>.pc in the…
profile	dft_energy	Selects the ORCA profile, and with it the step chain and the method. The 14 profiles are dft_energy, dft_gradient, dft_dipole, dft_opt_min…
orca_project	ORCA	Sets the project name in the framework spelling of the section. A non-empty value takes precedence over project and every file in the calculation…
orca_path	''	Names the ORCA installation root. The run script exports it as ORCA and starts $ORCA/orca on the input, and a path that does not exist is refused at…
mpi	''	Names the MPI library directory of a parallel ORCA build. The run script prepends $MPI/lib to LD_LIBRARY_PATH and $MPI/bin to PATH so the launcher…
orca_mem	1000	Sets the memory of one ORCA process in MB and is written as %maxcore. It is a per-core budget, so the memory of the job is this value times the rank…
orca_nproc	1	Sets the number of ORCA processes and is written as nprocs in the %pal block. It is the framework spelling of the rank count, and when nproc is also…
nproc	1	Sets the process count requested by the %pal block. It is the spelling a standalone configuration writes, and the larger of nproc and orca_nproc is…
thread	1	Sets the thread count of one ORCA process. The slurm script passes the value as #SBATCH --cpus-per-task, while a local run leaves the thread count to…
orca_thread	1	Sets the thread count in the framework spelling of the section. It folds into thread and the larger of the two values is used, so a request made in…

&ORCA		
keep_tmp	0	Keeps the calculation directory after the run. 1 leaves the input, the output and the scratch files in place, and 0 removes the directory once the…
slurm_time	01:00:00	Sets the batch time limit in HH:MM:SS and is written as #SBATCH --time; the default is 01:00:00.
slurm_partition	default	Sets the batch partition and is written as #SBATCH --partition; the default is default.
slurm_ntasks	1	Sets the number of batch tasks and is written as #SBATCH --ntasks; the default is 1. The ORCA process count is requested separately by nproc in the…
slurm_memory	4000	Sets the memory of the batch allocation in MB and is written as #SBATCH --mem; the default is 4000. It is the budget of the whole job rather than of…
slurm_command	''	Names the submission command of a slurm run. The default is empty, which leaves the sbatch line to the runner.
method	dft	Sets the electronic-structure method: dft for a ground-state calculation or tddft for excited states. The method must match the profile prefix and…
functional	pbe0	Names the exchange-correlation functional at the head of the ORCA keyword line. The name is lowercase free text and is not checked against a list, so…
basis	def2-svp	Sets the orbital basis set. The accepted names are sto-3g, 3-21g, 6-31g, 6-31g*, 6-31g**, 6-311g, 6-311g*, 6-311g**, def2-svp, def2-tzvp, def2-tzvpp…
density_fitting	true	Turns resolution-of-the-identity density fitting on or off. true writes RIJCOSX and aux_basis after the basis set, and false omits both keywords; the…
aux_basis	def2/J	Names the auxiliary basis set used with RIJCOSX. It is written after the RIJCOSX keyword and is free text, so a basis the installation lacks is…

&ORCA		
dispersion_correction	d3	Sets the Grimme dispersion correction: d3 writes D3BJ, d3zero writes D3ZERO, and d4 writes D4. An empty value or none writes no correction keyword…
grid_level	2	Sets the integration grid. 0 keeps the ORCA default and writes no grid line, while 1, 2 and 3 write a separate ! DEFGRID1, DEFGRID2 or DEFGRID3 line…
nstate	1	Sets the number of electronic states in the ladder, ground state included. The value must be at least 1, must agree with the length of state_spin…
tddft_nroots	0	Sets the root count of the %tddft block. Normalization derives it as the larger of the excited singlet and triplet counts, so the value is a result…
nsing_root	0	Counts the excited singlet roots of the ladder. Normalization derives it from state_spin and it decides which target states go to irootlist; the…
ntrip_root	0	Counts the triplet roots of the ladder. Normalization derives it from state_spin and it offsets triplet targets into trootlist; the default 0 means…
tda	false	Applies the Tamm-Dancoff approximation to the TDDFT response. true writes TDA true in the %tddft block and false leaves the ORCA default response…
num_freq	false	Chooses the frequency keyword of a ground-state run. false writes AnFreq and asks for the analytical Hessian, true writes NumFreq and asks for…
scf_converge	1e-08	Records the SCF convergence threshold. Every deck carries TightSCF on the keyword line, so the threshold the ORCA run applies comes from that keyword…
scf_maxit	200	Limits the SCF iterations through MaxIter in the %scf block. The same

&ORCA		
		block also asks for the MO coefficients with Print[P_MOs] 1; the default is 200.
opt_converge	false	Declares the optimization convergence request. The value is not written into the deck; the optimization is settled from the opt_converge value the…
opt_maxit	100	Limits the optimization cycles through MaxIter in the %geom block. The block is written only for the optimization profiles; the default is 100.
ir_width †	100	Sets the half-width of the Lorentzian broadening of the IR spectrum in cm^-1; the default is 100.0.
ir_scale †	1	Scales the intensities of the IR spectrum. The default is 1.0.
uv_width †	100	Sets the broadening width of the UV/visible spectrum in cm^-1; the default is 100.0.
nics_ring †	[0]	Lists the ring indices used for an NICS scan. The default [0] names no ring.
nics_scan †	[0]	Lists the scan distances of an NICS calculation in Angstrom. The default is [0.0].
charges	0	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
multiplicity	1	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
ci	[1]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
spin	[0]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
coupling	[]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
target_states	[0]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

&ORCA		
temperature †	298.15	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
solvent †	gas	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
verbose	4	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
shape †	ellipsoid	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
factor †	40	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
scale †	1	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
pbc	[0, 0, 0]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
cell	[99, 99, 99, 90, 90, 90]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
lattice_vector	[[0, 0, 0], [0, 0, 0], [0, 0, 0]]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
kpoint	[1, 1, 1]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

Full descriptions for all available keywords are summarized below.

execution_mode Selects the run arm. local writes a bash script and runs it in the calculation directory, and slurm writes a batch script and submits it with sbatch. The default is local.

shell Prepends extra shell commands, such as module loads, to the run script. A list is joined one command per line and a string is split on semicolons; the commands run before ORCA starts. The default is empty.

project Sets the file stem of the run. The input, the output and the point-charge side file are named <project>.inp, <project>.out and <project>.pc in the calculation directory; the default is orca_calc.

profile Selects the ORCA profile, and with it the step chain, the method and the capability the skill reports. The 14 profiles split into six ground-state ones (dft_energy, dft_gradient, dft_dipole, dft_opt_min, dft_opt_ts, dft_frequency) and eight excited-state ones (tddft_energy, tddft_gradient, tddft_transition_dipole, tddft_nac, tddft_soc, tddft_opt_min, tddft_opt_ts, tddft_frequency). The profile must agree with method, and each one compiles to a fixed chain of ORCA steps, so the jobtype and the profile answer the same question; a name outside the list is refused at validation.

orca_project Sets the project name in the framework spelling of the section, which is the spelling a converted or classic input writes. A non-empty value is copied onto project, so it wins over whatever project carries, and ORCA_PROJECT is exported with the result. All file names in the calculation directory follow it; the default is ORCA.

orca_path Names the ORCA installation root. The run script exports it as ORCA, then prepends the MPI directories and starts $ORCA/orca on <project>.inp with the output redirected to <project>.out and stderr merged into the same log. A path that does not exist is refused at validation rather than failing remotely; the default is empty.

mpi Names the MPI library directory of a parallel ORCA build. The run script prepends $MPI/lib to LD_LIBRARY_PATH and $MPI/bin to PATH so the launcher and its libraries resolve; the default is empty, which suits a serial ORCA build.

orca_mem Sets the memory of one ORCA process in MB and is written as the %maxcore line of the deck. The budget is per process, not per job, so a run with nproc ranks asks the machine for nproc times this value. The default is 1000.

orca_nproc Sets the number of ORCA processes and is written as nprocs in the %pal block of the deck. It is the framework spelling of the rank count; nproc is the spelling a standalone configuration writes, and the two fold to their larger value so neither can shrink a request made in the other. The default is 1.

nproc Sets the process count written to the %pal block. It is the spelling a standalone configuration writes, while orca_nproc is what the framework forwards from &ORCA. The pair folds to its larger value once, and the folded count is what both the deck and the resource arithmetic use. The default is 1.

thread Sets the thread count of one ORCA process. The slurm script passes it as #SBATCH --cpus-per-task, so the scheduler allocates that many cores per task, while a local run leaves the thread count to the machine it lands on. orca_thread is the framework spelling and the larger of the pair wins; the default is 1.

orca_thread Sets the thread count in the framework spelling of the section. It is folded into thread once, and the larger of the two values is kept, so a request made through either spelling survives. The default is 1.

keep_tmp Keeps the calculation directory after the run. 1 leaves the input, the output and the scratch files in place, and 0 removes the directory once the results are parsed; the default is 0.

slurm_time Sets the batch time limit in HH:MM:SS and is written as #SBATCH --time; the default is 01:00:00.

slurm_partition Sets the batch partition and is written as #SBATCH --partition; the default is default.

slurm_ntasks Sets the number of batch tasks and is written as #SBATCH --ntasks; the default is 1. The ORCA process count is requested separately by nproc in the %pal block.

slurm_memory Sets the memory of the batch allocation in MB and is written as #SBATCH --mem; the default is 4000. It is the budget of the whole job rather than of one ORCA process.

slurm_command Names the submission command of a slurm run. The default is empty, which leaves the sbatch invocation to the runner, and a non-empty value is carried with the job settings.

method Sets the electronic-structure method. dft covers the ground state: energy, analytical gradient, dipole, optimization and frequencies. tddft covers the excited states: excitation energies, transition dipoles, excited-state gradients, NACME, spin-orbit coupling and numerical frequencies. The method must match the profile prefix and must support the profile task, the legacy sf_tddft method is refused, and a tddft run whose ladder holds no excited state is refused rather than degraded to a ground-state run.

functional Names the exchange-correlation functional written at the head of the ORCA keyword line, ahead of the basis set and the job keywords. The value is lowercase free text and is not checked against a list, so any functional the installed ORCA build knows is accepted and an unknown name is reported by ORCA itself. The default is pbe0.

basis Sets the orbital basis set, written after the functional on the main keyword line. The accepted names are the minimal sto-3g, the Pople sets 3-21g, 6-31g, 6-31g*, 6-31g**, 6-311g, 6-311g* and 6-311g**, the Karlsruhe sets def2-svp, def2-tzvp, def2-tzvpp, def2-qzvp and def2-qzvpp, and the Dunning sets cc-pvdz, cc-pvtz, cc-pvqz, aug-cc-pvdz, aug-cc-pvtz and aug-cc-pvqz. The name is lowercased and stripped of spaces before the check, and the default is def2-svp.

density_fitting Turns resolution-of-the-identity density fitting on or off. true appends RIJCOSX and then aux_basis to the keyword line, after the functional and the basis set. false omits both, which leaves a plain four-centre SCF. RIJCOSX is compatible with TDDFT gradients and numerical Hessians, so the default true is kept for every profile; the value must be a boolean.

aux_basis Names the auxiliary basis set that RIJCOSX fits the Coulomb and exchange terms against. It is only written when density_fitting is true, and an empty value drops it while RIJCOSX stays. The name is free text, so it must be one the installation provides; the default is def2/J.

dispersion_correction Sets the Grimme dispersion correction appended after the basis and density-fitting keywords: d3 becomes D3BJ, d3zero becomes D3ZERO and d4 becomes D4. An empty value or none writes no correction at all, and any other spelling is refused. The value is lowercased and stripped before the mapping, and the default is d3.

grid_level Sets the integration grid of the SCF. 0 keeps the ORCA default and writes no extra line. 1, 2 and 3 write a second keyword line, ! DEFGRID1, ! DEFGRID2 or ! DEFGRID3, so the main keyword line keeps its usual layout. Any other integer is refused, and the default is 2.

nstate Sets the number of electronic states in the ladder, ground state included. The value must be at least 1, must agree with the length of state_spin, and bounds target_states; the default is 1, a ground-state run.

tddft_nroots Sets the root count of the %tddft block. Normalization derives it as the larger of the excited singlet and triplet counts, so the value is a result of the build rather than a hand setting; the default 0 becomes that derived count, and a tddft run that still has no excited state is refused.

nsing_root Counts the excited singlet roots of the ladder. Normalization derives it from state_spin and it decides which target states go to irootlist; the default 0 means the value has not been derived yet.

ntrip_root Counts the triplet roots of the ladder. Normalization derives it from state_spin and it offsets triplet targets into trootlist; the default 0 means the value has not been derived yet.

tda Applies the Tamm-Dancoff approximation to the TDDFT response. true writes TDA true in the %tddft block and false leaves the ORCA default response; the default is false.

num_freq Chooses the frequency keyword. On the ground state false writes AnFreq and true writes NumFreq. On the excited state the keyword is always NumFreq, because ORCA has no analytical TDDFT second derivatives, so the switch only controls the DFT frequency profiles. The value must be a boolean and the default is false; the .hess file is written by ORCA automatically and no %output Print[P_Hessian] block is requested.

scf_converge Records the SCF convergence threshold. Every deck carries TightSCF on the keyword line, so the threshold the ORCA run applies comes from that keyword rather than from this value; the default is 1e-8.

scf_maxit Limits the SCF iterations through MaxIter in the %scf block. The same block also asks for the MO coefficients with Print[P_MOs] 1; the default is 200.

opt_converge Declares the optimization convergence request. The value is not written into the deck; the optimization is settled from the opt_converge value the skill returns once ORCA finishes, and the default false accepts a run that stopped early.

opt_maxit Limits the optimization cycles through MaxIter in the %geom block. The block is written only for the optimization profiles; the default is 100.

ir_width Sets the half-width of the Lorentzian broadening of the IR spectrum in cm^-1; the default is 100.0. This keyword is accepted but has no effect in v3.0.0.

ir_scale Scales the intensities of the IR spectrum. The default is 1.0. This keyword is accepted but has no effect in v3.0.0.

uv_width Sets the broadening width of the UV/visible spectrum in cm^-1; the default is 100.0. This keyword is accepted but has no effect in v3.0.0.

nics_ring Lists the ring indices used for an NICS scan. The default [0] names no ring. This keyword is accepted but has no effect in v3.0.0.

nics_scan Lists the scan distances of an NICS calculation in Angstrom. The default is [0.0]. This keyword is accepted but has no effect in v3.0.0.

charges Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

multiplicity Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

ci Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

spin Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

coupling Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

target_states Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

temperature Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2. This keyword is accepted but has no effect in v3.0.0.

solvent Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2. This keyword is accepted but has no effect in v3.0.0.

verbose Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

shape Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2. This keyword is accepted but has no effect in v3.0.0.

factor Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2. This keyword is accepted but has no effect in v3.0.0.

scale Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2. This keyword is accepted but has no effect in v3.0.0.

pbc Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

cell Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

lattice_vector Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

kpoint Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

The skill drives ORCA 5.0 or newer through the $ORCA/orca command and nothing else. Only the dft and tddft methods exist, the legacy sf_tddft method is refused, and the 14 profiles are the whole jobtype surface, so a jobtype without a matching profile is not offered.

The molecular restriction is absolute: a non-zero pbc is refused by both the framework capability gate and the skill validation, and cell, lattice_vector and kpoint are never read. Background point charges travel in a <project>.pc side file that the deck references through %pointcharges, because ORCA rejects inline rows, and NACME is not written at all for a run that carries point charges.

Excited-state limits follow ORCA. Nonadiabatic couplings are computed for S0-SN pairs only, a pair between two excited states is refused, and a SOC request needs a closed-shell singlet reference with excited singlet-triplet pairs. State spin labels must ascend in 2S and must agree with the declared pair maps, because ORCA orders the states that way and the skill refuses rather than reordering them.

Frequencies come from AnFreq or NumFreq on the ground state and from NumFreq on every TDDFT run; the dft_frequency profile deliberately carries no gradient step, because ORCA rejects EnGrad in a deck that also asks for a Hessian.

5.19. OPENQP

The openqp section configures the OpenQP layer of a run. It selects the profile, the method, the basis set and functional, the SCF and response controls, the guess and NAC settings, and the runtime that launches openqp. The classic spelling of the section is &OPENQP; in YAML and agent JSON it is the openqp mapping.

Every profile compiles to an OpenQP runtype and a chain of input sections, and the skill writes the geometry file and the deck itself, so no user-side OpenQP input file exists. Section 5 describes how a jobtype reaches a profile.

The table lists the execution and program keys first, then the method, response and optimizer keys, then the spectrum-output keys, and the framework-parity keys last.

The skill exposes the following profiles. A profile fixes the method and the set of properties a calculation returns.

Profile	Capability	Method
hf_energy	energy	hf
hf_gradient	gradient	hf
hf_geometry_optimization	geometry_optimization	hf
hf_frequency	vibrational_frequency	hf
tddft_energy	energy	tddft
tddft_gradient	gradient	tddft
tddft_geometry_optimization	geometry_optimization	tddft
tddft_nonadiabatic_dynamics	nonadiabatic_dynamics	tddft
tddft_frequency	vibrational_frequency	tddft
mrsf_energy	energy	mrsf
mrsf_gradient	gradient	mrsf
mrsf_nac	nonadiabatic_coupling	mrsf
mrsf_soc	soc_coupling	mrsf
mrsf_soc_gradient	soc_coupling	mrsf
mrsf_geometry_optimization	geometry_optimization	mrsf
mrsf_nonadiabatic_dynamics	nonadiabatic_dynamics	mrsf
mrsf_frequency	vibrational_frequency	mrsf

The keywords, default values, and short descriptions are listed below. A † marks a keyword that is accepted but has no effect in v3.0.0; ‡ marks one value that is honoured; § marks an alias; * marks a derived quantity.

&OPENQP		
execution_mode	local	Selects the run arm. local writes a bash script and runs it in the calculation directory, and slurm writes a batch script and submits it with sbatch…
shell	''	Prepends extra shell commands, such as module loads, to the run script. A list is joined one command per line and a string is split on semicolons…
project	openqp_calc	Sets the file stem of the run. The skill writes <project>.xyz and <project>.inp and reads <project>.out and <project>.json from the same directory…
profile	tddft_energy	Selects the OpenQP profile, and with it the runtype, the input sections and the method. The 17 profiles are hf_energy, hf_gradient…
openqp_project	OQP	Sets the project name in the framework spelling of the section. A non-empty

&OPENQP		
		value takes precedence over project, is exported as OPENQP_PROJECT and…
openqp_path	''	Names the OpenQP installation root. The run script exports it as OPENQP_ROOT and prepends $OPENQP_ROOT/lib to LD_LIBRARY_PATH, and a path that does…
use_mpi	false	Chooses the launch mode. false runs openqp with --nompi and uses OpenMP threads only, and true launches the job with mpirun and starts openqp_nproc…
mpi_command	''	Overrides the launcher prefix of an MPI run, for example mpiexec -n 4. An empty value lets the script build mpirun -n <openqp_nproc>; the default is…
openqp_nproc	1	Sets the number of MPI ranks, passed as -n to the launcher and, in slurm mode, as #SBATCH --ntasks. It is the only rank spelling this skill accepts…
thread	1	Sets the OpenMP threads per process through OMP_NUM_THREADS. It is the spelling a standalone configuration writes, while openqp_thread is the…
openqp_thread	1	Sets the thread count in the framework spelling of the section. It folds into thread and the larger of the two values is used, so neither spelling…
keep_tmp	0	Keeps the calculation directory after the run. 1 leaves the input, the log, the orbital json and the export files in place, and 0 removes the…
slurm_time	01:00:00	Sets the batch time limit in HH:MM:SS and is written as #SBATCH --time; the default is 01:00:00.
slurm_partition	default	Sets the batch partition and is written as #SBATCH --partition; the default is default.
slurm_ntasks	1	Sets the number of batch tasks and is written as #SBATCH --ntasks. In MPI mode the submitted count is the larger of this value and openqp_nproc; the…

&OPENQP		
slurm_memory	4000	Sets the memory of the batch allocation in MB and is written as #SBATCH --mem; the default is 4000.
slurm_command	''	Names the submission command of a slurm run. The default is empty, which leaves the sbatch line to the runner.
method	tddft	Sets the electronic-structure method: hf for a Hartree-Fock ground state, tddft for a TD-DFT or TD-HF response, or mrsf for spin-flip MRSF-TDDFT on a…
basis	6-31g*	Sets the orbital basis set from the OpenQP library. The accepted names are 3-21g, 4-31g, 6-31g, 6-31g*, 6-31g**, 6-31+g*, 6-311g, 6-311g*, 6-311g**…
functional	bhhlyp	Names the exchange-correlation functional, written on the [input] line for tddft and mrsf. The accepted names are the libxc spellings bhhlyp, b3lyp…
td_type	tda	Selects the response type of a tddft run: tda for the Tamm-Dancoff approximation or rpa for the full response. The mrsf method forces mrsf regardless…
scf_type	''	Holds the reference type written as [scf] type, one of rhf, rohf or uhf. Normalization derives it from the charge and multiplicity, the mrsf method…
scf_maxit	100	Limits the SCF iterations through [scf] maxit; the default is 100.
scf_converge	1e-06	Sets the SCF energy threshold through [scf] conv, in Hartree; the default is 1e-6.
td_maxit	50	Limits the response iterations through [tdhf] maxit; the default is 50.
td_converge	1e-06	Sets the response convergence threshold through [tdhf] conv, in atomic units; the default is 1e-6.
guess_type	auto	Sets the initial-guess type through [guess] type, for example auto for the Huckel fallback of a first step. Every run also saves the orbital json…
nac_type	ktdc	Selects the nonadiabatic-coupling flavour. ktdc runs the data route for the

&OPENQP		
		framework kTDC integrator and needs no orbital continuation, dcm asks for…
nac_dt	20.67	Sets the time step of the numerical NAC vector differentiation, in atomic units of time; the default is 20.67.
nac_dx	0.0001	Sets the displacement of the numerical NAC vector differentiation, in Bohr; the default is 0.0001.
nac_bp	true	Also computes the branching-plane quantities of a NAC vector run. The default is true.
dosoc	0	Requests spin-orbit coupling for the MRSF response, the request side of the framework &md dosoc flag. 1 selects the two-manifold run that solves the…
nstate_s	0	Sizes the singlet ladder of a two-manifold run through [tdhf] nstate_s. 0 means the value is derived from state_spin, and a non-zero value must cover…
nstate_t	0	Sizes the triplet ladder of a two-manifold run through [tdhf] nstate_t. 0 means the value is derived from state_spin, and a non-zero value must cover…
scal_rel	0	Sets the scalar-relativistic DKH order of the reference, written only when it differs from OpenQP's default. 0 keeps that default; the default is 0.
opt_maxit	100	Limits the geometry optimization cycles through [optimize] maxit; the default is 100.
opt_optimizer	bfgs	Selects the native optimizer written as [optimize] optimizer; the default is bfgs.
opt_rmsd_grad	0.0001	Sets the RMS gradient threshold of the optimization through [optimize] rmsd_grad, in Hartree/Bohr; the default is 1e-4.
opt_max_grad	0.0003	Sets the maximum gradient threshold of the optimization through [optimize] max_grad, in Hartree/Bohr; the default is 3e-4.

&OPENQP		
hess_dx	0.01	Sets the displacement of the numerical Hessian, written as [hess] dx in Bohr; the default is 0.01.
nstate	1	Sets the number of electronic states in the ladder. It is written as [tdhf] nstate and must agree with the length of state_spin; a two-manifold run…
ir_width †	100	Sets the half-width of the Lorentzian broadening of the IR spectrum in cm^-1; the default is 100.0.
ir_scale †	1	Scales the intensities of the IR spectrum. The default is 1.0.
uv_width †	100	Sets the broadening width of the UV/visible spectrum in cm^-1; the default is 100.0.
nics_ring †	[0]	Lists the ring indices used for an NICS scan. The default [0] names no ring.
nics_scan †	[0]	Lists the scan distances of an NICS calculation in Angstrom. The default is [0.0].
charges	0	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
multiplicity	1	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
ci	[1]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
spin	[0]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
coupling	[]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
target_states	[0]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
temperature †	298.15	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

&OPENQP		
verbose	4	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
solvent †	gas	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
pbc	[0, 0, 0]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
cell	[99, 99, 99, 90, 90, 90]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
lattice_vector	[[0, 0, 0], [0, 0, 0], [0, 0, 0]]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
kpoint	[1, 1, 1]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

Full descriptions for all available keywords are summarized below.

execution_mode Selects the run arm. local writes a bash script and runs it in the calculation directory, and slurm writes a batch script and submits it with sbatch; in slurm mode openqp_nproc also becomes #SBATCH --ntasks. The default is local.

shell Prepends extra shell commands, such as module loads, to the run script. A list is joined one command per line and a string is split on semicolons; the commands run before openqp starts. The default is empty.

project Sets the file stem of the run. The skill writes <project>.xyz and <project>.inp and reads <project>.out and <project>.json from the same directory; the fixed-name export files carry no stem. The default is openqp_calc.

profile Selects the OpenQP profile, and with it the runtype, the input sections and the method. The 17 profiles are hf_energy, hf_gradient, hf_geometry_optimization, hf_frequency, tddft_energy, tddft_gradient, tddft_geometry_optimization, tddft_nonadiabatic_dynamics, tddft_frequency, mrsf_energy, mrsf_gradient, mrsf_nac, mrsf_soc, mrsf_soc_gradient, mrsf_geometry_optimization, mrsf_nonadiabatic_dynamics and mrsf_frequency; the default is tddft_energy.

openqp_project Sets the project name in the framework spelling of the section. A non-empty value is copied onto project and exported as OPENQP_PROJECT, and because the log is always $OPENQP_PROJECT.log, the orbital json is always $OPENQP_PROJECT.json and the automatic continuation follows that fixed name. The default is OQP.

openqp_path Names the OpenQP installation root. The run script exports it as OPENQP_ROOT and prepends $OPENQP_ROOT/lib to LD_LIBRARY_PATH ahead of the MKL and GCC directories,

so an installation that ships its own libraries resolves them. The openqp executable itself is looked for in PATH, not under this root, and a path that does not exist is refused at validation. The default is empty.

use_mpi Chooses the launch mode. false runs openqp with --nompi and uses OpenMP threads only, and true launches the job with mpirun and starts openqp_nproc ranks; the default is false.

mpi_command Overrides the launcher prefix of an MPI run, for example mpiexec -n 4. An empty value lets the script build mpirun -n <openqp_nproc>; the default is empty.

openqp_nproc Sets the number of MPI ranks. With use_mpi true the run script launches mpirun -n <openqp_nproc>, and in slurm mode the value also raises #SBATCH --ntasks to match. It is the only rank spelling this skill reads: neither nproc nor mpi_nproc is accepted, because the rank count has no second name here. The count also bounds the displacement pool of a numerical Hessian or NAC vector, one displacement per rank. The default is 1.

thread Sets the OpenMP threads per process through OMP_NUM_THREADS. It is the spelling a standalone configuration writes, while openqp_thread is the framework spelling, and the larger of the pair is used; the default is 1.

openqp_thread Sets the thread count in the framework spelling of the section. It folds into thread and the larger of the two values is used, so neither spelling can shrink a request made in the other; the default is 1.

keep_tmp Keeps the calculation directory after the run. 1 leaves the input, the log, the orbital json and the export files in place, and 0 removes the directory once the results are parsed; the default is 0.

slurm_time Sets the batch time limit in HH:MM:SS and is written as #SBATCH --time; the default is 01:00:00.

slurm_partition Sets the batch partition and is written as #SBATCH --partition; the default is default.

slurm_ntasks Sets the number of batch tasks and is written as #SBATCH --ntasks. In MPI mode the submitted count is the larger of this value and openqp_nproc; the default is 1.

slurm_memory Sets the memory of the batch allocation in MB and is written as #SBATCH --mem; the default is 4000.

slurm_command Names the submission command of a slurm run. The default is empty, which leaves the sbatch invocation to the runner, and a non-empty value is carried with the job settings.

method Sets the electronic-structure method and the OpenQP input it compiles to. hf writes method=hf and takes no functional. tddft writes method=tdhf with the chosen functional and td_type tda or rpa. mrsf writes method=tdhf with [tdhf] type=mrsf and a triplet ROHF reference, which is the only reference OpenQP accepts for a spin-flip response. The method must agree with the profile prefix, and a method that cannot answer the profile task is refused.

basis Sets the orbital basis set, written as basis= on the [input] line. The names come from the OpenQP basis library: the Pople sets 3-21g, 4-31g, 6-31g, 6-31g*, 6-31g**, 6-31+g*, 6-311g, 6-311g* and 6-311g**, the Dunning sets cc-pvdz, cc-pvtz, cc-pvqz, aug-cc-pvdz and aug-cc-pvtz, the Karlsruhe sets def2-svp, def2-tzvp and def2-tzvpp, and sto-3g. The value is lowercased and stripped of spaces before the check, and any other name is refused rather than passed to the engine. The default is 6-31g*.

functional Names the exchange-correlation functional, appended to the [input] line for tddft and mrsf. The list is closed and holds the libxc names bhhlyp, b3lyp, b3lyp5, pbe, pbe0, bp86, blyp, b97, b97m, m06, wb97, wb97x and cam-b3lyp; hf takes an empty value and refuses a functional, while tddft and mrsf refuse an empty one. The default is bhhlyp, and the name reaches libxc as given.

td_type Selects the response type written as [tdhf] type. tda applies the Tamm-Dancoff approximation and rpa solves the full response; the mrsf method forces mrsf and ignores the request, and hf carries no response block at all, so the value is emptied. A value other than rpa or tda on a tddft method is refused. The default is tda.

scf_type Holds the reference type written as [scf] type, one of rhf, rohf or uhf. Normalization derives it from the charge and multiplicity, the mrsf method forces rohf, and the default empty value means it has not been derived yet.

scf_maxit Limits the SCF iterations through [scf] maxit; the default is 100.

scf_converge Sets the SCF energy threshold through [scf] conv, in Hartree; the default is 1e-6.

td_maxit Limits the response iterations through [tdhf] maxit; the default is 50.

td_converge Sets the response convergence threshold through [tdhf] conv, in atomic units; the default is 1e-6.

guess_type Sets the initial-guess type of the [guess] block. Every run also sets save_mol=true, so OpenQP stores its orbitals as <project>.json beside the log, and [guess] file declares the same file with type=auto, which means the json is read when present and the Huckel guess is built when it is not. The default auto therefore continues the previous step in a multi-step run and starts from Huckel on the first one.

nac_type Selects the nonadiabatic-coupling flavour, and with it the runtype of a dynamics or NAC profile. ktdc keeps a dynamics run on runtype=data with per-step energies and gradients only, which is what the framework kTDC integrator needs and what needs no orbital continuation. dcm switches to runtype=prop for the nacme and MO overlap of the previous step, which a single calculation cannot supply and which is therefore refused outside the dynamics path. nac keeps a single-point mrsf_nac run on runtype=nac for one state pair, and inside a dynamics profile it adds [properties] nac=nac for numerical vectors. The framework spelling nactype is accepted as an alias, the default is ktdc, and a spin-orbit run refuses any of them.

nac_dt Sets the time step of the numerical NAC vector differentiation through [nac] dt, in atomic units of time. The NAC vector is a difference quotient over a displaced geometry and a finite time step, so the two controls nac_dt and nac_dx define the approximation. The default is 20.67, and the section is written only for a runtype=nac run.

nac_dx Sets the nuclear displacement of the numerical NAC vector differentiation through [nac] dx, in Bohr. It is the geometry step that the finite difference is taken over, and it is written in the same [nac] section as nac_dt. The default is 0.0001.

nac_bp Asks for the branching-plane quantities of a NAC vector run through [nac] bp. The flag rides in the same section as nac_dt and nac_dx and is only written for a runtype=nac run. The default is true.

dosoc Requests spin-orbit coupling, the framework spelling of &md dosoc carried into the skill section. 1 selects the two-manifold run: the singlet and triplet ladders are solved and coupled, and nstate_s and nstate_t size them. 0 leaves the run single-manifold. A spin-orbit run needs method mrsf, at least one singlet and one triplet row, and refuses every NAC flavour, because the analytic vector route answers the singlet fold only. The default is 0.

nstate_s Sizes the singlet ladder of a two-manifold run, written as [tdhf] nstate_s. 0 derives the size from the declared singlet rows of state_spin, and a non-zero value must cover them. The state list is sorted by spin, so singlets come first; the default is 0.

nstate_t Sizes the triplet ladder of a two-manifold run, written as [tdhf] nstate_t. 0 derives the size from the declared triplet rows of state_spin, and a non-zero value must cover them. Triplet states follow the singlets in the sorted state list; the default is 0.

scal_rel Sets the scalar-relativistic DKH order of the reference, written only when it differs from OpenQP's default. 0 keeps that default; the default is 0.

opt_maxit Limits the geometry optimization cycles through [optimize] maxit; the default is 100.

opt_optimizer Selects the native optimizer written as [optimize] optimizer; the default is bfgs.

opt_rmsd_grad Sets the RMS gradient threshold of the optimization through [optimize] rmsd_grad, in Hartree/Bohr; the default is 1e-4.

opt_max_grad Sets the maximum gradient threshold of the optimization through [optimize] max_grad, in Hartree/Bohr; the default is 3e-4.

hess_dx Sets the displacement of the numerical Hessian, written as [hess] dx in Bohr; the default is 0.01.

nstate Sets the number of electronic states in the ladder, written as [tdhf] nstate. It must be at least 1, must agree with the length of state_spin, and bounds target_states. In a two-manifold run the value stays the configured ladder count rather than the total across both ladders, because each ladder is sized by nstate_s or nstate_t. The default is 1.

ir_width Sets the half-width of the Lorentzian broadening of the IR spectrum in cm^-1; the default is 100.0. This keyword is accepted but has no effect in v3.0.0.

ir_scale Scales the intensities of the IR spectrum. The default is 1.0. This keyword is accepted but has no effect in v3.0.0.

uv_width Sets the broadening width of the UV/visible spectrum in cm^-1; the default is 100.0. This keyword is accepted but has no effect in v3.0.0.

nics_ring Lists the ring indices used for an NICS scan. The default [0] names no ring. This keyword is accepted but has no effect in v3.0.0.

nics_scan Lists the scan distances of an NICS calculation in Angstrom. The default is [0.0]. This keyword is accepted but has no effect in v3.0.0.

charges Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

multiplicity Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

ci Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

spin Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

coupling Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

target_states Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

temperature Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2. This keyword is accepted but has no effect in v3.0.0.

verbose Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

solvent Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2. This keyword is accepted but has no effect in v3.0.0.

pbc Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

cell Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

lattice_vector Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

kpoint Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

The skill runs the openqp command and never imports pyoqp, so the installation must provide that executable in PATH; openqp_path only exports OPENQP_ROOT and its lib directory. Every run writes <project>.xyz and <project>.inp and then reads the fixed-name export files energies, grad_<i>, nacme, dcme, nac_<i>_<j>, opt.xyz, opt_status.txt, <project>.hess.json and socme.

The exchange-correlation family is closed. functional takes the libxc names bhhlyp, b3lyp, b3lyp5, pbe, pbe0, bp86, blyp, b97, b97m, m06, wb97, wb97x or cam-b3lyp, hf takes none, and any other name is refused instead of being handed to the engine. The response type is limited to TDA and RPA for tddft.

MRSF covers the spin-flip side. MRSF-TDDFT runs on a triplet ROHF reference, it is the only method that answers NAC vectors and spin-orbit couplings, and one NAC vector calculation covers one state pair. A spin-orbit run cannot carry NAC of any flavour, needs at least one singlet and one triplet row, and pins ispher=false; nac_type dcm is refused for a single calculation because it continues from the previous step orbital json. Advanced runtypes (MECI, MECP, MEP, TS) are not in the profile set, and a non-zero pbc is refused because the backend is molecular.

5.20. XTB

The xtb section configures the xTB layer of a run. It selects the profile and the tight-binding or force-field Hamiltonian, and the runtime that launches the xtb executable. The classic spelling of the section is &XTB; in YAML and agent JSON it is the xtb mapping.

The eight profiles pair each Hamiltonian with an energy or an energy-plus-gradient step. GFN2-xTB, GFN1-xTB and GFN0-xTB serve the qm and qm2 roles of a composite, while GFN-FF serves the mm role and is the only member that accepts a periodic cell. Section 5 describes how a jobtype reaches a profile.

The table lists the execution and program keys first, then the method keys, and the framework-parity keys last.

The skill exposes the following profiles. A profile fixes the method and the set of properties a calculation returns.

Profile	Capability	Method
gfn2_energy	energy	gfn2
gfn2_gradient	gradient	gfn2
gfn1_energy	energy	gfn1
gfn1_gradient	gradient	gfn1

Profile	Capability	Method
gfn0_energy	energy	gfn0
gfn0_gradient	gradient	gfn0
gfn_ff_energy	energy	gfn_ff
gfn_ff_gradient	gradient	gfn_ff

The keywords, default values, and short descriptions are listed below. A † marks a keyword that is accepted but has no effect in v3.0.0; ‡ marks one value that is honoured; § marks an alias; * marks a derived quantity.

&XTB		
execution_mode	local	Selects the run arm. local writes a bash script and runs it in the calculation directory, and slurm writes a batch script and submits it with sbatch…
shell	''	Prepends extra shell commands, such as module loads, to the run script. A list is joined one command per line and a string is split on semicolons…
project	xtb_calc	Holds the run label of the configuration. File naming follows xtb_project, which is the authoritative spelling and is mirrored into project at parse…
profile	gfn2_energy	Selects the xTB profile, and with it the Hamiltonian and the step chain. The eight profiles are gfn2_energy, gfn2_gradient, gfn1_energy…
xtb_project	XTB	Sets the file stem of the run, exported as XTB_PROJECT. The geometry, the optional input template, the output, the engrad, the point-charge file and…
xtb_path	''	Locates the xTB executable. A directory is read as an installation root and the script runs $xtb_path/bin/xtb, a path that names a file is used as…
xtb_mem	4000	Sets the stack size of one xTB process in MB, exported as OMP_STACKSIZE with KMP_STACKSIZE following it. It sizes the thread stack rather than a heap…
thread	1	Sets the OpenMP threads through OMP_NUM_THREADS. It is the spelling

&XTB		
		a standalone configuration writes, while xtb_thread is the framework spelling…
xtb_thread	1	Sets the thread count in the framework spelling of the section. It folds into thread and the larger of the two values is used, so neither spelling…
keep_tmp	0	Keeps the calculation directory after the run. 1 leaves the geometry, the input, the output and the engrad in place, and 0 removes the directory once…
slurm_time	01:00:00	Sets the batch time limit in HH:MM:SS and is written as #SBATCH --time; the default is 01:00:00.
slurm_partition	default	Sets the batch partition and is written as #SBATCH --partition; the default is default.
slurm_ntasks	1	Sets the number of batch tasks and is written as #SBATCH --ntasks; the default is 1. xTB starts no MPI ranks, so a larger count requests resources…
slurm_memory	4000	Sets the memory of the batch allocation in MB and is written as #SBATCH --mem; the default is 4000.
slurm_command	''	Names the submission command of a slurm run. The default is empty, which leaves the sbatch line to the runner.
method	gfn2	Selects the xTB Hamiltonian. gfn2, gfn1 and gfn0 are semiempirical tight-binding methods that serve the qm and qm2 roles, and gfn_ff is the GFN-FF…
stress	false	Requests the whole-system stress tensor. It is meaningful only for a periodic gfn_ff run, where xTB writes the lattice gradient and the S0 block is…
keep_topology	false	Reuses the GFN-FF topology file gfnff_topo that the previous evaluation left in the calculation directory. A dynamics run sets it so the bond list…
nstate	1	Sets the number of electronic states. Normalization forces the value to 1, because xTB computes the ground state only and no excited surface exists…

&XTB		
charges	0	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
multiplicity	1	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
ci	[1]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
spin	[0]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
coupling	[]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
verbose	4	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
solvent †	gas	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
temperature †	298.15	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
shape †	ellipsoid	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
factor †	40	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
scale †	1	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
pbc	[0, 0, 0]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
cell	[99, 99, 99, 90, 90, 90]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
lattice_vector	[[0, 0, 0], [0, 0, 0], [0, 0, 0]]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

&XTB		
kpoint	[1, 1, 1]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

Full descriptions for all available keywords are summarized below.

execution_mode Selects the run arm. local writes a bash script and runs it in the calculation directory, and slurm writes a batch script and submits it with sbatch. The default is local.

shell Prepends extra shell commands, such as module loads, to the run script. A list is joined one command per line and a string is split on semicolons; the commands run before xtb starts. The default is empty.

project Holds the run label of the configuration. File naming follows xtb_project, which is the authoritative spelling and is mirrored into project at parse time; the schema default is xtb_calc.

profile Selects the xTB profile, and with it the Hamiltonian and the step chain. Each of the four methods has an energy profile and a gradient profile: gfn2_energy and gfn2_gradient, gfn1_energy and gfn1_gradient, gfn0_energy and gfn0_gradient, and gfn_ff_energy and gfn_ff_gradient. The energy profiles run xtb with the method flag alone, and the gradient profiles add --grad and parse the engrad file. The profile must agree with method, and the default is gfn2_energy.

xtb_project Sets the file stem of the run and is exported as XTB_PROJECT. The geometry file is named <project>.xyz, or <project>.coord for a periodic GFN-FF run in Turbomole format; an optional user template <project>.inp in the launch directory is prepended to the generated input; the output, the engrad, the point-charge file and the GFN-FF topology follow the same stem. The default is XTB.

xtb_path Locates the xTB executable and, when it can be identified, the installation root. A directory is read as the root and the run script executes $xtb_path/bin/xtb; any other path is used as the binary itself; an empty value falls back to xtb in PATH. When a root is known, the script also exports XTBHOME to it and XTBPATH to $XTBHOME/share/xtb, which is where the parameters live. The default is empty.

xtb_mem Sets the stack size of one xTB process in MB and is exported as OMP_STACKSIZE, with KMP_STACKSIZE copied from it. The budget is a thread stack rather than a heap limit, so a large value is cheap and does not reserve memory the calculation does not touch. The default is 4000.

thread Sets the OpenMP threads of the single xTB process through OMP_NUM_THREADS. xTB starts no MPI ranks, so this is the only parallel axis the skill has; xtb_thread is the framework spelling and the larger of the pair wins. The default is 1.

xtb_thread Sets the thread count in the framework spelling of the section. It is folded into thread once and the larger value is kept, so a request made in either spelling survives. The default is 1.

keep_tmp Keeps the calculation directory after the run. 1 leaves the geometry, the input, the output and the engrad in place, and 0 removes the directory once the results are parsed; the default is 0.

slurm_time Sets the batch time limit in HH:MM:SS and is written as #SBATCH --time; the default is 01:00:00.

slurm_partition Sets the batch partition and is written as #SBATCH --partition; the default is default.

slurm_ntasks Sets the number of batch tasks and is written as #SBATCH --ntasks; the default is 1. xTB starts no MPI ranks, so a larger count requests resources the calculation cannot use.

slurm_memory Sets the memory of the batch allocation in MB and is written as #SBATCH --mem; the default is 4000.

slurm_command Names the submission command of a slurm run. The default is empty, which leaves the sbatch invocation to the runner, and a non-empty value is carried with the job settings.

method Selects the Hamiltonian written on the xTB command line: --gfn 2 for GFN2-xTB, --gfn 1 for GFN1-xTB, --gfn 0 for GFN0-xTB and --gfnff for GFN-FF. GFN2 and GFN1 are the two full parameterizations and answer energies and analytical gradients, while GFN0 keeps the EEQ charge model only, so it is not a full GFN Hamiltonian. GFN-FF is the force field member: it is the only method that writes a periodic cell, and it is the method that serves the mm role of a composite. The method must agree with the profile prefix, and the default is gfn2.

stress Requests the whole-system stress tensor. It is meaningful only for a periodic gfn_ff run, where xTB writes the lattice gradient and the S0 block is filled in Hartree/Bohr^3; every other run returns a zero tensor. The default is false.

keep_topology Reuses the GFN-FF topology that the previous evaluation left in the fixed calculation directory, so this geometry is scored on that bond list. The framework sets it for a dynamics run, whose surface must not change under the trajectory, and leaves it false for a search, where each trial geometry is scored with its own topology; a stale gfnff_topo is removed otherwise. The default is false.

nstate Sets the number of electronic states. Normalization forces the value to 1, because xTB computes the ground state only and no excited surface exists; the default is 1.

charges Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

multiplicity Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

ci Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

spin Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

coupling Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

verbose Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

solvent Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2. This keyword is accepted but has no effect in v3.0.0.

temperature Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2. This keyword is accepted but has no effect in v3.0.0.

shape Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2. This keyword is accepted but has no effect in v3.0.0.

factor Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2. This keyword is accepted but has no effect in v3.0.0.

scale Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2. This keyword is accepted but has no effect in v3.0.0.

pbc Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2. The three flags are the periodic axes, and any non-zero flag selects the periodic arm: a cell is written and the geometry

goes to a Turbomole coord file with $periodic 3. Only gfn_ff accepts it, and point-charge embedding is refused alongside it.

cell Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

lattice_vector Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

kpoint Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

The skill wraps the xTB 6.x executable and is a ground-state engine: the capabilities are energy, gradient and stress, and there are no excited states, no NAC, no SOC and no Hessian or frequency profile. Energies and gradients come back from <project>.engrad in Hartree and Hartree/Bohr, and a parse failure points at <project>.out.

GFN-FF is the only periodic method. A non-zero pbc is refused for gfn2, gfn1 and gfn0, and point-charge embedding cannot be combined with periodic boundaries at all. Only a periodic GFN-FF run fills the stress tensor, from the lattice gradient, and only its S0 block.

Parallelism has one axis. xTB starts no MPI ranks, so thread (or xtb_thread) and the stack size set through xtb_mem are the whole resource story, and a user template <project>.inp in the launch directory is prepended to the generated input, which is how extra xTB keywords such as a solvent model are supplied.

5.21. CP2K

The cp2k section configures the cp2k skill, which runs DFT and TDDFPT (linear-response) calculations with CP2K/Quickstep, for molecular and fully periodic systems alike. It is read when cp2k appears in the theory queue of &CONTROL and is spelled &CP2K in the classic input, cp2k in YAML and agent JSON. The section is optional; an absent section runs the skill on the defaults listed below.

The keys are grouped as the run arm and program paths, the method and model, the iteration and output controls, and finally the state, cell and environment keys the framework forwards from &MOLECULE and &MD. Those last keys are documented once in Section 5.2 and repeat here only so the section reads in full. Section 4 shows the same section in the three input carriers.

The skill exposes the following profiles. A profile fixes the method and the set of properties a calculation returns.

Profile	Capability	Method
dft_energy	energy	dft
dft_gradient	gradient	dft
dft_opt_min	geometry_optimization	dft
dft_md	abinitio_dynamics	dft
tddft_energy	energy	tddft
tddft_gradient	gradient	tddft
tddft_transition_dipole	transition_dipole	tddft
tddft_opt_min	geometry_optimization	tddft
tddft_md	abinitio_dynamics	tddft

The keywords, default values, and short descriptions are listed below. A † marks a keyword that is accepted but has no effect in v3.0.0; ‡ marks one value that is honoured; § marks an alias; * marks a derived quantity.

&CP2K		
execution_mode	local	Chooses the run arm. local starts CP2K as a subprocess of the framework; slurm writes a batch script and submits it with sbatch. The arm changes only…
shell	''	Sets the skill-level shell commands, one command per line, copied verbatim into the run or batch script before the CP2K call. It carries the module…
project	cp2k_calc	Sets the stem of the run artifacts. The CP2K input, log and coordinate file carry it, for example <project>.inp, <project>.log and <project>.xyz. The…
cp2k_project	CP2K	Sets the project stem through the classic-input spelling. It is copied into project, which is the single value used for <project>.inp, <project>.log…
cp2k_path	''	Sets the CP2K installation directory CP2K_HOME. The run script exports it and resolves the executable from it, and validation refuses a directory…
cp2k_exe	''	Sets the CP2K executable. An empty value resolves to $CP2K_HOME/exe/local/cp2k.psmp; an MPI build is expected because the run starts through mpirun.
setup_script	''	Names a toolchain setup script that is sourced before the calculation starts, for example the install/setup file of a CP2K build. An empty value…
keep_tmp	0	Keeps the calculation directory when 1, and deletes it once the results have been parsed when 0. A kept directory holds the input, the log, the force…
thread	1	Sets the OpenMP threads of one rank, exported as OMP_NUM_THREADS. cp2k_thread is the framework spelling,

&CP2K		
		and a written pair folds to the larger value…
cp2k_thread	1	Sets the OpenMP thread count through the spelling the framework forwards from the &CP2K section. It is folded into thread, and the larger of the pair…
nproc	1	Sets the number of MPI ranks, passed to mpirun as -n. cp2k_nproc is the framework spelling, and the larger value of the pair is used.
cp2k_nproc	1	Sets the MPI rank count through the framework spelling. It folds into nproc, the larger value winning, so a request made in either spelling is…
slurm_time	01:00:00	Sets the wall-clock limit of the submitted job in hh:mm:ss. It is read only when execution_mode is slurm.
slurm_partition	default	Sets the scheduler partition of the submitted job. An empty value writes no partition directive, so the cluster default answers.
slurm_ntasks	1	Sets the number of tasks of the submitted job, which the batch template forwards as the MPI rank count.
slurm_memory	4000	Sets the memory per node of the submitted job in MB.
slurm_command	''	Sets the submission command of the batch script, for example sbatch or a site wrapper. An empty value uses the default.
profile	dft_energy	Selects the calculation profile, which fixes the CP2K RUN_TYPE chain. The nine values are dft_energy, dft_gradient, dft_opt_min, dft_md…
method	dft	Selects the electronic-structure method, dft for the ground state or tddft for the excited states. Only these two values are accepted, and the…
functional	pbe	Selects the exchange-correlation functional, pbe or pbe0. pbe0 is assembled as PBE with SCALE_X 0.75

&CP2K		
		plus a 25 percent Hartree-Fock fraction; any…
cutoff	400	Sets the plane-wave cutoff of the Quickstep grid in Ry, written as &MGRID CUTOFF. The value must be positive, and the default is 400 Ry.
ref_cutoff	50	Sets the refined-grid cutoff in Ry, written explicitly as &MGRID REL_CUTOFF so the engine default never applies. The value must be positive, and the…
basis_file	[BASIS_MOLOPT]	Lists the CP2K basis-set library files, written one BASIS_SET_FILE_NAME line each. The default is BASIS_MOLOPT; a second entry such as BASIS_ADMM_UUZ…
potential_file	[POTENTIAL]	Lists the pseudopotential library files, written one POTENTIAL_FILE_NAME line each. The default is POTENTIAL.
auxiliary_basis_file	[]	Lists the auxiliary-basis library files of an ADMM run, for example BASIS_ADMM_UUZ. The list is independent of whether ADMM is active, which the…
basis	{}	Sets the basis set per element as a name-to-keyword mapping, for example C: DZVP-MOLOPT-SR-GTH-q4. Elements left out are filled with…
potential	{}	Sets the pseudopotential per element, for example O: GTH-PBE-q6. Missing elements are filled with GTH-PBE, and an explicit entry takes priority over…
auxiliary_basis	{}	Sets the auxiliary basis per element. A non-empty mapping switches ADMM on, adding a basis-projection auxiliary-density method with an AUX_FIT set…
dispersion	''	Adds a Grimme dispersion correction through &VDW_POTENTIAL. d3 writes DFTD3 and d3bj writes DFTD3(BJ), while an empty value or none omits the section…
dipole_correction	false	Turns on the slab dipole correction, written as SURF_DIP .TRUE. with

&CP2K		
		SURF_DIP_DIR Z. It requires a periodic slab whose normal is the z axis; a…
tddft_dipole_form	length	Selects the gauge of the transition dipole moments, length or velocity, written as DIPOLE_FORM in the TDDFPT section. The default is length.
tddft_maxiter	100	Sets the maximum number of TDDFPT iterations, written as MAX_ITER in the TDDFPT block of the CP2K input.
tddft_converge	1e-05	Sets the TDDFPT eigenvalue convergence threshold. The default 1e-5 is the value the skill convergence table uses for the response solver.
nstate	1	Sets the number of electronic states of the run. The TDDFPT root count is derived from it when tddft_nstates is unset, and every target state index…
stress	false	Requests the whole-system stress tensor, returned as an (nstate, 3, 3) array, tensile-positive, in GPa. The framework planning route injects the…
stress_method	''	Selects the stress route. analytical asks CP2K for its STRESS_TENSOR ANALYTICAL, the ground-state tensor the engine prints; num differentiates the…
stress_delta	0.001	Sets the dimensionless strain step of the numerical stress route. A value outside the interval (0, 1e-2] is refused, and 1e-4 to 1e-3 is the…
stress_scope	full	Selects the strain directions the numerical route samples. full perturbs the six independent tensor components, twelve single points; iso dilates the…
scf_converge	1e-06	Sets the SCF energy convergence threshold in Hartree, written as EPS_SCF in &SCF. The default is 1e-6.
scf_maxit	200	Sets the maximum number of SCF cycles, written as MAX_SCF in &SCF.
opt_converge	false	Carries the convergence verdict of the internal geometry optimization back to

&CP2K		
		the framework. The parser fills it from the GEO_OPT block, and false…
opt_maxit	100	Sets the maximum number of geometry-optimization steps, written as MAX_ITER in the GEO_OPT block of &MOTION.
md_nstep	100	Sets the number of steps of a standalone MD profile, written into &MOTION &MD. Framework-driven dynamics does not read it, because there the…
md_timestep	0.5	Sets the timestep of a standalone MD profile in fs. Like md_nstep it belongs to the skill-driven MD profiles only.
md_ensemble	NVT	Selects the ensemble of a standalone MD profile, NVE or NVT. NVT uses the CSVR thermostat, whose temperature is the framework temperature key.
tddft_nstates	0	Sets the number of TDDFPT roots per spin manifold. Zero leaves the count to the normalization, which derives it from nstate, the target states and…
nsing_root	0	Counts the excited singlet roots of the state ladder. The normalization fills it from the state spins, and the TDDFPT block solves that many singlet…
ntrip_root	0	Counts the triplet states of the ladder. A positive value switches RKS_TRIPLETS on, so the TDDFPT block solves triplet roots as well as singlet roots.
ir_width †	100	Sets the FWHM broadening in cm-1 that an IR spectrum drawn from a frequency run would use. It is carried for the property pipeline; the CP2K skill…
ir_scale †	1	Sets the scale factor applied to harmonic frequencies before an IR spectrum is drawn; 1.0 leaves computed wavenumbers unscaled. The CP2K skill…
uv_width †	100	Sets the FWHM broadening in cm-1 of an absorption spectrum built from

&CP2K		
		excitation energies. The skill reports transition dipoles and oscillator…
nics_ring †	[0]	Lists the ring atom indices of a NICS calculation. The default [0] is the unset marker, and no CP2K run consumes the key in this version, which…
nics_scan †	[0]	Lists the probe positions in Angstrom of a NICS height scan. The default [0.0] is the unset marker, and no CP2K run consumes the key in this version.
charges	0	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
multiplicity	1	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
ci	[1]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
spin	[0]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
coupling	[]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
target_states	[0]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
verbose	4	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
solvent	gas	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
temperature	298.15	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
shape	ellipsoid	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

&CP2K		
factor	40	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
scale	1	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
pbc	[0, 0, 0]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
cell	[99, 99, 99, 90, 90, 90]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
lattice_vector	[[0, 0, 0], [0, 0, 0], [0, 0, 0]]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
kpoint	[1, 1, 1]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

Full descriptions for all available keywords are summarized below.

execution_mode Chooses the run arm. local starts CP2K as a subprocess of the framework; slurm writes a batch script and submits it with sbatch. The arm changes only the launcher, because both arms run the same generated input.

shell Sets the skill-level shell commands, one command per line, copied verbatim into the run or batch script before the CP2K call. It carries the module loads and exports a site needs before CP2K starts; the framework environment section is not forwarded here.

project Sets the stem of the run artifacts. The CP2K input, log and coordinate file carry it, for example <project>.inp, <project>.log and <project>.xyz. The classic input spells the same value cp2k_project, and the two are kept equal.

cp2k_project Sets the project stem through the classic-input spelling. It is copied into project, which is the single value used for <project>.inp, <project>.log and the {project}-pos-1.xyz and {project}-force-1_0.xyz files.

cp2k_path Sets the CP2K installation directory CP2K_HOME. The run script exports CP2K_HOME, resolves the executable from it when cp2k_exe is empty, and puts OMP_NUM_THREADS on the environment. Validation refuses a directory that does not exist, so a misspelt installation is reported before a job is written. The value is the same one the toolchain build exports, which is what makes a run script portable between the demo and the local installation.

cp2k_exe Sets the CP2K executable. An empty value resolves to $CP2K_HOME/exe/local/cp2k.psmp; an MPI build is expected because the run starts through mpirun.

setup_script Names a toolchain setup script that is sourced before the calculation starts, for example the install/setup file of a CP2K build. An empty value sources nothing.

keep_tmp Keeps the calculation directory when 1, and deletes it once the results have been parsed when 0. A kept directory holds the input, the log, the force and trajectory files, and the strain directories of a numerical stress run.

thread Sets the OpenMP threads of one rank, exported as OMP_NUM_THREADS. cp2k_thread is the framework spelling, and a written pair folds to the larger value so neither spelling can shrink the request.

cp2k_thread Sets the OpenMP thread count through the spelling the framework forwards from the &CP2K section. It is folded into thread, and the larger of the pair is what the run script exports.

nproc Sets the number of MPI ranks, passed to mpirun as -n. cp2k_nproc is the framework spelling, and the larger value of the pair is used.

cp2k_nproc Sets the MPI rank count through the framework spelling. It folds into nproc, the larger value winning, so a request made in either spelling is honoured.

slurm_time Sets the wall-clock limit of the submitted job in hh:mm:ss. It is read only when execution_mode is slurm.

slurm_partition Sets the scheduler partition of the submitted job. An empty value writes no partition directive, so the cluster default answers.

slurm_ntasks Sets the number of tasks of the submitted job, which the batch template forwards as the MPI rank count.

slurm_memory Sets the memory per node of the submitted job in MB.

slurm_command Sets the submission command of the batch script, for example sbatch or a site wrapper. An empty value uses the default.

profile Selects the calculation profile, which fixes the CP2K RUN_TYPE chain. The nine values are dft_energy, dft_gradient, dft_opt_min, dft_md, tddft_energy, tddft_gradient, tddft_transition_dipole, tddft_opt_min and tddft_md; a profile that does not belong to the chosen method is refused.

method Selects the electronic-structure method, dft for the ground state or tddft for linear-response excited states. The method decides which tasks exist: both carry energy, gradient, geometry optimization and ab-initio dynamics, tddft adds transition dipoles, and neither carries nonadiabatic coupling, spin-orbit coupling, dipoles or frequencies. A profile has to belong to the chosen method, so dft_gradient with method tddft is refused rather than silently rewritten.

functional Selects the exchange-correlation functional, pbe or pbe0 only. pbe is written as a PBE functional section. pbe0 is assembled from the same section with SCALE_X 0.75 plus a Hartree-Fock section with fraction 0.25, the layout the verified bulk demo uses. Any other name raises an unsupported-functional error, and the dispersion reference functional follows this value.

cutoff Sets the plane-wave cutoff of the Quickstep grid in Ry, written as &MGRID CUTOFF. Both cutoffs are always written explicitly, so a CP2K default never decides the grid. The value must be positive; 400 Ry is the shipped default and what the demo inputs use.

ref_cutoff Sets the refined-grid cutoff in Ry, written as &MGRID REL_CUTOFF. Raising it widens the region in which a basis function is mapped onto the reference grid, which is the usual remedy for a noisy force or a slowly converging SCF. The value must be positive, and 50 Ry is the default.

basis_file Lists the CP2K basis-set library files, written one BASIS_SET_FILE_NAME line each. The default is BASIS_MOLOPT; a second entry such as BASIS_ADMM_UUZ is added for an ADMM run.

potential_file Lists the pseudopotential library files, written one POTENTIAL_FILE_NAME line each. The default is POTENTIAL.

auxiliary_basis_file Lists the auxiliary-basis library files of an ADMM run, for example BASIS_ADMM_UUZ. The list is independent of whether ADMM is active, which the element-level auxiliary_basis mapping decides.

basis Sets the basis set per element as a name-to-keyword mapping, for example C: DZVP-MOLOPT-SR-GTH-q4. Normalization fills every element the mapping leaves out with DZVP-MOLOPT-SR-GTH plus the valence-electron suffix of that element, and a MOLOPT entry given here wins over the filled value. Elements outside the valence table fall back to the unsuffixed name and rely on the override. The names are resolved by CP2K from the library files named in basis_file.

potential Sets the pseudopotential per element, for example O: GTH-PBE-q6. Every element the mapping leaves out is filled with GTH-PBE, and an explicit entry takes priority. The potential has to come from the library file named in potential_file, which defaults to POTENTIAL.

auxiliary_basis Sets the auxiliary basis per element and, by being non-empty, switches ADMM on: the input gains an auxiliary-density-matrix method with basis projection and an AUX_FIT basis set per kind, and a TDDFPT run adds a no-purification method because ADMM and the response solver do not combine otherwise. Missing elements are filled with admm-dzp plus the valence suffix. An empty mapping leaves ADMM off and the run is a plain Quickstep calculation.

dispersion Adds a Grimme dispersion correction through &VDW_POTENTIAL. d3 writes DFTD3 and d3bj writes DFTD3(BJ); an empty value or none omits the section, which is the default. The reference functional of the correction always follows the active functional, so pbe0 with d3 writes PBE0, and an unknown token is refused.

dipole_correction Turns on the slab dipole correction, written as SURF_DIP .TRUE. with SURF_DIP_DIR Z. It requires a periodic slab whose normal is the z axis; a non-periodic cell, a tilted cell, and a numerical full-strain stress request are refused.

tddft_dipole_form Selects the gauge of the transition dipole moments, length or velocity, written as DIPOLE_FORM in the TDDFPT section. The default is length.

tddft_maxiter Sets the maximum number of TDDFPT iterations, written as MAX_ITER in the TDDFPT block of the CP2K input.

tddft_converge Sets the TDDFPT eigenvalue convergence threshold. The default 1e-5 is the value the skill convergence table uses for the response solver.

nstate Sets the number of electronic states of the run. The TDDFPT root count is derived from it when tddft_nstates is unset, and every target state index has to stay below it.

stress Requests the whole-system stress tensor. The result arrives in stress_tensor as an (nstate, 3, 3) array, tensile-positive, in GPa, and the framework converts it centrally to Hartree/Bohr^3. The framework planning route injects the request for NPT dynamics or a variable-cell optimization, while a standalone test input may set it directly. The request is orthogonal to the profile: the tensor keywords are added on top of the active profile and the parser reads the block non-blocking, so an absent tensor is zeros rather than a failed step. The best route for a ground state is the analytical one, because the engine already reports that tensor.

stress_method Selects the stress route. analytical asks CP2K for its STRESS_TENSOR ANALYTICAL, the ground-state tensor the engine prints; num differentiates the energy over an external strain scan and sends no stress keyword at all. An empty value is resolved from the method, tddft to num and dft to analytical.

stress_delta Sets the dimensionless strain step of the numerical stress route. A value outside the interval (0, 1e-2] is refused, and 1e-4 to 1e-3 is the recommended starting range.

stress_scope Selects the strain directions the numerical route samples. full perturbs the six independent tensor components, twelve single points; iso dilates the cell uniformly in two points and reports the result on the diagonal.

scf_converge Sets the SCF energy convergence threshold in Hartree, written as EPS_SCF in &SCF. The default is 1e-6.

scf_maxit Sets the maximum number of SCF cycles, written as MAX_SCF in &SCF.

opt_converge Carries the convergence verdict of the internal geometry optimization back to the framework. The parser fills it from the GEO_OPT block, and false means the optimization stopped before it converged.

opt_maxit Sets the maximum number of geometry-optimization steps, written as MAX_ITER in the GEO_OPT block of &MOTION.

md_nstep Sets the number of steps of a standalone MD profile, written into &MOTION &MD. Framework-driven dynamics does not read it, because there the framework steps through the gradient profile.

md_timestep Sets the timestep of a standalone MD profile in fs. Like md_nstep it belongs to the skill-driven MD profiles only.

md_ensemble Selects the ensemble of a standalone MD profile, NVE or NVT. NVT uses the CSVR thermostat, whose temperature is the framework temperature key.

tddft_nstates Sets the number of TDDFPT roots per spin manifold. Zero leaves the count to the normalization, which derives it from nstate, the target states and the triplet count.

nsing_root Counts the excited singlet roots of the state ladder. The normalization fills it from the state spins, and the TDDFPT block solves that many singlet roots.

ntrip_root Counts the triplet states of the ladder. A positive value switches RKS_TRIPLETS on, so the TDDFPT block solves triplet roots as well as singlet roots.

ir_width Sets the FWHM broadening in cm-1 that an IR spectrum drawn from a frequency run would use. It is carried for the property pipeline; the CP2K skill computes no frequencies in this version. This keyword is accepted but has no effect in v3.0.0.

ir_scale Sets the scale factor applied to harmonic frequencies before an IR spectrum is drawn; 1.0 leaves computed wavenumbers unscaled. The CP2K skill computes no frequencies in this version. This keyword is accepted but has no effect in v3.0.0.

uv_width Sets the FWHM broadening in cm-1 of an absorption spectrum built from excitation energies. The skill reports transition dipoles and oscillator strengths but draws no spectrum. This keyword is accepted but has no effect in v3.0.0.

nics_ring Lists the ring atom indices of a NICS calculation. The default [0] is the unset marker, and no CP2K run consumes the key in this version, which computes no shielding tensors. This keyword is accepted but has no effect in v3.0.0.

nics_scan Lists the probe positions in Angstrom of a NICS height scan. The default [0.0] is the unset marker, and no CP2K run consumes the key in this version. This keyword is accepted but has no effect in v3.0.0.

charges Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

multiplicity Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

ci Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

spin Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

coupling Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

target_states Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

verbose Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

solvent Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

temperature Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

shape Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

factor Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

scale Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

pbc Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

cell Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

lattice_vector Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

kpoint Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

The skill runs nine profiles and nothing else. dft_energy, dft_gradient, dft_opt_min and dft_md cover the ground state; tddft_energy, tddft_gradient, tddft_transition_dipole, tddft_opt_min and tddft_md cover the excited state. Framework-driven dynamics does not use the *_md profiles: with driver main the framework propagates and asks for dft_gradient one step at a time, and an md jobtype handed to the skill itself is refused because it would resolve to a single point on a frozen geometry.

Full three-dimensional periodicity is supported through pbc and lattice_vector. Partial periodicity such as [1, 1, 0] is refused because the low-dimensional electrostatics mapping is not defined in this version, and a periodic run needs a non-singular lattice. Without a cell the skill builds a vacuum box and solves it with the WAVELET Poisson solver, which is what a molecular run uses.

Requests the emitted input cannot honour are refused instead of approximated: a multiplicity other than 1, because the SCF block carries no unrestricted keyword; a functional other than pbe or pbe0; a triplet state row under method dft; an analytical stress tensor under method tddft, which CP2K aborts with "Stress Tensor not available for TDDFT calculations"; and the slab dipole correction for a non-periodic or tilted cell, or together with a numerical full-strain stress request.

The skill computes no nonadiabatic coupling, no spin-orbit coupling and no frequencies, so those jobtype tokens are refused rather than returned as zeros. A numerical stress scan never relaxes its strain points, so it reports the instantaneous stress of the configuration handed in, and a point whose state order changes fails the tensor by name because character-based state tracking is not implemented. A non-zero CP2K return code raises immediately instead of masking the failure.

5.22. LAMMPS

The lammps section configures the lammps skill, the classical force-field layer of the package. It is read when lammps appears in the theory queue of &CONTROL and is spelled &LAMMPS in the classic input, lammps in YAML and agent JSON.

The skill never builds a force field. The topology file <project>.data and the optional input templates are supplied by the user in the launch directory, and the section selects only the profile, the QM/MM partition and the runtime. The keys are grouped as the run arm and program, the engine controls, the output and selection keys, and the framework parity keys documented in Section 5.2.

The skill exposes the following profiles. A profile fixes the method and the set of properties a calculation returns.

Profile	Capability	Method
ff_energy	energy	ff
ff_gradient	gradient	ff
ff_optimize	geometry_optimization	ff
ff_optimize_cell	geometry_optimization	ff
ff_nve	classical_dynamics	ff
ff_nvt	classical_dynamics	ff
ff_npt	classical_dynamics	ff
qmmm_field	gradient	ff
qmmm_energy	gradient	ff

The keywords, default values, and short descriptions are listed below.

&LAMMPS		
execution_mode	local	Chooses the run arm. local runs LAMMPS as a subprocess, or inside the resident service when python_enabled asks for it; slurm submits one blocking…
shell	''	Sets the skill-level shell commands, one command per line, copied into the run or batch script before LAMMPS starts. The module loads that put libmpi…
project	lammps_calc	Sets the artifact stem of the run. It follows lammps_project, which is the

&LAMMPS		
		single source of truth, so the two spellings always name the same files.
lammps_project	LAMMPS	Sets the project stem of the user files and of every artifact: the topology file <project>.data, the optional templates <project>.in…
lammps_path	''	Sets the LAMMPS installation: a directory that is expected to hold bin/lmp, a path to the binary itself, or an empty value that searches lmp in PATH.
python_enabled	false	Asks for resident execution, where a service process holds one LAMMPS instance and receives each step of geometry over a Unix socket. It takes effect…
python_path	''	Sets the interpreter of the resident service process, that is the python that can import lammps and matches the MPI family of lammps_path. An empty…
debug	false	Writes the cold-path result files even on the resident path, so the two trajectories can be compared file for file. It costs the per-step I/O the…
driver	skill	Records who owns the geometry, skill or main. The framework sets it and a skill section never writes it; a section read on its own has no framework…
keep_tmp	0	Keeps the calculation directory when 1, and clears it when the backend or the resident service retires when 0. The directory holds the compressed…
thread	1	Sets the OpenMP threads per rank, exported as OMP_NUM_THREADS. lammps_thread is the framework spelling, and the larger value of a written pair is…
lammps_thread	1	Sets the OpenMP thread count through the framework spelling. It folds into thread, the larger value winning, so a request made in either spelling is…
nproc	1	Sets the number of MPI ranks. Above 1 the run starts with mpirun -np N;

&LAMMPS		
		lammps_nproc is the framework spelling and the larger of the pair wins.
lammps_nproc	1	Sets the MPI rank count through the framework spelling. It folds into nproc, and in resident mode it is the world of the service process.
slurm_time	01:00:00	Sets the wall-clock limit of the submitted job in hh:mm:ss. It is read only when execution_mode is slurm.
slurm_partition	''	Sets the scheduler partition of the submitted job. An empty value writes no partition directive, so the cluster default answers.
slurm_ntasks	1	Sets the number of tasks of the submitted job, which the batch template forwards as the MPI rank count.
slurm_memory	4000	Sets the memory per node of the submitted job in MB.
slurm_command	''	Sets the submission command of the batch script, for example sbatch or a site wrapper. An empty value uses the default.
profile	ff_energy	Selects the task: ff_energy, ff_gradient, ff_optimize, ff_optimize_cell, ff_nve, ff_nvt, ff_npt, qmmm_field or qmmm_energy. The profile fixes what…
method	ff	Selects the method. Only ff is accepted, because LAMMPS serves the classical force-field layer and never a QM layer.
lammps_units	real	Declares the unit style of the user input: real (kcal/mol, Angstrom, fs, atm) or metal (eV, Angstrom, ps, bar). It is not a conversion switch; the…
ewald_accuracy	1e-8	Sets the accuracy token written onto every kspace_style line the skill re-issues after read_data, in scientific notation such as 1e-8. It is not a…
lammps_shake	0	Turns the shake fix defined by the input template on (1) or off (0). It is refused for ff_energy and ff_gradient, where the framework propagates and…

&LAMMPS		
shake_fix_id	fShakeTIP3P	Names the shake fix the input template defines, fShakeTIP3P by default. The skill renumbers or removes that fix instead of adding a second one.
shake_keywords	''	Holds a raw fix ... shake ... command that is injected when lammps_shake is 1 and the templates define no such fix. An empty value in that situation…
box_relax	off	Selects the lattice degrees of freedom an engine-owned minimization may move: off keeps the box, iso scales it, aniso lets each axis answer for…
box_relax_pressure	0	Sets the target pressure in bar of the box-relax fix, converted to the pressure unit the run style reads at the point of writing.
min_style	cg	Sets the minimization algorithm written as min_style, cg by default. Any style the installed LAMMPS build accepts may be named, for example sd…
min_etol	0.0001	Sets the energy tolerance of minimize, in the native energy unit of the run. The default is 1e-4.
min_ftol	1e-08	Sets the force tolerance of minimize, in the native force unit of the run. The default is 1e-8.
min_maxiter	1000	Caps the number of minimization iterations, passed as the maxiter argument of minimize. The default is 1000.
min_maxeval	1000000	Caps the number of force evaluations, passed as the maxeval argument of minimize. The default is 1000000.
barostat_mode	isotropic	Selects how the NPT fix couples to the cell: isotropic scales the volume as one scalar, anisotropic lets the three axes move on their own. The…
thermo_damp	100	Sets the thermostat damping time of the NVT and NPT fixes, in the native time unit of the run.

&LAMMPS		
pressure	1	Sets the target pressure of the NPT fix, read in the pressure unit of the run style: atm for real and bar for metal.
pressure_damp	1000	Sets the barostat damping time of the NPT fix, in the native time unit of the run. It is validated as positive for ff_npt.
md_nstep	100	Sets the number of dynamics steps of the MD profiles, written as run N. The value must be positive.
md_timestep	1	Sets the timestep of the MD profiles in the native time unit of the run: fs for real and ps for metal. The value must be positive.
md_seed	12345	Seeds the initial velocities through velocity all create when the system carries no velocities of its own.
dump_freq	100	Sets how often the trajectory and the thermo output are written, in steps. The value must be positive.
stress	false	Requests the whole-system stress tensor. The deck adds a virial-only pressure compute whose six components are returned tensile-positive in bar; the…
atom_selection	[]	Lists the 1-based topology rows this task evaluates; an empty list means the whole system. A multi-center run gives every task its own subsystem, and…
qm_atoms	[]	Lists the QM atoms as 0-based indices in the framework order. The partition must be non-empty, must not cover the whole system, and is mutually…
qm_probe	[]	Lists the atoms a qmmm_field probe measures, as a subset of qm_atoms. Empty measures the whole QM set, which is the single-center case.
qm_centers	[]	Tiles qm_atoms into centers, one group per center, so qmmm_energy removes the in-cell Q-Q pairs inside each center only. Empty means one center.
q_qm	[]	Gives the QM zero-order charges in electron units, in qm_atoms order; the

&LAMMPS		
		length has to equal the QM set. qmmm_energy writes them into the data file.
qmmm_field_file	''	Points at a four-column ESPF file holding precomputed V and E for a standalone qmmm_energy run. A runtime caller passes the same arrays through the…
charges	0	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
multiplicity	1	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
nstate	1	Sets the number of electronic states. The force-field layer solves one surface, so the normalization sets the value to 1 and states other than the…
ci	[1]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
spin	[0]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
coupling	[]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
verbose	4	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
solvent	gas	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
temperature	298.15	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
shape	ellipsoid	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
factor	40	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

&LAMMPS		
scale	1	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
pbc	[0, 0, 0]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
cell	[99, 99, 99, 90, 90, 90]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
lattice_vector	[[0, 0, 0], [0, 0, 0], [0, 0, 0]]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
kpoint	[1, 1, 1]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

Full descriptions for all available keywords are summarized below.

execution_mode Chooses the run arm. local runs LAMMPS as a subprocess, or inside the resident service when python_enabled asks for it; slurm submits one blocking sbatch job per calculation.

shell Sets the skill-level shell commands, one command per line, copied into the run or batch script before LAMMPS starts. The module loads that put libmpi on the search path belong here, because the resident service inherits them.

project Sets the artifact stem of the run. It follows lammps_project, which is the single source of truth, so the two spellings always name the same files.

lammps_project Sets the project stem of the user files and of every artifact: the topology file <project>.data, the optional templates <project>.in, <project>.in.init and <project>.in.settings, the generated decks, the logs and the dumps.

lammps_path Sets where LAMMPS lives. A directory is read as an installation whose bin/lmp is the binary, a file path is taken as the binary itself, and an empty value searches lmp in PATH. lammps_path and python_path have to name one installation when the resident path is used: liblammps links exactly one MPI implementation, and a launcher from another family leaves every rank its own singleton world, which the service detects and refuses.

python_enabled Asks for resident execution, where a service process holds one LAMMPS instance and receives each step of geometry over a Unix socket. It takes effect only with a non-slurm arm, and only ff_energy, ff_gradient, qmmm_field and qmmm_energy are served that way.

python_path Sets the interpreter of the resident service process, that is the python that can import lammps and matches the MPI family of lammps_path. An empty value uses the interpreter running PyRAI3MD.

debug Writes the cold-path result files even on the resident path, so the two trajectories can be compared file for file. It costs the per-step I/O the resident path exists to avoid and is inert on the subprocess path.

driver Records who owns the geometry, skill or main. The framework sets it and a skill section never writes it; a section read on its own has no framework in it, so the engine integrates and its own constraint keywords apply.

keep_tmp Keeps the calculation directory when 1, and clears it when the backend or the resident service retires when 0. The directory holds the compressed force field, the generated decks, the log and the dumps, and both execution paths read this one key.

thread Sets the OpenMP threads per rank, exported as OMP_NUM_THREADS. lammps_thread is the framework spelling, and the larger value of a written pair is used.

lammps_thread Sets the OpenMP thread count through the framework spelling. It folds into thread, the larger value winning, so a request made in either spelling is honoured.

nproc Sets the number of MPI ranks. Above 1 the run starts with mpirun -np N; lammps_nproc is the framework spelling and the larger of the pair wins.

lammps_nproc Sets the MPI rank count through the framework spelling. It folds into nproc, and in resident mode it is the world of the service process.

slurm_time Sets the wall-clock limit of the submitted job in hh:mm:ss. It is read only when execution_mode is slurm.

slurm_partition Sets the scheduler partition of the submitted job. An empty value writes no partition directive, so the cluster default answers.

slurm_ntasks Sets the number of tasks of the submitted job, which the batch template forwards as the MPI rank count.

slurm_memory Sets the memory per node of the submitted job in MB.

slurm_command Sets the submission command of the batch script, for example sbatch or a site wrapper. An empty value uses the default.

profile Selects the task: ff_energy, ff_gradient, ff_optimize, ff_optimize_cell, ff_nve, ff_nvt, ff_npt, qmmm_field or qmmm_energy. The profile fixes what the generated deck does, and a profile that does not belong to the method is refused.

method Selects the method. Only ff is accepted, because LAMMPS serves the classical force-field layer and never a QM layer.

lammps_units Declares the unit style the user template is written in, real or metal. real reads energies in kcal/mol, distances in Angstrom, time in fs and pressure in atm; metal reads eV, Angstrom, ps and bar. Every quantity the backend returns is labelled with the style in _native_units and the framework scales it, so a wrongly declared style shifts every result instead of raising. The value also decides the time unit of md_timestep and the pressure unit of pressure.

ewald_accuracy Sets the accuracy token written onto every kspace_style line the skill re-issues after read_data, in scientific notation such as 1e-8. It is not a speed dial: the periodic QM/MM field probe takes its potential from the difference of two evaluations, and at 1e-6 that difference was short by 5.3e-4 relative on a 1581-atom box.

lammps_shake Turns the shake fix defined by the input template on (1) or off (0). It is refused for ff_energy and ff_gradient, where the framework propagates and the framework constraint block owns the constraint, and for both QM/MM tasks; the engine-integrated profiles may keep it.

shake_fix_id Names the shake fix the input template defines, fShakeTIP3P by default. The skill renumbers or removes that fix instead of adding a second one.

shake_keywords Holds a raw fix ... shake ... command that is injected when lammps_shake is 1 and the templates define no such fix. An empty value in that situation is an error.

box_relax Selects which lattice degrees of freedom an engine-owned minimization may move. off keeps the box the framework handed over, iso scales the volume, aniso lets each axis relax on its own. The box is written as a fix box/relax line under minimize, so the engine owns the cell for the whole relaxation; that is why a value other than off requires the ff_optimize_cell profile, and why ff_optimize_cell with off is refused. The relaxed cell comes back through fin_cell.

box_relax_pressure Sets the target pressure in bar of the box-relax fix, converted to the pressure unit the run style reads at the point of writing.

min_style Sets the minimization algorithm written as min_style, cg by default. The choice is passed through to the engine, so any style the installed build understands may be named, including sd, quickmin, fire and hftn. It only affects the ff_optimize and ff_optimize_cell profiles.

min_etol Sets the energy tolerance of minimize, in the native energy unit of the run. The default is 1e-4.

min_ftol Sets the force tolerance of minimize, in the native force unit of the run. The default is 1e-8.

min_maxiter Caps the number of minimization iterations, passed as the maxiter argument of minimize. The default is 1000.

min_maxeval Caps the number of force evaluations, passed as the maxeval argument of minimize. The default is 1000000.

barostat_mode Selects how the NPT fix couples to the cell: isotropic scales the volume as one scalar, anisotropic lets the three axes move on their own. The framework integrator reads the same word when it drives an NPT step.

thermo_damp Sets the thermostat damping time of the NVT and NPT fixes, in the native time unit of the run.

pressure Sets the target pressure of the NPT fix, read in the pressure unit of the run style: atm for real and bar for metal.

pressure_damp Sets the barostat damping time of the NPT fix, in the native time unit of the run. It is validated as positive for ff_npt.

md_nstep Sets the number of dynamics steps of the MD profiles, written as run N. The final structure, the final cell and the final velocities are returned, so the value is also the length of the trajectory a run hands back. It is validated as positive and read only by ff_nve, ff_nvt and ff_npt.

md_timestep Sets the timestep of the MD profiles. The unit follows lammps_units: fs for real and ps for metal. The value is validated as positive. Dropping a constraint is not free, because the timestep then has to resolve the constrained motions itself; the bundled TIP3P water box has an O-H period of about 14 fs, so 1 fs keeps roughly fourteen points per period.

md_seed Seeds the initial velocities through velocity all create when the system carries no velocities of its own.

dump_freq Sets how often the trajectory dump and the thermo output are written, in steps. A value coarser than md_nstep leaves a run with its final dumps only. It is validated as positive and read by the MD profiles.

stress Requests the whole-system stress tensor. The input gains a pressure compute with the kinetic term excluded, and its six components are written after run 0 to {title}.stress and read back from there instead of from the eight-digit thermo table. The parser negates the compressive pressure

into the tensile-positive convention and normalizes the result into bar, the unit the manifest declares. A step that was asked for a tensor and wrote no file is a failed step, never a zero.

atom_selection Lists the 1-based atom ids of the topology template that this task evaluates, in the order they are handed over. An empty list is the whole system, and an absent key means the same thing. The selection is what lets one multi-center run give every task its own subsystem, so it joins the force-field cache identity: a cache blind to it could answer one task with another task's rows. It is mutually exclusive with qm_atoms.

qm_atoms Lists the QM atoms as 0-based indices in the framework atom order. The partition has to be non-empty, must not cover the whole system, and every index has to be inside it. Both QM/MM profiles read the key, and both refuse a template without a charge column or a bonded row that crosses the partition. It is mutually exclusive with atom_selection.

qm_probe Lists the atoms a qmmm_field probe measures, as a subset of qm_atoms. An empty list measures the whole QM set, which is what every single-center run does. Under several centers the atoms a probe does not measure are environment charges on the same side of the A/B difference as the MM subsystem, so each center has to name itself.

qm_centers Tiles qm_atoms into centers, one group per center, and qmmm_energy uses the tiling to remove the in-cell Q-Q pairs inside each center only, which leaves the inter-center pair in the classical surface where both centers feel its force. Every center has to be non-empty and the groups have to tile qm_atoms. An empty list is one center, the single-center behavior.

q_qm Gives the QM zero-order charges in electron units, in qm_atoms order; the length has to equal the QM set. qmmm_energy writes them into the data file, evaluates the full force field once, and forms the electrostatic term from the Flow-1 arrays. A runtime caller may pass the per-step charges through the calculate call instead, which is what a dynamics run does.

qmmm_field_file Points at a four-column ESPF file, one row per QM atom holding V and the three field components, for a standalone qmmm_energy run. The file has to exist when it is named. A runtime caller never reads it back per step, because the V and E arrays travel in memory.

charges Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

multiplicity Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

nstate Sets the number of electronic states. The force-field layer solves one surface, so the normalization sets the value to 1 and states other than the ground state have no meaning here.

ci Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

spin Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

coupling Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

verbose Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

solvent Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

temperature Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

shape Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

factor Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

scale Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

pbc Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

cell Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

lattice_vector Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

kpoint Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

LAMMPS is the classical mm layer and never a qm or qm2 layer: the manifest declares the mm role only, so a theory queue cannot place it in a QM slot. A force field is user data. <project>.data is mandatory and a missing topology file is a validation error; <project>.in and the legacy <project>.in.init and <project>.in.settings are optional, and without them the deck is generated from the profile. The skill analyses the templates once into <project>.ff.json and renders every deck from that model, and a template it cannot prove equivalent falls back to the verbatim files with a warning.

QM/MM is two standalone profiles, never a switch inside ff_gradient. qmmm_field probes the MM potential and field at charged probes through an A/B difference and writes <title>.esfp; qmmm_energy evaluates the full force field with the QM charges and returns the reduced classical surface U_main minus U_gas minus the electrostatic term. A v1 limit is that Q-Q non-Coulomb coefficients are not zeroed, because quantum and MM atoms commonly share LAMMPS types; the shipped water fixture simply carries zero epsilon on those rows.

SHAKE belongs to whoever integrates the positions. lammps_shake is refused for ff_energy and ff_gradient, where the framework propagates and the framework constraint block declares the constraint, and for both QM/MM profiles. For the engine-integrated profiles the constraint decision is recorded once in the force-field model, so the deck emits neither fix nor unfix and the legacy mechanism only applies to a template the model refused.

The default unit style is real, with kcal/mol, Angstrom, fs and atm. Results leave the backend in native units with _native_units set and the framework scales them from the manifest; the .esfp file is the atomic-unit exception, and .qmmm, .grad and the auxiliary gradients stay native because they are composite input and audit data. The stress tensor is the potential part only, on a single-point evaluation, in bar: the internal-iteration profiles report no tensor, and restart.data is written for sequential task chaining, never as a framework continuation entry.

5.23. QDM

The qdm section configures the qdm skill, which evaluates one-dimensional analytic nonadiabatic model Hamiltonians and propagates a fully quantum reference dynamics on them. The skill is pure Python and numpy, so the section names no program, no installation path and no input file.

It is read when qdm appears in the theory queue of &CONTROL and is spelled &QDM in the classic input, qdm in YAML and agent JSON.

The geometry convention is the skill's own. Every model is one dimensional and is carried by a single dummy atom of symbol X, of which only the x component is read, so a structure is one line such as X -7.937658 0.0 0.0. The coordinate enters in Angstrom and is converted to Bohr internally, and y and z are ignored.

The skill exposes the following profiles. A profile fixes the method and the set of properties a calculation returns.

Profile	Capability	Method
tully1_energy	energy	tully1
tully1_gradient	gradient	tully1
tully1_nac	nac	tully1
tully1_reference	md	tully1
tully2_energy	energy	tully2
tully2_gradient	gradient	tully2
tully2_nac	nac	tully2
tully2_reference	md	tully2
tully3_energy	energy	tully3
tully3_gradient	gradient	tully3
tully3_nac	nac	tully3
tully3_reference	md	tully3
lz1_energy	energy	lz1
lz1_gradient	gradient	lz1
lz1_nac	nac	lz1
lz1_reference	md	lz1
lz2_energy	energy	lz2
lz2_gradient	gradient	lz2
lz2_nac	nac	lz2
lz2_reference	md	lz2
lz3_energy	energy	lz3
lz3_gradient	gradient	lz3
lz3_nac	nac	lz3
lz3_reference	md	lz3

The keywords, default values, and short descriptions are listed below.

&QDM		
execution_mode	local	Chooses the run arm. local evaluates the model in process with numpy; slurm serializes the configuration into the calculation directory, submits a…
project	QDM	Sets the stem of the configuration, the reference output files and the results file, for example {project}_population.dat and {project}_norm.dat. The…

&QDM		
qdm_project	QDM	Sets the project stem through the classic-input spelling. It is copied into project, which names {project}_population.dat, {project}_norm.dat…
scratch_path	''	Sets the parent directory of the calculation directory. An empty value places it under the run scratch root, and the directory is created when it…
keep_tmp	0	Keeps the calculation directory when 1, so the .dat dumps of energy, gradient, population and norm survive the call, and removes it once the results…
thread	1	Sets OMP_NUM_THREADS for the process. The skill starts no MPI ranks, so threads are its only parallel axis; qdm_thread is the framework spelling and…
qdm_thread	1	Sets the thread count through the framework spelling. It folds into thread, the larger value being used.
slurm_time	01:00:00	Sets the wall-clock limit of the submitted job in hh:mm:ss. It is read only when execution_mode is slurm.
slurm_partition	default	Sets the scheduler partition of the submitted job. An empty value writes no partition directive, so the cluster default answers.
slurm_ntasks	1	Sets the number of tasks of the submitted job. The skill starts no MPI ranks, so the value only describes the allocation.
slurm_memory	4000	Sets the memory per node of the submitted job in MB.
slurm_command	''	Sets the submission command of the batch script, for example sbatch or a site wrapper. An empty value uses the default.
profile	tully1_energy	Selects the model together with the output set. Every method carries four profiles, {method}_energy, {method}_gradient, {method}_nac and…

&QDM		
method	tully1	Selects the analytic model: tully1, tully2 or tully3 for the Tully 1990 single, dual and extended-coupling examples, or lz1, lz2 or lz3 for the…
model_mass	2000	Sets the wavepacket mass in atomic units, used for the kinetic energy and for the momentum handed to the reference solver. The name avoids mass…
tully_a	None	Overrides the first Tully parameter of the selected model: the asymptotic diabatic splitting in Hartree for tully1, the Gaussian barrier height in…
tully_b	None	Overrides the second Tully parameter: the wall steepness in 1/Bohr for tully1, the barrier width in 1/Bohr^2 for tully2, and the coupling strength in…
tully_c	None	Overrides the third Tully parameter: the coupling strength in Hartree for tully1 and tully2, and the coupling decay rate in 1/Bohr for tully3. Null…
tully_d	None	Overrides the fourth Tully parameter, the width in 1/Bohr^2 of the Gaussian coupling, for tully1 and tully2. tully3 has no fourth parameter and…
tully_e	None	Overrides the fifth Tully parameter, the asymptotic energy shift in Hartree of the second diabatic surface, for tully2 only. The other Tully models…
lz_a	None	Overrides the slope in Hartree/Bohr of the two linear diabatic surfaces of the Landau-Zener models, which cross at the origin with slopes plus and…
lz_b	None	Overrides the constant off-diagonal coupling in Hartree of the Landau-Zener models, which is half the adiabatic gap at the crossing. Null keeps the…
target_states	[0, 1]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
target_nac_pair	[[]]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

&QDM		
ref_ngrid	2048	Sets the number of grid points of the reference split-operator propagation. The value must be at least 2, and the default is 2048.
ref_xmin	None	Sets the lower edge of the reference grid in Bohr. A null value keeps the fixed default window that starts at -30 Bohr, and a wavepacket start…
ref_xmax	None	Sets the upper edge of the reference grid in Bohr. A null value keeps the default window ending at +80 Bohr, and a lower edge that is not below the…
ref_dt	20	Sets the time step of the reference propagation in atomic units of time. The value must be positive, and the default is 20.
ref_nsteps	500	Sets the number of propagation steps of the reference run. The value must be at least 1, and the default is 500.
ref_save_every	1	Sets the stride at which density snapshots are stored, in steps. The value must be at least 1, and 1 stores every step.
wavepacket_sigma	0.5	Sets the width in Bohr of the initial Gaussian wavepacket. The value must be positive, and the default is 0.5.
wavepacket_k0	10	Sets the fallback initial momentum in 1/Bohr, used when the run carries no velocities. With velocities present the solver takes model_mass times the…
charges	0	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
multiplicity	1	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
ci	[1]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
spin	[0]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

&QDM		
coupling	[]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
pbc	[false, false, false]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
cell	[99, 99, 99, 90, 90, 90]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
lattice_vector	[[0, 0, 0], [0, 0, 0], [0, 0, 0]]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
kpoint	[1, 1, 1]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

Full descriptions for all available keywords are summarized below.

execution_mode Chooses the run arm. local evaluates the model in process with numpy; slurm serializes the configuration into the calculation directory, submits a blocking sbatch job, and collects {project}_results.json afterwards.

project Sets the stem of the configuration, the reference output files and the results file, for example {project}_population.dat and {project}_norm.dat. The classic input spells the same value qdm_project, and the two are kept equal.

qdm_project Sets the project stem through the classic-input spelling. It is copied into project, which names {project}_population.dat, {project}_norm.dat, {project}_grid.dat, {project}_density.npy and {project}_results.json.

scratch_path Sets the parent directory of the calculation directory. An empty value places it under the run scratch root, and the directory is created when it does not exist.

keep_tmp Keeps the calculation directory when 1, so the .dat dumps of energy, gradient, population and norm survive the call, and removes it once the results are collected when 0. The next step rebuilds the directory, so clearing cannot strand a stepped run.

thread Sets OMP_NUM_THREADS for the process. The skill starts no MPI ranks, so threads are its only parallel axis; qdm_thread is the framework spelling and the larger value of a written pair wins.

qdm_thread Sets the thread count through the framework spelling. It folds into thread, the larger value being used.

slurm_time Sets the wall-clock limit of the submitted job in hh:mm:ss. It is read only when execution_mode is slurm.

slurm_partition Sets the scheduler partition of the submitted job. An empty value writes no partition directive, so the cluster default answers.

slurm_ntasks Sets the number of tasks of the submitted job. The skill starts no MPI ranks, so the value only describes the allocation.

slurm_memory Sets the memory per node of the submitted job in MB.

slurm_command Sets the submission command of the batch script, for example sbatch or a site wrapper. An empty value uses the default.

profile Selects the model together with the output set. Every method carries four profiles, {method}_energy, {method}_gradient, {method}_nac and {method}_reference: the first three answer per-geometry properties, and _reference runs the split-operator propagation.

method Selects the analytic model. tully1 maps the Tully 1990 example I, a single avoided crossing built from two exponential diabatic surfaces coupled by a Gaussian; tully2 maps example II, a flat surface crossed twice by a Gaussian barrier; tully3 maps example III, whose coupling grows and then plateaus so the nonadiabatic region stays extended on one side. lz1, lz2 and lz3 are the Landau-Zener linear-crossing models, in which two straight diabatic surfaces cross at the origin under a constant coupling; lz1 is the calibration target of the Zhu-Nakamura surface-hopping integrator because both premises of that reconstruction hold exactly there. The method has to match the profile, and every model carries two adiabatic states.

model_mass Sets the wavepacket mass in atomic units, which is the mass the kinetic energy and the classical equations of motion use. The reference solver also forms its initial momentum as this mass times the first atom x velocity when velocities are present. The key is named model_mass because mass already means the framework atomic masses, and it is validated as positive. The standard Tully value is 2000 atomic units.

tully_a Overrides the first Tully parameter, whose meaning follows the model: the asymptotic diabatic splitting A in Hartree for tully1, the Gaussian barrier height A in Hartree for tully2, and the constant diabatic splitting A in Hartree for tully3. A null value keeps the standard Tully 1990 parameter, which is 0.01, 0.1 and 0.0006 Hartree respectively. A written value must be positive, and a model that declares no such parameter ignores the key.

tully_b Overrides the second Tully parameter: the wall steepness B in 1/Bohr for tully1, the barrier width B in 1/Bohr^2 for tully2, and the coupling strength B in Hartree for tully3. Null keeps the standard value of 1.6, 0.28 and 0.1 in those units. A written value must be positive.

tully_c Overrides the third Tully parameter: the coupling strength C in Hartree for tully1 and tully2, and the coupling decay rate C in 1/Bohr for tully3. Null keeps the standard value of 0.005, 0.015 and 0.9 in those units. A written value must be positive.

tully_d Overrides the fourth Tully parameter, the width D in 1/Bohr^2 of the Gaussian coupling, for tully1 and tully2. Null keeps the standard 1.0 and 0.06 1/Bohr^2. tully3 has no fourth parameter, so the key is silently ignored there rather than refused.

tully_e Overrides the fifth Tully parameter, the asymptotic energy shift E in Hartree that raises the second diabatic surface of tully2. Null keeps the standard 0.05 Hartree. The two other Tully models declare no fifth parameter, so the key is ignored by them; lz_a and lz_b are ignored by all three Tully models for the same reason.

lz_a Overrides the slope A in Hartree/Bohr of the two linear diabatic surfaces of the Landau-Zener models. The surfaces cross at the origin with slopes plus and minus A, so the adiabatic gap at the crossing is twice lz_b and the character-swap width scales as lz_b over lz_a. Null keeps the model class default of 0.04 Hartree/Bohr.

lz_b Overrides the constant off-diagonal coupling B in Hartree of the Landau-Zener models, which is half the adiabatic gap at the crossing. Null keeps the model class default of 0.02

Hartree. A run that wants a narrower gap widens the swap region in proportion, and the calibration fixture pins a gap threshold that admits the hop evaluation.

target_states Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

target_nac_pair Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

ref_ngrid Sets the number of grid points of the reference propagation, which has to be at least 2. Together with ref_xmin and ref_xmax it fixes the grid spacing, and the density snapshot of one save is an array of shape (ngrid, 2). The default 2048 matches the reference fixtures.

ref_xmin Sets the lower edge in Bohr of the reference grid. A null value keeps the fixed default window that starts at -30 Bohr. The wavepacket start has to lie inside the window, so a model whose initial coordinate falls below the edge raises instead of propagating on a truncated grid.

ref_xmax Sets the upper edge in Bohr of the reference grid. A null value keeps the default window that ends at +80 Bohr. It has to be above ref_xmin, otherwise validation refuses the pair; the asymmetric window reflects the direction a Tully wavepacket travels.

ref_dt Sets the time step of the reference propagation in atomic units of time. The value must be positive, and 20 atomic units is the default the fixtures use. The products ref_dt times ref_nsteps is the total propagation time.

ref_nsteps Sets the number of propagation steps of the reference run; the value must be at least 1 and the default is 500. The step count together with ref_dt gives the total propagation time, and ref_save_every decides how much of it is stored.

ref_save_every Sets the stride at which density snapshots are stored; the value must be at least 1, and 1 stores every step. The written {project}_density.npy has one frame per save, and the population and norm files follow the same cadence.

wavepacket_sigma Sets the width in Bohr of the initial Gaussian wavepacket. The value must be positive, and the default is 0.5.

wavepacket_k0 Sets the fallback initial momentum in 1/Bohr, used when the run carries no velocities. With velocities present the solver takes model_mass times the first atom x velocity instead.

charges Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

multiplicity Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

ci Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

spin Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

coupling Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

pbc Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

cell Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

lattice_vector Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

kpoint Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

The skill evaluates analytic models and no electronic structure: there is no molecule to build and no basis, functional or SCF cycle to converge, so a run needs no external program. The geometry is one dummy atom X, only its x component is read, and y and z are ignored. At least one atom still has to be supplied, because the coordinate has to come from somewhere; the symbol itself is never looked up in an element table.

Two states and one dimension are the model rather than a setting. Every Hamiltonian is 2x2, so nstate is 2 and any other value raises, and pbc, cell, lattice_vector and kpoint carry no meaning here: a periodic request is refused outright instead of being silently ignored. The default periodicity of the section is therefore non-periodic on all three axes.

Surface hopping is not part of the skill. The framework MD workflow consumes the per-step energy, gradient and NAC of a {method}_nac profile and integrates the trajectory itself; the skill's own propagation is the split-operator reference, selected as {method}_reference when the skill drives the run. An optimize or surface_hopping jobtype has no profile here and is refused by name under a skill driver, so an iteration the models cannot serve is never answered by a single point.

The reference run is strict about its own settings: a wavepacket start outside the grid window raises, a lower grid edge that is not below the upper edge raises, and a non-positive model_mass, ref_dt, wavepacket_sigma, ref_ngrid below 2, or a step or stride count below 1 is refused. Negative or zero Tully parameters are refused in the same way when they are written.

5.24. NN_PES

The nn_pes section configures the nn_pes skill, which serves a trained neural-network potential energy surface and also owns the training run that produces it. It is the implementation of the nn theory label and is spelled &NN_PES in the classic input, nn in YAML and agent JSON. It is read when nn appears in the theory queue of &CONTROL.

The model container is a folder tree. Three ensembles live under modeldir, one folder per role: energy_grad for energies and gradients, nac for coupling vectors and soc for spin-orbit scalars. Each ensemble carries its own hyperparameter block, &EG, &NAC or &SOC, and this section selects the data, the ensemble size, the device and the runtime. The framework parity keys at the end of the list are documented once in Section 5.2.

The skill exposes the following profiles. A profile fixes the method and the set of properties a calculation returns.

Profile	Capability	Method
mlp_e_energy	energy	mlp_e
mlp_e_train	energy	mlp_e
mlp_e_soc	soc	mlp_e
mlp_e_soc_train	soc	mlp_e
mlp_eg_energy	energy	mlp_eg
mlp_eg_gradient	gradient	mlp_eg
mlp_eg_train	energy	mlp_eg
mlp_g2_gradient	gradient	mlp_g2
mlp_g2_train	gradient	mlp_g2

Profile	Capability	Method
mlp_nac_nac	nac	mlp_nac
mlp_nac_train	nac	mlp_nac
mlp_nac2_nac	nac	mlp_nac2
mlp_nac2_train	nac	mlp_nac2
eg_nac_energy	energy	eg_nac
eg_nac_gradient	gradient	eg_nac
eg_nac_nac	nac	eg_nac
eg_nac_train	energy	eg_nac
eg_nac2_energy	energy	eg_nac2
eg_nac2_gradient	gradient	eg_nac2
eg_nac2_nac	nac	eg_nac2
eg_nac2_train	energy	eg_nac2
eg_nac_soc	soc	eg_nac
eg_nac_soc_train	soc	eg_nac
eg_nac2_soc	soc	eg_nac2
eg_nac2_soc_train	soc	eg_nac2

The keywords, default values, and short descriptions are listed below.

&NN_PES		
execution_mode	local	Chooses the arm of a training run. local starts one subprocess per ensemble member in place; slurm writes the same command into a job script and…
project	NNPES	Sets the stem of the run artifacts, for example {project}_train.summary.txt and {project}_{profile}.log. It follows nn_project, which is the single…
nn_project	NNPES	Sets the project stem through the classic-input spelling. It is copied into project, which names the training summary, the per-step log and the…
scratch_path	''	Sets the parent directory of the calculation directory, which holds the step log and the uncertainty dump. The trained ensemble lives in modeldir…
keep_tmp	0	Keeps the calculation directory when 1. A value of 0 clears it when the backend is gone rather than after each step, because the log and the…
thread	1	Sets the thread count of the skill. nn_pes_thread is the framework spelling and the larger value wins, and the two lifetime budgets start from this…

&NN_PES		
nn_pes_thread	1	Sets the thread count through the framework spelling. It folds into thread, the larger value being used.
train_thread	0	Sets the threads of each training member subprocess, which gets its own budget. Empty or 0 inherits thread.
predict_thread	0	Sets the intra-op and inter-op pool size of the TensorFlow instance used for serving. Empty or 0 inherits thread.
slurm_time	01:00:00	Sets the wall-clock limit of the submitted training job in hh:mm:ss. It is read only when execution_mode is slurm.
slurm_partition	default	Sets the scheduler partition of the submitted training job. An empty value writes no partition directive, so the cluster default answers.
slurm_ntasks	1	Sets the number of tasks of the submitted training job. The skill starts no MPI ranks, so the value describes the allocation only.
slurm_memory	4000	Sets the memory per node of the submitted training job in MB.
slurm_command	''	Sets the submission command of the batch script, for example sbatch or a site wrapper. An empty value uses the default.
profile	mlp_eg_gradient	Selects family and capability. {method}_energy, _gradient, _nac and _soc serve properties, and {method}_train fits an ensemble. Twenty-five profiles…
method	mlp_eg	Selects the network family: mlp_e for scalars, mlp_eg for energies with differentiated gradients, mlp_g2 for gradients computed directly, mlp_nac and…
modeldir	''	Sets the model root, the folder holding energy_grad, nac and soc. A training run writes the role folders inside it and a serving run reads one a…
train_data	''	Names the training file, which is read as Hartree energies with Hartree/Bohr

&NN_PES		
		gradients and couplings. A training profile needs exactly one of…
train_dataset	{}	Carries the same training data inline as a mapping instead of by file name, which is what the framework writes when it expands a permutation map. It…
pred_data	''	Names a test set. A serving request that carries it is a test-set evaluation, one pass over the frames with an error table instead of one geometry…
gl_seed	1	Sets the master random seed. Each ensemble member is seeded with this value plus its index, which is what makes a fit reproducible, because a child…
train_mode	training	Selects whether the fit starts from scratch, training, or refits an existing folder, retraining. A hyperparameter written once is mirrored into the…
mult_nn	2	Sets the ensemble size, which is also how many semicolon-separated member groups of a hyperparameter are read: one group serves every member, surplus…
nn_predict_mode	0	Selects the serving predictor. 0 returns the member mean, and k from 1 to mult_nn returns member k as the prediction while the other members go to…
gpu	-1	Lists one device index per ensemble member, separated by semicolons; a single value broadcasts to every member and -1 asks for no device. The index…
eg	{}	Holds the hyperparameters of the energy ensemble in the per-member form of the container, for example depth 3 and nn_size 100;60. The vocabulary and…
nac	{}	Holds the hyperparameters of the coupling ensemble, in the same per-member form. The flat keyword nac

&NN_PES		
		means this block, never the couplings a run…
soc	{}	Holds the hyperparameters of the spin-orbit ensemble, which is trained by the energy family with its derivative head off, because a spin-orbit value…
nstate	2	Sets the number of electronic states, which is the channel count of the energy ensemble and the ladder the couplings are declared over. The value…
nnac	0	Sets the number of declared coupling channels, normally resolved from the framework coupling declaration. One declared pair is one channel, and the…
target_states	[0, 1]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
target_nac_pair	[[]]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
target_soc_pair	[[]]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
write_step_log	0	Writes two per-step diagnostic files, {project}_{profile}.log and {project}_{profile}.uncertainty.txt, when 1. The ensemble spread itself is computed…
predict_std	0	Reports the post-fit prediction spread on the training frames when 1. It belongs to training and prediction only; a serving call never enters that…
charges	0	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
multiplicity	1	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
ci	[1]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

&NN_PES		
spin	[0]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
coupling	[]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
pbc	[false, false, false]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
cell	[99, 99, 99, 90, 90, 90]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
lattice_vector	[[0, 0, 0], [0, 0, 0], [0, 0, 0]]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.
kpoint	[1, 1, 1]	Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

Full descriptions for all available keywords are summarized below.

execution_mode Chooses the arm of a training run. local starts one subprocess per ensemble member in place; slurm writes the same command into a job script and submits one blocking sbatch job per member. Serving is always in process and ignores this key.

project Sets the stem of the run artifacts, for example {project}_train.summary.txt and {project}_{profile}.log. It follows nn_project, which is the single source of truth.

nn_project Sets the project stem through the classic-input spelling. It is copied into project, which names the training summary, the per-step log and the uncertainty dump.

scratch_path Sets the parent directory of the calculation directory, which holds the step log and the uncertainty dump. The trained ensemble lives in modeldir, outside it.

keep_tmp Keeps the calculation directory when 1. A value of 0 clears it when the backend is gone rather than after each step, because the log and the uncertainty dump are appended across a run; it also removes {project}_train.summary.txt.

thread Sets the thread count of the skill. nn_pes_thread is the framework spelling and the larger value wins, and the two lifetime budgets start from this value.

nn_pes_thread Sets the thread count through the framework spelling. It folds into thread, the larger value being used.

train_thread Sets the threads of each training member subprocess, which gets its own budget. Empty or 0 inherits thread.

predict_thread Sets the intra-op and inter-op pool size of the TensorFlow instance used for serving. Empty or 0 inherits thread.

slurm_time Sets the wall-clock limit of the submitted training job in hh:mm:ss. It is read only when execution_mode is slurm.

slurm_partition Sets the scheduler partition of the submitted training job. An empty value writes no partition directive, so the cluster default answers.

slurm_ntasks Sets the number of tasks of the submitted training job. The skill starts no MPI ranks, so the value describes the allocation only.

slurm_memory Sets the memory per node of the submitted training job in MB.

slurm_command Sets the submission command of the batch script, for example sbatch or a site wrapper. An empty value uses the default.

profile Selects family and capability. {method}_energy, _gradient, _nac and _soc serve properties, and {method}_train fits an ensemble. Twenty-five profiles are declared, and a profile that does not belong to the method is refused.

method Selects the network family: mlp_e for scalars, mlp_eg for energies with differentiated gradients, mlp_g2 for gradients computed directly, mlp_nac and mlp_nac2 for couplings by a virtual potential or by propagation, and eg_nac or eg_nac2 for an energy ensemble together with a coupling ensemble.

modeldir Sets the model root. The folder holds one subfolder per role, energy_grad, nac and soc, and each subfolder carries the hyper_v{i}.json, weights_v{i}.weights.h5, scaler_v{i}.json, fiterr_train_v{i}.npy and fiterr_valid_v{i}.npy files of every member. A training run writes those folders; a serving run reads them and refuses a folder without a valid-error file, because the parent writes freshly initialized weights before the fit starts. The folder is also what a scaler is read from when a test-set evaluation wants physical units.

train_data Names the training file for a training profile. The file is read as Hartree energies with Hartree/Bohr gradients and couplings, the convention the engine backends report; there is no unit keyword, and a file carrying a unit block for another scheme is refused at read time. The trainer converts the derivative onto the Angstrom axis the network is fed, so the one rule the pair has to satisfy is that the gradient is the derivative of the energy the same surface reports. Exactly one of train_data and train_dataset has to be set.

train_dataset Carries the training set inline as a mapping, in the same shape and under the same unit rules as a file. The framework expands a permutation map into this mapping before the backend is initialized, so the skill never reads a map and never reorders atoms itself. It is mutually exclusive with train_data; setting both, or neither, is refused before any folder is created.

pred_data Names a test set for a serving profile. The request then becomes a test-set evaluation: one pass over the file frames and an error table, instead of one geometry and its properties. The file is read by the same loader and unit convention as a training file, and no training file is needed for it, because the scaler a metric needs lives in the model folder. A training profile that carries pred_data is refused, since a fit and a measurement are two different calls.

gl_seed Sets the master random seed. Each ensemble member is seeded with this value plus its index, which is what makes a fit reproducible, because a child process inherits no random state.

train_mode Selects training, a fit from scratch, or retraining, a refit of an existing folder. The hyperparameter schema mirrors every keyword written once into the retraining section as well, because this key decides which of the two sections the fit reads. A value outside the two names is refused.

mult_nn Sets the ensemble size. It decides how many member groups of a hyperparameter are read, where the separator between members is a semicolon and the member axis is always the outer one. One group serves every member and more groups than members keep the leading ones,

while fewer groups than members raise when more than one was written, because silently truncating an ensemble would train a different model than the one asked for. With two or more members the skill also reports the member-to-member standard deviation as the error bar of the surface.

nn_predict_mode Selects which member a serving call reports. 0 returns the member mean and is the default; k from 1 to mult_nn returns member k as the prediction and writes the other members to the alternative channels, which is what the adaptive analysis layer reads. A negative value, a value beyond mult_nn, or a value above 0 with a single-member ensemble is refused, because there is no second model to compare with.

gpu Lists the device index of each training subprocess, one per ensemble member, separated by semicolons; a single value broadcasts to every member, and -1 asks for no device at all. The entry is a device number, not the original project mode number, so gpu 1 means device 1. The parameter is validated per member and must not be below -1; the GPU code paths are present but unverified, because the development host is a CPU-only wheel.

eg Holds the hyperparameters of the energy ensemble, one block per member with semicolon as the member separator, for example depth 3 with nn_size 100;60 or loss_weights 1 1;1 10. The block is read when the run fits the energy family, and the container keeps one section for training and one for retraining. The vocabulary, the value shape and the broadcast rule belong to the hyperparameter schema: an unknown keyword is refused with the accepted list before a training folder is created, and a scalar broadcasts while an index table or a list needs one row per member.

nac Holds the hyperparameters of the coupling ensemble in the same per-member form. The coupling family has its own constraints: mlp_nac builds one output per electronic state and is therefore accepted only for a declaration of exactly nstate pairs, while a wider declared pair set needs mlp_nac2. The training file channel count has to equal the declared count, so this block and the framework coupling declaration are read together.

soc Holds the hyperparameters of the spin-orbit ensemble, in the same per-member form. The role is fitted by the energy family with the derivative head off, because a spin-orbit matrix element is a scalar that depends on geometry rather than a derivative of anything, and it is reported in cm-1 with no gradient channel. A soc jobtype token resolves to the mlp_e_soc profile, and a dynamics run that needs all three roles asks for the composite eg_nac_soc profile.

nstate Sets the number of electronic states, which is the channel count of the energy ensemble and the ladder the couplings are declared over. The value must be at least 1.

nnac Sets the number of declared coupling channels, normally resolved from the framework coupling declaration. One declared pair is one channel, and the width of the loaded ensemble is compared with this count at serving.

target_states The framework declares which states a run reports, and the skill serves one channel of the energy ensemble per state, so a target outside the ladder is refused instead of being answered with a neighbouring channel. The state set also sizes the coupling declaration: the framework splits the declared pairs on the state spins and hands the resolved nac_pair_map and soc_pair_map over, which is what the coupling channels are compared with at serving.

target_nac_pair Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

target_soc_pair Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

write_step_log Writes two per-step diagnostic files, {project}_{profile}.log and {project}_{profile}.uncertainty.txt, when 1. The ensemble spread itself is computed either way, because it is the adaptive sampling signal.

predict_std Reports the post-fit prediction spread on the training frames when 1. It belongs to training and prediction only; a serving call never enters that analysis.

charges Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

multiplicity Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

ci Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

spin Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

coupling Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

pbc Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

cell Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

lattice_vector Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

kpoint Framework keyword; the skill receives it from &MOLECULE and &MD. See Section 5.2.

TensorFlow is an optional dependency of the package. Install it as pyrai3md[nn] or tensorflow>=2.20 on Python 3.11 or newer; without it the skill is still listed by discovery and any use fails with "tensorflow is not installed" instead of looking absent. The backend is molecular: supports_pbc is false and any non-zero periodicity is refused, because the geometric features and the index weights were built and measured for molecules only.

One training term runs per process. A fit reconfigures TensorFlow device visibility for the whole process, so two terms that train cannot share a thread pool, and the thread count is process-scoped rather than term-scoped: a second term asking for a different count is told so rather than silently given one.

A fit has no numeric convergence verdict. The framework judges whether the training record came back, marking a run failed when a promised channel is missing or empty, but no threshold is applied to the reported validation MAE: whether the surface is good enough for a given dynamics run is the caller's judgement from the summary. The summary prints metrics in eV and eV/Angstrom, and spin-orbit values in cm-1.

The coupling and spin-orbit channels are declared, never derived, and channels are never zero-filled, because a silently missing coupling changes a surface-hopping trajectory. The shipped fixtures are demonstrations rather than converged surfaces, the coupling and spin-orbit ensembles rest on synthetic data, and the GPU paths are present but unverified on a CPU-only development host.

6. Workflows

This chapter follows the jobtype. It states what each token selects, what a property run plans, how each iterative workflow moves the structure it is given, and what a run leaves on disk. The keyword tables behind these descriptions are in Chapter 5.

6.1. Job types and workflow resolution

The control keyword jobtype selects the workflow and, for a property run, the capabilities planned. Basic tokens name a property and combine freely; an advanced token names a workflow and stands alone. Section 5.1 lists the keyword itself.

Token	What the run returns
energy	Total energies of the state ladder, Hartree.
gradient	Energies and the nuclear gradient of each state, Hartree/Bohr.
nac	Derivative couplings of the declared pairs, Hartree/Bohr.
soc	Spin-orbit couplings of the declared pairs, cm-1.
frequency	Force constants read into frequencies, modes and thermochemistry.
absorption	Transition dipoles, oscillator strengths and the spectrum.
dipole	State dipole moments.
transition	Transition dipole moments of the declared pairs.
hessian	The force-constant matrix, without the vibrational reading.
num_freq	The frequency output, by the framework displacement batch.
num_hess	The matrix alone, by the framework displacement batch.

Advanced token	What the run returns
md	A trajectory: frames, velocities, energies, populations, optional hops, a runtime record.
surface_hopping	The same dynamics loop, spelled for a run whose subject is the hops.
adaptive	Rounds of data, training, dynamics, screening and reference labelling.
train	One fit: the training record returned and the model folder written.
prediction	One served pass over a test set and its error statistics.
search	A grid of training runs, ranked after the grid finishes.
optimize	A final structure, a per-step convergence record and the accepted steps.
read	Frames extracted from finished calculations, written nowhere.

Eight aliases resolve to the canonical spelling: sp is energy, opt is optimize, freq and frequence are frequency, hess is hessian, numfreq is num_freq, numhess is num_hess, namd is md and predict is prediction. hop and hybrid are retired, refused with a message naming md with a hopping algorithm and a composite queue; Appendix E lists both. Mixing a basic token with an advanced one is refused, as are two advanced tokens.

A driver other than main short-circuits resolution: every jobtype reaches the skill unchanged and the run is a single shot, one skill, one evaluation, the outcome settled from what the skill returned.

A path method and a variable-cell search are refused there, and exactly one theory skill is required. With driver main the workflow comes from the token, and a basic token resolves none, so a property run is one calculator call. No path jobtype or section exists: optimize.method routes the three path names.

There is no scan section either, and jobtype scan is refused as an unknown jobtype because the token belongs to neither group. The registered Scan class is a placeholder that would repeat the input geometry and record each energy; its coordinate generation was never written.

6.2. Single-point and property calculations

A run holding only basic tokens plans one capability set and evaluates it once, at one geometry. The planner collects the requests, adds their dependencies and hands the set to the theory queue as one call: a gradient needs the energy of its state, a frequency needs the gradient, absorption and transition ask for transition dipoles, a numerical frequency needs only energies and gradients. The coupling and transition tokens take their pairs from the molecule section, where ci and coupling declare the state ladder and the pairs returned.

The set is spelled into the token order the skills expect (energy, gradient, nac, soc, dipole, transition, frequency), and each skill maps it onto one of its profiles. A combination the skill has no profile for is refused at planning, in a message naming the jobtype string and the profile it resolved to; a density functional run asking for soc is refused this way.

```
# Gradient and nonadiabatic coupling in one evaluation, C2H4
# Run: pyrai3md ethylene_grad_nac.yaml

control:
  title: ethylene_grad_nac
  jobtype: gradient nac
  theory: [molcas]
  driver: main

atoms: [C, C, H, H, H, H]
coordinates:
  - [-1.120191050,  0.000000000, -2.661941306]
  - [-1.120191050,  0.000000000, -1.331783306]
  - [-0.590584410,  0.756356668, -3.234891306]
  - [-1.649797690, -0.756356668, -3.234891306]
  - [-1.876547718, -0.529606640, -0.758833306]
  - [-0.363834382,  0.529606640, -0.758833306]

molecule:
  charges: 0
  multiplicity: 1
  ci: 2
  spin: 0
  coupling: 1 2

molcas:
  method: casscf
  basis: ano-s-mb
  nact_elec: 2
```

```
nact_orb: 2
inactive_orb: 7
```

A non-iterative run ends with one result block in the log. The energy is always present, because it is the run's progress. Verbose 1 adds the structure and the cell; verbose 2, the default, adds the gradient, couplings, dipole, transition dipole, frequency and thermochemistry sections and the velocities, each only when requested. Verbose 3 adds the Hessian and a not-computed line for every absent property. Coupling sections are titled by pair, and a multi-center run prints them per center.

6.3. Geometry optimization

jobtype optimize runs a search for one structure, and optimize.method selects the algorithm. Section 5.4 is the keyword reference; the path names are described in Section 6.4.

Method	Character	Requirements
minima	Inverse-Hessian BFGS step under a trust radius, RFO optional.	Energies and gradients. The default.
gdiis	DIIS extrapolation preconditioned by the BFGS model.	As minima. No RFO variant.
ts	Partitioned RFO climbing one mode and descending the rest.	An initial Hessian before the first step; Bofill updates after it.
meci	Two-surface crossing search, branching plane from the gradient difference and the derivative coupling.	A pair (state_i, state_j) and a coupling for both plane directions.
mecp	The same machinery across multiplicities.	A pair. The coupling is not used.

A search ends when five criteria hold at the same step; the progress block prints each threshold beside its value with a True or False judgement. The defaults are repeated here as the reference a run is compared with.

Keyword	Default	Quantity and unit
opt_energy	1.0e-6	Change in the electronic energy, Hartree.
opt_grms	3.0e-4	Root-mean-square gradient, Hartree/Bohr.
opt_gmax	4.5e-4	Largest gradient component, Hartree/Bohr.
opt_drms	1.2e-3	Root-mean-square atomic step, Bohr.
opt_dmax	1.8e-3	Largest atomic step component, Bohr.

A variable-cell run is judged twice: the atomic half keeps those five criteria, and the lattice half is read on its own pair, a stress in bar and a dimensionless strain. Both must clear their thresholds, because a box under stress has not converged however quiet the atoms are.

Keyword	Default	Quantity and unit
opt_csrms	40.0	Root-mean-square cell stress, bar.
opt_csmax	60.0	Largest cell stress component, bar.
opt_cerms	5.0e-5	Root-mean-square cell strain, dimensionless.
opt_cemax	7.0e-5	Largest cell strain component, dimensionless.

The step is bounded by a trust radius that belongs to the algorithm: 0.3 Bohr to start for minima and gdiis, 0.05 for the saddle search, 0.5 or 0.1 as a ceiling and 1e-4 as a floor. The radius is resized on the ratio of predicted to returned energy change, quartered below a quarter, doubled above three quarters and otherwise left alone so a run cannot oscillate. opt_trust_radius overrides the start; opt_use_rfo takes the step from the augmented Hessian instead of the BFGS model.

The curvature model starts as the identity, is updated by the BFGS formula on accepted pairs, skips an update whose curvature product is non-positive, and resets after repeated failure. The saddle search mixes the rank-one and symmetric-Broyden updates by Bofill's rule, so its model can acquire a negative eigenvalue and keep it. A step is accepted on the actual energy: a rise clearing a small floor is rejected, the geometry restored, the radius shrunk and a shorter step taken along the same direction, and four rejections in a row stop the run. Saddle and crossing searches skip that test. A run ending on maxit is marked failed.

optimize_cell adds the six movable lattice entries to the stepped vector, with the atoms riding the box at fixed fractional coordinates and the stress conjugate to each entry as its force. opt_pressure, in bar, makes the objective the enthalpy E + pV; without it the run asks for a zero-stress box. opt_cell_coupling chooses between an isotropic scale of the box and six independent entries. Either keyword without optimize_cell is refused, and so is a variable-cell run whose backend declares no stress, or a saddle whose curvature comes from the backend.

With driver main the framework steps and writes opt.xyz (latest structure), opt_geom.xyz (a frame per accepted step) and opt_status.txt (one row per step: energy, change, displacement and gradient readings, then the cell columns). With a skill driver the engine iterates and the framework writes the two xyz files from the structure returned through fin_coord; opt_status.txt is not written, and the outcome is settled from opt_converge.

```
# Minimum-energy conical intersection of two singlets, C2H4
# Run: pyrai3md ethylene_meci.yaml

control:
  title: ethylene_meci
  jobtype: optimize
  theory: [molcas]
  driver: main

optimize:
  method: meci
  state_i: 0
  state_j: 1
  maxit: 60
  opt_grms: 1.0e-3
  opt_gmax: 2.0e-3
```

```
molecule:
  charges: 0
  multiplicity: 1
  pbc: [0, 0, 0]
  ci: 2 1
  spin: 0 0
  coupling: 1 2

molcas:
  method: casscf
  basis: ano-s-mb
  nact_elec: 2
  nact_orb: 2
  inactive_orb: 7
```

6.4. Reaction paths (MEP, IRC, NEB)

A path is asked for with optimize.method mep, irc or neb: the keyword selects it, not the jobtype, and the run enters the path workflow while each name refuses a driver other than main for the reason given in Section 6.1. A walker, serving mep and irc, evaluates one probe at a time on the calculator's state, because a probe's gradient decides the next probe. A band, serving neb, evaluates every image of a round as one batch, so images run in parallel under multipool, each in its own runtime directory.

A minimum energy path walks a mass-weighted hypersphere of radius path_radius re-centred on every accepted point, so one point is one arc length. The sphere model is the Gonzalez-Schlegel construction: the probe is held on the shell and rotated towards minus the tangential gradient, and the rotation shrinks whenever the energy fails to fall. The relu model minimizes the energy plus a one-sided penalty that vanishes inside the shell. A point ends when its reading crosses path_inner_tol within path_inner_maxit probes; the walk ends at a stationary point, at an uphill point, whose predecessor is the minimum, or at its step bound.

An IRC is that walk seeded one radius along the imaginary mode of a transition state. irc_mode hessian takes the mode from the framework's Hessian, run with the vibrational analysis off, and irc_mode molden reads it from the file named by irc_mode_file. irc_direction both runs one walker per side from the one saddle, restoring it before each branch; forward or backward runs one side. The names label the ends, because a mode read from a file has an arbitrary sign.

A band relaxes a chain of images between the structures the input names. The force on an interior image is a tangent, the energy-weighted average of the two segment directions, a parallel spring of constant neb_spring that equalizes the spacing, and the physical force with only its perpendicular component kept, so the band does not slide off the path. The endpoints carry nothing, and relaxation is FIRE at the initial timestep neb_step. With neb_climb, the first round whose largest perpendicular force falls below neb_climb_threshold inverts the highest image's parallel force and drops its spring, one-way. A band ends when the interior forces are below opt_grms and opt_gmax, with a climbing image judged on its own force.

linear interpolation interpolates fractional coordinates, which keeps every image of a periodic band inside the cell; geodesic interpolates in a Morse-scaled internal-coordinate metric, the better guess when connectivity changes, and needs two interior images. The validator refuses each keyword group on a method that cannot read it, a band without both endpoints, a geodesic band with fewer than two images, a climb threshold without the climb switch, and irc_mode molden without a file.

A path writes its own records: path.xyz holds the latest structure, path_geom.xyz every accepted frame or the band once per round, path_profile.txt one row per accepted point or image, and path_status.txt one row per evaluation or round, with path_forward.xyz and path_backward.xyz for a two-sided IRC. None is opt_status.txt: that reader classifies a record by its column count, and a path table under that name would be read as another writer's. A periodic frame carries its own box, and for a band that is the image's box.

```
# Nudged elastic band between the HCN and HNC minima
# Run: pyrai3md path_neb.yaml

control:
  title: path_neb
  jobtype: optimize
  theory: [pyscf]
  driver: main

optimize:
  method: neb
  maxit: 100
  neb_reactant: path_reactant.xyz
  neb_product: path_product.xyz
  neb_images: 8
  neb_interpolate: linear
  neb_spring: 0.01
  neb_step: 0.1
  neb_climb: true
  neb_climb_threshold: 1.0e-2
  opt_grms: 3.0e-3
  opt_gmax: 4.5e-3

atoms: [H, C, N]
coordinates:
  - [0.000000000, 0.000000000, 1.055827000]
  - [0.000000000, 0.000000000, -0.010776000]
  - [0.000000000, 0.000000000, -1.145051000]

molecule:
  charges: 0
  multiplicity: 1

pyscf:
  method: hf
  basis: sto3g
  scratch_path: ./scratch
```

6.5. Ab initio MD and NAMD

jobtype md and surface_hopping select the same dynamics workflow, which integrates the nuclei with velocity Verlet in two half kicks around one electronic evaluation per step. A step applies the first half of the ensemble operator, propagates and wraps the coordinates, evaluates the target states, merges deviations, completes the velocity from the previous and current gradients, corrects coupling phases, projects the constraints, corrects the velocities, applies the second half of the ensemble operator, projects again after a cell rescale, samples the kinetic energy, records the pre-hop state, attempts a hop, judges the guards, synchronizes the tasks, advances the counter and writes the checkpoint. The kinetic energy is sampled last because every operation before it may write velocities. Step 1 records the unpropagated condition, so the first propagated frame is step 2.

The ensemble keyword selects nve, nvt or npt. An nve run requires the thermostat off and refuses dissipation, nvt requires a thermostat, and npt requires both. Undefined pairings are refused at validation rather than left to drift:

Barostat	Thermostats accepted	Note
berendsen	nose-hoover, nose-hoover-chain, csvr	Weak coupling; constrains no thermostat choice.
parrinello-rahman	nose-hoover, nose-hoover-chain	The extended cell is derived with a particle chain.
mttk	nose-hoover, nose-hoover-chain	Parrinello-Rahman plus a chain on the cell momentum.
scr	nose-hoover, nose-hoover-chain, csvr	First-order stochastic cell equation; isotropic only.

The starting point comes from initcond. 0 or none uses the geometry and velocities carried by the state; read or load takes ninitcond frames from <title>.init.xyz, each atom line carrying element, coordinates and velocities; wigner and boltzmann sample from <title>.freq.<format>, where the format is molden, g16, orca or bagel, seeded from gl_seed and the temperature. Fewer frames than ninitcond is an error. The first step's kinetic energy is set by excess, scale or target, one request written three ways, and two at once is refused. A restarted segment applies neither the draw nor the scaling.

Hopping is selected by sfhp, and the algorithms differ in what they need:

sfhp	Data requirement	Character
nosh	Energies and gradients of the active state.	No hopping; the default, a pass-through in the step.
gsh	Gradients of every state, so activestate 0.	Global hopping from a pure-state density; no decoherence or substeps.
fssh	Energies, gradients and a coupling, as a NAC vector or from the energy curvature.	Fewest switches, with substeps, decoherence and an acceptance test.

sfhp	Data requirement	Character
lzsh	State energies, plus the previous step couplings for a cross-spin hop.	Landau-Zener; the active state is irrelevant and every hop rescales the velocity isotropically.

The switches refine any of them. nactype names the coupling source of an FSSH run: nac takes the derivative coupling vector, ktdc estimates it from the energy curvature, needing no vector. phasecheck corrects the coupling sign between steps, substep sets the electronic substeps of one nuclear step, and rk4 picks forward Euler or a fourth-order integrator for the amplitudes. deco sets decoherence and off disables it. adjust picks the velocity policy after an accepted hop, none, a global rescale or a rescale along the coupling; reflect picks what a frustrated hop does. maxh bounds the highest state a hop may target. gap and gapsoc are the windows in eV for internal-conversion and intersystem-crossing hops. dosoc requests the couplings, and addsoc biases every coupling of one evaluation by a constant in cm-1.

Four resident guards end a trajectory, each after the offending frame is written. md_maxpop bounds the population, and md_maxenergy, md_maxgrad, md_maxnac and md_maxsoc bound the prediction deviation of a machine-learned surface, an unset value leaving that channel unjudged. A tracked geometry, track_type with track_index and track_thrhd in Angstrom, ends the run when its bound is crossed, track_stop choosing between a report and a stop. Cell divergence and any non-finite number are always fatal. The outcome is carried as a stop code:

Code	Meaning
0	Ran to the planned end.
1	Ill population: outside the bound, or not a number.
2	A tracked geometry bound was crossed.
3	A prediction error above one of the md_max thresholds.
4	The calculation failed.

A checkpoint <title>.rt is written on the cadence named by checkpoint, and the last step always writes one. It carries the state, the counter, the generator state and the runtime record, and a run is continued by launching that file, which bypasses the input, planning and dispatch stages. restart and addstep are refused keywords, because step already means the steps the new process adds. With ninitcond above one each trajectory is dispatched as a child with its own runtime directory, files and log.

```
# NAMD with FSSH on C2H4, Molcas CASSCF, NVT thermostat
# Run: pyrai3md ethylene_namd.yaml

control:
  title: ethylene_namd
  jobtype: md
  theory: [molcas]
  driver: main
  gl_seed: 20260926
```

```
atoms: [C, C, H, H, H, H]
coordinates:
  - [-1.120191050,  0.000000000, -2.661941306]
  - [-1.120191050,  0.000000000, -1.331783306]
  - [-0.590584410,  0.756356668, -3.234891306]
  - [-1.649797690, -0.756356668, -3.234891306]
  - [-1.876547718, -0.529606640, -0.758833306]
  - [-0.363834382,  0.529606640, -0.758833306]

molecule:
  charges: 0
  multiplicity: 1
  ci: 3
  spin: 0
  coupling: 1 2, 1 3, 2 3

md:
  root: 2
  initcond: wigner
  ninitcond: 2
  temperature: 298.15
  ensemble: nvt
  thermostat: nose-hoover-chain
  thermostat_chain: 3
  thermostat_tau: 100.0
  sfhp: fssh
  nactype: nac
  substep: 20
  deco: 0.1
  gap: 0.5
  step: 2000
  size: 20.67
  buffer: 500
  checkpoint: 100
  md_maxenergy: 0.05
  md_maxgrad: 0.15

molcas:
  method: casscf
  basis: ano-s-mb
  nact_elec: 4
  nact_orb: 4
  inactive_orb: 7
```

6.6. Hessian, frequencies and thermochemistry

The four force-constant tokens reach one workflow that answers two questions: which matrix the run wants, spelled by the jobtype token, and who produces it, spelled by hess_driver, which is not control.driver. The token also decides whether the matrix is read.

Jobtype	Matrix route	Vibrational reading
hessian, hess	auto: the backend matrix when the evaluation returns one, the displacement batch when it does not.	Off unless hess_vibration asks for it.
frequency, freq	auto, as above.	On: frequencies, modes and thermochemistry.
num_hess	numerical: the framework displaces and differences.	Off unless hess_vibration asks for it.
num_freq	numerical, as above.	On.

hess_mode and hess_driver are cross-checked. main displaces and differences the gradients the framework already has, so main with analytic is refused, and so is main with auto, because auto is the statement that no producer has been settled on. skill with numerical is legal and means the engine displaces its own atoms. A route the run did not get is written into the log, with the number of evaluations an auto fallback will spend.

The numerical route is a batch of central differences: each Cartesian component of each mobile atom is displaced by plus and minus hess_step, 0.005 Angstrom by default, and the gradient difference divided by twice the step in Bohr is one column, so the default step costs 2 times 3N evaluations for N mobile atoms, one child run each in its own runtime directory. A displaced point without a gradient fails the run and raises, naming the atom and axis. The matrix is assembled on the mobile block and symmetrized there, the held rows zero.

hess_atoms selects a partial Hessian in the one-based input numbering. During a search the held set belongs to the constraint plan, because fix is the one statement about what may move, so a hess_atoms beside a fix must agree with it: a mismatch is refused, and so is hess_atoms with no fix behind it, since the matrix would carry zero rows on atoms the run may move. The batch works per component, and the analysis re-reads the mobile set from the displacement rows so a held atom cannot appear as a soft mode.

The analysis mass-weights the matrix and removes the rigid directions by rank, so a linear molecule keeps four modes and a single atom none. Frequencies are signed, imaginary modes negative, modes normalized. The thermochemistry is the rigid rotor harmonic oscillator model with the quasi-RRHO free-rotor blend below hess_cutoff; hess_temperature, hess_pressure, hess_scale and hess_symmetry set the standard state. The result carries zero_point_energy, internal_energy, enthalpy, entropy, free_energy, heat_capacity_cv and heat_capacity_cp. The log prints frequencies, modes, corrections and the matrix at verbose 3, and hess_molden writes <title>.freq.molden, the file an initial-condition sampler reads.

Two restrictions follow. A vibrational analysis cannot be combined with a variable cell, because the analysis builds frequencies on a mass-weighted basis and the six lattice entries have no mass to be weighted by; a box's curvature is read by the search that asked for it. And an analytical matrix cannot be asked of a queue holding more than one backend, because no single skill owns a composite's second derivative.

6.7. Reading finished calculations

jobtype read extracts labels from calculations that have already run. It starts no engine, builds no calculator and evaluates nothing: it opens the paths the read section names, hands each to the file reader of the single theory binding, and returns the frames to its caller. No training-data file, trajectory or result artifact is written, so the working directory is untouched. The dataset belongs to the training-data tool, which calls this flow; Section 10.1 describes it.

The read section names the paths, an optional project stem overriding the one discovered from the files, an optional profile used to parse an optimization or dynamics directory, and one flag per label: energy, gradient, nac and soc. An entry may be a directory, an output file, or a file listing one path per line. A read run binds exactly one theory skill, no reference and no multiscale partition; in-process engines such as pyscf, qdm and nn_pes are refused by name. Building a label set for a later training run is the recommended use. Section 5.10 lists the section.

```
&control
title read_run
jobtype read
theory molcas

&read
path dirs.txt
energy 1
gradient 1
nac 0
soc 0

&molecule
charges 0
multiplicity 1
ci 1
spin 0
```

6.8. Artifacts and restart per workflow

Each workflow owns its file names, and this table is the single owner of them. Files are named after control.title and written in the run directory unless a row says otherwise. A dispatched child restamps its log to its own unit directory and leaves a runtime file and a result record there.

Workflow	Files	Restart and notes
Single point or property run	<title>.log carries the result block.	Nothing to restart; <title>.rt exists only in a dispatched child.
optimize	opt.xyz, opt_geom.xyz, opt_status.txt, and the log.	Rewritten by every run. A skill-driven run writes the two xyz files only.
path (mep, irc, neb)	path.xyz, path_geom.xyz, path_profile.txt, path_status.txt, and path_forward.xyz with path_backward.xyz for a two-sided IRC.	No opt_status.txt. A resumed path walks again from its new start.

Workflow	Files	Restart and notes
md and surface_hopping	<title>.md.xyz, .md.velo, .md.energies, .md.pop, .md.ensemble for nvt and npt, .md.stress when the backend supplied one, the <title>.sh.* set on hop steps, the charge files when charge management is on, and <title>.rt.	Continue by launching <title>.rt. Each trajectory child gets its own runtime/<skill>_runtime_<idx>/ with its files, log and record.
hessian, frequency, num_hess, num_freq	<title>.freq.molden when hess_molden is true, and the matrix, modes and corrections in the log.	The molden file is the input an initial-condition sampler looks for.
train	The model folder named by modeldir, one subfolder per fitted role, holding hyper_v<i>.json, scaler_v<i>.json, weights_v<i>.weights.h5, fiterr_valid_v<i>.npy, fitlog_<i>.txt and fit_stats/, plus <project>_train.summary.txt.	A continuation is declared with train_mode retraining and cannot change the architecture.
prediction	{project}_{profile}.{channel}.pred.txt, {project}_{profile}.frames.txt and the summary, in the run directory.	Kept out of the calculation directory so a scratch cleanup cannot delete them.
adaptive	The <title>.adaptive tree with <title>.adaptive.json, the per-iteration data, model and selection records, and the log.	The summary carries every iteration record, so a round reads back without its trajectory.
search	summary.csv, crashed.txt when a point crashed, and the grid record in the log.	summary.csv is the ranking; a rerun skips complete model folders and re-ranks them.

7. Multiscale calculations

A multiscale run takes its energy and gradient from more than one calculation. The system is split into regions, each is queued with its own method, and the results are assembled by the coefficients of the scheme before any workflow sees them. Both schemes are selected in the control section listed in Section 5.1 and take their partition from the molecule section.

7.1. Additive QM/MM

Sets multiscale_type to qmmm with exactly two theory entries, the quantum method first and the classical provider last. molecule.highlevel names the quantum atoms as a 1-based index list, and the union of the regions must be a strict subset. Three entries always select ONIOM3, so qmmm with three entries and qmmm with a reference queue are both refused.

The scheme is additive: E = E_QM + E_classic. The classical provider returns one whole-system evaluation U_main, and the composite subtracts the in-cell point-charge sum U_gas and the QM-MM electrostatics qmmm_el = sum over quantum atoms of q_i V_i, so E_classic = U_main - U_gas - qmmm_el. The quantum rows of the gradient also carry the classical rows; every other row is classical.

With embedding true the plan is qmmm_field, then QM, then qmmm_energy, and the quantum calculation is embedded in the classical field at its nuclei, which the Molcas ESPF deck consumes. The field comes from an A/B Coulomb probe whose two legs must converge to the same reciprocal sum, and its values are converted to atomic units at the handoff. read_charge is not read for QM/MM: the classical stage supplies the field at every step, so both values are cleared without a warning. The classical provider owns the cell; a quantum region is cut out as a cluster and receives none.

q_qm holds the zero-order charges of the quantum atoms in highlevel order, one finite value per atom, and a list of any other length is refused; the key is refused under several centres. A backend that returns computed charges overrides q_qm at run time. qm_contains_field_gradient states who carries the field force: true, the default, means the quantum gradient already contains it; false means the composite adds the removed term back onto the quantum rows.

The point-charge interaction inside the cell is removed analytically, and the non-Coulomb part of the quantum-quantum interaction is removed by cloning the quantum LAMMPS types on the classical side, which leaves the QM-MM and MM-MM terms untouched. Supported jobtypes are energy, gradient, optimize including optimize_cell, md in the nve, nvt and npt ensembles, surface_hopping, adaptive sampling and the constraint block. A Hessian is numerical only, with hess_driver main or the num_hess and num_freq tokens; an analytic matrix is refused.

```
control:
  title: qmmm_water_box
  jobtype: energy gradient
  theory: [molcas, lammps]
  multiscale_type: qmmm

molecule:
  charges: 0
  multiplicity: 1
  ci: "2 1"
  spin: "0 1"
  coupling: "1 2"
  highlevel: [1, 2, 3]
  q_qm: [-0.83, 0.415, 0.415]
  embedding: true
  pbc: [1, 1, 1]
  cell: [8.399091, 8.399091, 8.399091, 90.0, 90.0, 90.0]

molcas:
  method: caspt2
  basis: ano-l-vdzp

lammps:
  method: ff
  lammps_project: water_box
```

7.2. ONIOM2 and ONIOM3

Reads the theory queue as a stack, written high to low. Two entries select ONIOM2, spelled by multiscale_type oniom while an empty value keeps the legacy meaning, and three entries always select ONIOM3. highlevel and midlevel are disjoint 1-based lists, and the working regions are derived: region_h is highlevel, region_m is highlevel union midlevel, and region_l is the whole system.

The two-layer energy is E_L(real) plus, per centre, E_H(model) minus E_L(model), the pattern +1, +1, -1. The three-layer energy adds the middle rung E_M(I) minus E_L(I), giving +1, +1, -1, +1, -1. Each term scatters its gradient to its own atoms before the coefficient applies.

embedding true computes the environment charges. High-level charges come only from theory[0]; mid-level charges come only from theory[1] and only on the disjoint middle shell; the low level and the corrections never update charges. read_charge true loads {title}.charge instead, whose first line declares point angstrom external, point angstrom atom or field au, one kind per file.

A charged centre of a three-layer run keeps its net charge in the high region: the middle shell inherits the centre charge one to one, so it has to be neutral and supplies the field only. A static charge file under several ONIOM2 centres is refused, as is a two-layer run with a classical outer method and embedding true, which points to qmmm.

7.3. Multi-centre plans

Several centres are written as one region per centre, with a semicolon between centres in charges, multiplicity, ci, spin and coupling and a comma between coupling pairs. Each centre keeps its own state ladder, and centre_groups is refused with a partition. The composite section sets execution, not physics: executor is serial or thread and max_workers sizes the pool.

sc_charge true needs two centres and embedding, and repeats probe and energy-only charge rounds until the largest per-centre move is under sc_charge_threshold or sc_charge_maxiter rounds have run; sc_charge_damping weights the fresh fit.

A partitioned run writes the global md files and, beside them, md.1.xyz, md.2.xyz and the rest, with a 1-based centre index. The optimizer boundary is narrower than the electronic one: it reads the global state axis and has no multi-centre higher-derivative contract, so a multi-centre optimize with a Hessian is not supported, while the framework numerical batch still covers a multi-centre saddle.

7.4. Embedding charges and self-consistency

point_charges is a whole-system array of [q, x, y, z] rows, one per atom in framework order, with the charge in elementary units and the position in Angstrom. Row i is atom i, and a length that disagrees with the atom count is refused.

external_charges names a legacy charge file instead, and is a standalone form: no highlevel or midlevel region and no periodic cell, because the file cannot be replicated across a boundary. It

excludes read_charge. The two differ in time: read_charge loads a static field once, while embedding recomputes the charges at every step.

Machine learning meets the same distinction at one rule. A model learns the map from a geometry to an energy, so the charge that enters a label must be fixed and declared. A static field meets the rule and the criterion judges a run trained under it; a computed field is read as not judged. The adaptive reference queue labels from the layer covering the neural-network surface.

7.5. Limitations

The table lists the refused and unimplemented combinations with the reason for each.

Combination	Status	Why
qmmm with three theory entries	Refused	three entries always select ONIOM3
qmmm with a reference queue	Refused	both entries are used at every geometry
q_qm under several centres	Refused	each centre is fed by its own fit
sc_charge with one centre or no embedding	Refused	the iteration exists for the other centres field
ONIOM2 with a classical outer method and embedding	Refused	use qmmm, whose field channel replaces it
Multi-centre ONIOM2 with read_charge	Refused	one static file has no per-centre binding
Analytic Hessian on a multiscale run	Refused	one geometry has one composite curvature
Per-term remote execution of QM/MM	Refused	the field handoff is local to the evaluation host
Multi-centre optimize with a Hessian	Not supported	no multi-centre higher-derivative contract
Link atoms, overlapping centres, mixed providers	Not implemented	a bond across a partition is not capped
Stress of a QM/MM box	Approximate	the cell response of qmmm_el and U_gas stays at point-charge level

8. Periodic systems

A periodic run adds a cell to every evaluation. The cell is declared once on the molecule section and travels with the coordinates of that evaluation, so a force, a virial and a recorded frame always describe the same box. The workflow does not change: the optimizer, the integrator and the composite layer ask for an energy and a gradient for a system that has a lattice.

8.1. Cell, lattice and periodic flags

pbc holds exactly three 0 or 1 flags and is the single periodicity authority. A partial spelling such as [1, 1, 0] is refused at the framework layer and again by the CP2K input writer, so only full 3D periodicity or a molecular run exists. A non-zero lattice no longer implies a periodic run, and a periodic run needs a real cell: with pbc on and no usable lattice, validation refuses the input.

cell takes six parameters, a b c alpha beta gamma in Angstrom and degrees, or nine matrix components. lattice_vector is the runtime authority, a 3x3 matrix of row vectors with R = S @ B between Cartesian and fractional coordinates, and lattice is the classic six-parameter spelling of the same thing. The six-parameter default 99 99 99 90 90 90 is the not-filled sentinel: no lattice is derived from it, so a molecular run never carries a fake 99 Angstrom cube, and a periodic run that leaves it has no cell. A cell and a lattice_vector that disagree are an error rather than a silent overwrite.

Minimum-image work is derived, not stored. Wrapping happens after position propagation and accumulates the crossings in image, and the continuous view is rebuilt from those counts, so a molecule that straddles a face is cut out whole for a region task. The minimum-image displacement is a certified search: it widens beyond the 27 nearest cells when a tilted shape demands it and raises rather than shortening the vector.

kpoint sets the k-point grid, which the backend reads. Every evaluation receives one frozen snapshot of coordinates, crossing counts and cell, and a region task is evaluated without the cell because a molecular engine gets a cluster.

8.2. Stress and variable-cell optimization

stress is a derived capability, asked for only in two situations: with ensemble npt, where the barostat needs the internal pressure, and with optimize_cell true, where the search needs the cell conjugate. Only the whole-system provider has to answer it. A backend reports its own declared unit, and the framework converts once to Hartree/Bohr^3 with tension positive; md.stress writes the nine components in that unit with the derived pressure in bar. The optimizer criteria are the pressure pair opt_csrms and opt_csmax in bar and the strain pair opt_cerms and opt_cemax, dimensionless.

opt_pressure, in bar, makes the optimizer objective an enthalpy, and opt_cell_coupling says how many directions that pressure may move: isotropic projects the lattice half of each step onto the shape-preserving direction and judges the mean normal stress, while anisotropic, the default, leaves all six relaxed components free. The conjugate is V sigma^T H^-T and not the volume times the stress, and the atoms ride the box: the fractional coordinates are held while the cell stretches.

Several combinations are refused rather than relaxed silently. opt_pressure or opt_cell_coupling without optimize_cell is refused. A backend that declares no stress raises by name, so a run cannot report cell rows for a box that never moved. A periodic run whose outermost layer is a molecular program is refused at validation. optimize_cell with a vibrational analysis is refused, because a lattice entry has no mass; a saddle with a backend Hessian is refused, because six

zero rows are six directions of zero curvature. A deformation beyond what a lattice can carry raises instead of drifting into a NaN box, the cumulative strain bound is ln 1.5, and an anisotropic commit re-triangularizes the deformed lattice.

8.3. Dynamics under PBC

NVE and NVT run at a fixed cell, and NPT adds a barostat that integrates the cell. The thermostat and barostat are a literature pairing, not a free grid: parrinello-rahman and mttk accept only the Nose-Hoover family, because the chain on the particle momenta is part of their derivation, while berendsen and scr accept any thermostat other than off. scr is isotropic only, since its tensor form injects shear strain that a liquid cannot store.

The run writes md.stress only when a backend supplied a stress tensor, and md.ensemble only for an extended ensemble. The cell of a frame is written in the Lattice attribute of the md.xyz comment, while image and pbc are not written to xyz at all: a reader that needs them reads the extended record or the checkpoint. An engine that integrates its own box reports it back through fin_cell, and the framework re-derives cell, lattice and volume from it so the next single point is not run on a stale box.

The cell history is recomputed from the products by tools/analyze_npt_frames.py, which reports the frame count, the volume excursion against the first frame, the accumulated rotation of the simulation frame, the extremes of the strain and its momenta, the kinetic, virial and total pressure and the instantaneous temperature, naming the file each number came from. It reads products only.

8.4. Which engines support PBC

Support is declared in each manifest as supports_pbc, with the stress unit beside it. A method that only serves a centre region may be molecular even when the environment layer is periodic, because the runtime cuts that task out of the continuous view and clears its cell.

Engine	Supports PBC	Stress	Notes
pyscf	No	No	Molecular only; frequency capability but no box
molcas	No	No	Molecular only; ESPF embedding is a cluster run
bagel	No	No	Molecular only
orca	No	No	Refused for periodic cells; molecular only
cp2k	Yes	GPa, converted	Analytical tensor, plus a numerical strain route for excited states
xtb	GFN-FF only	Hartree/Bohr^3	gfn0, gfn1 and gfn2 are molecular; GFN-FF reads gradlatt
lammps	Yes	bar, converted	Classical provider and the outermost layer of periodic composites
qdm	No	No	Molecular only

Engine	Supports PBC	Stress	Notes
nn_pes	No	No	Refuses any pbc: features are molecular

9. Constraints

The constraint block adds potentials and geometric conditions to a run without touching the electronic structure calculation. It is closed by default: an empty block builds no plan and changes no number. The block is written under molecule.constraints in YAML, as a top-level constraint or system_constraint section, or as the classic &constraint section.

9.1. Constraint families

Five kinds of action are organized together and implemented separately. A bias adds a harmonic or flat-bottomed potential on a bond, an angle, a dihedral or a named collective variable; it never moves a geometry. A wall adds a confining potential around a sphere, an ellipsoid, a box or a triclinic cell, and acts on each atom or on a group centre. A fix removes selected Cartesian components from the motion and restores the anchors it captured. SHAKE projects the geometry onto a manifold of fixed bond lengths, angles or dihedrals and carries no force constant; its RATTLE option projects the velocities as well. Metadynamics accumulates Gaussian hills along a collective variable or an aligned RMSD, so the surface a trajectory sees depends on where it has been.

The families act at two points of a step, and that placement is physical, not an implementation detail. Hard projection and freeze happen between the position update and the electronic evaluation, so the reported energy belongs to the projected geometry. A bias, a wall and a metadynamics hill are additive, so they are applied in the calculator as the result is finalized: later would miss the velocity kicks.

A collective variable is a named coordinate, not a potential. Three kinds exist, bond, angle and dihedral. A CV may be referenced by several biases and by a metadynamics entry, and the same atoms may be named inline instead.

9.2. Writing the block

The key tables, their defaults and their one-line descriptions are in Section 5.13. The block has eight subsections: units, groups, cvs, biases, walls, fixes, constraints, metadynamics. units applies to the whole block and is converted once when the plan is built: a force constant divides by the square of the length factor. Every atom number in the block is 1-based in all carriers. The example declares one group, one variable, one ramped bias, one wall, one fix, one RATTLE bond condition and one metadynamics entry.

```
molecule:
```

```
  constraints:
    units:
      length: angstrom
      angle: degree
      energy: hartree
    groups:
      - name: solute
        atoms: 1-6
    cvs:
      - name: oh
        kind: bond
        atoms: [1, 2]
    biases:
      - id: hold_oh
        cv: oh
        target: 0.96
        force_constant: 0.02
        force_constant_start: 0.02
        force_constant_end: 0.2
        force_constant_delay: 100
    walls:
      - id: cage
        group: solute
        geometry: sphere
        center: [0.0, 0.0, 0.0]
        radius: 8.0
        force_constant: 0.1
    fixes:
      - id: anchor
        atoms: [1]
        direction: xyz
    constraints:
      - id: oh_bonds
        kind: bond
        bond_type: O-H
        tolerance: 1.0e-8
        velocity_constraint: rattle
    metadynamics:
      - id: explore
        cvs: [oh]
        height: 0.0005
        width: [0.05]
        pace: 100
```

The classic spelling carries the same actions as one flat list of keyword lines. Object lines declare what the action acts on, parameter lines follow in the same order, and a semicolon separates objects on one line while a comma separates fields of one object.

```
&constraint
  unit_length angstrom
  unit_energy hartree
  group_name solute
  group_atoms 1-6
  cv_bond 1 2
  cv_name oh
  bias_cv oh
```

```
bias_cv_ref 0.96
bias_cv_fc 0.02
wall_type sphere
wall_sphere_radius 8.0
wall_group solute
wall_fc 0.1
fix_atoms 1
shake_bond 1 2
shake_vel rattle
meta_cv oh
meta_height 0.0005
meta_width 0.05
meta_pace 100
```

9.3. Where constraints act

Support is a property of the workflow: a condition needs a step, and a schedule needs a clock. The table reads yes where the workflow honours the family and static where the potential is present but nothing propagates it.

Jobtype	fix	wall	bias	metadynamics	SHAKE
md, nve and nvt	yes	yes	yes	yes	yes
md, npt	yes	yes	yes	yes	yes
optimize	yes	static	static	not claimed	yes
adaptive	inherits	inherits	inherits	inherits	inherits
single point	no	static	static	no	no
hessian, search, read	no	no	no	no	no

An optimization honours a fix as a mask on the step and a strict condition as a projected step, and judges convergence on the tangential gradient. Adaptive sampling runs its trajectories through the MD workflow, so they carry the MD set, while its labels come from the base electronic result and a wall or bias energy never enters a training set. MECI, MECP, saddle and band searches do not claim the block, and a constrained surface hop is not supported because a hop rescales velocities inside the feasible space.

9.4. Refused combinations

A declaration that cannot be served is refused at input rather than approximated at run time.

A soft wall in a periodic system is refused. Its law is a spring toward one point, so there is no shell to check against a cell face, and a wrapped coordinate jumps its offset by a whole lattice vector on every crossing. A bias region of the inside or outside kind is refused, and so is a one-sided upper or lower region on a dihedral: a one-sided region of a periodic coordinate needs a declared origin and direction. A collective variable whose atoms are not in one connected component is refused when the run is periodic, because there is no image rule to derive for it.

A ramp interpolates linearly between declared endpoints. _scale multiplies the declared value and cannot be combined with _start and _end. _delay and _stop bound a ramp and are refused when no endpoint exists, and endpoints must have the shape of the value they replace. A quantity that has to stay positive, such as a force constant or a radius, may not ramp to zero or below; the endpoint is refused at input and the same check runs at every step. A wall power below 2 is refused because the potential would not be differentiable at the wall. bond_type perceives the bond graph and expands bond pairs only, so it is refused for an angle or a dihedral condition.

The flat restraint keywords of the previous version, among them freeze, cbond, cavity, shape, factor and compress, are refused by name with the replacement spelling, and Appendix D collects these refusals beside their modern spellings.

10. Machine learning potentials

This chapter covers the machine-learning side: the label file, training and prediction, the hyperparameter scan, and the adaptive loop that uses a reference engine to improve a potential where it is weakest.

10.1. Training data

A training run starts from one label set: a JSON mapping of geometries and their computed values. train_data names the file and train_dataset hands the same mapping inline; both reach one reader and pass the same checks.

Key	Shape	When it is needed
xyz	(nframe, natom, 3)	always; rows carry the symbol or not
symbols	(nframe, natom)	when the xyz rows carry no symbol
energy	(nframe, nstate)	the energy role
grad	(nframe, nstate, natom, 3)	the mlp_eg and mlp_g2 families
nac	(nframe, nnac, natom, 3)	the coupling roles
soc	(nframe, nsoc)	the soc role
transition_dipole	(nframe, npair, 3)	optional; carried, not fitted
osc_strength	(nframe, npair)	optional; carried, not fitted
natom, nstate, nnac, nsoc, npair	counts	optional; derived from the arrays
unit	channel to unit spelling	optional; checked

A single-state file may omit the state axis of grad and a single-pair file the channel axis of nac. The canonical file also carries a format name, a version and any per-frame field the run produced (charge, cell, pbc).

```
{
  "format": "pyrai3md-training-data", "version": 1,
  "natom": 3, "nstate": 2, "nnac": 0, "nsoc": 0, "npair": 0,
  "xyz": [[[0.000000, 0.000000, 0.000000],
```

```
          [0.000000, 0.000000, 1.889726],
          [1.700000, 0.000000, -0.500000]], "..."],
  "symbols": [["O", "H", "H"], "..."],
  "energy": [[-76.0201000000, -75.9011000000], "..."],
  "grad": [[[[0.0, 0.0, 1.204e-03], [0.0, 0.0, -6.02e-04],
             [0.0, 0.0, -6.02e-04]], "..."], "..."],
  "unit": {"energy": "hartree", "gradient": "hartree/bohr"}
}
```

Energies are Hartree, derivatives Hartree per Bohr, spin-orbit couplings cm-1. The reader puts every derivative on the Angstrom axis the network is fed, once, so energy and gradient describe one surface; a spin-orbit value is not a derivative and gets no length factor. There is no unit keyword, because a switch could state a scheme the numbers do not follow; the optional unit block is checked instead.

Labels come from finished calculations. A jobtype read run starts no engine: it parses the read paths with the single theory skill and returns the frames. Section 10.6 wraps it with extract, shuffle, split, merge and select. A channel is written only when every selected frame carries it.

10.2. Training

A training run is one call of the nn_pes skill under a profile ending in _train: the skill writes the data and hyperparameters, runs one fit per member, waits and reads the result back. The framework makes the call once and states whether the record arrived; no geometry is evaluated, so the log carries a training report only.

modeldir names the durable folder; one subfolder per role lands inside it, and each role reads its hyperparameters from a section named after the network family.

Role	Folder under modeldir	Hyperparameter section
energy and gradient	energy_grad	eg
nonadiabatic couplings	nac	nac
spin-orbit couplings	soc	soc

mult_nn is the ensemble size and the axis every hyperparameter is written against: one value for the ensemble or one per member, semicolon in the classic text and outer list in YAML, and fewer groups than members is refused. gl_seed seeds the fit, each member receiving gl_seed plus its index, because a child inherits no random state.

train_mode selects the section the fit reads: training fits a new folder, retraining continues from the weights on disk and cannot change the architecture. A member folder holds hyper_v<i>.json, scaler_v<i>.json, weights_v<i>.weights.h5, the fiterr arrays, fitlog_<i>.txt and fit_stats/.

The completion certificate is fiterr_valid_v<i>.npy, written last: initialised weights are written before the fit starts, so without it a folder loads and predicts noise. The report multiplies the stored error by the folder's scaler spread and prints eV, eV per Angstrom and cm-1.

Note. *A fit has no numeric convergence verdict: the framework judges whether the record arrived, and whether the error suits a dynamics run is the caller's judgement.*

10.3. Prediction

A prediction run serves a trained folder over a file of geometries that carry reference values. pred_data names that file, and the skill repeats its serving call over every frame; an absent channel is absent rather than zero.

The model folder computes every number. Four statistics are reported per channel: mae, rmse, the largest absolute deviation, and spread_max, the largest member disagreement. spread_max is a disagreement, not an error, and no relative figure is reported, because a derivative component can be zero.

Two raw products are written in the run directory: {project}_{profile}.{channel}.pred.txt, four column blocks in one unit each (reference, prediction, absolute deviation, spread), and {project}_{profile}.frames.txt. Printed values are eV, eV per Angstrom and cm-1.

nn_predict_mode selects the answering member: zero is the member mean, k a one-based member, and the others are mirrored into alternative channels. Fewer than two members refuses a non-zero mode.

10.4. Hyperparameter search

A grid search varies the training of one model family over a declared grid and ranks what came back. The skill says which keywords may be varied and what a candidate means; the run says which values to try. The grid maps keyword to candidate list and the scan is their Cartesian product, with no start, stop and step generator.

Each point is stamped on a copy of the parent state with the child jobtype train and an absolute model directory. Its key joins its values in keyword order by a tilde, as in depth=1~nn_size=24; its folder is NN-<title>-<key>.

Keyword	Role in the scan
grid	keyword to candidate list; an empty grid is refused
retrieve	read the folders a previous scan left and rank them; submit nothing
sort_by	channel the ranking reads; default the profile's first channel
sort_stat	max ranks on the worst member, mean on the average
sort_order	ascending or descending
grid_root	root of the scan directories
max_grid	refuse a scan larger than this; zero states no limit

A complete folder is skipped, read back and marked, so a rerun fills in what is missing; a scan planned under retraining is refused, because a continuation cannot apply the architecture the grid varies. summary.csv holds the ranking and crashed.txt the points with no record.

The scan ranks and does not judge: no threshold keyword exists, because a validation error is not a convergence criterion. One skill per scan.

10.5. Adaptive sampling

Adaptive sampling is a round-based loop that retrains the potential on what it got wrong. Iteration 1 is the external start: an external data set and network run the dynamics and the screens are computed, with no reference and no training. Each later round labels what the previous one selected, appends the labels, fits a network and runs the dynamics with it.

```
iteration 1   external data + external network -> MD -> screen
               -> iter-1/selected_iter-1.xyz

iteration k   reference labels on iter-(k-1) selection
               -> data_iter-<k>_<nframe>.json
               -> train NN-<title>-iter-<k>
               -> MD with that network -> screen
               -> iter-<k>/selected_iter-<k>.xyz
```

The terminal round still labels, trains and runs dynamics, and its own selection has no consumer; maxiter counts the rounds. sample_energy and sample_grad are the baseline and stay on, while sample_nac, sample_soc, sample_dipole, sample_transition_dipole and sample_oscillator_strength are switches; Section 5.7 lists their defaults. A channel that is off has no row in the sampling table.

Screening reads each outcome from its history over a window of record_step steps. The deviation is the ensemble standard deviation, read over states for the energy and over states, atoms and components for the gradient. A frame above a max threshold is a trigger, one above the matching min threshold joins the pool, and candidates are taken newest first up to maxsample per trajectory. Every row is kept in status_iter-<k>.json.

Two checks promote a frame. check_population collects the structure a dynamics run stopped on when the population went ill; the parent only decides whether it is worth labelling. check_energy_order promotes a frame in which a same-spin level lies below an earlier one by more than the tolerance, the signature of a crossing the dynamics missed.

maxmae arms the training-data quality gate. When the fit's validation energy error is above the ceiling, the frames this round appended are tested against the frames held before it, one state at a time, with a robust z score. A flagged frame is removed from a candidate set, a fresh network is fitted, and the removal is kept only when the error strictly falls; otherwise the run stops.

Four analysis switches read the round without changing it and are closed by default: analyze_pes for local curvature and a low-rank gradient reconstruction, analyze_gap for gap statistics, analyze_trace for scalars along a trajectory, and analyze_coverage for covered descriptor space. Two exploration switches add a bias: explore_udd builds a potential or force bias from the member spread (udd_mode, udd_amplitude, udd_width, udd_start_iteration), and explore_gap pushes a chosen pair apart (gap_method, gap_pair, gap_threshold), its antisymmetric term leaving the pair mean unchanged.

Products land under <title>.adaptive/: md/traj_<idx>/ and selected_iter-1.xyz in iteration 1, then the unpruned data snapshot, the model folder with its log, reference/ref_<idx>/, new-samples.json, analysis_iter-<k>.json and prune/ for a gate retrain. Section 5.7 lists the keywords and Section 13.5 walks a run through its rounds.

10.6. Data tools

tools/training_data.py turns finished calculations into a maintained label set: extraction is the framework feature jobtype read, and the tool adds the filtering, the statistics and the file.

Command	What it does	Output
extract	runs jobtype read over a read input and keeps the requested channels	data.json
shuffle	permutes the frames with a seed; an optional group count shuffles a prefix separately	data-shuffled.json
split	shuffles, then writes a prefix and the remainder	data-set-1.json, data-set-2.json
merge	shuffles the first set and appends the others	data-merged.json
select	keeps one-based indices and inclusive ranges	data-selected.json
describe	prints counts, shapes and units	no file

```
python tools/training_data.py extract read.yaml -o egs.json
python tools/training_data.py shuffle egs.json -s 1234
python tools/training_data.py split   egs.json -p 200 -r 0
python tools/training_data.py merge   egs.json extra.json
python tools/training_data.py select  egs.json --index 1-100,150
```

11. Parallel and remote execution

This chapter describes how a workflow spreads its work across processes and machines, how a scheduler submission is configured, and how a runtime file makes a finished or interrupted run restartable.

11.1. The two dispatch axes

Two orthogonal switches and one pool size decide how a workflow spreads its work. multitask runs the units of one workflow as a spawn process pool on this machine; remotetask submits each unit as its own child runtime, normally with Slurm; multipool is how many workers run at once. The three are written in the workflow section that fans out, never in a calculation section.

At the workflow level all three are tri-state: an absent value leaves the decision to the skill sections and then to the workflow class default. A skill section declares its own value, and planning aggregates the declarations of every skill in one role. All unset means false, true beside unset

means true, false beside unset means false, and true beside false is refused as a conflicting configuration. A node-chain declaration outranks both, so a workflow whose units are its own steps stays serial whatever the section says: the adaptive and the single-structure optimize nodes force multitask false, while the md, hessian, path, search, train and prediction nodes take the keyword.

Adaptive sampling runs its stages one after another, so the node itself is pinned serial and each stage carries its own triple. md_multitask, md_remotetask and md_multipool steer the dynamics stage and reference_multitask and reference_multipool the labelling stage; a stage value that is left empty falls back to the generic triple. The reference stage never submits a framework runtime: its remote switch turns each reference skill into its own Slurm client instead, and the training stage uses remote_train. The parent executes one stage at a time, so the stage flags change the width of a stage and never the order of the rounds.

A workflow section cannot carry thread, nproc or a skill-prefixed spelling of them. A workflow parallelises with processes, and threads belong to the calculation, so the keyword is written in the skill section where the engine actually runs; the misplaced key is refused by name and the message names the section that owns it.

Workflow	What one unit is	Keyword that widens the batch
md	one trajectory of a multi-trajectory run	&md multitask, &md multipool
optimize	one evaluation of the loop, which stays serial	none; the node is pinned
path	one image of a band, once per round	&optimize multipool
hessian	one displaced geometry of the finite-difference batch	&hessian multitask, &hessian multipool
search	one grid point, dispatched as a training child	&search multitask, &search multipool
adaptive	one trajectory or one reference frame, per stage	the md_ and reference_ triples
train, prediction	one skill call, so the batch has one unit	the flags reach the node but have one unit to spread

Inside one evaluation the backend terms are a second, finer axis. A multi-trajectory dynamics run and a single-structure optimization pin the theory role serial; the hessian, path, search, train and prediction nodes let the skill-section tri-state through. Per-term remote execution is selected from the system section or from a skill tri-state, never from the workflow section.

11.2. Slurm child runtimes

A remote unit becomes one submission. The framework creates runtime/<skill>_runtime_<idx>/ under the working directory, serializes the child state into a .rt file there, writes slurm.sh and submits it with sbatch from that directory, so the child runs with the directory as its working directory.

```
runtime/
  nn_pes_skill_runtime_3/
    nn_pes_skill_runtime_3.rt     serialized child PyRAIState
    slurm.sh                      #SBATCH lines, shell, exports, launch
    result.json                   written by the child
    slurm-<job>.out, slurm-<job>.err
```

The script holds, in this order, the directives the slurm section renders, the environment shell commands, the exported variables with the package path prepended to PYTHONPATH, and the launch line python3 -m pyrai3md <name>.rt. The parent then polls sacct until the job reaches a terminal state, reads result.json and returns it. A missing or unparsable result is reported with the tail of the child's slurm-*.err, because that file is the only view the parent has of what went wrong on the node.

The slurm section supplies the allocation: partition, nodes, task and CPU counts, memory, wall time, job name, output and error files, account, quality of service, constraint, mail events, array and gres. Section 5.12 lists the keys and their defaults, an empty value writes no directive, an unknown key is refused by the script builder, and extra is the only pass-through. gres requests generic resources such as a GPU, which is how a training job asks the scheduler for the device its gpu keyword selects.

Two submission routes share that section and differ in who owns the job. A framework runtime is submitted by the parent: the whole child state travels in the .rt file, and the child writes result.json. A self-submitting skill writes its own script and returns a result; an adaptive reference stage and a training run whose environment selects the slurm execution mode take that route, the training arm submitting one blocking job per ensemble member. Per-term remote workers are the finest route: one backend term becomes a .rtt payload and answers with term_outcome.json.

> **Note.** *max_in_flight and environment.scheduler are pmd_env settings, not run input. The first bounds how many submitted jobs a qualification round keeps in the queue, the second names the scheduler that round talks to; neither is read from an input file.*

11.3. Runtime files and restart

A .rt file is not another input format; it is another source of the runtime state. Launching pyrai3md <title>.rt loads the state and enters the unchanged execution path, so interface parsing, schema validation and planning are bypassed. Every dispatched child leaves its own .rt beside its result.json, which is what makes a child reproducible without its input file.

A .rtt file is the per-term analogue: one backend term, its frozen geometry and electronic request, and one term_outcome.json back. It runs one term and never the whole workflow, so it can neither sum a composite nor advance the parent state.

md.checkpoint names the cadence of the runtime file. Zero, the default, writes none; a value of n writes one every n steps and always on the last step, stamped with the generator state at the moment the geometry and the history were written. The trajectory frame set follows the same arithmetic, so a resumed run continues the file instead of repeating a frame.

A restart preserves the coordinates, velocities, electronic state, history frames, constraint plan, task queue, accumulated results and the random number state, so a resumed trajectory continues the draw the first process was making instead of reseeding. It does not preserve the backends: loggers, file handles, caches and the live skill objects are excluded and rebuilt when the calculator initializes. It also does not re-read the input file, so an edited keyword has no effect on a resumed run; changing a setting is a new input, not a restart. A file whose stored partition fingerprint no longer matches the state it carries is refused rather than resumed.

12. Output and analysis

This chapter describes what a run reports: the log and its levels, the trajectory and status files a workflow writes, and the tools that read them afterwards.

12.1. The run log

The run log is <title>.log in the run directory. A dispatched child restamps its logger into its own unit workspace, so two processes never append to one file. The log opens with the start line and the version banner, then echoes the input as it was read: the control, system and environment sections, the sections the jobtype reads, and the skills that answered.

Every step writes a block in one grammar: a step header, the energies, and then whatever the verbose level allows. The levels are cumulative. Level 0 keeps the progress and error blocks; 1 adds the structures; 2 adds the property arrays and is the default; 3 adds the Hessian and an explicit not-computed marker for every absent property; 4 or more also dumps the per-step surface-hopping amplitudes to <title>.fssh. silent governs the screen rather than the file: at its default the log is the only destination, and zero echoes the same blocks to the terminal.

The last line is PyRAI3MD End: <timestamp> Total: <elapsed> Status: <status>. A reader who only tails the file can therefore tell whether the run succeeded, what it ended with and how long it took.

An optimization writes &optimization setup once, with the method, the optimizer, the step bound and the cell settings, then &optimization progress per step with each criterion beside its threshold and whether it is met, and closes with a banner sentence: converged in n steps, or did not converge after n steps with the reason. The path family writes the same two blocks. Its setup rows name the band size, the spring, the climbing image and the IRC direction, and its progress rows carry the arc, residual and radius of the point or of the round.

A grid search writes &grid search before the fan-out, with the number of points and one row per planned point, and &grid search summary afterwards, with the ranking, a unit row per column and a dash for a channel a point does not carry. An adaptive run adds &data and &nn training per later round, &nn validation gate when the quality gate is armed, &adaptive sampling per round, and a closing &adaptive sampling summary with one trend row per iteration. A training run prints &nn training with &nn validation mean absolute error; a prediction run prints &nn prediction error analysis, headed by the test frame count and the raw comparison files, and one table per statistic after it. Both end with a banner naming the record that came back.

12.2. Result and trajectory files

Section 6.8 lists the files each workflow leaves. The dynamics products need a column-by-column reading, because they are the ones a reader parses years later.

Every frame of <title>.md.xyz and <title>.md.velo carries the comment line <title> coord <step> state <k>, extended with to <m> CI on a step that hopped. Geometries are Angstrom and velocities atomic units. A periodic geometry frame also carries Lattice="a b c alpha beta gamma", because the coordinates are the wrapped main image and the cell is the missing fact.

<title>.md.energies holds one row per recorded step: the time as (step - 1) times the time step in atomic units, the potential energy of the active state, the kinetic energy, their sum, and then the energy of every electronic state. All energies are Hartree. The potential, kinetic and total columns describe the state the step ran, so they are not the sum of the state columns.

<title>.md.pop holds the same time column, the population of every state and a zero-or-one vector that marks the active state after the hop. A run that never propagates a density writes no population rows. <title>.md.stress appears only when a backend supplied a non-zero whole-system stress, with nine tensor components and the derived pressure in bar; <title>.md.ensemble appears only for an extended ensemble, with the per-step thermostat and barostat record.

<title>.md.charges carries the managed point charges [q, x, y, z] of each recorded frame and exists only when charge management is bound. A partitioned run adds one set of per-centre files, <title>.md.<n>.* with a one-based centre index, whose frames hold the atoms of that centre only, while the global files carry the whole system. The <title>.sh.* family is the same matrix for hopping steps only and is written whenever a hop happened, outside the direct and buffer cadence that thins the md.* records.

12.3. Trajectory and structure tools

Four command line tools read finished products. They write in the directory they are run in and never into the trajectory they read.

Command	Reads	Writes
traj_analyzer.py analysis.in	trajectory directories, one keyword per line in the input	<title>.traj.log, and the products of the mode
traj_generator.py sampling.in	a frequency or initial-condition file and the program templates	one directory per trajectory, runset-<n>.sh, runall.sh
multiscale_generator.py creation.in	solute, solvent and the packing parameters	the combined environment and its initial conditions
analyze_npt_frames.py --root <tree>	md.xyz cell comments and md.ensemble columns	nothing; the report goes to the terminal

tools/traj_analyzer.py walks one stage at a time. Mode 1 counts rows per trajectory and writes the complete list; mode 2 measures energy drift and writes the conserved list with a per-step drift

file; mode 3 extracts frames to JSON and snapshot files; mode 4 averages populations and writes the hop energy gaps; mode 5 classifies snapshots by geometrical parameters; mode 6 writes plot data; mode 7 fetches single trajectories; mode 8 reads initial, final and hopping snapshots. The reader for this program is the default, so the legacy programs are reached by naming them.

tools/traj_generator.py samples initial conditions and builds one calculation directory per sample, each with its own script, for a list of programs including this one; the number of trajectories is the node count times the job count, and an index continues an existing set instead of overwriting it. tools/multiscale_generator.py builds a solvent or aggregate environment, prepares the classical files and merges initial conditions, in one of seven modes from create to mm. tools/analyze_npt_frames.py recomputes the cell history of a variable cell run from the frame comments and the ensemble columns and reports the volume excursion, the accumulated rotation of the simulation frame, the strain extremes, the pressure components and the temperature; it writes nothing.

```
python tools/traj_analyzer.py analysis.in
python tools/traj_generator.py sampling.in
python tools/multiscale_generator.py creation.in
python3 tools/analyze_npt_frames.py --root run_tree --detail
```

A trajectory analysis is usually a chain: mode 1 keeps the complete trajectories, mode 2 removes the ones that lost energy, and the later modes read the resulting lists instead of the whole set, so every average describes the same set of trajectories.

13. Examples and tutorials

This chapter works from the inputs that ship with the package. The examples directory is the reference for every workflow keyword combination that the released version supports. Reading one example beside Section 6 is usually faster than composing an input from the keyword tables alone.

13.1. The shipped examples

The directory holds 213 YAML inputs and 254 files in total. Almost all of the extra files are companion data that one or more inputs read: xyz endpoints for a band, a charge file, the LAMMPS topology triple of .data, .in.init and .in.settings, an .init.xyz batch of starting points, or a JSON training set. Each YAML input is runnable as it stands, from that directory, with pyrai3md followed by the file name.

The files are grouped by the prefix before the first underscore. The table counts every file of the family, inputs and companions together, and gives the number of YAML inputs in parentheses where the two differ.

Family	Files	What it demonstrates	Representative files
path	48 (32 inputs)	Path searches: minimum energy path, intrinsic reaction coordinate and nudged elastic band walks, in the gas phase, in a periodic cell, and across a composite; the shipped endpoint files carry the reactant, product and transition-state guesses.	path_neb, path_irc, path_mep_pbc, path_cineb, path_neb_xtb
multiscale	41 (37 inputs)	Two- and three-layer composites: ONIOM2 and ONIOM3 with mechanical or electrostatic embedding, additive QM/MM, charge management from a file, and the composite versions of the searches (mecp, meci, mep, neb, ts).	multiscale_qmmm_embedding_sp, multiscale_oniom2_orca_embedding_sp, multiscale_oniom3_orca_embedding_npt, multiscale_qmmm_neb, multiscale_qmmm_ts
constraint	37	The constraint block in every role: harmonic and flat-bottomed biases, confining walls, fixed atoms and directions, SHAKE and RATTLE, metadynamics including the RMSD form, and the same terms inside a periodic NVE, NVT or NPT trajectory.	constraint_md_bias, constraint_md_shake, constraint_md_periodic_npt_wall, constraint_md_metad_rmsd, constraint_opt_surface
multicenter	30	Several QM centers in one system: one high-level region per center, the per-center state ladders, the fitted charge rounds, and the periodic box that carries all of them.	multicenter_qmmm_sp, multicenter_oniom3_orca_embedding_npt, multicenter_pbc_nve, multicenter_qmmm_sc_charge, multicenter_qmmm_num_freq
md	30 (27 inputs)	The trajectory itself: NVE, NVT and NPT with each thermostat and barostat pairing, surface hopping from four engines, velocity handling, batch and parallel dispatch, and integer-step restarts.	md_local, md_bagel_fssh, md_lammps_nvt_nhc, md_lammps_npt_nhc_mttk, md_parallel
optimize	25	Single-structure searches: minima, transition states, crossings of two surfaces, GDIIS, partial optimizations that move a subset of atoms, and variable-cell relaxation under an external pressure.	optimize_minima, optimize_ts, optimize_cell_gdiis, optimize_cell_meci, optimize_partial_ts

Family	Files	What it demonstrates	Representative files
adaptive	20 (18 inputs)	Adaptive sampling rounds over a reference and a neural network: the single-method carrier, the QM/MM and ONIOM2 carriers, the local and slurm dispatch pairs, the training-data quality gate, and one carrier per analysis switch.	adaptive_single_local, adaptive_qmmm_local, adaptive_quality_gate_local, adaptive_analysis_gap_local, adaptive_oniom2_slurm
nn	4	The machine-learning side without a trajectory: training from a dataset, prediction from a trained ensemble, and the hyperparameter grid scan that ranks the resulting models.	nn_train, nn_predict, nn_grid_search, nn_grid_search_local
hessian	1	The framework numerical Hessian and vibrational analysis on a three-atom system, sized to run inside a short qualification budget.	hessian_num
single point	2	The two baselines every other family is read against: one energy and one gradient on the same ethylene geometry, with a minimal theory section.	energy.yaml, gradient.yaml

The path family is the largest because each walk exists in the gas phase, in a periodic cell and across a composite. path_neb and path_irc show the two walkers with framework gradients, path_neb_xtb repeats the band on the xTB skill, and path_mep_pbc shows a walk whose points carry a cell. The multiscale family covers the same ground for composites: multiscale_qmmm_embedding_sp is the smallest readable QM/MM input, multiscale_oniom3_orca_embedding_npt adds the three-layer form and a moving box, and multiscale_qmmm_neb and multiscale_qmmm_ts show that a composite is an ordinary theory queue as far as the workflow is concerned.

The constraint family is organised by term and then by ensemble: constraint_md_bias and constraint_md_metad_rmsd show the two bias forms, constraint_md_shake shows a hard constraint, and constraint_md_periodic_npt_wall and constraint_opt_surface put a wall and a bias into a periodic NPT trajectory and into a surface search. The multicenter family answers the other structural question: multicenter_qmmm_sp is two centers with one charge round, multicenter_qmmm_sc_charge iterates the charges to a fixed point, and multicenter_oniom3_orca_embedding_npt carries three centers through a barostatted run.

The md family is the one to read before writing any trajectory input. md_local is the smallest runnable trajectory, md_bagel_fssh is the surface-hopping skeleton that this chapter repeats with PySCF, and the md_lammps_nvt_nhc and md_lammps_npt_nhc_mttk pair show a legal thermostat and a legal thermostat-barostat pairing on a classical engine. The optimize family reads the same way: optimize_minima, optimize_ts, optimize_cell_gdiis and optimize_cell_meci name one search each, and optimize_partial_ts shows hess_atoms holding all but a few atoms.

The adaptive family gives each feature its own carrier. adaptive_single_local is the reference chain in its shortest form, adaptive_qmmm_local puts a neural network in the QM role of a solvent

box, adaptive_quality_gate_local is the only input that sets maxmae, and adaptive_analysis_gap_local sets analyze_gap. The four nn inputs cover the model lifecycle without a trajectory, hessian_num is the numerical Hessian carrier, and energy.yaml and gradient.yaml are the two baselines.

> **Note.** *The examples assume the engines they name are installed and that the scratch directory exists. A family that needs an engine the host does not have fails at its first skill call, not at input validation, so read the header comment of a file before treating that failure as an input error.*

13.2. Tutorial: energy, optimization and frequencies

The three inputs below are one water molecule carried through the three single-structure workflows. They share the electronic structure section, so a mistake in one run is the same mistake in the next. The first input asks for the energy only.

```
control:
  title: h2o_energy
  jobtype: energy
  theory: [pyscf]

atoms: [O, H, H]
coordinates:
  - [0.000000000, 0.000000000, 0.117300000]
  - [0.000000000, 0.757200000, -0.469200000]
  - [0.000000000, -0.757200000, -0.469200000]

molecule:
  charges: 0
  multiplicity: 1
  ci: [1]
  spin: [0]

pyscf:
  method: hf
  basis: sto-3g
  scratch_path: ./scratch
```

The second input relaxes the geometry. The optimize section is the one from optimize_minima.yaml, and the jobtype token selects the iterative workflow rather than the single-point one.

```
control:
  title: h2o_opt
  jobtype: optimize
  theory: [pyscf]

optimize:
  method: minima
  maxit: 100
  opt_energy: 1e-6
  opt_grms: 3e-4
  opt_gmax: 4.5e-4
```

```
  opt_drms: 1.2e-3
  opt_dmax: 1.8e-3

atoms: [O, H, H]
coordinates:
  - [0.000000000, 0.000000000, 0.117300000]
  - [0.000000000, 0.757200000, -0.469200000]
  - [0.000000000, -0.757200000, -0.469200000]

molecule:
  charges: 0
  multiplicity: 1
  ci: [1]
  spin: [0]

pyscf:
  method: hf
  basis: sto-3g
  scratch_path: ./scratch
```

The third input asks for the frequencies at the optimized geometry, which is copied from the opt.xyz the second run leaves behind. The hessian section asks for the molden record and for the thermochemical corrections at room conditions.

```
control:
  title: h2o_freq
  jobtype: frequency
  theory: [pyscf]

hessian:
  hess_step: 0.005
  hess_molden: true
  hess_temperature: 298.15
  hess_pressure: 1.0

atoms: [O, H, H]
coordinates:
  - [0.000000000, 0.000000000, 0.119400000]
  - [0.000000000, 0.761500000, -0.474200000]
  - [0.000000000, -0.761500000, -0.474200000]

molecule:
  charges: 0
  multiplicity: 1
  ci: [1]
  spin: [0]

pyscf:
  method: hf
  basis: sto-3g
  scratch_path: ./scratch
```

The three commands are run in this order from one directory.

```
pyrai3md h2o_energy.yaml
pyrai3md h2o_opt.yaml
pyrai3md h2o_freq.yaml
```

The energy run writes one result block to h2o_energy.log. The block opens with the electronic energy in Hartree, and because the input asked for nothing else, the remaining sections either carry the gradient twice or say that a property was not computed.

The optimization run repeats the block once per evaluation, under a step header, and keeps four products. opt.xyz is overwritten every step and holds the structure the run would stop on. opt_geom.xyz is appended every step and is the trajectory of the search. opt_status.txt carries one row per accepted step with the columns step, energy, shift, max_d, rms_d, max_g and rms_g, in Hartree, Angstrom and Hartree/Bohr. The fourth product is the run log, whose final section states which criterion stopped the run.

The frequency run writes h2o_freq.freq.molden. The sampler looks for that exact name, so it has to keep it when the file is moved beside a later dynamics input. The log carries the vibrational frequencies in cm-1, the normal modes and the thermochemical corrections; the molden record carries the frequencies and mass-weighted modes in the layout the Wigner sampler reads. Water has three real frequencies at a minimum, and an imaginary mode would be printed as a negative one.

13.3. Tutorial: surface hopping

The input below runs a ten-step FSSH trajectory on ethylene with three singlet states and PySCF TDA. It follows md_bagel_fssh.yaml, with the same geometry, the same starting velocities and the same state ladder, so the two inputs differ only in the electronic structure engine. The coupling is requested as a list of pairs and the dynamics estimates each coupling from the energy curvature, so the engine only has to return state energies and gradients.

```
control:
  title: md_ethylene_fssh
  jobtype: md
  theory: [pyscf]

atoms: [C, C, H, H, H, H]
coordinates:
  - [-1.120191050, 0.000000000, -2.661941306]
  - [-1.120191050, 0.000000000, -1.331783306]
  - [-0.590584410, 0.756356668, -3.234891306]
  - [-1.649797690, -0.756356668, -3.234891306]
  - [-1.876547718, -0.529606640, -0.758833306]
  - [-0.363834382, 0.529606640, -0.758833306]

velocities:
  - [0.000233177, -0.000042299, -0.000208531]
  - [-0.000217441, 0.000195315, 0.000185800]
  - [-0.000893166, -0.000054466, 0.000176273]
  - [0.001361820, -0.000285727, 0.000253295]
```

```
  - [-0.000024165, -0.000666228, 0.000131482]
  - [-0.000631993, -0.000816868, -0.000290190]

molecule:
  charges: 0
  multiplicity: 1
  ci: [3]
  spin: [0]
  coupling: "1 2, 2 3, 1 3"

md:
  ensemble: nvt
  thermostat: nose-hoover
  temperature: 300
  initcond: 0
  ninitcond: 1
  step: 10
  size: 20.67
  root: "2"
  sfhp: fssh
  nactype: ktdc
  substep: 20
  deco: "0.1"

pyscf:
  method: tda
  functional: pbe
  basis: sto-3g
  nstate: 3
  target_states: [0, 1, 2]
  execution_mode: local
  keep_tmp: 0
  thread: 1
```

The md block answers four separate questions. ensemble, thermostat and temperature fix the nuclear bath. step and size fix the propagation, with step counting steps and size giving the timestep in atomic units. root names the state the trajectory starts on, in the 1-based spelling the input uses everywhere, so root two is the second state. sfhp, nactype, substep and deco fix the electronic side: the fewest switches algorithm, couplings estimated from the energy curvature, twenty electronic substeps per nuclear step, and the decoherence time in atomic units. Section 5.3 gives the full vocabulary and Section B gives the index bases.

The run writes six md files and, whenever a hop happens, the matching six sh files. md.xyz holds every recorded frame in Angstrom with a comment line naming the title, the step and the active state. md.velo holds the velocities of the same frames in atomic units. md.energies holds the step, the potential energy of the active state, the kinetic energy and the per-state energies. md.pop holds the populations followed by a one-hot vector that marks the post-hop active state. md.ensemble is written only for a thermostatted or barostatted ensemble and carries the thermostat and barostat readings of the step. md.stress appears only when the engine supplied a stress tensor. The sh files carry the same columns at hopping steps only, and the writer emits them immediately rather than on the direct and buffer cadence that governs the md files.

A hop is read in three places. The md.xyz comment of the step gains a suffix of the form to 2 CI, so a frame that changed state is recognisable without the populations. The active-state block of md.pop switches its one-hot vector from the old state to the new one. The sh.xyz frame for the same step appears with the post-hop velocities in sh.velo, which is the velocity the integrator rescaled or reflected to conserve energy across the gap. In the log, the per-step header is followed by the same numbers the records carry, so a hop can be found by searching for the comment suffix once the step is known.

The run also writes md_ethylene_fssh.rt on the cadence of the md checkpoint keyword, and by default it writes none. A trajectory that was interrupted is continued by launching that file, not by editing the input: pyrai3md md_ethylene_fssh.rt. The restored state carries the step count, the history window, the electronic density and the random generator position, so the resumed segment continues the same hopping sequence instead of re-seeding it. The input is read once at the start; changing the dynamics section afterwards does not alter the state a runtime file restores.

> **Note.** *The default verbosity does not write per-step amplitudes. Set verbose 4 in the control section when the amplitudes themselves have to be inspected, which writes <title>.fssh; the file grows with every electronic step, so it is a diagnosis tool rather than a production record.*

13.4. Tutorial: QM/MM

A QM/MM run is a two-entry theory queue with multiscale_type qmmm. The first entry is the QM provider and the second is the classical provider, which also owns the box. The highlevel list names the QM region, and the embedding switch decides whether the QM calculation sees the classical charges as a field or only through the additive energy. The input below puts one water in the QM region and one in the classical region, on a six-atom cluster.

```
control:
  title: qmmm_water_dimer
  jobtype: energy gradient
  theory: [pyscf, lammps]
  multiscale_type: qmmm

atoms: [O, H, H, O, H, H]
coordinates:
  - [1.000000000, 1.000000000, 1.000000000]
  - [2.000000000, 1.000000000, 1.000000000]
  - [1.000000000, 2.000000000, 1.000000000]
  - [5.000000000, 5.000000000, 5.000000000]
  - [6.000000000, 5.000000000, 5.000000000]
  - [5.000000000, 6.000000000, 5.000000000]

molecule:
  charges: 0
  multiplicity: 1
  pbc: [0, 0, 0]
  ci: [1]
  spin: [0]
  highlevel: [1, 2, 3]
  q_qm: [-0.83, 0.415, 0.415]
```

```
  embedding: true

pyscf:
  method: hf
  basis: sto-3g
  execution_mode: local
  thread: 1

lammps:
  method: ff
  lammps_project: qmmm_cluster_test
  lammps_shake: 0
  python_enabled: true
  execution_mode: local
```

The pbc line is not decoration. The classical topology declares a ten Angstrom box, and an input that leaves the periodicity to the default reads as a molecular run in its own log while the engine still loads the box. Stating pbc as three zeros declares this run a cluster, which is what the shipped QM/MM examples do when they reuse a periodic topology. The lammps_project value names the topology stem the classical stage reads.

A larger environment is built rather than typed. The multiscale generator packs a solute into solvent shells, writes the classical files and produces the combined starting conditions:

```
title         waterbox
mode          create
env_type      solvent
solute        solute.xyz
solvent       solvent.xyz
nshell        2
density       1.0
solbox        60
mass          18.0
packmol       /path/to/packmol
cpus          8
```

```
python tools/multiscale_generator.py creation.in
```

The tool writes the packed system and the classical files beside the creation file; the merge and edit modes then adjust a starting geometry or a velocity set without rebuilding the box. Section 10.6 describes the modes and Section 7.1 describes the composite itself.

A QM/MM run reports the composite in its own log block, headed &composite energy decomposition in Hartree. Each term occupies one line, ordered high to low, carrying the raw energies of that term in state order, and a final total line carries the additive QM/MM energy of each state. The classical term is the full force-field evaluation of the whole system, not the MM correction alone, so the terms do not sum to the total by eye; the coefficients that make them sum are the plan's, and the block states them by order. Beside it, the &qmmm 0-order multipole charges block reports the charges the classical field was built from, and the &electronic energy

block reports the assembled total that the workflow judged. A run with embedding off prints no charge block, which is the quickest way to confirm which embedding the run actually used.

13.5. Tutorial: adaptive sampling

Adaptive sampling alternates between running trajectories on the current neural network, labelling the selected structures with the reference method, and refitting the network. The input below runs two iterations over a pending ethylene model, with PySCF as the reference, the quality gate armed at 0.0082 eV, and the gap analysis restricted to the ground and first excited state. It is adaptive_quality_gate_local.yaml with one analysis switch added.

```
control:
  title: adaptive_ethylene
  jobtype: adaptive
  theory: [nn]
  reference: [pyscf]
  gl_seed: 20261105

atoms: [C, C, H, H, H, H]
coordinates:
  - [-1.1201910500, 0.0000000000, -2.6619413060]
  - [-1.1201910500, 0.0000000000, -1.3317833060]
  - [-0.5905844100, 0.7563566680, -3.2348913060]
  - [-1.6497976900, -0.7563566680, -3.2348913060]
  - [-1.8765477180, -0.5296066400, -0.7588333060]
  - [-0.3638343820, 0.5296066400, -0.7588333060]

molecule:
  charges: 0
  multiplicity: 1
  ci: [2]
  spin: [0]

md:
  initcond: read
  ninitcond: 5
  step: 10
  size: 20.67
  root: 1
  sfhp: nosh
  nactype: ktdc
  rk4: 1
  format: molden

adaptive:
  maxiter: 2
  maxsample: 1
  record_step: 4
  maxmae: 0.0082
  keep_md: 1
  load: 1
  md_multitask: true
  md_multipool: 5
  reference_multitask: true
  reference_multipool: 5
```

```
  maxenergy: 0.0
  minenergy: 0.0
  maxgrad: 0.0
  mingrad: 0.0
  gap_tolerance: 1.0e-6
  sample_energy: 1
  sample_grad: 1
  analyze_gap: s0s1

nn:
  method: mlp_eg
  modeldir: ./nn_ethylene_init
  train_data: ./egs_ethylene_gate_100.json
  mult_nn: 2
  nstate: 2
  target_states: [0, 1]
  execution_mode: local
  scratch_path: ./scratch
  eg:
    depth: 2
    nn_size: 16
    batch_size: 5
    epo: 30
    learning_rate: 1.0e-3
    epostep: 1
    loss_weights: '1 10'
    val_split: 0.1

pyscf:
  method: tda
  functional: pbe
  basis: sto-3g
  nstate: 2
  target_states: [0, 1]
  execution_mode: local
  thread: 1
```

Four settings carry the workflow. theory names the surface the trajectories run on and reference names the method that labels them; nn in the reference queue is refused, because a network cannot label itself. maxiter bounds the rounds and maxsample bounds how many structures one round may keep. load is the statement that the round starts from the data and the network the input names rather than from a previous run. The md block is an ordinary dynamics block, because every round launches ordinary trajectories.

The products collect under a directory named after the title, not in the launch directory:

```
adaptive_ethylene.adaptive/
  adaptive_ethylene.adaptive.json      overall round record
  adaptive_ethylene.log                run log, one sampling block per
round
  iter-1/
    md/traj_<idx>/                     the round's trajectories
    selected_iter-1.xyz                the structures the round selected
    population_iter-1.json             the population record
    status_iter-1.json                 the round's counts and verdict
```

```
iter-2/
  data_iter-2_<nframe>.json          the accumulated training set
  NN-adaptive_ethylene-iter-2/       the refitted ensemble
  NN-adaptive_ethylene-iter-2-r1/    a gate retrain, when one ran
  reference/ref_<idx>/               the labelling calculations
  reference/new-samples.json         the labels this round added
  analysis_iter-2.json               only when an analysis switch is on
  md/traj_<idx>/
  selected_iter-2.xyz
```

The first iteration is the external round: it runs trajectories on the network supplied in the input, selects structures and labels them, but has no model of its own to write. Every later iteration fits a model, stores it beside the data snapshot and the labels, and hands it to the next round. The data file name carries the frame count, so data_iter-2_106.json is the 106-frame set of the second round and the number can be checked against the two carrier counts without opening the file.

A round is judged on three things before its trajectories are trusted. The maxmae gate compares the fitted model's validation energy MAE against the ceiling in eV; a fit above the ceiling is retried after the worst new frames are removed one at a time, and the retry is kept only if it lowers the MAE. The gate writes a &nn validation gate block and records the decision, so a passing fit and a pruned fit are distinguishable from the log alone. The gap analysis, when analyze_gap is not off, reports the minimum gap, the first crossing and the fraction of frames below a threshold for the chosen state scope, which is how a round that collected only high-energy structures is noticed. The physical screens are the third check: maxenergy, maxgrad and their minima bound the accepted structures, and check_population decides whether the structure a trajectory stopped on is collected.

> **Note.** *The analysis switches default off and the analysis file is written only when at least one is on. A missing analysis_iter-2.json is therefore a statement about the input, not about the round.*

14. Qualification and verification

This chapter describes the test suite that ships with the package and the two remote drivers. The suite runs through the same entry points a user uses, so a passing round is evidence that the installation and the examples agree.

14.1. The qualification round

pyrai3md test plans and runs a round from ~/.pyrai3md/pmd_env.yaml over the package template. It covers four categories: the framework unit tests and integration tests, the shipped-example carriers, the skill certification cases and the benchmark calibrations.

The framework category runs the modules under qualification/verification in one process and adds the cross-mode files under qualification/integration when those are opened. The example category renders every YAML under pyrai3md/examples into two user carriers, the YAML itself and the classic section input, so one example is two tasks. The skill category takes each skill tests directory: the pytest scripts first, then every case YAML expanded into three carriers, the classic

input run by pyrai3md, the sectioned JSON run by pyrai3md_agent and the flat skill JSON run by pyrai3md_skill. The benchmark category runs the FSSH, GSH, LZSH, kTDC and performance calibrations against their analytic references.

The carriers are the point. An example or a skill case is written to disk and run through the command a user would type, so the parser, normalization, validation, planning, dispatch and logging of a real run are all exercised. A defect that appears only in the classic writer, or only in the agent schema, is a round failure.

Planning queries the skill registry, generates the inputs and opens the work directory tmp_YYYYMMDD_HHMMSS in the current directory. The round writes one record per task to results.json and the terminal state to status.json. A local round leaves one row fragment per task under jobs/; a grouped Slurm round leaves the job specs, the batch scripts and the fragments there and collects them in plan order.

Three levels of switches decide what runs, all read from the same file. The master _runall sits above everything: true opens every example, skill case and calibration, false closes them and leaves the tests that need no engine, and condition passes the decision down. The middle level is one switch per bucket or skill: test_integration, the six example family keys (optimize_test, path_test, md_lammps_test, constraint_md_test, multiscale_test and adaptive_test), skills.<name>.test, skills.<name>.runtime_tests and the benchmark entries. The bottom level is one key per case: examples.<name>, skills.<name>.tests.<case> and the enabled flag of a benchmark entry. Each takes true, false or condition, and a missing key reads as condition. The master is consulted above the families, so false there closes a family whose own key reads true; an optimize round is _runall: condition with the optimize family on.

```
_runall: condition
test_integration: false
examples:
  optimize_test: true
  energy: false
skills:
  molcas_skill:
    test: condition
    tests:
      casscf_gradient: true
benchmark:
  fssh:
    - method: tully1
      enabled: false
```

A closed case stays in the plan as a skipped task whose reason names the key that closed it, and the report folds the closed rows of each bucket into one row carrying the count and the reasons.

14.2. Remote rounds

The deploy tool runs the same round on a configured host. It reads the deploy section, asks for anything incomplete, resolves the SSH route (direct, then the jump host) and the conda

environment, syncs the checkout, installs it with pip, and starts pyrai3md test in the background so the run survives the starting session.

The sync excludes version-control files, byte-code caches, egg metadata, editor folders, earlier rounds, run logs and build products, and it deletes no excluded file on the host, so earlier artifacts survive. The install removes the build tree first, so a stale build cannot be reused.

The tool polls status.json, reports progress against the task count the round publishes, then fetches results.json and downloads the log as test_<stamp>.log. The generated carriers and calculation directories stay in the round directory on the host. The force form cancels the jobs a grouped Slurm round recorded and stops the orchestrator.

The deploy section holds the host, user, port, install path and optionally a jump host; environment.scheduler chooses a local or a grouped Slurm round. Section 3.6 covers the client side.

14.3. Post-round verification

The verifier audits a finished round from its files alone.

```
python tools/verify.py tmp_20261112_090000
python tools/verify.py --selftest
```

It never starts an engine, a skill entry point or a calculation, and writes nothing into the round; where it needs a skill reader it calls that parser on a file. Its sections are lettered A to K.

Section	What it cross-checks
A	Recomputes each recorded example metric from the frame records and flags a record older than the round.
B	Compares each logged step's electronic and kinetic energy with the same step's record row.
C	Requires the recorded potential energy to be a number a surviving engine file printed; C2 does the same for an optimize leg's status table, and C3 requires a composite row's term columns to add to the energy beside them.
D	Re-derives a variable-cell verdict from the status record, the cell metrics and the printed cell sentence, and re-reads each periodic path leg's status table by column name.
E	Compares one example's two carriers on the recorded metrics and on each record's term decomposition.
F	Re-derives the path family's claims: a held coordinate, frozen atoms, and the energy a composite leg and its single-layer counterpart record for one geometry.
G	Reads one OpenMolcas case's three carrier logs with the skill's own parser and requires the four arrays to agree.
H	Re-derives the periodic field probe's cross energy with a converged Ewald sum, which names a deck that lost its pinned k-space line.
I	Checks the QM/MM force and gradient files atom by atom against the assembled gradient on the QM rows and elsewhere.

Section	What it cross-checks
J	Audits an adaptive run's two-round product tree: selections, label counts, a servable model folder, the alignment record, the structural counts of both carriers and the sidecar rule.
K	Compares every skill case's three carriers file-only, the engine deck and the final energy each printed, and runs no skill.

The selftest plants a clean round, corrupts one number per section and requires every corruption to be named. The report also states its coverage: which skills have readers, which rows it refused to compare, and which channels have no artifact to check.

15. Extending PyRAI3MD

This chapter describes the contract a new engine must satisfy and points to the development documents behind each layer. It is meant for readers who want to add a skill, not for users writing an input.

15.1. The skill contract

The framework and a skill meet at three declared places: the skill manifest, the query module and the skill configuration schema. The import tree is one-way. A skill may import pyrai3md.utilities and nothing else from the framework; it may not import the schema, planning, workflow or kernel layers. The framework never imports an engine: the backend import path is stored as a string and resolved at calculation start.

The query module is the skill's only exit point. The framework calls five public names from it: get_profiles and list_profiles describe the calculation profiles, resolve_profile maps a method, jobtype and driver onto one, get_registry names the backend module and class, and get_defaults returns the schema defaults used to build the skill configuration. Paths it returns are skill-relative; the framework prepends its package prefix at run time.

A calculation is one call to backend.calculate with the atoms, the coordinates, the target states and, for a periodic system, the cell. It returns a plain dictionary keyed by the property names the framework reads. The dictionary is not validated against a model: the contract is by name, and a key spelled differently is not read.

Four channels carry state rather than properties. fin_coord carries the final structure, one row per atom in Angstrom, after the skill ran its own loop; a length that disagrees with the atom count raises. fin_velocity carries final velocities in Bohr per atomic unit of time. fin_cell carries the box the returned coordinates live in, as six cell parameters or nine lattice components, and a value returned for a non-periodic system raises. opt_converge carries the optimization verdict when the driver is the skill. The framework consumes each channel and drops the key; a different spelling of the last one is not read.

Units at the boundary are Hartree, Hartree per Bohr and Hartree per cubic Bohr, with coordinates in Angstrom, velocities in Bohr per atomic unit of time and spin-orbit couplings in cm-1. A gradient is a derivative, never a force. A skill converts its native units and declares them in its manifest; the framework applies that declaration, and an unknown unit name raises.

15.2. Writing a skill

A skill is a directory under pyrai3md/skills/. The manifest declares what the framework may ask for: the profiles and the capability and method of each, the capability list, the backend module and class, the units of the returned arrays, the roles the skill can fill, whether it supports periodic systems, and its tunables, which are the control parameters a user may write, the resource keywords that size a run, the hyperparameters a grid scan may vary and the result channels a training or prediction call must return.

The schema package owns everything the framework communicates with: the configuration model, the supported methods and capabilities, the constants and defaults, the adapter that turns a framework task into a skill configuration, the normalization step and the validation rules. The service package owns the computation: the workflow compiler that turns a profile into a graph, the builders that write the engine input files, the runner, the output parsers and the templates.

```
<skill>_skill/
  manifest.yaml
  query.py
  backend.py
  agent.py
  skill_guide.md
  schema/
    config.py
    method.py
    reference.py
    adapter.py
    normalize.py
    validate.py
  service/
    workflow/
    builder/
    runner/
    parser/
    templates/
  tests/
    <case>.yaml
    test_<module>.py
```

One rule has no diagnostic and no second chance: every configuration field must be registered in get_defaults. The framework generates the skill configuration from that table, so a field that is not registered is silently dropped between the input file and the backend, and the run proceeds without it. Registering a field also puts it in the vocabulary the query module reports.

The tests directory carries the offline pytest scripts and the case YAMLs the certification planner expands into three carriers (Section 14.1). A skill is complete when pyrai3md_skill runs one of its JSON configs and one case appears in a round.

15.3. Where to read more

The repository carries the design documents this manual summarizes. They are written for developers and are the authority when the manual and the code disagree.

Document	Covers
dev/ARCHITECTURE.md	The layer map, capability names, skill discovery and resolution, the runtime modes and the dependency rules.
dev/SKILL_APP_DEVELOPMENT.md	Skill package layout, manifest fields, the query mechanism, backend initialization, the result contract and the schema and service layers.
dev/skills/*_SKILL_DEVELOPMENT.md	One guide per engine: its input deck, profiles, parser, units and the limits measured on it.
dev/methods/*.md	One document per workflow family: MD, FSSH, GSH, LZSH, optimization, cell optimization, path, periodic systems, multiscale, Hessian, adaptive, training, search and constraints.
dev/QUALIFICATION_DEVELOPMENT.md	The four plan categories, every switch in the environment file, the deployment workflow and the verifier sections.
pyrai3md/skill_guide.md	The framework's own usage guide: entry points, input carriers, command reference, output layout and development rules.

The keyword chapters are generated from the same configuration models, so a default quoted in Section 5.1 is the value the program loads.

A. Units and sign conventions

The framework converts every quantity once, at its boundary, into the atomic unit set the workflows integrate in: Bohr, Hartree, the atomic unit of time and the electron mass. An input is written in the units this appendix lists, a skill may compute in its own units if its manifest says so, and the log reports in the conventional unit named in each block title. The table below is the whole contract; where an input unit differs from the boundary unit, the conversion and its direction are named.

Quantity	Unit at the boundary	Input unit	Note
Cartesian coordinates	Angstrom	Angstrom	Input, xyz files and every trajectory record use Angstrom; the geometry is converted to Bohr once when the state is built.

Quantity	Unit at the boundary	Input unit	Note
Cell parameters	Angstrom and degree	Angstrom and degree	cell is a, b, c, alpha, beta, gamma. lattice_vector is Angstrom and wins over cell when both are written.
Energy	Hartree	Hartree	Skills that declare eV or kcal/mol in their manifest are converted by the framework, not by the caller; the value the workflow sees is always Hartree.
Energy threshold	Hartree	Hartree	maxenergy, minenergy and the md md_max* thresholds are Hartree. The one exception is adaptive maxmae, which is a validation energy MAE in eV.
Gradient	Hartree/Bohr, dU/dR	Hartree/Bohr	The stored array is the derivative of the energy with respect to the coordinate, not a force. An engine that returns forces has them negated and converted once, so a downhill direction is a negative gradient.
Force (engine native)	Hartree/Bohr after conversion	engine native	A classical engine returns force, not gradient. The manifest unit table converts the energy-per-length factor and the sign together; a caller that also negates a converted array inverts the dynamics.
Stress tensor	Hartree/Bohr^3	bar, as a criterion	The full 3x3 tensor is Hartree/Bohr^3. opt_pressure and md pressure are read in bar, and the optimizer and barostat compare against a pressure derived from the tensor. Engines that print GPa are converted centrally.
MD time	atomic unit of time	atomic unit of time	md size is the timestep in au and the trajectory time column is au. One au is 0.02418884254 fs.
Thermostat and barostat time	atomic unit	fs	thermostat_tau and barostat_tau are written in fs, which is the literature unit, and converted to au. The legacy constant is about 41.34 au per fs.
Temperature	K	K	temperature is kelvin at the boundary and in the log; the thermostat and the Boltzmann factors work in Hartree internally.
Pressure	bar	bar	pressure, opt_pressure and every pressure column of md.ensemble are bar. One Hartree/Bohr^3 is 2.9421e7 bar, so a barostatted run reads very small numbers in atomic units.
Mass	amu	amu	mass is amu in the input and in the log. Internally the mass is in electron masses; one amu is 1822.8885 electron masses.
Velocity	Bohr per atomic time	Bohr per atomic time	velocities in the input, in <title>.init.xyz and in md.velo are the same unit, which

Quantity	Unit at the boundary	Input unit	Note
			is what lets a sampled batch be re-read without rescaling.
Helicity and spin-orbit coupling	cm-1	cm-1	The &spin-orbit coupling block, the soc array and the soc_pair_map are in cm-1 at every layer, so no conversion is applied.
State gap	Hartree	Hartree	State energies, gaps and the analysis gap records are Hartree. Neural-network error metrics, including maxmae, are reported in eV.
Dipole moment	Debye in the log	not an input	One atomic unit of dipole is 2.541746 Debye. The log prints Debye; the result array keeps the skill value the log was formatted from.
Transition dipole	Debye in the log	not an input	Ordered by transition_pair, which is 1-based in the input and printed 1-based. The oscillator strength beside it is dimensionless.
Frequency	cm-1	not an input	Vibrational frequencies are cm-1 and an imaginary mode is printed as a negative number, in the log, in the molden record and in the frequency array.
Hessian	Hartree/Bohr^2	Angstrom, as a step	The matrix is Hartree/Bohr^2. hess_step is a displacement in Angstrom, so the difference quotient carries the Bohr conversion and a step written in Bohr gives a matrix off by 1.89.
Nonadiabatic coupling	Hartree/Bohr	not an input	The nac array and the &nonadiabatic coupling block use the energy derivative coupling convention, in Hartree/Bohr, not the wavefunction-overlap convention.

Two sign conventions are worth stating separately because they are the two that produce silently wrong physics rather than a refused input. The first is the gradient sign: the framework stores dU/dR, so the force an engine reports is negated on the way in and never again. The second is the stress sign: the tensor is the derivative of the energy with respect to strain, and the pressure a barostat targets is minus one third of its trace, so a positive pressure compresses the cell. A run that moves the box the wrong way usually has a sign flipped on one of these two, not a unit error.

Masses, charges and the electron count follow the same rule. A charge is in units of the elementary charge everywhere; multiplicity is a positive integer, never a spin; and the total electron count that the state derives is the number the check compares against the multiplicity. Section 5.2 lists the fields and Section B lists the quantities the state derives from them.

B. Index bases and derived quantities

Two different questions are answered here. The first is which index base a keyword uses, because the framework accepts atom and state numbers in more than one spelling and a reader who assumes one base for all of them will be wrong about at least one keyword. The second is which fields of the state are derived rather than read, because a derived field written into an input is either ignored or refused, and the number the run reports is the one it computed.

The rule for atom numbers is 1-based outside and 0-based inside. Every atom selection in an input counts from one, is converted once when the state is built, and is printed 1-based again. The two documented exceptions are center_groups, whose numbers have always been 0-based, and the molecule list of a wall, whose parsed form holds 0-based atom numbers. State numbers follow the opposite convention for the optimizer: opt_state_i and opt_state_j are 0-based in the input itself, while root, coupling and every state number in a log line are 1-based.

Keyword	External base	Internal base	Note
highlevel, midlevel	1-based	0-based region index	The QM and mid regions of a layered run. Both accept a flat list or one list per center; the per-center form keeps the same base.
highref, midref	1-based	0-based region index	The reference regions used by adaptive sampling. They are parsed like highlevel and midlevel but never enter the theory center plan.
coupling	1-based	0-based pair map	Each pair is two state numbers written as i j, separated by commas. The pair maps nac_pair_map and soc_pair_map carry the converted pairs.
root	1-based	0-based active state	The starting state of a trajectory. The external spelling is 1-based, so root two is the second state; the state field carries the decremented value.
current_state	no external spelling	0-based	The active state the run is on. It is set by root or by a hop, printed 1-based in the log and in the trajectory comment, and is never an input keyword.
opt_state_i, opt_state_j	0-based	0-based	The optimized surface and, for a crossing search, the second surface. Zero is the ground state, so naming an excited state means writing one, not two.
track_index	1-based	0-based pair list	The atom pairs or fragment groups of the geometry tracker. Each number is an atom index

Keyword	External base	Internal base	Note
			in the input and is decremented once.
center_groups	0-based	0-based	Fragment groups of a multi-center plan. This keyword is the inherited exception: its numbers were 0-based in the legacy input and remain so.
hess_atoms	1-based	0-based hessian_atoms	The mobile atoms of a partial Hessian in the input. The result field hessian_atoms is the converted 0-based list, and the matrix keeps zero rows and columns for the held atoms.
constraint atoms	1-based	0-based	groups atoms, bias atoms, wall atoms and ref_atoms, fix atoms and every atoms list of a hard constraint or a metadynamics entry count from one.
wall groups (molecules)	1-based selection, 0-based molecules	0-based molecules	A wall selection is 1-based like every other selection. The parsed molecules list, which the legacy wall_groups line fills, holds 0-based atom numbers and a molecule count per group.
permutation_map	1-based file	0-based expansion	A training permutation map. The file holds one atom order per line in the 1-based numbering of the structure, and the expansion converts it to the internal order.

The derived table below is generated from the System model, so every row names a field that exists in this version. A field listed here is not an input keyword: writing one is either absorbed and overwritten or refused, and neither outcome is what an input author wanted. Persisted fields are the ones a runtime file restores; transient fields are recomputed inside the step that uses them.

Derived field	What it holds
total_electron	Total electron count, the sum of the atomic numbers minus the total charge. It is the number the multiplicity check compares against.
nalpha	Number of alpha electrons, (total_electron + 2S) / 2, derived from the total electron count and the aggregate multiplicity.
nbeta	Number of beta electrons, (total_electron - 2S) / 2.
nstate	Number of electronic states on the state ladder, the sum of the ci entries across the spin multiplicities.
state_label	Per-state label, for example S0 or T1, built from the spin block and the position of the state inside it.
state_spin	Per-state 2S, so 0 is a singlet and 2 is a triplet. It is the value that decides which states may couple by SOC.

Derived field	What it holds
state_density	Electronic state density matrix, (nstate, nstate, 2) with the last axis carrying the real and imaginary parts. It is propagated by the hopping integrator and persisted across steps.
npair	Number of state pairs the input requested, the length of transition_pair.
nnac	Number of requested pairs that ask for a nonadiabatic coupling, the length of nac_pair_map.
nsoc	Number of requested pairs that ask for spin-orbit coupling, the length of soc_pair_map.
transition_pair	The requested pairs as they were written, as 0-based state indices in a (npair, 2) array.
nac_pair_map	Row index to state pair for the nac array, (nnac, 2). The coupling row of a pair is found here, never by guessing the pair order.
soc_pair_map	Row index to state pair for the soc array, (nsoc, 2).
target_coord	A target structure carried with the state for a workflow that needs one. It is empty unless a target was supplied, and it is never a second copy of the geometry.
boundary	Atom groups at a partition boundary, carried with the state so a layered run can name the atoms on the edge of a region. It is empty for an unpartitioned system.
volume	Cell volume in cubic Angstrom, derived from the lattice. Zero when the run is not periodic.
image	Per-atom count of cell lengths crossed by the coordinate wrap, (natom, 3). The coordinates stay wrapped, so this array is what makes a continuous trajectory and a cross-frame displacement recoverable.
thermostat_dof	Particle thermostat chain state, the chain positions followed by their momenta. Persisted across steps and restored by a runtime file; never sent to a skill.
barostat_strain	Full-tensor cell strain, (3, 3). Persisted, and the quantity the cell drift integrates.
barostat_momentum	Conjugate cell momentum, (3, 3), the variable the cell equation of motion advances.
barostat_velocity	Instantaneous cell velocity, momentum divided by the cell mass. It is transient inside one step and is not a checkpoint field.
barostat_thermostat_dof	Chain state of the thermostat that acts on the cell momentum, in the same layout as thermostat_dof.

The pair maps deserve one sentence of emphasis. The nac and soc arrays are ordered by the maps, not by the order the pairs were typed, and a run that requests a subset of pairs has a shorter array than the state ladder suggests. Reading the coupling of a pair from an index into the array without going through nac_pair_map is therefore wrong as soon as the subset is not the full ladder. Section 5.2 lists the input fields these quantities are derived from, and Section 12.1 shows where each one is printed.

C. Keyword status index

This appendix is generated from the same keyword specifications as the tables in Section 5, so a mark in a table and a row here come from one declaration. Three questions are answered:

which keywords are accepted but have no effect in this version, which accept several values but honour one, and which names are aliases stored under a modern name. The marks used in the keyword tables are the same three: a dagger, a double dagger and a section sign.

Keywords with no effect

A keyword in this table parses, validates and appears in the run log under its own name, and no workflow reads it. It is kept so that an older input still loads; a new input should not use one.

Keyword	Section	Note
qmmm_xyz	&SYSTEM	accepted but not read
solvent	&SYSTEM	accepted but not read; implicit solvation is set by the solvent key of the engine section
method	&DYNAMICS	accepted but not read
pop_step	&ADAPTIVE	accepted but not read
maxdiscard	&ADAPTIVE	accepted but not read
refine_num	&ADAPTIVE	accepted but not read
refine_start	&ADAPTIVE	accepted but not read
refine_end	&ADAPTIVE	accepted but not read
refine_gap	&ADAPTIVE	accepted but not read
dynenergy	&ADAPTIVE	accepted but not read
inienergy	&ADAPTIVE	accepted but not read
fwdenergy	&ADAPTIVE	accepted but not read
bckenergy	&ADAPTIVE	accepted but not read
dyngrad	&ADAPTIVE	accepted but not read
inigrad	&ADAPTIVE	accepted but not read
fwdgrad	&ADAPTIVE	accepted but not read
bckgrad	&ADAPTIVE	accepted but not read
dynnac	&ADAPTIVE	accepted but not read
ininac	&ADAPTIVE	accepted but not read
fwdnac	&ADAPTIVE	accepted but not read
bcknac	&ADAPTIVE	accepted but not read
dynsoc	&ADAPTIVE	accepted but not read
inisoc	&ADAPTIVE	accepted but not read
fwdsoc	&ADAPTIVE	accepted but not read
bcksoc	&ADAPTIVE	accepted but not read
program	&ENVIRONMENT	accepted but not read
version	&ENVIRONMENT	accepted but not read
device	&ENVIRONMENT	accepted but not read
program_path	&ENVIRONMENT	accepted but not read
slurm_time	&ENVIRONMENT	accepted but not read
slurm_partition	&ENVIRONMENT	accepted but not read
slurm_ntasks	&ENVIRONMENT	accepted but not read
num_gpu	&ENVIRONMENT	accepted but not read
slurm_memory	&ENVIRONMENT	accepted but not read
slurm_command	&ENVIRONMENT	accepted but not read
nics_ring	&PYSCF	accepted but not read
nics_scan	&PYSCF	accepted but not read
thermostat	&PYSCF	accepted but not read
barostat	&PYSCF	accepted but not read
ensemble	&PYSCF	accepted but not read

Keyword	Section	Note
nics_ring	&MOLCAS	accepted but not read
nics_scan	&MOLCAS	accepted but not read
ir_width	&BAGEL	accepted but not read
ir_scale	&BAGEL	accepted but not read
uv_width	&BAGEL	accepted but not read
nics_ring	&BAGEL	accepted but not read
nics_scan	&BAGEL	accepted but not read
solvent	&ORCA	accepted but not read
temperature	&ORCA	accepted but not read
shape	&ORCA	accepted but not read
factor	&ORCA	accepted but not read
scale	&ORCA	accepted but not read
ir_width	&ORCA	accepted but not read
ir_scale	&ORCA	accepted but not read
uv_width	&ORCA	accepted but not read
nics_ring	&ORCA	accepted but not read
nics_scan	&ORCA	accepted but not read
solvent	&OPENQP	accepted but not read
temperature	&OPENQP	accepted but not read
ir_width	&OPENQP	accepted but not read
ir_scale	&OPENQP	accepted but not read
uv_width	&OPENQP	accepted but not read
nics_ring	&OPENQP	accepted but not read
nics_scan	&OPENQP	accepted but not read
solvent	&XTB	accepted but not read
temperature	&XTB	accepted but not read
shape	&XTB	accepted but not read
factor	&XTB	accepted but not read
scale	&XTB	accepted but not read
ir_width	&CP2K	accepted but not read
ir_scale	&CP2K	accepted but not read
uv_width	&CP2K	accepted but not read
nics_ring	&CP2K	accepted but not read
nics_scan	&CP2K	accepted but not read

Keywords that honour one value

A keyword in this table accepts a list or a compound spelling but reads one member of it in v3.0.0. The table names the keyword, not the value: the description in Chapter 5 says which member survives.

Keyword	Section	Note
load	&ADAPTIVE	several values accepted, one honoured
sample_energy	&ADAPTIVE	several values accepted, one honoured
sample_grad	&ADAPTIVE	several values accepted, one honoured
sample_dipole	&ADAPTIVE	several values accepted, one honoured
sample_transition_dipole	&ADAPTIVE	several values accepted, one honoured
sample_oscillator_strength	&ADAPTIVE	several values accepted, one honoured

Keyword	Section	Note
dynsample	&ADAPTIVE	several values accepted, one honoured
refine	&ADAPTIVE	several values accepted, one honoured
transfer	&ADAPTIVE	several values accepted, one honoured

Aliases

An alias is read and stored under its modern name, and the run log prints the modern name. The table below joins the aliases the specifications declare with the legacy spellings the package accepts at the control and dynamics layer.

Alias	Stored as	Scope
qm	theory	&CONTROL - legacy keyword spelling
abinit	reference	&CONTROL - legacy keyword spelling
sp	energy	jobtype - canonical jobtype token
opt	optimize	jobtype - canonical jobtype token
freq, frequence	frequency	jobtype - canonical jobtype token
hess	hessian	jobtype - canonical jobtype token
numfreq	num_freq	jobtype - canonical jobtype token
numhess	num_hess	jobtype - canonical jobtype token
namd	md	jobtype - canonical jobtype token
predict	prediction	jobtype - canonical jobtype token
nh, nose	nose-hoover	&MD - thermostat spelling
nhc, nose-hoover-chains	nose-hoover-chain	&MD - thermostat spelling
csnr	csvr	&MD - thermostat spelling
pr, parrinello	parrinello-rahman	&MD - barostat spelling
parrinello-rahman-npt	parrinello-rahman	&MD - barostat spelling
c-rescale	scr	&MD - barostat spelling
stochastic-cell-rescaling	scr	&MD - barostat spelling
MD	DYNAMICS	section name - the section is read under either name
MOLECULE	SYSTEM	section name - MOLECULE remains the historical spelling

The three tables are built from the same keyword specifications as the tables in Chapter 5, so a mark in a keyword table and a row here come from one declaration. The index covers every framework section and every skill section. A keyword that appears in none of the three tables is either an active input or a retired one; retired keywords are refused rather than accepted, and Appendix Section E lists their replacements.

> **Note.** *A retired keyword is deliberately absent from all three tables. It is refused at validation with a message that names its replacement, and Appendix Section E collects those replacements. A keyword that is merely out of date is an alias; a keyword whose meaning no longer exists is a refusal.*

D. Refusal and limitation matrix

Most input mistakes are refused at validation, before any engine is started. This appendix collects those refusals by area, so that a run which stops at input can be diagnosed without reading the validator. The third column quotes the sentence the refusal opens with, shortened where the message names a list of options, and the section column points at the prose that explains the rule. The matrix is a map of the rules, not a substitute for them: a rule whose explanation matters is in the section named beside it.

Area	Refused combination	Reason or message
jobtype	A basic token together with an advanced token, as in energy md, or two advanced tokens, as in train adaptive	Basic options are freely composable and advanced options must be used alone, so either combination is refused with both vocabularies printed (Section 5.1).
jobtype	The retired tokens hop and hybrid	Both tokens have been removed, and the message names the replacement: surface hopping is md with sfhp, and a hybrid is a composite run (Section 5.1).
jobtype	A token that is not in either vocabulary	Unknown jobtype, with the basic and advanced vocabularies printed in the message (Section 6.1).
theory	A label that is not a registered skill	The unknown label is refused at validation, so a typo fails at input rather than at the first dispatch (Section 5.1).
reference	nn in the reference queue, or more than three layers, or a layered theory with more than one reference layer outside adaptive sampling	The reference labels structures and cannot be a network; the layer count is bounded, and a layered theory accepts one reference layer unless the jobtype is adaptive (Section 5.1).
multiscale_type	A value other than qmmm or oniom, or qmmm with three theory entries	The key names a two-entry scheme only; three theory entries are always ONIOM3 (Section 7).
multiscale_type	qmmm with a control.reference queue that is not an adaptive reference	The two providers come from control.theory; control.reference must be empty (Section 7.1).
multiscale_type	A legacy qmmm_key declaration	qmmm_key is retired as a QMMM declaration; write multiscale_type qmmm with two theory methods and molecule.highlevel (Section 7.5).
QMMM	No highlevel, or a highlevel that covers every atom	QMMM needs a proper QM region and a non-empty classical region (Section 7.1).
QMMM	A q_qm set whose length is not the QM set, q_qm with several centers, or sc_charge with one center or without embedding	The charge order is the whole QM set in global order; a pinned set answers for one center only, and the charge iteration needs a field and a second center (Section 7.4).
embedding	ONIOM2 with a classical outer method and embedding true	Not supported; the message sends the run to multiscale_type qmmm, where the QM calculation receives the MM field (Section 7.2).
embedding	ONIOM2 with several centers and read_charge, or read_charge together with external_charges	One static charge file has no single binding for per-center environments, and two background-field declarations are one too many (Section 7.4).

Area	Refused combination	Reason or message
PBC	Mixed pbc flags, or a molecular backend as the whole-system layer	Partial periodicity is not supported; the outermost layer must carry the box, and a molecular backend is allowed only as a center region (Section 8.4).
PBC	A soft wall in a periodic system	A soft wall is a spring toward one point and has no shell to check against a cell face; keep the hard law or run without periodicity (Section 5.13).
md	nve with a thermostat, nvt without one, npt without a barostat or thermostat, or dissipate on an nve run	Each ensemble states which components it owns; the message names the ensemble that would accept the keyword (Section 8.3).
md	A barostat paired with a thermostat its derivation does not accept, or scr in the anisotropic cell mode	Parrinello-Rahman and MTTK are extended-Lagrangian schemes with a defined pairing; scr shears a liquid and supports the isotropic cell only (Section 8.3).
adaptive	dynsample, refine or transfer set to a non-zero value	The keyword is not ported in this adaptive build and must be 0 (Section 10.5).
adaptive	load other than 1, zero sample_energy or sample_grad, a maximum threshold below its minimum, or an unknown analysis scope	A round starts from the supplied network, trains on energy and gradient, and its screens are ordered (Section 10.5).
hessian	hess_driver main with hess_mode analytic or auto, or hess_driver skill on a backend without a frequency capability	The framework has no analytical second derivative; main is the displacement batch, and a skill driver needs a backend that produces the matrix (Section 6.6).
driver	A non-main driver with more than one theory skill, a driver name that is not the first skill, or a non-main driver with a path method or optimize_cell	A skill owns the whole iteration, so exactly one skill can own it; a variable-cell search is a framework loop (Section 5.1).
optimize	opt_pressure or opt_cell_coupling without optimize_cell	With the box held there is no lattice derivative for a pressure to act on (Section 5.4).
optimize	A hypersphere, IRC or band key on a method that cannot read it, a band without both endpoints, or irc_mode molden without a mode file	Each path keyword answers a question exactly one walk asks, and a walk without its endpoints or its imaginary mode has nothing to start from (Section 6.4).
search	sort_stat or sort_order outside its vocabulary	A misspelled statistic is not allowed to fall back to the default, because the default answers a different question (Section 10.4).
read	path, project or profile without jobtype read, a read run without a path, a read run that asks for no label, or a read run with a non-main driver	Extraction never starts an engine: it needs one theory, no reference, the framework driver and at least one requested label (Section 6.7).
constraint	A retired flat restraint key, a ramp that drives a positive parameter to zero, endpoints shaped unlike the value they replace, or an inside or outside bias region	A restraint that arrives nowhere is worse than a refusal, so the old spellings name their block; the ramp and region rules are stated with the block (Section 9.4).

Two refusals are worth separating from the list because they are capability limits rather than vocabulary errors. A molecular backend cannot carry a periodic box, and a stress-driven barostat needs a backend that returns a stress tensor; both are refused where the plan is built, with a message naming the layer that must change. A run that reaches an engine and fails there is a different class of problem, and Section G covers it.

Warnings are not refusals. An ONIOM2 run with a classical outer method and embedding off is accepted with a warning that the same additive surface is spelled qmmm, because the calculation is well defined even though the spelling is not the recommended one. A skipped capability gate, which happens when skill discovery fails during a partial installation, is also a warning; it is printed so that a periodic run does not silently proceed on a backend nothing checked.

E. Migrating PyRAI2MD inputs

PyRAI2MD is the predecessor of PyRAI3MD, and most of its input vocabulary survives in 3.0.0 under a modern spelling. This appendix maps the sections and keywords that moved, so that an old input can be brought forward deliberately instead of by trial and error.

The molecular section is the part that changed least. The atomic symbols, coordinates, velocities, charges, multiplicity, ci, spin, coupling, highlevel and midlevel keep their names, their meaning and their index bases; only the section name moved, and the old name is still read. What changed around it is the method queue: qm and abinit became theory and reference, and the labels inside the queue are now skill names.

2.x spelling	3.0.0 spelling	What changed
&MOLECULE	&SYSTEM	SYSTEM is the new name; MOLECULE is still accepted and is the spelling most examples use. The fields inside keep their names and 1-based atom numbers.
&MD	&DYNAMICS	The section is read under either name. In YAML and agent JSON it is the md mapping; the dynamics spelling is the long form.
qm	theory	The method queue, written high to low. Each entry is a skill label; qm.pyscf and abinit.orca are stripped once and read as pyscf and orca.
abinit	reference	The labelling queue of adaptive sampling. It accepts the same labels as theory except nn.
jobtype train, adaptive, md	unchanged	The three workflow tokens keep their spelling and their meaning.
jobtype hop	refused	Surface hopping is md with sfhp set to fssh, gsh or lzsh and the nactype keyword beside it. There is no separate hop jobtype.
jobtype scan	refused	There is no scan jobtype and no scan section in 3.0.0; the Scan class is a registered placeholder. A reaction path is written as an optimize run with the path family, and a hyperparameter scan is the search jobtype.

2.x spelling	3.0.0 spelling	What changed
NN, MLP, E2N2, SCHNET sections	nn_pes section	One skill section replaces the per-architecture sections. profile selects the computation chain and method selects the architecture inside it.
EG, NAC, SOC hyperparameter sections	nn_pes role blocks	The eg, nac and soc blocks of the nn_pes section carry the architecture and training settings for the energy-and-gradient, coupling, and spin-orbit roles.
MOLCAS, ORCA, BAGEL, XTB sections	skill sections	Each keeps its own name as a skill section and gains execution_mode and profile. The section is read only when the skill is in the theory or reference queue.
use_hpc	multitask and remotetask	use_hpc 0 is multitask without remotetask; use_hpc 1 is both. The two switches answer the two questions separately, which is what lets a run fan out on one host without a scheduler.
ml_ncpu	multipool	The pool size of the workflow section that fans out. It is a per-section keyword now, so the md fan-out and the reference fan-out can differ.
control ml_ncpu, control qc_ncpu	removed	Neither is a control keyword. Parallelism is declared by the section that owns the fan-out, as multipool, and a control keyword is no longer read for it.
initcond 1	initcond wigner	The numerical spelling of the sampling modes is gone. initcond names the source: none, read, wigner or boltzmann.
&FILE and FILE extraction	jobtype read	The read section names the path, the profile and the labels to extract; a dataset built from finished artefacts is then written by tools/training_data.py.
restraint keys in &MOLECULE	the constraint block	The flat freeze, cbond, cavity and shape keys are refused with the block keyword that replaces each one. In YAML the block is molecule.constraints; in classic input it is &CONSTRAINT.

Three habits carry over unchanged and are worth keeping. A title is still the stem of every product, so two runs in one directory still overwrite each other. The state ladder is still written as ci and spin, with coupling naming the pairs, and the pair maps are still derived rather than written. The highlevel and midlevel regions are still 1-based atom lists.

Two habits have to change. A method queue is now a skill queue, so a label that was a program name in 2.x is a skill name in 3.0.0, and an unknown label is refused at validation rather than at the first calculation. A workflow token is now the only thing that selects a workflow: composition of basic property tokens is allowed, and mixing a property token with a workflow token is refused, which catches an input that expected the old precedence rules.

The keyword status marks in Section 5 are the quickest way to tell a moved keyword from a retired one. A dagger means the keyword is accepted and read by nothing, a section sign means it is an

alias, and a refusal means the replacement is named in the error. Section C lists the aliases, and Section D lists the refusals.

F. Output file index

Every file the framework writes is named after the run title, except the three fixed names a child runtime uses and the scheduler logs, which carry the job identifier. The table below is alphabetical by the name as it appears on disk; a title in angle brackets stands for the control title, so a run titled water writes water.md.xyz. Section 12.2 describes the same set by producer, and Section 12.1 describes the log blocks that carry the same numbers.

File	Writer	Contents
<title>.adaptive/	adaptive workflow	The run directory of an adaptive run: the round record, the run log, and one iter-<k> directory per round with the data snapshot, the model, the reference labels, the MD folders and the round status.
<title>.charge	hand-written or certification converter	Background charges or an external field: a header naming the kind, the unit and the row role, then q x y z rows. Read by a run that sets read_charge.
<title>.freq.molden	hessian workflow	Frequencies in cm-1 and the mass-weighted modes in the Molcas layout. The initial-condition sampler looks for this exact name.
<title>.fssh	hopping integrator	Per-step FSSH amplitudes, written only at verbose 4 or above. It is a diagnosis record and grows with every electronic step.
<title>.init	sampling and adaptive warm-up	The sampled batch in the legacy layout. It is the same batch the sampler hands to the trajectories.
<title>.init.xyz	sampling and adaptive warm-up	The multi-frame starting conditions with velocities, one frame per trajectory. Read by initcond read, and it is the file to edit when a run must start from chosen structures.
<title>.json	tools/traj_analyzer.py	The arrays of a trajectory extracted for later analysis, written when save_traj is on. Modes 4 to 7 read it back instead of re-parsing the trajectory.
<title>.log	run logger	The run log: setup, one result block per evaluation under a step header, warnings, and the conclusion. Every other file carries a subset of these numbers.
<title>.md.energies	md workflow writer	One row per recorded step: step, active-state potential energy, kinetic energy and the per-state energies.
<title>.md.ensemble	md workflow writer	One row per recorded step of a thermostatted or barostatted run: temperatures, kinetic energy, volume, chain and barostat state, and the pressures in bar. An NVE run writes no such file.
<title>.md.pop	md workflow writer	State populations followed by a one-hot vector marking the post-hop active state. The two blocks together are how a hop is read from disk.
<title>.md.stress	md workflow writer	Step, the nine components of the active-state stress tensor in Hartree/Bohr^3, and the derived

File	Writer	Contents
		pressure in bar. Written only when the engine supplied a stress tensor.
<title>.md.velo	md workflow writer	Velocities of every recorded step in Bohr per atomic time, in the same atom order as the geometry.
<title>.md.xyz	md workflow writer	Structure of every recorded step in Angstrom. A periodic frame states its cell in the comment as a Lattice key, and a frame on which the active state changed carries a to <n> CI suffix.
<title>.rt	md checkpoint	The serialized runtime state: geometry, velocities, history, electronic density, step count and generator position. Launching it continues the trajectory; the md checkpoint keyword sets the cadence, and no file is written when it is zero.
<title>.sh.energies	md workflow writer	The energies record at hopping steps only, with the same columns as the md file.
<title>.sh.pop	md workflow writer	The population record at hopping steps only, carrying the post-hop active state.
<title>.sh.velo	md workflow writer	Post-hop velocities at hopping steps, which is where the rescaling or reflection across the gap is visible.
<title>.sh.xyz	md workflow writer	Structure at hopping steps only. The sh files are written immediately on a hop, independent of the direct and buffer cadence that governs the md files.
md.charges, sh.charges	md workflow writer	The managed-region point charges q x y z of each recorded frame and each hopping frame. Written only when charge management is active; a partitioned run also writes md.<n>.charges per center.
opt.xyz	optimize workflow	The latest accepted structure, overwritten every step. It is the file to copy into the next input when a run is chained.
opt_geom.xyz	optimize workflow	Every accepted frame appended in order, which is the trajectory of the search rather than its latest state.
opt_status.txt	optimize workflow	The convergence table: a fixed header, then one row per accepted step with step, energy, shift, max_d, rms_d, max_g and rms_g. A cell-relaxing run adds the cell columns after the fixed ones.
path.xyz	path workflow	The latest frame of a path search, rewritten as the walk advances.
path_geom.xyz	path workflow	Every accepted frame of the walk or every image of the band, which is the path itself.
path_profile.txt	path workflow	One row per accepted point or per image, carrying the profile the search reports at the end of the run.
path_status.txt	path workflow	One row per evaluation for a walker, or per round for a band. It is deliberately not opt_status.txt, because its columns answer different questions.
path_forward.xyz, path_backward.xyz	path workflow	The two sides of an IRC walk when both directions are requested.

File	Writer	Contents
pyrai3md_step.perf	step profiler	Phase timings of the md step when perf_profile is on. It is a performance record, not a trajectory record.
result.json	child runtime	The result payload a dispatched runtime returns to its parent: energies, gradients and the metadata the parent needs to continue. Its absence is how a failed child is detected.
slurm-%j.err, slurm-%j.out	scheduler	Standard error and standard output of a submitted child, named with the job identifier unless the slurm section sets explicit names.
summary.csv	grid search	The ranked table of a hyperparameter scan, one row per grid point with the ranking statistics of its ensemble.
term_outcome.json	dispatch term worker	The unified outcome of one remote term: the result the worker produced plus its status. The parent reads it instead of parsing the worker log.

Three naming rules explain most of the surprises. A record that is rewritten every step has no step number in its name, so opt.xyz and path.xyz always describe the newest state. A record that is appended carries the step in its first column rather than in its name, so the file stays one table. And a record that is conditional is absent rather than empty: no stress tensor means no md.stress, no extended ensemble means no md.ensemble, and no checkpoint keyword means no runtime file at all.

A run directory that is reused can therefore contain a mixture of records from two runs. The framework truncates the files it owns at startup, but it does not delete a file a previous run wrote under a different title, and it does not delete the adaptive directory of an earlier run with the same title. Check the title and the modification times before reading a record whose step numbers do not start where the log says they should.

G. Troubleshooting

This appendix pairs the failures a new user meets with the cause and the action. The messages are quoted from validation and from the run log; the sections named in the action column hold the full rule.

Symptom	Likely cause	What to do
pyrai3md: command not found	the package is not installed in the active environment	run pip install . in the checkout, or activate the environment that has it
the input stops at "missing atoms or coordinates"	no atoms array, no coord path and no same-name .xyz file	add top-level atoms and coordinates, or point coord at a file, or place <input stem>.xyz next to the input
the input stops at "unknown jobtype"	a retired or unsupported token such as hop, hybrid or scan	use md with a surface-hopping algorithm, a composite method queue, or an optimize path method; see Section 6.1

Symptom	Likely cause	What to do
"theory label is not registered"	a misspelt method label, or a skill whose directory is absent	check the label spelling, then list the installed skills with the skill query module
"capability is not available from the selected skill"	the jobtype asks for a property the engine does not declare	consult Section 2.2 and the skill limitations, or choose another engine
"executable not found" or the engine starts and exits at once	the engine path is empty or wrong, or the module environment is not loaded	set the path in ~/.pyrai3md/pmd_env.yaml under the skill section, load the site module the engine needs, and rerun
a periodic request is refused	the chosen engine does not declare periodic support	use cp2k, xtb with GFN-FF, or lammps; Section 8.4 lists the support
"partial periodicity is not supported"	pbc was given one or two flags	give three flags, all zero or all one, and a real cell
npt is refused	the barostat and thermostat pair is not allowed, or one of them is off	use the pairing table in Section 6.5, for example Berendsen with Nose-Hoover
the Hessian request is refused at validation	hess_driver main combined with analytic or auto, or a variable cell with a vibrational analysis	use numerical for main, skill for a backend matrix, and drop the cell relaxation while analysing vibrations
a path method is refused	driver is not main, or keywords of another path method are present	set driver main and keep only the keywords of the chosen method; the groups are listed in Section 6.4
adaptive stops at startup	load is not 1, transfer is not 0, or refine or dynsample is non-zero	remove those keys; this version does not implement those branches
read keys are rejected	a read section is present without jobtype read	set jobtype read, or remove the read section
the table of contents is empty or its page numbers are stale	the contents is a Word field and has not been updated	open the file in Word and update the field, or accept the prompt on open
no log file appears	the run wrote into another directory, or the scratch path changed	look for <title>.log beside the scratch root printed by --info
a Slurm child fails immediately with an invalid partition	the partition in pmd_env.yaml does not exist on the host	set partition to names the scheduler accepts; an empty value writes no directive
a trajectory ends before the requested number of steps	a guard fired: population, energy, gradient, coupling or tracked geometry	read the stop code in the log (Section 6.5) and widen the corresponding md_max* or track threshold
the engine runs out of memory	too many ranks or threads per node, or too small a memory directive	lower thread or nproc, raise mem in the slurm section, or reduce the system
a model cannot be found	modeldir does not contain the profile or the trained folder	set modeldir and profile in the nn_pes section and check the model folder layout in Section 10.2
training does not use the GPU	gpu is -1, or the submission has no gres	set the GPU index and add gres to the slurm section
two runs of the same input differ slightly	different thread counts, or a seed that was not fixed	fix gl_seed, keep the thread count equal, and compare on the same build

Symptom	Likely cause	What to do
a shipped example fails on this machine	the example binds an engine that is not installed here	run it with --info first to see the resolved plan, then switch the theory queue to an installed engine

When a message is not listed here, read it literally: validation names the offending key and often the replacement, and the run log ends with a status line that separates a refusal, a failed calculation and a guard stop. Appendix Section D collects the refusal matrix.

H. Glossary

The terms below are used with this meaning throughout the manual.

Term	Meaning
ab initio dynamics	molecular dynamics whose energies and gradients come from an electronic-structure calculation at every step
adiabatic state	an eigenstate of the electronic Hamiltonian at the current nuclear geometry
adaptive sampling	an iterative loop that runs trajectories on the current model, selects uncertain structures, labels them with a reference method and retrains
barostat	the part of an NPT integrator that changes the cell to hold a target pressure
capability	a property or operation an engine declares, such as energy, gradient, nac, soc, stress or field; the planner reads these declarations
child runtime	a serialized run that the parent launches as a separate process or scheduler job
CI space	the configuration-interaction ladder given by ci: how many states each spin multiplicity contains
composite	a method queue of two or three engines whose results are combined, as in ONIOM or QM/MM
constraint	an imposed geometric condition: a bias, a wall, a fixed atom, SHAKE or a metadynamics bias
decoherence	a correction that removes unphysical population transfer between trajectories in surface hopping
driver	the owner of an iteration: main for the framework, skill for the engine itself
ensemble	the statistical-mechanical setting of a trajectory: NVE, NVT or NPT
ESPF	electrostatic potential fitting, the way the classical environment is passed to the quantum region in QM/MM
FSSH	fewest-switches surface hopping, a stochastic algorithm that distributes trajectories over electronic states
GSH	the surface-hopping variant that propagates a pure-state density and needs all-state gradients
kTDC	curvature-driven time-derivative coupling, a gap-based approximation to the nonadiabatic coupling
LZSH	Landau-Zener surface hopping, which needs state energies rather than couplings
MECI	minimum-energy conical intersection, the lowest point of a crossing seam between two states
MECP	minimum-energy crossing point between two spin states
minimum image	the convention that a pair interaction uses the closest periodic copy of the partner

Term	Meaning
multiscale	a calculation that combines methods of different cost on different regions of one system
ONIOM	a subtractive composite scheme in which a small region is treated at a high level and the rest at a lower level
PBC	periodic boundary conditions
plan	the resolved list of tasks, engines and resources that a request produces before any calculation starts
profile	the computation chain an engine runs, for example a gradient profile or an optimization profile
reference queue	the method queue used to label structures during adaptive sampling
skill	a plugin that owns one engine: its schema, input builder, runner and parser
surface hopping	a mixed quantum-classical dynamics method in which nuclei follow one adiabatic surface and occasionally switch
thermostat	the part of an NVT integrator that holds a target temperature
TS	transition state, a first-order saddle point of the potential energy surface
UDD	undersampling-driven deviation, a bias added to a region the training data covers poorly